

当 AI 成为执行者，研发组织必须重新设计

Agentic Agile

智能体敏捷

从“碳基协作”到“硅基自治”

构建 AI 时代的人机协同研发操作系统

王立杰 著

AI 不是在替你写几行代码，
而是在重构整个研发体系。

Agentic Agile 智能体敏捷：从碳基协作到硅基自治

构建 AI 时代的人机协同研发操作系统

人员、组织、流程与工具的系统重构

作者：王立杰（无敌哥）

适用人群：CTO、研发管理者、AI 转型负责人、产品与技术负责人、架构师、研发效能与敏捷负责人、FDE、AI 工程团队及一线开发者

核心方法与工具：Agentic Agile 3-4-3、SCOPE-V、验证工程与配套 Skill

书稿版本：v4.0 工作稿

本版本强化 AI First / AI Native 设计原则、组织级分形扩展。

前言 | 当 AI 成为执行者，研发组织必须重新设计

AI 带来的变化，已经不只是“写代码更快”。

今天，一个能够读取仓库、调用工具、修改文件、运行测试并持续修复的 Agent，正在从代码助手变成研发执行者。过去按天计算的分析、编码和验证，开始按小时甚至分钟完成；过去需要多个岗位接力的任务，也开始被一个人和一组 Agent 并行推进。

这是一场真实的生产力跃迁。可很多企业从采购 AI 工具起步，最后只得到零散的个人提效，组织效率并没有明显改善。

代码生成得更多，需求仍在排队；Agent 执行得更快，决策仍在会议中等待；个人产出上升，测试、审核和返工却成为新的瓶颈；工具数量不断增加，目标、权限、责任和证据仍然散落在聊天、文档和个人经验里。

问题不只是模型偶尔出错，而是企业把新的智能生产力塞进了旧的岗位、旧的组织、旧的流程和旧的工具链。

三次“换操作系统”的生产力跃迁

历史上的通用技术，很少在“装上去”的那一刻兑现全部生产力。真正的跃迁，通常发生在组织围绕新能力重新设计生产方式之后。

第一次，是工厂电气化：从替换动力到重构空间。 早期工厂只是用大型电动机替换蒸汽机，仍然保留中央传动轴、皮带、楼层和机器布局。动力源变了，生产系统没有变，因此并未立即出现与新技术相称的生产率跃迁。后来，“单机驱动”让每台机器拥有独立电机，机器不再围绕传动轴摆放，工厂得以采用更轻的单层建筑，并按照材料和工艺流重新配置机器。电力的价值由此从“更方便的动力”变成了“重新设计工厂的自由”。¹

第二次，是福特的移动装配线：从重构空间到重构工作流。 机器可以灵活布置，并不意味着生产会自动形成连续流。福特进一步改变的，是工作本身的组织方式。1913 年，海兰帕克工厂将可互换零件、任务分解、标准作业、工位衔接和连续物料流组合成一套新的生产系统。产品开始移动，工人在固定位置完成明确任务，前一道工序的产出按照稳定节拍进入下一道工序。到 1914 年初，生产一辆 Model T 所需的时间由固定工位方式下的约 12.5 小时缩短到 93 分钟，效率接近原来的八倍。² 工人没有突然快八倍，真正被重构的是等待、搬运、切换和协作方式。

电气化让机器可以重新摆放；移动装配线让工作可以重新流动。

第三次，是集装箱化：从优化单点到重构价值网络。 20 世纪 50 年代，远洋运输已经拥有更大的船、更强的起重设备和更高效的发动机，货物抵达港口后却仍需逐件装卸、清点和仓储，再分别转入卡车或铁路。真正昂贵的往往不是海上航行，而是货物在船舶、港口、仓库、铁路和公路之间反复交接。当时，装卸环节一度占到运输成本的近一半。

1956 年，Malcolm McLean 的 *Ideal-X* 携带 58 个集装箱从新泽西驶往得克萨斯。真正产生革命性影响的，不只是一个标准化钢箱，而是一整套围绕标准箱体重新设计的系统：船舶结构、港口吊机、码头布局、卡车底盘、铁路运输、装卸规则、安全标准和责任交接都随之改变。货物不必在每次换乘时重新拆装，而可以用同一个标准化载体穿过海运、公路和铁路。集装箱没有让船航行得更快，却大幅降低了转运成本和装卸时间。³

集装箱的价值不只来自标准化一个箱子，而来自围绕这个箱子重新设计整个运输网络。

三次变化说明同一个规律：**替换工具，只会得到局部效率；重构生产方式，才会释放通用技术的系统价值。** 这才是“创造性破坏”真正可见的部分：旧系统不是被一个新部件替换，而是整套生产方式被重新组合。这也正是本书所说的“换操作系统”。

AI 时代同样如此。给每位工程师安装 Copilot，类似于用电动机替换蒸汽机却保留旧传动轴；让 Agent 在旧需求、旧审批和旧交接中跑得更快，也可能只是把堵点推向测试、审核和管理者。**真正的转型，是围绕 Agent 的新能力重新设计人员责任、组织决策、研发流程和工程工具。**

回到组织架构上，今天很多公司的结构仍然是 CEO → VP → 总监 → 经理 → 员工。这是一种从顶向下的组织设计：信息和决策沿着层级传递，执行能力被固定在岗位和团队里。AI 原生组织未必需要简单地减少层级，**更关键的是把任务、决策权和执行能力重新组合**：少量人类责任节点，可以在清晰边界内调度多个 Agent 执行单元，并行推进不同任务。

为了说明这种变化，可以用一个启发式表达：

$$\text{产出} \approx \text{碳基时间} + \text{硅基时间} \times \text{Agent 并行数量}$$

这里的“碳基时间”指人类用于意图澄清、价值判断、系统设计、异常处理和责任裁决的投入；“硅基时间”指 Agent 在授权边界内完成检索、生成、测试、验证和修复的执行时间；“Agent 并行数量”指同时工作的 Agent 或执行单元数量。这个表达只用来解释执行能力如何扩展，不是组织产出的预测公式。真实产出还要扣除上下文准备、协调、验证、返工、等待和风险成本。

再看传统研发组织的工作模式，默认人是主要执行者：岗位承载能力，团队承载协作，流程和会议控制节奏，工具辅助人完成局部任务。当 Agent 开始承接完整任务、跨边界调用工具并以远高于人类流程的速度持续行动，这些默认假设就需要重新审视。

沿着这个思路，AI 研发转型必须同时回答四个问题：

- 1. **人员如何重构**：当大量执行交给 Agent，人还负责什么？谁定义意图、设计边界、处理异常并承担最终责任？
- 2. **组织如何重构**：如何从固定岗位和部门协作，走向“人类责任节点 + Agent 能力网络”？决策权和授权边界放在哪里？
- 3. **流程如何重构**：当机器高速并发执行，哪些 Sprint 仪式、审批和交接仍有价值，哪些应被事件、证据和实时反馈替代？
- 4. **工具如何重构**：如何从彼此孤立的 Copilot，走向具备上下文、编排、约束、验证、记忆、证据和遥测的 Agent 工程系统？

这就是本书重新定义 Agentic Agile 的起点。

Agentic Agile 不是传统敏捷加 AI，也不是一套更重的治理流程。它是面向智能体时代，对研发人员、组织、流程与工具进行系统重构的人机协同研发操作系统。

本书用“从碳基协作到硅基自治”概括这套操作系统的演进方向：从人类承担绝大多数理解、执行、检查和协调，逐步走向 Agent 在已签署意图、可执行约束和明确责任边界内，自主完成执行、验证、恢复与进化。硅基自治不是 AI 获得组织主权，更不是人类退出研发；执行权可以逐步交给硅基系统，价值定义权、不可逆决策权和责任承担仍由人类掌握。

这里的“操作系统”并非某一套软件平台。它描述的是 AI 进入研发执行系统后，价值、责任、协作、运行和反馈怎样共同工作；企业战略、产品管理、HR、DevOps 和行业治理仍有各自的职责。

AI First 与 AI Native：本书的两个设计原则

AI First 要求组织面对新任务、新价值流或新组织设计时，先重新判断人和 Agent 的分工：哪些执行可以授权给机器，哪些价值判断、不可逆决策和最终责任必须由人掌握。它绝不等于“所有事情先让 AI 做”。

AI Native 也不是多采购一个模型或建立统一平台。它要求人员责任、组织边界、流程状态和工具基础从设计之初就适配 Agent 的持续执行、验证、恢复与反馈，而不是把 Agent 外接到旧岗位、旧工单和旧审批链上。

由此可以区分三种状态：

状态	主要特征	不能误判成什么
AI Enabled	在既有岗位、流程或工具中加入 AI 能力	使用了 AI 就完成了转型
AI First	每次新设计先判断合理的人机分工与责任边界	凡事先让 AI 做
AI Native	生产系统从设计之初就适配 Agent 的执行、验证、恢复和反馈	所有旧方法都必须推倒

企业可以让低风险重复任务先进入 AI Native 运行方式，同时让高风险场景保持人主导。是否转型成功，最终由真实价值、工程证据、风险和长期遥测判断。

在这套操作系统中：

- 三大理论基石解释为什么需要外部化认知、建立反馈并治理复杂协作；
- 五条 Agentic Agile 宣言说明发生冲突时优先保护什么；
- 3-4-3 用三个超级角色、四大动态工件和三大自治机制建立运行架构；
- SCOPE-V 把意图、约束、编排、证明、进化与裁决连接成持续控制回路；
- Harness、验证工程与证据包（Evidence Bundle）让 Agent 的完成声明变成可以复查的事实；
- 算力与价值遥测（Compute & Value Telemetry）判断 AI 是否真正创造价值、形成受控自治、提高人机效率并推动系统进化。

这套操作系统还必须能够从任务级别向组织级别扩展。其组织级结构不是传统汇报层级，而是三个分辨率：

战略意图层：企业要让 AI 放大什么价值，拒绝什么风险
↓ 约束继承、投资边界

价值网络层：多产品、多团队、多平台如何围绕价值流协同
↓ 能力编排、协议、证据聚合

任务运行层：一项真实工作如何由人和 Agent 分工、协同、验证、恢复和裁决
↑ Evidence 向上聚合，Telemetry 进入组织慢外环

所谓分形，不是把同一套表格复制到每个层级，而是让各层都以相同的方式处理意图、约束、证据、裁决和反馈。任务层用它判断一项工作是否完成，价值网络层用它判断端到端交付是否改善，战略层则据此决定是否继续扩大投入。控制逻辑可以复用，但每一层都必须回答自己的问题。

治理在这里不是目的，而是操作系统的控制机制。它不是为了减慢 AI，而是让机器的速度能够被组织安全地转化为业务结果。验证工程也不是全书的终点，而是这套操作系统不可缺少的可信执行内核。

提效则是 AI 转型必须兑现的结果。如果组织不能以更少投入、更短周期或更低边际成本创造更多有效价值，就没有理由把工具采用称为转型。但提效不能用代码量、Agent 调用次数或一场漂亮演示证明，它必须由真实业务结果、工程证据和持续遥测共同支持。

因此，本书不从遥远的“全自动研发”开始，而从一个今天就可以运行的最小闭环开始：

业务意图与人类责任

→ 可签署的契约与可执行的边界

→ 人与 Agent 的受控协作

→ 基于真实反馈的证明与修复

→ 独立证据与责任裁决

→ 价值与效能遥测

→ 下一轮人员、组织、流程与工具改进

读完本书，你应当能够回答八个问题：

1. 为什么 AI 工具提效没有自动变成组织提效？
2. 人员、组织、流程和工具分别需要重构什么，四者如何协同？
3. 三大角色（IO、OA、AS）如何重新分配价值、治理和执行责任？
4. 四大动态工件如何把战略意图、任务边界、组织约束和完成证据连接起来？
5. SCOPE-V 与 Harness 如何让 Agent 高速执行、自我纠偏并在越界时停止？
6. 如何用验证工程、Evidence 与 4 层 9 维遥测证明 AI 真正兑现了提效？
7. 如何从一个真实任务扩展到既有系统、多团队、多工具和研发组织转型？
8. 如何把任务运行层的闭环扩展到价值网络层和战略意图层，同时避免把 Agent 数量、Token 或工具激活率误当成组织转型？

本书负责建立完整认知与方法体系，Agentic Agile 3-4-3 Skill 提供跨 AI 工具的可运行能力，课程通过真实项目训练四维转型，而企业实践负责用事实检验这套操作系统的真实成果。

不要只让 AI 生成更多。

让人与 Agent 共同交付更快、更可信、可以持续进化的价值。

给读者的一个建议

读到这里，建议你做一次 **碳基·硅基十问**。

这十问讨论的不是“AI 会不会替代人”这么简单的问题。

真正的问题是：当执行、判断、协作、记忆、授权、优化，越来越多地进入硅基系统之后，组织本身会不会发生一次“换芯”？

碳基的人类还在组织里，但未必继续站在执行中心。硅基的系统开始承担越来越多任务，但它也不能自己证明自己应当做什么、做到哪里、错了由谁负责。

所以，这十问不是技术问卷，而是一张未来组织的认知地图。它的目的不是给出标准答案，而是逼近未来组织最难回避的十个判断。

你可以访问 agentic.iloveagile.me/questionnaire 参加，也可以扫描下方二维码进入：



作者简介



王立杰，AI治理架构师，资深研发效能顾问，工信部研发效能工程师认证专家讲师、PMI-ACP授权讲师、企业级规模化敏捷SAFe认证咨询师(SPC6)、华为云MVP（最有价值专家），曾任京东首席敏捷创新教练、IBM客户技术专家、DNV高级咨询师等，帮助小米、OPPO、海康威视、京东方、吉利亿咖通、晓羊教育集团、中远海运、京东购物APP、京东到家、招商银行、工商银行等组织实现研发效能提效。畅销书《敏捷无敌》、《京东敏捷实践指南》作者，江湖人称“无敌哥”。

教授经典课程：

- 《Agentic Agile（智能体敏捷）：从碳基协作到硅基自治——AI时代研发组织重构与转型沙盘》
- 《Agentic AI 项目管理：企业项目管理的人机协同新范式》
- 《AI时代研发治理与领导力：从“驭人”到“驭智”高管研修班》
- 《乌托邦计划沙盘：跨部门协同与高效项目管理特训营》
- 《敏捷项目管理实战沙盘演练》
- 《DevOps黑客马拉松》
- 《创新设计思维Design Thinking》
- 北大光华管理学院/新华都商学院《创业机会分析与识别》
- 京东大学《京东创新之路》、《从0到1商业模式快速探索》

曾经提供培训与咨询的企业包括宝马、博世、吉利、上汽、广汽传祺等车企，百度、京东、小米、Oppo、微博、360、58同城、美团等互联网企业，中国移动、联通、Agilent、IBM、阿朗、爱立信、诺基亚、东软、华为等传统电信/IT/软件企业，招商银行、工商银行、中信银行、中国银行、山东城商行等金融企业，E人E本、长虹、海尔、美的等白电企业，海天建筑、同方威视、中大医疗等传统行业；曾经在“AgileChina敏捷中国、RSG、51CTO、MPD、质量竞争力大会/TiD”等大会做过多次演讲，被评为质量竞争力大会/TiD 2014最受欢迎10大讲师；目前专注于企业AI组织转型、研发效能提升、企业产品创新。

全书结构

Agentic Agile 智能体敏捷：从碳基协作到硅基自治

- 构建 AI 时代的人机协同研发操作系统
- 前言 | 当 AI 成为执行者，研发组织必须重新设计
 - 三次“换操作系统”的生产力跃迁
 - AI First 与 AI Native：本书的两个设计原则
 - 给读者的一个建议

作者简介

全书结构

- 一条主线
- 六部分分别回答什么
- 三条阅读路径

第一部分 | 为什么 AI 时代需要新的研发操作系统

- 本部分阅读路径

第一章 | 新生产力进入旧组织：为什么局部提效没有变成组织提效

- 1.1 AI 改变的不只是工具，而是执行主体
- 1.2 局部提效为什么没有自动成为组织提效
 - 1.2.1 Anthropic 案例：代码生成率不等于研发效率
 - 这条信息是怎样被误解的
 - 先把三个数字分开
 - 应该怎样理解这个案例
- 1.3 从工具采用到智能体组织：四种不同变化
- 1.4 旧研发系统的四个默认假设
- 1.5 氛围编程的真正风险：探索模式进入正式交付
- 1.6 更快的执行，需要更快而不是更多的控制
- 1.7 同一个报销需求，为什么“全绿”仍然失败
- 1.8 判断转型是否发生，不能只看 AI 使用量
- 1.9 本章小结：AI 首先是一场生产方式变革

第二章 | 四维重构：人员、组织、流程与工具必须同时改变

- 2.1 一张图看懂四维重构
- 2.2 必须协同改变，但不能“大爆炸式转型”
- 2.3 人员重构：迁移任务，不是预测岗位消亡
- 2.4 组织重构：从岗位接力到人机责任网络
- 2.5 流程重构：让机器运行快内环，让人承担责任外环
- 2.6 工具重构：从 Copilot 孤岛到可替换的工程系统
- 2.7 治理与度量为什么不是第五、第六维
- 2.8 用报销价值流检验四维是否连通
- 2.9 四维转型的最小诊断法
- 2.10 本章小结：四维不是四条线，而是一套系统

第三章 | 理论基石：认知科学、控制论与系统论

- 3.1 认知科学：复杂度不能只放在人脑里
 - 3.1.1 工作记忆为什么重要
 - 3.1.2 认知负荷的三种来源
 - 3.1.3 邓巴数与协作网络
 - 3.1.4 大上下文不等于全局理解
 - 3.1.5 从认知外包到认知治理
 - 3.1.6 人的意图与判断如何被增强
- 3.2 控制论：没有可靠反馈，速度只会放大偏差
 - 3.2.1 从开环执行到闭环控制
 - 3.2.2 负反馈、强化回路与稳定性
 - 3.2.3 从敏捷反馈到 Agent 运行时反馈
 - 3.2.4 反馈时延、噪声、增益与覆盖
 - 3.2.5 闭环自治何时必须让人接管（HITL）
- 3.3 系统论：以受治理的多样性应对复杂性
 - 3.3.1 必要多样性：治理能力必须匹配环境复杂性
 - 3.3.2 多 Agent 提供多样性，但不会自动形成涌现

3.3.3 协调债：Agent 多样性也会制造新的复杂度

3.3.4 Work Graph：把协调关系变成可执行结构

3.3.5 系统对齐：防止局部最优破坏整体目标

3.3.6 韧性闭环：为无法消除的失败设计恢复能力

3.4 三大理论的融合：从理论启示到工程机制

3.5 本章实践：用三副镜子诊断一个 AI workflow

3.6 本章小结

第四章 | Agentic Agile 五条宣言：先有价值取舍，再有方法

4.1 意图锚定，胜过详尽规约

4.2 多模态涌现协作，胜过仅依赖跨职能人类团队

4.3 算法规则与架构韧性，胜过仅依赖流程纪律与仪式

4.4 实时遥测与闭环对抗，胜过仅依赖阶段性复盘

4.5 基于验证的系统工程，胜过随性提示与氛围编程

4.6 五条宣言如何共同工作

4.7 把宣言用于一项真实转型决策

4.8 本章小结

第二部分 | Agentic Agile：人机协同研发操作系统架构

本部分阅读路径

第五章 | 系统总览：四维转型、3-4-3、SCOPE-V 与遥测如何组成整体

5.1 总装图与职责分层：从价值意图到组织进化

5.2 从任务闭环到企业扩展：三个运行范围

5.3 3-4-3：相对稳定的运行架构

5.4 SCOPE-V：把架构变成持续控制回路

5.5 三种时间尺度：机器快内环、任务控制闭环、组织慢外环

5.6 Evidence 与 Telemetry：一次裁决和长期进化

5.7 四维重构怎样落到运行系统

5.8 一个报销规则如何穿过整套操作系统

5.9 最小可运行操作系统

5.10 总览阶段最常见的三种误判

5.11 本章小结：从模型集合到运行系统

第六章 | 三个超级角色：从岗位分工到人机责任网络

6.1 从岗位名称转向任务责任

6.2 IO：意图主理人

6.2.1 IO 的判断能力如何被增强

6.3 OA：编排架构师

6.4 AS：自治蜂群

6.5 分开执行、判断与责任

6.6 显式签署契约为什么重要

6.7 三个角色之间的制衡

6.8 授权原则：运行时的最小分界

6.9 最小人机责任单元

6.10 一个任务中的角色协作

6.11 角色反模式

IO 只提需求，不承担签署

OA 退化为 Prompt 工程师或流程协调者

AS 被描述为确定性机器

人类只在出事后介入

用自治率评价角色成功

6.12 角色能力如何成长

6.13 本章小结

第七章 | 四大动态工件：让组织共同语言可以被机器执行

本章导读：三层架构中的工件分辨率

7.1 四大动态工件的闭环关系

7.2 意图图谱：全局导航与组织记忆

为什么传统需求列表在 AI Coding 时代还不够

为什么 AI 越强，越需要意图图谱

意图图谱包含什么

意图图谱怎样产生

一个费用报销图谱是怎样长出来的

图谱的常见失效

7.3 意图契约：单次任务的执行基准

为什么有了图谱，还需要契约

契约怎样从图谱产生

契约至少包含什么

契约如何防止两个方向的失控

契约与传统 PRD、用户故事有什么不同

契约什么时候需要重新签署

契约也是人的判断增幅器

7.4 约束矩阵：可执行的安全边界

为什么契约明确了，还需要约束矩阵

约束怎样产生

从一句原则到一条可执行约束

六个核心域

Block、Warn、Escalate、Log 有什么区别

约束矩阵的生命周期

7.5 Evidence Bundle：支持裁决的证据包

为什么测试报告还不够

Evidence Bundle 怎样产生

五类证据

证据不是越多越好，而是要映射到判断

最低内容

一个证据包怎样改变发布决定

证据包如何被阅读

证据包的常见失效

7.6 工件一致性：从意图到证据

7.7 工件维护链：生命周期、版本时效与粒度

7.8 工件反模式

7.9 本章小结

第八章 | 三大自治机制与 Harness：让 Agent 获得有边界的执行权

8.1 三大自治机制

机制一：意图注入与对话到契约——给自治执行一个稳定起点

机制二：高频对抗与自净化——让偏差在小范围内尽早暴露

机制三：人类异常裁决与证据包验收——把机器不该决定的事情交还责任主体

三大机制如何连成一条运行链

8.2 从三大自治机制到 Harness 工程承载

8.3 Harness 的六大能力支柱

支柱一：上下文管理——让 Agent 看到“足够且正确”的信息

支柱二：工具系统——区分“知道什么”与“能够做什么”

支柱三：执行编排——把复杂任务从聊天顺序变成工作关系

支柱四：状态与记忆——让跨步骤、跨会话的执行保持连续

支柱五：评估与观测——既判断结果，也看见过程

支柱六：约束与恢复——给高速执行装上护栏、刹车和逃生通道

六大支柱的木桶效应

8.4 四类工程方法：实现手段，不是第四套架构

8.5 轻量模式与完整模式：按风险裁剪

8.6 风险驱动的最小建设路径：先闭环，再扩大自治

8.7 常见概念混淆与机制反模式

8.8 管理者如何判断 Harness 是否有效

8.9 本章小结：自治必须被环境承载

第三部分 | 验证工程与 3-4-3 落地：让操作系统真正运行

本部分阅读路径

第九章 | SCOPE-V：人机协同研发的持续控制回路

9.1 六个持续运行的控制面

9.2 SCOPE-V 与传统 SDLC 的分工

9.3 为什么人机协作时代需要这样的控制回路

9.4 六个控制面分别控制什么

- 9.4.1 S: Specification——控制方向
- 9.4.2 C: Constraints——控制可行动空间
- 9.4.3 O: Orchestration——控制执行关系
- 9.4.4 P: Proof——控制反馈真实性
- 9.4.5 E: Evolution——控制修复与学习
- 9.4.6 V: Verification——控制最终裁决

- 9.5 为什么 E 必须在 V 之前
- 9.6 快内环: $P \rightleftharpoons E$ 的诊断与自愈
- 9.7 独立裁决: V 的审计与放行
- 9.8 慢外环: Telemetry $\rightarrow$ S/C/O 的组织学习
- 9.9 每个控制面都必须产生可检查工作件
- 9.10 五道机械门把控制状态变成不可跳过的事实
- 9.11 一个完整控制回路示例
- 9.12 SCOPE-V 设计的四条底线
- 9.13 与三大自治机制的关系
- 9.14 本章小结

第十章 | Specify: 从模糊需求到可签署意图

- 10.1 需求本身就是第一类治理对象
- 10.2 批判性校准: 只追问会改变结果的问题
- 10.3 从事实与意图图谱裁剪任务上下文
- 10.4 意图契约: 把决定冻结为执行基准
- 10.5 Example Mapping 与 AC: 把规则推进到可签署的验证协议
 - Example Mapping 的四种基本元素
 - 怎样开展一次最小 Example Mapping
 - 从示例走向可执行规格
 - 什么时候需要更强的规格建模
 - Example Mapping 的常见误区
- 10.6 显式签署与前置门: 确认 AC 已可执行
- 10.7 规格漂移
 - 漂移是怎样发生的
 - 先区分发现与漂移
 - 规格漂移的治理流程
 - 如何发现漂移
 - 规格版本不是文档管理小事
- 10.8 签署后的 AC: 保持受控并进入证据链
 - 从“可读”到“可执行”的三个层级
 - AC 要形成覆盖组合, 而不是只写“快乐路径”
 - AC 如何贯穿 SCOPE-V 与证据链
 - 防止 AC 被 Agent 反向操纵
- 10.9 Spec 不是越详细越好
- 10.10 Specify 常见失败
- 10.11 本章小结

第十一章 | Constrain: 把组织原则变成可执行边界

- 11.1 从原则提醒到运行边界
- 11.2 六大核心约束域: 从分类目录到工程控制面
 - STRUCT: 确保治理骨架没有被拆掉
 - DATA: 限制 Agent 可以接触和改变的数据世界
 - BEHAVE: 把业务语义和系统不变量锁进运行行为
 - QUAL: 定义“做到了”之外的工程可接受程度
 - PROC: 证明关键治理动作真实发生, 而非只留下结果
 - STYLE: 限制高概率生成下的代码熵增
 - 六域如何协同: 一项变更往往同时穿过多个控制面
- 11.3 按风险扩展 NFR
- 11.4 MUST、SHOULD、MAY
- 11.5 Tools Manifest 与最小权限
- 11.6 上下文安全
- 11.7 数据安全: 在任务上执行, 在组织层治理
- 11.8 例外、降级与恢复

- 11.9 约束冲突
- 11.10 约束的经济性
- 11.11 Constrain 常见失败
- 11.12 本章小结

第十二章 | **Orchestrate**: 从团队协作到可计算的人机能力网络

- 12.1 从任务列表到 Work Graph
 - 12.1.1 一张工作图由什么组成
 - 12.1.2 它与任务清单、流程图和项目计划有什么不同
 - 12.1.3 为什么经常使用 DAG，但又不能只理解成 DAG
 - 12.1.4 它与 Intent Graph 的区别
 - 12.1.5 费用报销案例：从清单变成可执行关系
- 12.2 每个 Agent 节点的执行契约
- 12.3 Handoff 与跨 Agent 协作
 - 一份可验证的 Handoff 示例
- 12.4 并行是否真的带来收益
- 12.5 编排中的三个关键决策
 - 决策一：串行还是并行
 - 决策二：自动重试还是升级人类
 - 决策三：共享上下文还是隔离上下文
- 12.6 Orchestrate 常见失败
- 12.7 一个最小 Work Graph
- 12.8 本章小结

第十三章 | **Prove**: 证明实现满足契约

- 13.1 AC 不是事后检查清单
- 13.2 编码门：先证明 Red 可信
- 13.3 最小实现：让证据收敛
- 13.4 选择证明组合
- 13.5 多层证明的职责
 - 单元与组件测试
 - 接口与集成测试
 - UI 与业务场景
 - 性能、安全与数据验证
 - 生产前验证
- 13.6 防止同源误判
- 13.7 证明门的状态
- 13.8 什么是无效全绿
- 13.9 证明强度，而不是测试数量
- 13.10 Prove 与后续控制面的边界
- 13.11 本章小结

第十四章 | **Evolve**: 让失败进入系统能力，而不是只进入复盘

- 14.1 失败不是例外，而是反馈
- 14.2 失败分类与回流路由
- 14.3 可自动修复的条件
- 14.4 修复证据：证明问题真的被消除
- 14.5 反思与 Failure Mining
- 14.6 规则债务与约束疲劳
- 14.7 Champion-Challenger 与有界演进
- 14.8 停止条件：自治系统也必须知道何时认输
- 14.9 Evolve 常见失败
- 14.10 Bug 回溯门：完成之后发现错误也必须回流
- 14.11 演进的长期资产
- 14.12 本章小结

第十五章 | **Verify**: 从全绿到基于证据的责任裁决

- 15.1 测试通过不等于上线批准
- 15.2 六维验证
- 15.3 Evidence Bundle 的证据映射
- 15.4 收尾门：没有完整证据就不能宣告完成
- 15.5 人类发布裁决

- 15.6 发布裁决示例
- 15.7 证据如何提高人的判断效率
- 15.8 Verify 常见失败
- 15.9 本章小结

第十六章 | 价值与效能遥测：证明 AI 转型真正兑现了提效

本章导读：遥测的三层聚合边界

- 16.1 从度量到遥测：Agentic Agile 的反馈控制系统
 - 16.1.1 度量纪律：从治理问题到可比较信号
 - 16.1.2 四层九维：观察人机系统是否形成能力
 - 16.1.3 指标联读与反指标：避免局部最优
- 16.2 一份契约一张仪表盘（Dashboard）：任务级治理快照
- 16.3 全局仪表盘（Dashboard）：从任务快照到项目价值与效能视图
 - 为什么双层视图不可缺少
- 16.4 Token、算力成本与价值归因
 - Token 为什么是治理指标
 - Token 成本怎样计算
 - 实测优先，估算必须标记
 - 怎样读每任务 Token
 - 从 Token 成本走向价值归因
- 16.5 遥测真实性与采集闭环
- 16.6 遥测如何驱动动作
- 16.7 遥测常见失败
- 16.8 遥测治理：让数字保持可解释
- 16.9 本章小结

第十七章 | AI 原生交付流水线：让每次发布携带意图、证据与回滚能力

- 17.1 DevOps 底座与 3-4-3 治理
- 17.2 为什么 AI 流水线必须前移到 Commit 之前
- 17.3 隔离与汇合：发布前先保证证据仍然有效
- 17.4 从代码版本升级为交付谱系
- 17.5 建立 Release Manifest
- 17.6 CI 如何承载五道机械门
- 17.7 Build Once, Verify Once, Promote Many
- 17.8 配置与非代码资产同样属于交付物
- 17.9 CD 中的人机权限边界
- 17.10 灰度、Feature Flag 与条件批准
- 17.11 回滚不只是重新部署旧版本
- 17.12 从发布版本回到契约级遥测
- 17.13 最小交付底线
- 17.14 常见失败
- 17.15 本章小结

第十八章 | 端到端案例：从一句话需求到可验证发布

- 18.1 任务入口：一句话不等于意图
- 18.2 Specify：形成可签署契约
- 18.3 Constrain：把底线写进执行环境
- 18.4 Orchestrate：组织有依赖的工作
- 18.5 Prove：先证明实现满足契约
- 18.6 Evolve：把失败变成下一轮能力
- 18.7 Verify：判断当前证据支持什么行动
- 18.8 流水线：把已经证明的能力送到交付系统
- 18.9 发布后：Telemetry 让组织继续学习
- 18.10 案例结论

第四部分 | 四维组织转型：把工程能力变成企业运行方式

本部分阅读路径

第十九章 | 人员与组织重构：从岗位团队到人机责任网络

- 19.1 重构单位是任务组合，不是岗位名称
- 19.2 IO、OA、AS 不是三个新职位
- 19.3 人类不可转移的五类责任
- 19.4 从固定团队到意图单元和动态能力网络

- 19.5 决策权必须与信息 and 风险一起前移
- 19.6 能力成长：从亲自执行到设计人机系统
- 19.7 一个团队如何完成第一次重组
- 19.8 授权矩阵：让执行速度不再等待层级速度
- 19.9 激励与反监控边界
- 19.10 本章小结

第二十章 | 流程与工具重构：从旧流程加 AI 到 AI 原生价值流

- 20.1 旧流程加 AI 为什么只会局部提速
 - 20.1.1 先测价值流，不要先画未来蓝图
 - 20.1.2 识别三类被 AI 放大的流程债
- 20.2 哪些敏捷与工程活动保留，哪些应被替代
 - 20.2.1 用事件替代状态汇报，用证据替代完成声明
 - 20.2.2 一项活动是否保留的四问
- 20.3 AI 原生价值流：从任务传递到持续控制
- 20.4 工具从 Copilot 孤岛走向最小工程系统
 - 20.4.1 六层工具架构：能力分层，事实贯通
 - 20.4.2 先连接现有系统，再决定是否建平台
 - 20.4.3 Build、Buy 与复用的决策原则
- 20.5 跨 AI 工具是架构要求，不是兼容性加分项
 - 20.5.1 稳定协议至少包含什么
 - 20.5.2 能力探测与降级必须诚实
- 20.6 流程与工具共同裁剪
- 20.7 谁运营这套流程和工具
- 20.8 费用报销规则：一条价值流怎样被真正重构
- 20.9 常见反模式
- 20.10 如何判断流程工具重构是否成功
- 20.11 本章小结

第二十一章 | 三层四维协同落地：转型画布、成熟度与 90 天首轮验证

- 21.1 三层四维协同转型
- 21.2 三层四维转型画布：在每一层检查四个改变对象
- 21.3 证据驱动的成熟度
- 21.4 四级演进路径
 - 21.4.1 L1：工具依附期——AI 是个人外骨骼
 - 21.4.2 L2：人机编队期——AI 成为可管理的虚拟同事
 - 21.4.3 L3：受控自治期——3-4-3 接管可验证工作流
 - 21.4.4 L4：硅基自治期——执行闭环自治，价值与责任仍归人
 - 21.4.5 等级跃迁必须同时满足四类门槛
- 21.5 90 天首轮验证：不是等 90 天才跑通闭环
 - 21.5.1 先写试点契约，而不是先安装工具
 - 21.5.2 试点准入：选择能被证明的场景
 - 21.5.3 最小角色与工件
 - 21.5.4 第1—7天：完成真实闭环并在企业场景复现
 - 21.5.5 第8—30天：稳定运行并验证可重复提效
 - 21.5.6 第31—60天：跨场景扩展并验证方法可迁移
 - 21.5.7 第61—90天：形成组织化与规模化决策
 - 21.5.8 四种试点结论
- 21.6 试点运行节奏
- 21.7 试点期的变革沟通
- 21.8 本章小结

第二十二章 | 企业转型路线：从任务运行到价值网络与战略意图组合

- 22.1 三层分辨率：任务运行、价值网络与战略意图层
 - 什么时候需要建立价值网络？
- 22.2 三层如何具体落地
 - 22.2.1 三层责任不是三套审批会
 - 22.2.2 三层工件的最小字段
 - 22.2.3 对照例子：同一个报销规则，什么时候需要建立价值网络
 - 情况 A：无需建立价值网络
 - 情况 B：必须建立价值网络

- 22.3 规模化扩展的单位不是 Agent 数量，而是责任单元
- 22.4 从首个闭环到第二责任单元
- 22.5 多责任单元的共享治理资产要像内部产品一样运营
- 22.6 什么时候值得建设平台管理共享治理资产
 - 22.6.1 从“描述世界”到“让组织能够行动”：本体的来龙去脉
 - 22.6.2 Palantir 的实践：Ontology 是数据之上的运营层
 - 22.6.3 Ontology对企业 AI 转型的五个启示
 - 22.6.4 本体与 Agentic Agile：不是替代关系，而是上下层关系
 - 22.6.5 什么时候企业值得建设本体平台
 - 22.6.6 一个最小本体的落地范围
 - 22.6.7 不应该建设本体平台的情况
- 22.7 连接责任单元：从局部闭环到价值网络
- 22.8 经营价值网络：扩大、保持、收缩与停止
- 22.9 分阶段扩展：一条可执行的路线
- 22.10 本章小结

第二十三章 | 管理与评价体系：从个人产出到人机系统能力

- 23.1 管理对象发生了变化
- 23.2 用系统指标替代工具指标
- 23.3 新管理者的五项责任
- 23.4 绩效评价的三层结构
- 23.5 一个反例
- 23.6 从遥测评审到组织行动
- 23.7 从局部行动到组织级裁决
- 23.8 让激励与评价不被异化
- 23.9 以信任换回真实反馈
- 23.10 本章小结

第二十四章 | 风险与行业裁剪：操作系统相同，参数与责任边界不同

- 24.1 不建立一套适用于所有行业的阈值
- 24.2 六个裁剪问题
- 24.3 金融场景
- 24.4 车载与安全关键系统
- 24.5 政企与涉敏环境
- 24.6 互联网与高频业务
- 24.7 医疗与生命科学
- 24.8 工业、能源与生产控制
- 24.9 把行业差异落到风险分层
- 24.10 Agentic 风险的七类来源
 - 意图风险
 - 上下文风险
 - 数据与隐私风险
 - 工具风险
 - 执行风险
 - 验证风险
 - 组织风险
- 24.11 把风险链转成威胁模型
- 24.12 把供应链纳入风险控制链
- 24.13 把控制失效纳入事故响应
- 24.14 在可验证边界内扩展自治
- 24.15 本章小结

第五部分 | 工程扩展与规模化实践

本部分阅读路径

第二十五章 | Knowledge Engineering：在任务开始前建立可信上下文

- 25.1 三种视图：Document Map、Code Map 与 Trace Link
- 25.2 五层知识结构
- 25.3 知识条目的最低身份
- 25.4 从检索结果到 Context Slice
- 25.5 知识如何进入 SCOPE-V
- 25.6 既有系统 Recon：先确认事实，再提出改变

25.7 常见失败

25.8 本章小结

第二十六章 | Agent Memory Engineering: 让任务经验安全回流下一次任务

26.1 State、Context、Memory、Knowledge

26.2 什么值得记住

26.3 记忆写入门 (Memory Write Gate)

26.4 记忆读取门 (Memory Read Gate)

26.5 记忆生命周期

26.6 记忆冲突与上下文污染

26.7 记忆如何贯穿 SCOPE-V

26.8 既有系统案例

26.9 验收清单

26.10 本章小结

第二十七章 | 规模化协作: 从单任务闭环到多人、多工具、多模块可信交付

27.1 先拆责任域, 再增加协作机制

27.2 所有权: 局部自主, 边界统一

27.3 Protocol 与跨模块契约 XC

27.4 集成 DAG: 把局部完成变成系统完成

27.5 多工具接力: 统一事实源, 不统一界面

27.6 多人共享会话: 协作权不能自动变成执行权

27.7 证据聚合与发布边界

27.8 何时继续扩大, 何时退回小闭环

27.9 本章小结

第六部分 | AI 组织案例

本部分阅读方法

本部分章节

第二十八章 | Kavak: 从二手车平台到 Agent 原生组织

28.1 Kavak 是谁: 它从来不只是一家二手车网站

28.2 转型前的组织与运营: 垂直一体化, 但客户仍在职能之间流动

28.2.1 高增长形成的专业化组织

28.2.2 增长掩盖不了运营断点

28.3 四阶段转型: 从垂直平台到 Agent 原生组织

28.3.1 第一阶段: 算法增强垂直一体化业务

28.3.2 第二阶段: 从“采用 AI”转向“围绕 Agent 重建”

28.3.3 第三阶段: 让 Eval 成为速度、授权与管理系统

28.3.4 第四阶段: 从工作流 Agent 到“一个客户一个长期 Agent”

28.4 组织怎样改变: 从岗位协作到人机能力网络

28.4.1 团队变平, 但不是组织自动消失

28.4.2 Jedi Academy: 转型不只培训 AI 工程师

28.4.3 AI CEO: 大胆实验不等于治理移交

28.5 结果: 经营改善与证据边界必须同时看

内部与外部声音对照

28.6 Kavak 与 Agentic Agile: 相通的是运行逻辑, 不是名词

28.7 六个核心论点: Kavak 究竟改变了什么

1. 从“旧组织加 AI”转向“围绕 Agent 重构组织”

2. 从“一个任务一个 Agent”转向“一个客户一个长期 Agent”

3. 从复制普通劳动力转向复制经过验证的最佳实践

4. 从“人调用 AI”转向“Agent 编排人机能力”

5. 从 Prompt 管理转向 Eval 驱动的管理系统

6. 从固定流程自动化转向可治理的自我改进

28.8 从案例到行动: 企业短期可以做什么

28.9 本章小结: 公司开始像软件一样演化, 但责任不能像代码一样自动部署

28.10 主要资料与证据说明

第二十九章 | EY (安永) 与 Factory Droids: 从编码助手到 Agent 原生工程组织

29.1 EY (安永) 是谁: 一家以专业标准和人才组织交付的全球网络

29.2 转型之前: 软件交付不是写代码, 而是穿越整个工程系统

29.2.1 谁在这个系统里承担什么责任

29.3 三阶段转型: 从个人辅助到半自主工程执行

- 29.3.1 第一阶段：先让工程师熟悉辅助式 AI
- 29.3.2 第二阶段：不由领导指定答案，让真实使用产生选择信号
- 29.3.3 第三阶段：连接“上下文宇宙”，再按工作负载授权
- 29.4 组织怎样改变：工程师从 Doer 走向 Orchestrator
 - 一个容易被忽视的问题：初级工程师如何成长
- 29.5 结果与证据边界：“4 倍”不是一个可独立归因的数字
 - 内部声音与证据属性
- 29.6 与 EY 整体 Agent 战略的关系：一个工程案例，不是全部转型
- 29.7 EY Droids 案例的六个核心论点
 - 1. 编码速度不是交付速度
 - 2. Agent 缺的通常不是 Prompt，而是组织上下文
 - 3. 不要把采用热度误当成授权证据
 - 4. 自治度应该跟随工作负载，而不是跟随工具名称
 - 5. 工程师从执行者转向编排者，但责任不会转移
 - 6. 生产率改善是社会技术系统的结果，不是模型的单独功劳
- 29.8 EY Droids 与 Agentic Agile：相通的是上下文与受控执行
- 29.9 从案例到行动：工程组织可以如何开始
- 29.10 本章小结：从会写代码的 Agent，到可交付变更的工程系统
- 29.11 主要资料与证据说明

第三十章 | 明略科技：从数据智能到 **Agentic Services** 组织

- 30.1 明略科技是谁：二十年都在连接数据与业务
- 30.2 转型之前：工具已经数字化，交付仍然依赖人海
- 30.3 四阶段演进：从数据工具到数字劳动力
 - 30.3.1 第一阶段：让广告效果首先变得可观测
 - 30.3.2 第二阶段：从线上数据走向企业知识和线下运营
 - 30.3.3 内部转型线：在 1500 人组织中平行内嵌 AI Native 体系
 - 30.3.4 外部业务线：从卖软件和项目，走向按结果收费的 Agentic Services
- 30.4 组织怎样改变：从专业分工链到“FDE + Agent 群”
- 30.5 从工具到结果：收费模式为什么会反过来塑造组织
- 30.6 结果：新业务已经产生收入，但距离重写公司仍有距离
- 30.7 六个核心论点：明略案例究竟改变了什么
 - 1. 从“交付洞察”转向“交付闭环结果”
 - 2. 从 SaaS 工具转向数字劳动力服务
 - 3. 从一个 Copilot 转向组织级 Agent 协作网络
 - 4. 从项目经理转向 FDE 责任节点
 - 5. 从按输入收费转向按可量化结果收费
 - 6. 从“向客户讲 AI”转向“先把自己变成 AI Native 组织”
- 30.8 与 Agentic Agile 的关系：FDE 是责任节点，不是超级个人
- 30.9 值得迁移的七个动作
- 30.10 不能回避的四个挑战
 - 挑战一：Agent 数量容易变成新的虚荣指标
 - 挑战二：按效果收费会刺激局部优化
 - 挑战三：自动化可能削弱专业人才的成长路径
 - 挑战四：经营改善不能全部归功于 AI
- 30.11 本章小结：当专业服务开始承诺结果
- 30.12 主要资料与证据说明

第三十一章 | **Spotify**：从敏捷研发到 **Agentic Fleet Engineering**

- 31.1 敏捷团队很快，舰队维护很慢
 - 31.1.1 为什么原有敏捷机制没有自动解决这个问题
 - 31.1.2 从“很多小任务”重新定义为“一次舰队变更”
 - 31.1.3 Fleet Management 到底是什么
 - 31.1.4 Fleet Management 与 Squad 的关系
- 31.2 四阶段演进
 - 31.2.1 从敏捷团队到平台化研发
 - 31.2.2 Fleet Management 把维护变成组织能力
 - 31.2.3 从脚本迁移到 Agent 迁移
 - 31.2.4 Agentic Fleet Engineering
- 31.3 四维转型

- 31.3.1 人员：从组件维护者到变更系统设计者
- 31.3.2 组织：产品团队加平台控制面
- 31.3.3 流程：从单仓库 PR 到全舰队闭环
- 31.3.4 工具：从脚本平台到 Agent Harness

31.4 与 3-4-3 的机制映射

31.5 用 SCOPE-V 重放一次 Fleet Change

31.6 真实声音与证据边界

- 31.6.1 这些数字为什么不能直接证明研发提效
- 31.6.2 关键转折：瓶颈从编码转移到判断
- 31.6.3 Spotify 案例的适用边界

31.7 六个核心启示

31.8 案例总结

案例资料

第三十二章 | GitLab：研发工具厂商如何把 Agent 纳入统一控制面

32.1 GitLab 是谁：把自己的研发方式做成产品

32.2 Agent 之前：组织已经共享一条数字交付链

32.3 两条演进线：产品能力与内部实践

- 32.3.1 第一阶段：先把工作变成机器可读的事实
- 32.3.2 第二阶段：Duo 进入人的工作环
- 32.3.3 第三阶段：从建议代码到承接完整任务
- 32.3.4 第四阶段：把 Agent 收入组织控制面

32.4 四维转型：改变的不只是工具

- 32.4.1 人员：从直接编码者到意图、架构与裁决者
- 32.4.2 组织：从 AI 小组到横跨产品和平台的能力网络
- 32.4.3 流程：从“写完再审”到双重反馈环
- 32.4.4 工具：从 AI 插件到研发控制面

32.5 为什么它与 3-4-3 高度同构

- 三个超级角色
- 四大动态工件
- 三大自治机制

32.6 用 SCOPE-V 重放一次 GitLab 式交付

32.7 真实结果：有高强度样本，还没有全局答案

32.8 内部声音：快不是唯一价值，责任也没有转移

32.9 六个核心论点

32.10 对研发组织的行动指南

32.11 还没有解决的四个问题

- 第一，吞吐遥测不是价值遥测
- 第二，人类评审可能成为新瓶颈和假门禁
- 第三，生成、测试与评审的独立性仍需证明
- 第四，上下文图展示事实关系，不是价值判断

32.12 案例总结：机器开始构建软件，组织必须重构责任

案例资料与证据边界

- 主要资料
- 证据边界

附录与行动入口

怎样使用附录

附录 A | Agentic Agile 宣言

附录 B | 开源治理资产速查

附录 C | 图书与课程学习地图

四门 Portal 在线课程

线下工作坊：把共识变成组织行动

从阅读到实践

图书、Skill、Portal 与工作坊的边界

附录 D | 术语表

附录 E | 五道门速查

附录 F | 公开资源与行动入口

公开资源

从今天开始

联系与共建

附录 G | 端到端案例工作表：从一句话需求到可验证发布

G.1 任务入口

G.2 意图与边界

G.3 Work Graph 与责任

G.4 Prove 证据表

G.5 Verify 裁决

G.6 发布后 Telemetry

G.7 复盘与回写

附录 H | 常见质疑与回答

H.1 “这不就是把敏捷流程重新命名吗？”

H.2 “小需求也要写这么多文档吗？”

H.3 “模型越来越强，以后还需要这些吗？”

H.4 “TDD 会不会拖慢 AI Coding？”

H.5 “Evidence Bundle 会不会变成新的形式主义？”

H.6 “为什么 IO 必须本人签署？”

H.7 “多 Agent 一定比单 Agent 快吗？”

H.8 “发生 Bug 是否说明证据失效？”

H.9 “已经有 DevOps、CI/CD 和平台工程，还需要 3-4-3 吗？”

H.10 “怎样证明这套方法值得投入？”

附录 I | 四大动态工件实战指南

I.1 Intent Graph：只画当前任务需要的关系

I.2 Intent Contract：写清会改变实现或裁决的内容

I.3 Constraint Matrix：让边界能够执行

I.4 Evidence Bundle：组织事实，不包装结论

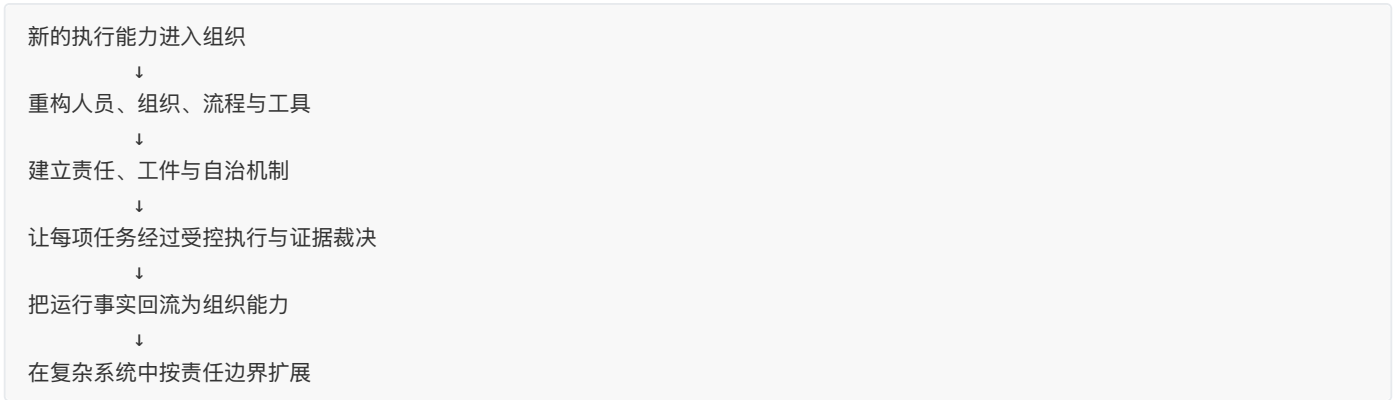
I.5 四个工件怎样一起检查

结语 | 从碳基协作到硅基自治

AI 已经能够理解、生成、执行和验证。研发组织真正面对的问题，不是还要不要使用 AI，而是如何让新的执行能力进入组织后，持续创造价值而不扩大失控。

本书围绕六个递进的问题展开：为什么必须重构研发组织，新的操作系统由什么构成，一次任务怎样可信运行，工程能力怎样成为组织能力，复杂系统怎样扩大而不失去控制，以及这些变化在真实企业中怎样发生。

一条主线



这条主线中的核心结构各有明确职责：

结构	它解决的问题
四维重构	人员、组织、流程与工具究竟要改变什么
3-4-3	谁负责、依靠哪些共同工件、自治如何发生
SCOPE-V	一项任务怎样从意图走到行动裁决
Harness	Agent 在什么权限、规则和停止条件内行动
Evidence	当前结论凭什么成立，哪些仍然未知
Telemetry	多次任务之后，组织是否真的变得更快、更稳、更可信

六部分分别回答什么

部分	核心问题	读完后的收获
第一部分 为什么要换	AI 为什么没有自动带来组织提效	看清旧研发体系的结构性瓶颈，建立四维重构与五条宣言的判断坐标
第二部分 换成什么	Agentic Agile 操作系统由什么构成	理解 3-4-3、SCOPE-V、Evidence 与 Telemetry 如何共同运行
第三部分 怎样运行	一项任务如何从模糊需求走向可信交付	掌握从意图、约束、执行、证明到裁决、发布与反馈的完整回路
第四部分 怎样成为组织能力	如何把工程闭环变成企业运行方式	形成组织、流程、试点、路线、评价与风险裁剪的转型方法
第五部分 怎样扩展	既有系统和复杂协作如何继续扩大	掌握 Recon、知识与记忆、跨模块契约、集成与协作边界
第六部分 真实组织怎样改变	AI 进入核心价值流后，企业如何重构人员、组织、流程与工具	从真实案例中识别转型路径、运行机制、结果、代价与证据边界

三条阅读路径

管理者与转型负责人：第一部分 → 第二部分 → 第四部分。先确定转型对象、责任结构与首轮验证，再进入管理与规模化决策。完成上述路径后，可以进入第六部分，用 AI 组织案例检验框架与真实企业之间的相通点、差异和适用边界。

工程负责人、架构师与研发效能负责人：第二部分 → 第三部分 → 第五部分。先建立运行架构，再把闭环落入真实交付系统，最后处理既有系统和多模块协作。

一线研发、产品、测试与 AI 实践者：第三部分 → 附录 → 第二部分。先从一项真实任务开始使用契约、约束、证据与五道门，再回看整体架构。

无论从哪条路径进入，全书最终都指向同一件事：让人定义价值和责任，让 Agent 在边界内执行，让证据决定行动，让遥测推动组织持续进化。

第一部分 | 为什么 AI 时代需要新的研发操作系统

第一部分从一个常被工具热潮遮蔽的事实开始：AI 已经改变研发执行能力，但大多数组织仍沿用以为人为唯一执行主体设计的岗位、团队、流程和管理节奏。个人效率先提高，组织瓶颈也随之转移。

本部分阅读路径

1. **第一章 | 新生产力进入旧组织**：为什么个人效率提升，常常没有变成端到端的组织提效？
2. **第二章 | 四维重构**：人员、组织、流程与工具为什么必须同时改变，不能只部署一种新工具？
3. **第三章 | 理论基石**：用认知科学、控制论与系统论理解外部化认知、反馈和复杂协作。
4. **第四章 | Agentic Agile 五条宣言**：当速度、控制、责任与价值冲突时，组织优先保护什么？

这里并不否定敏捷、DevOps 或探索性的氛围编程。它们仍然提供重要的价值观、协作机制和工程基础。需要重构的，是它们形成时默认的执行主体、反馈速度和责任载体。当 Agent 从“给建议”走向“能行动”，研发组织就需要一套新的运行方式，把机器速度转化为可持续、可验证的组织提效。

读完这一部分，你应能判断自己的团队到底卡在什么地方，以及为什么下一步不是先选模型或工具，而是进入第二部分设计一套人机协同研发操作系统。

第一章 | 新生产力进入旧组织：为什么局部提效没有变成组织提效

许多企业已经不再需要证明 AI 能不能提效。工程师用它理解代码、生成实现、补充测试和修复缺陷，产品经理用它整理需求，测试人员用它构造场景，管理者用它汇总信息。过去按天完成的工作开始按小时完成，提效是真实的，也是企业投入 AI 转型必须获得的结果。

但另一个事实同样真实：个人完成任务更快，不必然让需求更早上线；代码生成更多，不必然让有效价值交付更多；一个环节节省的时间，也可能转化为下游更长的等待、审核和返工。

AI 已经产生效率，只是效率首先出现在局部，组织尚未完成对新生产力的吸收。**AI 转型最先暴露的，往往不是模型能力上限，而是旧研发系统的吞吐上限。**

1.1 AI 改变的不只是工具，而是执行主体

传统软件工具主要等待人发出明确操作：编辑器编辑、编译器编译、流水线执行预设步骤。它们提高人的局部能力，但任务分解、上下文拼接、异常判断和下一步选择主要由人完成。

Agent 开始改变这种分工。一个 Agent 可以读取仓库和项目文档，搜索依赖关系，修改多个文件，运行测试，根据失败继续修复，调用外部工具，甚至与其他 Agent 分工协作。人不再逐步完成每一个动作，而是开始把一个包含目标、边界和验收条件的任务交给机器执行。

这带来两项同时发生的变化：

- 1. **执行能力跃迁**：大量分析、生成、检查和修复可以高频并发进行，单位任务的边际成本和等待时间显著下降。
- 2. **行动半径扩大**：一个模糊意图、错误假设或过宽权限，也可以被快速扩展为跨文件、跨模块、跨环境的连续操作。

因此，AI 研发的关键问题不再只是“怎样让模型写得更好”，而是：

当机器开始承接任务并持续行动，组织怎样重新定义价值、责任、边界、协作和完成？

1.2 局部提效为什么没有自动成为组织提效

研发交付不是个人产出相加，而是一条包含需求澄清、决策、实现、验证、发布和反馈的价值流。AI 可以把其中一个环节压缩十倍，但整条价值流的速度仍受制于没有同步改变的瓶颈。

实现从 3 天缩短到 3 小时

≠ 需求可以更早进入实现

≠ 测试与审核能够承接更多变更

≠ 发布决策更快发生

≠ 用户更早获得有效价值

当生成速度突然提高，瓶颈通常沿价值流迁移：

局部变化	新出现或被放大的瓶颈	组织表象
代码生成更快	需求边界没有同步澄清	做得快，返工也快
并行任务增多	架构、测试与集成吞吐不足	在制品增加，交付周期不降
Agent 自动修复	失败原因和修复过程不可追溯	全绿更多，信任反而下降
工具数量增加	上下文、权限和规则彼此割裂	每个人都高效，团队难以接力
执行持续加速	人类决策仍依赖会议与层层审批	机器已经完成，结果仍在等待

局部提效与组织提效是前后两个层级。前者证明 AI 具备生产力；后者要求组织重新设计整条价值流，让节省出来的时间穿过决策、验证和发布系统，最终成为业务结果。

1.2.1 Anthropic 案例：代码生成率不等于研发效率

2026 年，Anthropic 在官方文章《When AI builds itself》中披露：截至 2026 年 5 月，**超过 80% 合并进入 Anthropic 代码库的代码由 Claude 编写**。这个数字统计的是已合并代码中可归因于 Claude 的代码行比例，并不表示 Claude 完成了 80% 的研发工作，更不能直接换算成 80% 的研发效率提升。

原文还给出了几个经常被传播时省略的背景：

- 1. 这个比例统计的是“合并进入代码库”的代码，不是所有生成后仍然被丢弃的代码；
- 2. 统计时间点是 2026 年 5 月，不能脱离团队、代码库、工具和时间窗口推广到所有企业；
- 3. Anthropic 同时观察到，2026 年第二季度典型工程师每天合并的代码量约为 2024 年的 8 倍，但原文明确提醒：代码行数是“数量而非质量”的不完美指标，8 倍几乎肯定高估了真实生产率提升；
- 4. Anthropic 对员工的调查显示，受访者主观估计使用内部模型后产出约为原来的 4 倍，但官方也承认这种自我估计可能高估真实提升；
- 5. 同一篇文章还说，Claude 在工程任务中可以由人给出目标、自己寻找实现方法，但在“应该选择什么目标、哪些问题值得解决”上仍存在明显差距。

原文真正描述的是一个**执行方式发生巨大变化**的组织：人不再需要亲自输入每一行代码，Agent 可以承担更多实现、测试、调试和实验执行；人则更多负责提出目标、审阅结果、选择方向、处理异常和承担后果。Anthropic 甚至直接指出，随着代码产出增加，人工代码审查已经成为新的瓶颈。[Anthropic, “When AI builds itself”](#)

这条信息是怎样被误解的

从原始披露到常见结论，中间至少发生了三次概念偷换：

原始信息	传播中的改写	偷换了什么
超过 80% 的合并代码由 Claude 编写	AI 完成了 80% 的研发	代码贡献 → 全流程工作
工程师每天合并的代码量约为 8 倍	研发效率提升了 8 倍	代码数量 → 生产率与价值
人提供目标，Claude 执行方法	工程师不再重要	执行权变化 → 价值判断和责任消失

第一个偷换忽略了需求、架构、方案选择、测试设计、代码审查、集成、发布和运行反馈。代码只是研发价值流中的一个产物，不是研发工作的全部。

第二个偷换把代码行数当成生产率。代码越多，可能意味着交付了更多能力，也可能意味着实现更复杂、重复代码更多、返工更多，甚至是 Agent 生成了本来不会产生的额外代码。真正需要观察的是：同类目标是否更快变成可接受结果，质量和风险是否保持，用户是否更早获得有效价值。

第三个偷换把“人不再亲自写代码”误读为“人不再需要判断”。恰恰相反，当执行成本下降，组织会产生更多候选方案、实验结果和变更请求；真正稀缺的能力变成选择哪个问题值得解决、哪个结果可信、哪个风险可以接受，以及何时停止一条看似有效的路径。

先把三个数字分开

代码贡献率：AI 写了多少代码？
研发执行效率：同类任务多快得到可验证结果？
组织研发效能：组织多快做出正确决策，并持续交付有效价值？

三者可能同时上升，但没有逻辑上的等号：

代码贡献率上升
≠ 研发执行效率按同等比例上升
≠ 组织研发效能按同等比例上升

组织效能还受到需求澄清、决策等待、集成能力、人工审查、发布风险、客户反馈和治理边界的限制。根据 Amdahl 定律，一条价值流中只有部分环节加速时，整体收益会被未加速的环节限制；Anthropic 自己观察到人工代码审查成为新瓶颈，正是这种瓶颈迁移的例子。

应该怎样理解这个案例

我们不应从这条新闻得出“AI 只是写代码工具”，也不应得出“人类工程师已经没有价值”。更准确的结论是：**代码实现正在从人的主要工作，变成 Agent 可以大规模承接的执行环节；人的工作上移到意图定义、方案取舍、结果判断和责任治理。**

这个变化可以概括为：

人负责意图与判断，Agent 负责被授权的执行与反馈，组织负责证据与治理。

这套分工回应的是已经出现的新瓶颈。代码生成率越高，组织越需要提高意图的准确性、判断的独立性、证据的可读性和授权的可撤回性。否则，机器只会更快地产生更多待审查、待集成和待解释的结果。

AI 使用量、Token、生成代码量和 Agent 数量只能作为过程信号，不能单独作为转型成功指标。判断转型是否有效，还要同时观察：

- 从意图提出到可接受结果的端到端周期；
- 首次成功率、返工和逃逸缺陷；
- 人工判断是否集中在真正的价值与风险节点；
- 证据是否足以支持继续、停止、回退或扩大；
- 组织是否把有效经验沉淀为可复用的契约、约束、测试和知识。

1.3 从工具采用到智能体组织：四种不同变化

企业经常把“已经购买 AI 工具”、“部分流程接入 AI”和“完成 AI 转型”混为一谈。为了避免这种概念膨胀，可以把变化区分为四个层级：

层级	主要表现	获得的价值	尚未解决的问题
L1 工具采用（碳基主导）	Prompt、Copilot、AI 写文档或代码	提升个人任务速度	工作和责任结构基本不变
L2 流程增强（人机编队）	AI 接入需求、开发、测试等局部环节	减少部分重复劳动	旧流程仍是主干，瓶颈容易转移
L3 可验证闭环（受控自治）	意图、约束、执行、验证、证据和反馈连通	形成可重复、可裁决的交付能力	仍需验证能否跨团队、跨场景复用
L4 硅基自治	Agent 在人类定义的意图、约束与责任边界内自主运行	形成组织级智能生产系统	价值定义、不可逆决策和责任承担仍由人类掌握

这不是要求所有企业立即追求最高自治，也不是一条只进不退的技术成熟度阶梯。不同业务可以停留在不同层级，安全关键任务甚至应主动保留更多人类责任节点。区分层级的意义在于：**不要用 L1 的工具使用量，宣称已经获得 L3、L4 的组织能力。**

这四级也可以按默认执行方式概括为：**碳基主导 → 人机编队 → 受控自治 → 硅基自治**。路径描述的是执行方式的变化，不是责任主体从人类转移给机器。

1.4 旧研发系统的四个默认假设

今天多数研发组织形成于“人是唯一通用执行者”的时代，其运行方式建立在四个默认假设之上。

- **人员假设：**能力绑定岗位，人负责理解、执行、检查和汇报，工具只提供辅助。
- **组织假设：**部门、团队和汇报线承载能力与协作，跨边界工作依赖交接和协调。
- **流程假设：**Sprint、会议、审批和阶段门控制节奏，因为人的执行速度和沟通带宽相近。
- **工具假设：**每个专业工具服务一个局部活动，上下文和责任链由人脑与文档连接。

这些假设并非错误。Scrum、看板、DevOps、代码评审和职责分工，都是人类组织处理复杂协作的有效机制。问题在于，当 Agent 能够跨步骤、高并发、持续运行时，旧机制的时间尺度和责任载体不再完全匹配。

如果岗位不变，人可能一边把执行交给 Agent，一边仍按亲自完成任务接受考核；如果组织不变，Agent 可以跨模块工作，授权和责任却卡在部门边界；如果流程不变，机器几分钟产生的结果要等待数天会议；如果工具不变，上下文、权限、证据和遥测就散落在多个 Copilot 与聊天窗口中。

此时继续采购更强模型，就像给拥堵的道路换上更快的汽车。车辆性能提高了，系统吞吐未必提高。

1.5 氛围编程的真正风险：探索模式进入正式交付

氛围编程适合原型探索、个人实验和低风险试验。在这些场景中，快速发散、快速看到结果、随时推倒重来，本身就是效率。

真正的问题，是组织把探索阶段的做法直接带进了正式交付。原型开始接触真实数据，试验代码未经治理便进入生产；验收标准不再独立定义，而是根据已有实现倒推；Agent 既编写代码又设计测试，最后沿用同一套假设证明自己已经完成。

当探索模式未经转换直接进入正式交付，会积累四类债务：

- **意图债：**团队没有说清要改变的业务结果、边界条件和非目标，Agent 只能补齐假设。
- **上下文债：**Agent 获得的信息过少、过期、冲突或越权；更大的上下文窗口并不会自动产生正确上下文。
- **约束债：**“注意安全”、“不要影响旧功能”仍是态度，没有成为机器可执行的权限、红线和停止条件。
- **证据债：**团队展示了代码和全绿结果，却不能证明目标、约束、测试、风险和发布裁决之间的对应关系。

这四类债务会被机器速度放大。Agent 越能自主补齐空白、连续执行和快速修复，组织越不能依赖“有经验的人最后看一眼”来兜底。氛围编程最大的幻觉，不是模型会产生幻觉，而是组织把生成完成误认为价值交付完成。

1.6 更快的执行，需要更快而不是更多的控制

面对风险，组织最熟悉的反应是增加审批。但如果每次 Agent 行动都等待人确认，机器速度会被重新锁回人类节奏；如果完全取消控制，错误又会以机器速度扩散。真正需要改变的不是控制数量，而是控制方式。

传统控制大量依赖会议、阶段检查和个人经验。AI 原生控制需要把可以明确表达的部分前移并自动化：目标在执行前校准，边界变成可运行约束，测试从验收标准出发，失败触发停止或修复，完成由异源证据支持，不可逆决策仍由有权的人承担。

探索可以发散，正式执行必须收敛
候选结果可以由 Agent 生成，是否完成必须由证据裁决
低风险动作可以自动化，价值判断与不可转移责任必须由人承担

验证工程由此成为新操作系统的可信执行内核：它让高速执行可以被检查和信任，却不包办全部转型。人员怎样分工、组织怎样授权、流程怎样闭环、工具怎样承载，仍要在同一套系统中共同设计。

1.7 同一个报销需求，为什么“全绿”仍然失败

某团队让 Agent 修改报销审批规则，需求只有一句：“超过 5000 元增加总监审批。”这句话看起来足够具体，实际上至少留下了两个没有回答的问题：5000 元是否包含边界值，以及外币报销应该按什么金额换算。业务部门后来确认，5000 元整不应进入总监审批；而外币报销的判断口径，在任务开始时始终没有被定义。

Agent 很快完成了代码，并根据自己的理解生成测试。测试全部通过，上线前也没有出现红灯。上线后，问题才以业务结果的形式暴露出来：恰好 5000 元的报销被送进了总监审批，外币报销又按原币金额判断，导致大量单据阻塞。

这不是“测试没有运行”，而是测试从一开始就没有独立回答“业务规则是否正确”。失败是这样一步步形成的：

需求中的“超过”没有被转化为明确边界

→ Agent 按自己的理解实现规则，并据此补出验收标准

→ 测试验证了实现是否符合这个理解，而不是规则是否符合业务

→ 外币场景没有明确纳入或排除，相关上下文因此保持空白

→ 代码评审检查了实现质量，却没有重新审查业务口径和范围

→ 上线前没有来自业务规则、独立场景或发布责任人的充分证据

→ 线上异常被拆成新的 Bug，原任务仍被记录为“首次成功”

如果只把它看成一个代码缺陷，修复边界判断和外币换算即可；但从生产系统看，代码只是最后一个显性故障点。更早之前，业务意图没有被确认，财务规则没有转化为机器可执行的边界，范围内的场景没有被声明，测试又沿用了实现者的假设。到了发布节点，团队既缺少足以支持裁决的独立证据，也没有把线上失败回写为原任务的交付结果，于是“测试全绿”和“任务成功”掩盖了真实的交付失败。

这也是本书沿用这个案例的原因。后续章节会把它从一个“5000 元判断”向外展开为员工提交、经理审批、财务审核、合规控制、支付和系统接口组成的端到端价值流。同一个案例将同时接受两种检验：任务尺度上能否正确交付，组织尺度上能否真正减少等待、返工和责任模糊。

1.8 判断转型是否发生，不能只看 AI 使用量

工具激活率、Prompt 数量、Token 消耗和生成代码量，可以说明 AI 被使用，却不能单独说明组织提效。甚至“完成任务数增加”也可能来自任务切得更碎、返工被另立为新任务，或风险被转移到下游。

更有意义的问题是：

- 相同或更少投入，是否交付了更多有效价值？
- 从意图提出到用户获得结果，端到端周期是否缩短？
- 首次通过率是否提高，返工和失败成本是否下降？
- 人是否从重复执行转向价值判断、系统设计和异常裁决？
- Agent 的自治是否建立在真实约束、证据和可恢复能力之上？
- 成功经验是否成为可复用的组织资产，而不是个人技巧？

这些问题不能由一次演示回答。组织需要同时观察业务价值、治理健康、工程效能与系统进化，并允许指标相互制衡。代码量上升而业务结果不变，不是成功；周期缩短但失败成本失控，也不是可持续的成功。后续的价值与效能遥测将把这套判断变成可签署、可采集和可复查的事实。

1.9 本章小结：AI 首先是一场生产方式变革

AI 带来的提效是真实的，但组织提效不会由个人效率自动累加而来。新执行主体进入旧岗位、旧组织、旧流程和旧工具链后，瓶颈会迁移，隐性债务会被放大，管理者也更容易把工具热度误认为转型成果。

因此，企业需要的不是给旧研发系统增加一个 AI 插件，而是围绕新的执行能力重新设计生产方式。治理在其中负责让速度不越界，验证工程负责让完成可被信任，遥测负责判断提效是否真实；它们都是操作系统的组成部分，而不是转型本身的全部。

下一章将把本章暴露的结构矛盾展开为人员、组织、流程与工具四个维度，说明它们为什么必须围绕同一条价值流协同重构。

本章最小实践：选择一条已经使用 AI 的真实研发价值流，不讨论“大家用了哪些工具”，只标出四类事实：AI 加速了哪里，瓶颈转移到哪里，新增了什么返工或风险，最终业务结果是否改善。

完成证据：一张“局部提效—瓶颈迁移—业务结果”诊断图。

延伸实践：推荐学习《Agentic Agile 认知课：AI 转型为什么不是工具升级》，先识别局部提效为什么没有转化为组织提效；线下工作坊第一天进入“AI 转型为什么总是停在工具提效”环节，并通过四维转型画布把工具问题还原为人员、组织、流程与工具的系统重构。

第二章 | 四维重构：人员、组织、流程与工具必须同时改变

第一章说明，AI 带来的局部提效会让瓶颈沿价值流迁移。代码生成加快之后，需求澄清、架构决策、测试验证、跨团队协作和发布审批可能成为新的等待点。企业如果只优化一个环节，往往只是把压力推向下游。

如果把 AI 研发转型拆成四个彼此独立的项目——人力资源部门做岗位培训，管理层调整组织架构，研发效能团队改流程，平台团队采购工具——四方即使各自完成任务，也未必拼得出一套能够运行的人机协同系统。

四维重构的对象不是四张管理清单，而是同一条价值流。 人员决定谁判断、谁执行、谁负责；组织决定能力怎样组合、决策权放在哪里；流程决定工作与反馈怎样流动；工具把意图、权限、执行、验证和证据变成可运行机制。任何一维缺位，其他维度的收益都会被削弱。

2.1 一张图看懂四维重构

转型维度	传统组织的默认假设	Agent 进入后的新矛盾	重构方向
人员 People	人是主要执行者，能力绑定岗位	Agent 承担大量执行后，人的价值、能力和责任容易混淆	人从任务执行者转向意图定义、系统编排、异常裁决与责任签署者
组织 Organization	固定团队、部门和汇报线承载协作	Agent 可以跨边界并发，授权、信任和决策仍停留在部门结构中	从岗位接力转向“人类责任节点 + Agent 能力网络”
流程 Process	Sprint、会议、审批和交接控制 workflow	机器内环远快于人类节奏，等待与人工审核成为瓶颈	从活动和阶段驱动转向意图—约束—执行—验证—反馈的持续闭环
工具 Technology	工具辅助人完成局部任务	Copilot 孤岛、上下文断裂、权限分散、结果无法独立证明	从单点助手转向 Intent + Context + Orchestration + Harness + Evidence + Telemetry

这张表描述的是方向，不是未来组织的固定模板。不同业务、风险和工程基础会产生不同设计。四维模型真正要求的是：每一次改变都能回答它如何改善价值流、如何与另外三维连接、由什么事实证明有效。

2.2 必须协同改变，但不能“大爆炸式转型”

四维协同并不要求先设计目标组织、重画全部流程、建设统一平台，再让所有员工一次性切换工作方式。这样的整体改造周期长、假设多，很容易在真实反馈出现之前投入过重。

更可行的做法，是围绕一条真实价值流形成**最小完整重构**：

选择一个必须改善的业务结果

→ 选择一段真实价值流和可交付切片

→ 明确人和 Agent 的任务、判断与责任

→ 调整完成该切片所需的授权和协作边界

→ 删除、自动化或重构一个关键等待节点

→ 接入最小上下文、约束、验证、证据与遥测能力

→ 用真实结果决定扩大、调整或停止

每一维都要有动作，但只改变足以跑通本轮闭环的部分。组织架构不必先整体重画，平台不必一次建全，流程也不必为了“AI 原生”全部废弃。**转型单元应当足够小，可以快速获得证据；又必须足够完整，不把成本和风险悄悄转移给闭环之外的人。**

2.3 人员重构：迁移任务，不是预测岗位消亡

“AI 会消灭哪些岗位”是一个容易吸引注意力、却不适合作为组织设计起点的问题。岗位由多种任务、判断、关系和责任组成。Agent 通常先改变其中的任务组合，而不是在某一天完整接管一个岗位。

更可执行的方法，是把一条价值流中的工作分成四类：

任务类型	分配原则	典型示例
交给 Agent 执行	高频、可表达、可验证、可恢复	检索、代码生成、测试执行、重复修复、证据收集
人机协同增强	Agent 提供候选与分析，人作判断或组合	需求澄清、方案比较、风险识别、测试设计、故障诊断
保留人类责任	涉及价值取舍、模糊冲突、组织承诺或不可逆后果	优先级、风险接受、例外批准、发布裁决、对外承诺
禁止 Agent 自治	法律或伦理不允许委托，或者当前无法验证和恢复	未授权生产写入、模型自批权限、替责任人签署、绕过监管控制

执行权与责任必须分开。Agent 可以完成一项工作，却不能因此承担组织责任；人不再亲手完成每一步，也不等于退出。人的重心将从重复执行转向意图定义、边界设计、系统编排、异常处理和结果裁决。

这种变化也会重组专业能力。产品人员要把价值和非目标表达得可判定，开发人员要设计规格、测试和 Agent 工作流，测试人员要建设独立证据与评估体系，架构师要设计上下文、权限和恢复边界，研发管理者要学会运营人机系统而不是只管理人的活动。

人员重构的完成证据不是“多少工作由 AI 生成”，而是**关键任务是否被分配给更合适的主体、人类责任是否更清楚、重复劳动是否下降、异常是否能够在正确位置交还给人**。

2.4 组织重构：从岗位接力到人机责任网络

传统组织通过稳定团队、专业部门和汇报关系降低协作复杂度。这些结构仍然有价值，但它们不应继续成为每一次信息、任务和决策流动的唯一通道。

Agent 可以按意图临时组合代码、测试、安全、数据和领域能力。相应的组织单元需要从“哪些岗位参加”转向“完成这个意图需要哪些能力，哪些节点必须由人负责”。本书把这种结构概括为：

围绕真实意图形成最小责任单元，由人类责任节点定义价值、边界与裁决，由可组合的 Agent 能力网络承担约束执行。

人类责任节点不是新的审批层。它们只应出现在机器不应自行决定的位置，例如业务价值冲突、权限扩大、约束例外、不可逆动作和发布承诺。其余可以被明确表达和验证的决定，应通过预先授权、机械规则和证据快速流动。

组织重构至少要处理四件事：

- 1. **责任可指认**：谁定义意图，谁设计运行控制，谁执行，谁接受剩余风险？
- 2. **决策权前移**：哪些决定可以交给最接近事实的人或规则，不再层层上报？
- 3. **能力可编排**：领域专家、平台能力和 Agent 如何按任务组合，而不是每次重新拉群协调？
- 4. **异常有去处**：越界、冲突、证据不足和无法收敛时，由谁在什么时限内裁决？

组织重构不是取消稳定团队，也不是让所有人进入临时项目。稳定团队可以继续承载产品知识、长期责任和心理安全；Agent 能力网络则降低跨专业任务每次都靠人工接力的成本。二者的边界应由价值流和责任设计决定，而不是由“固定”或“动态”哪一个听起来更先进决定。

2.5 流程重构：让机器运行快内环，让人承担责任外环

传统流程常用 Sprint、站会、评审、审批和交接解决信息同步、工作协调与质量控制。这些活动形成于人的执行与反馈节奏。当 Agent 可以分钟级生成、验证和修复时，继续让每个动作等待下一场会议，会主动放弃机器速度；把全部检查取消，又会失去边界和责任。

AI 原生流程需要区分两个时间尺度：

- **机器快内环**：在已签署意图和权限范围内执行、测试、诊断、修复、重试和停止，反馈以秒、分钟或小时发生。
- **人类责任外环**：决定价值、签署边界、处理异常、接受风险、裁决发布，并根据遥测调整下一轮规则和授权。

流程重构不是把所有会议变成自动化，而是逐项判断它们承担什么功能：

现有活动	应保留的条件	可以重构的方向
Sprint 计划	仍需要阶段性投资和组合承诺	将任务级执行转为持续拉动，不等待下一代才纠偏
站会	需要处理真实障碍、冲突和责任	状态汇报由事件和看板生成，会议只处理例外
代码评审	需要架构判断、领域知识和责任确认	机械检查前移，评审对象扩展到意图、差异、风险和证据
阶段审批	涉及不可逆风险或法定责任	低风险重复审批转为预授权规则，高风险保留 HITL
回顾	需要共同解释系统性问题	高频失败自动回流，人工复盘聚焦模式、激励和规则改变

判断一个流程是否应该保留，不看它属于传统还是 AI 原生，而看它是否承担不可自动化的价值与责任。如果只是传递机器已经知道的状态，它应被事件化；如果承担风险接受和组织承诺，它就不能因为追求“零 HITL”被删除。

2.6 工具重构：从 Copilot 孤岛到可替换的工程系统

单点 AI 工具能够提高个人效率，却通常只看见局部上下文，也不天然理解组织意图、权限、历史决策和完成标准。企业如果不断增加 Agent，却没有共同的运行底座，会形成新的工具孤岛：上下文重复投喂，规则各自维护，执行无法追踪，证据无法合并，成本和风险难以比较。

工具重构不是先采购一个“大一统 Agent 平台”，而是逐步补齐六类可以组合的能力：

1. **Context**：按任务提供最小、可信、可追溯的代码、业务和知识上下文；
2. **Orchestration**：描述依赖、并发、交接、超时、重试、停止和人工升级；
3. **Harness**：把权限、约束、验证、恢复和运行规则加载到 Agent 周围；
4. **Evidence**：把验收标准、测试、约束、风险和执行事实组织成可裁决证据；
5. **Telemetry**：连接算力投入、自治过程、工程质量和业务结果；
6. **Knowledge & Memory**：只把经过验证的经验晋升为可复用知识、规则、测试或 Skill。

这些能力必须与具体模型和 AI 宿主保持适度解耦。企业可以同时使用 Codex、Claude、WorkBuddy 或其他工具，真正需要稳定的是契约、约束、证据、遥测和交接协议。否则每次模型或供应商变化，都要重新建设一套组织工作方式。

工具维度的目标也不是让人接触更多控制台。好的工程系统应把规则嵌入日常工作，让低风险任务快速通过，让高风险或证据不足的任务准确停止，并把人带到需要判断的位置。

2.7 治理与度量为什么不是第五、第六维

四维模型之外，企业还需要治理和度量，但不应把它们设计成两个独立工作流。

治理横贯四维：它明确人类不可转移的责任，规定组织授权和例外，定义流程中的机械门与 HITL，并通过工具执行权限、约束和恢复。治理如果只由一个委员会或平台团队承担，就会变成外置审批，而不是系统能力。

度量同样横贯四维：它观察人的时间是否转向高价值判断，组织等待和交接是否减少，流程周期与首次通过率是否改善，工具、上下文和算力是否产生净收益。度量如果只统计工具激活率、代码量或 Token，就会奖励局部产出，进一步掩盖价值流瓶颈。

因此：

四维重构决定生产系统怎样运行
治理决定它在什么边界内运行
Evidence 证明一次运行是否成立
Telemetry 判断长期运行是否真正提效并持续进化

Dashboard 是管理者观察和决策的入口，不是另一个“AI 绩效考核系统”。它评估的是人、Agent、上下文、工具链和算力共同构成的生产系统，不能把系统结果粗暴归因到个人，更不能用来逼迫团队减少必要的 HITL 或隐藏失败。

2.8 用报销价值流检验四维是否连通

第一章的“超过 5000 元增加总监审批”看似只是一个规则变更。把观察尺度扩大后，它属于一条更长的价值流：员工提交报销，经理审批，财务审核，合规检查，支付执行，移动端与财务系统同步状态。

如果要让 Agent 参与这条价值流，四维必须同时回答：

维度	本案例需要作出的决定
人员	谁定义“以上”、币种和例外规则？哪些测试与实现交给 Agent？谁接受上线风险？
组织	业务、财务、合规和技术决策权如何分配？规则冲突由谁裁决？
流程	哪些审批可以根据金额和风险自动路由？哪些异常必须进入人工复核？
工具	规则、汇率、权限和历史决策怎样进入上下文？如何测试、留证、发布、回滚和观测？

只改工具，Agent 会更快地产生一个可能错误的实现；只改流程，自动路由可能把错误规则传播得更快；只培训人员，责任仍可能卡在旧部门边界；只调整组织，没有可执行契约和证据，协作仍然依赖口头解释。

四维连通后的最小结果，不是“建成智能报销平台”，而是完成一个真实规则切片：责任清楚、边界可执行、流程减少等待、工具能够验证和留证，并且上线后的业务结果可以被观察。这才是可以继续扩展的转型起点。

2.9 四维转型的最小诊断法

四维讨论很容易滑向“加强协同、提升能力、建设平台”之类无法验收的口号。为了让诊断进入行动，每个维度只写五项：

必填项	要回答的问题
当前事实	当前价值流中真实发生了什么，有什么数据或样本？
目标改变	本轮具体要改变哪项行为、结构或能力？
最小动作	七天内可以完成什么最小动作？
责任人	谁对动作与结果负责，Agent 不能替谁签署？
完成证据	看到什么事实，才算改变已经发生？

诊断顺序也很重要：先选择业务结果和目标价值流，再看四维断点；先找当前瓶颈，再决定需要什么 Agent 和平台能力。若从工具清单出发，团队很容易为已有产品寻找场景，而不是为真实问题设计系统。

2.10 本章小结：四维不是四条线，而是一套系统

AI 研发转型不是工具升级，也不是组织架构调整、流程再造或人员培训中的任何单项工程。它要求人员、组织、流程和工具围绕同一条价值流形成最小完整重构，并用真实证据验证这种新组合是否产生了更快、更好、更低成本或更可持续的业务结果。

协同重构不等于大爆炸。正确起点是一条真实价值流、一个可交付切片和一个四维闭环：任务与责任重新分配，授权与协作边界得到调整，关键等待点被删除或重构，最小工程系统能够执行、验证、留证和反馈。

后续章节将先用三大理论基石和五条宣言建立设计依据与价值取舍，再通过 3-4-3、SCOPE-V、验证工程、Evidence 和 Telemetry 把四维重构变成可运行机制。

本章最小实践：选择一条真实研发价值流，为人员、组织、流程和工具分别写出“当前事实、目标改变、第一步动作、责任人、完成证据”。

完成证据：一张 Agentic Agile 四维转型诊断图，其中四维围绕同一个业务结果，不允许各自选择不同试点。

延伸实践：推荐学习《Agentic Agile 认知课：AI 转型为什么不是工具升级》，从知识工作、组织惯性和价值兑现的角度理解为什么 AI 转型不是采购工具。

第三章 | 理论基石：认知科学、控制论与系统论

Agentic Agile 面对三个长期存在的系统问题：有限认知怎样承载复杂信息，反馈系统怎样发现并纠正偏差，组织怎样获得处理复杂环境所需的多样性。

认知科学、控制论与系统论分别提供一副观察镜，帮助我们识别纯人类协作、单模型执行和多 Agent 系统各自的能力边界，并据此设计更可靠的治理机制。

上一章的四维模型回答**要重构什么**，这三种理论则帮助我们判断**为什么要这样重构，设计是否合理**。两者不是一一对应，而是彼此交叉：

理论视角	首先揭示的问题	主要四维落点	对其他维度的影响
认知科学	人脑与模型上下文都有限，隐性知识难以稳定传递	人员、工具	推动组织共同语言外部化，减少流程中的寻找、解释和交接负担
控制论	高速执行会放大偏差，慢反馈使错误扩散	流程、工具	重新安排人类责任节点，并要求组织明确停止、恢复与升级权
系统论	单一主体响应不足，多主体协作又会制造协调债	组织、流程	影响人员责任分工，并要求工具承载依赖、协议、证据聚合与韧性

这组理论的价值，在于防止局部设计自洽、整体系统失衡。例如，工具维度建设了更大的上下文窗口，却没有处理知识权威和人员责任，认知负荷只是从人脑转移到了模型；流程维度增加自动反馈，却没有可靠信号和停止条件，闭环反而可能加速错误。

3.1 认知科学：复杂度不能只放在人脑里

3.1.1 工作记忆为什么重要

1956 年，认知心理学家乔治·米勒在《神奇的数字7，加或减2》中讨论了人类处理信息的容量限制。后续研究对“7±2”进行了修正，指出人在没有充分组块帮助的情况下，能够稳定保持和操作的信息单元可能更少。具体数字并不是本书的重点，真正重要的结论是：**工作记忆有限，而且容易受到任务切换、干扰和信息结构的影响。**



软件研发恰好是一项高认知负荷工作。工程师不仅要理解当前代码，还要同时处理：

- 用户真正想解决的问题；
- 业务规则和边界值；
- 模块依赖与数据流；
- 历史设计决策；
- 安全、性能和合规要求；

- 当前版本和环境状态；
- 团队尚未说出口的隐性约定。

当这些内容主要依赖个人记忆时，组织会自然采用分工、文档、评审、待办列表、迭代计划、任务拆解和完成定义来降低认知压力。这些机制并不是“落后”，而是人类面对复杂协作时形成的有效外部化工具。

3.1.2 认知负荷的三种来源

认知负荷理论通常将认知负荷分为三类。把这三类负荷放回研发场景考察，有助于我们更准确地设计和组织上下文。

内在负荷来自问题本身。例如财务审批包含金额、币种、组织层级、预算、税务和例外规则，问题天然复杂。

外在负荷来自信息呈现和工作方式。例如规则散落在会议记录、聊天记录、代码注释和个人记忆中；工程师需要不断切换工具和寻找上下文。

有益负荷用于建立真正的理解，例如形成领域模型、识别不变量、设计测试和抽象可复用模式。

有效治理不试图降低问题本身的复杂度，而是减少不必要的信息处理成本，让团队把注意力投入到价值判断和系统设计上。

3.1.3 邓巴数与协作网络

工作记忆解释了个人为什么难以同时处理大量信息，但软件开发还存在另一层限制：复杂知识分散在许多人之间，团队必须通过沟通才能形成共同理解。罗宾·邓巴关于社会关系规模的研究提醒我们，人能够长期维持的稳定关系数量存在现实边界；常被引用的约150人及其若干内层圈，是统计意义上的经验尺度，而不是适用于所有组织的硬性上限。

对研发治理更直接的启示是：团队规模扩大后，共同上下文和高信任协作不会自动按人数增长。若任意两人都可能直接协作，潜在的两两沟通关系可用 $N(N-1)/2$ 表示。它是二次增长，而不是指数增长，但已经足以说明协调负担为什么会迅速上升：10人团队有45对潜在关系，20人团队有190对。实际沟通不一定覆盖所有关系，组织也会用模块边界、稳定团队和接口机制降低负担，但不能假设所有成员始终拥有同一份上下文。



当 Agent 加入团队，沟通成本不会消失，而是改变形态：

- 人与人之间的口头解释，部分转化为 Agent 对上下文的解析；
- 团队之间的接口协商，部分转化为工具协议和 Handoff；
- 依赖个人经验形成的信任，需要由契约、约束和证据补充；
- 信息不同步，会表现为上下文、代码和知识版本漂移；
- 职责不清，会变成人与 Agent 都以为对方会负责。

Agent 的加入不会让协作消失。它只是进一步暴露了隐性默契无法无限扩展的问题，迫使组织把目标、边界、交接和完成依据转成机器与人都能读取和验证的结构。

3.1.4 大上下文不等于全局理解

大模型拥有越来越长的上下文窗口，但窗口容量不是理解质量。把整个代码库、所有文档和全部历史记录塞进一次任务，会产生新的问题：

- 重要信息被噪声淹没；
- 新旧规则互相冲突；
- 不应暴露的数据进入上下文；
- 模型无法判断哪个来源更权威；
- Token 成本和推理时延上升；
- 过期信息被当成当前事实。

Agentic Agile 因此采用分层上下文和任务裁剪，而不是追求“什么都给模型看”。意图图谱保存全局方向，约束矩阵保存共同底线，意图契约保存当前任务，代码发现与检索提供必要实现上下文。

3.1.5 从认知外包到认知治理

认知外包的目的，是把检索、比较、执行、重复验证等可自动化认知活动交给机器，把价值判断、责任承担、冲突取舍和未知情境留给人，而不是让人停止思考。

对应的认知治理包括：

- 意图图谱：防止局部任务失去全局方向；
- 意图契约：防止当前任务无限发散；
- 上下文裁剪（Context Cropping）：提供最小必要上下文；
- 工具约束清单（Tools Manifest）：限制 Agent 能够触达的能力；
- 证据包（Evidence Bundle）：把执行过程重新压缩成人可审阅的证据。

3.1.6 人的意图与判断如何被增强

“人负责意图与判断”不意味着人要重新亲自完成检索、比较和执行，也不意味着把所有决定都交给少数经验丰富的人。人的判断质量可以被设计和训练，关键是把判断从个人直觉变成一个有事实、有反例、有证据的闭环：

定义要改变的价值

→ 暴露假设、未知和非目标

→ 让 Agent 展开事实、影响、方案与反例

→ 人做价值、风险和成本取舍

→ Agent 执行并返回结果与证据

→ 人判断是否接受、停止、回退或扩大

→ 将稳定经验沉淀为规则、测试或知识

这套闭环同时提升准确性和效率：Agent 减少寻找信息、枚举方案和重复验证的时间；人把有限注意力集中到不可转移的价值冲突、风险接受和责任判断。判断质量至少要检查四件事：目标是否可度量，边界是否可解释，反例是否被主动覆盖，结论是否有独立证据支持。

AI 时代，人的核心能力不在于“比 Agent 写得更多”，而在于更快识别真正的问题、更早发现错误假设、更清楚地做出取舍，并为结果承担责任。

认知科学为 Agentic Agile 提供的不是“机器优于人”的结论，而是一个设计原则：**复杂度必须被外部化、结构化和裁剪，不能仅靠人脑记忆，也不能仅靠模型上下文硬吞。**

认知治理回答了“系统在行动前应该知道什么”，却还没有回答：Agent 行动之后，偏差怎样被发现、修正和停止。这正是控制论要解决的问题。

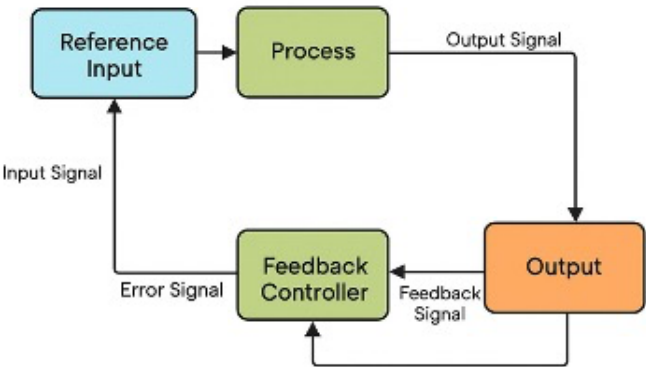
3.2 控制论：没有可靠反馈，速度只会放大偏差

认知外部化能够让目标和边界更清楚，但清楚的输入并不保证执行永远正确。模型输出具有概率性，工具和环境会失败，需求与代码也可能在执行过程中发生变化。控制论提供的关键视角是：**系统不能只根据指令向前运行，还必须观测结果、比较偏差并采取纠正动作。**

3.2.1 从开环执行到闭环控制

开环系统根据输入执行动作，却不利用结果持续调整行为；闭环系统则把输出重新带回控制过程，与目标或参考状态比较，再决定保持、纠正、降级或停止。闭环并不保证结果必然正确，但它使系统具备发现偏差和恢复的结构。

How Feedback Works in a Closed Loop Control System



一个基本闭环通常包含六个要素：

控制要素	回答的问题	AI Coding 中的映射
目标或参考状态	期望结果是什么？	已签署意图契约、AC、业务不变量
被控对象	哪个系统正在变化？	代码、配置、数据、运行环境与 Agent 状态
执行动作	系统可以怎样改变对象？	编辑、测试、工具调用、迁移、部署准备
观测	实际发生了什么？	测试、构建、扫描、日志、差异和生产信号
比较与判断	与目标和约束相差多少？	AC 映射、约束门禁、Evidence Bundle、Verify
控制动作	偏差出现后怎么办？	修复、重试、回退、降级、补证或升级人类

氛围编程一次性给模型 Prompt、得到代码、直接合并，接近开环执行：它可能成功，却没有建立稳定发现偏差的能力。验证工程要求形成可运行的闭环：

目标与约束

→ 被授权的执行

→ 真实观测

→ 比较目标、约束与结果

→ 纠正、回退、停止或升级

→ 再次观测

瀑布、敏捷或其他方法都可能包含反馈，也都可能因为反馈太晚、太弱或不可信而在局部退化成开环。判断的关键，是反馈是否真正进入当前执行路径，而不只是写在流程名称中。

3.2.2 负反馈、强化回路与稳定性

负反馈通过抵消偏差帮助系统趋于稳定。例如单元测试失败、CI 构建中断、安全扫描阻断、权限门拒绝越权操作，都会迫使执行回到允许范围。负反馈不是“否定创新”，而是在当前目标和边界内抑制失控扩散。

强化回路则会放大已有趋势。成功模式被固化为模板并被更多任务复用，可能形成有益强化；未经验证的错误规则进入知识库并反复影响后续任务，也会形成有害强化。因此，经验沉淀不能只看是否被重复使用，还要经过验证、版本、适用范围和撤销机制。

稳定系统通常需要两者配合：**用负反馈控制偏差和风险，用经过验证的强化回路扩大有效能力**。任何反馈都不是越快、越强越好；信号错误、增益过大或修复范围失控，同样可能造成振荡和新的故障。

3.2.3 从敏捷反馈到 Agent 运行时反馈

每日同步、迭代评审、用户验收和回顾都在缩短反馈周期；持续集成、测试自动化和 DevOps 又把反馈推进到提交、构建、发布和生产运行阶段。Agentic Agile 并不是第一次发现反馈的重要性，也不需要否定这些已有实践。

变化在于时间尺度和执行密度。Agent 可能在一次人工评审之前完成多轮修改、测试和工具调用。如果关键反馈仍主要等待会议或阶段末评审，执行吞吐就会远高于验证吞吐，早期误解会迅速扩散。因此，可自动化的反馈需要嵌入 Agent 的执行路径，高风险价值判断和责任裁决则保留给人。

3.2.4 反馈时延、噪声、增益与覆盖

反馈系统不仅怕慢，也怕错。

时延过长：Agent 已经连续修改多个模块，团队才发现最初目标理解错误，返工范围巨大。

噪声过高：测试不稳定、日志缺失或 LLM-as-Judge 未校准，系统可能根据错误信号反复修改正确代码。

增益过大：一次局部失败触发大范围重构，纠正动作本身引入更多不确定性。

覆盖不足：只验证正常路径，没有边界、权限、恢复和业务不变量，系统会对未观测风险保持盲目乐观。

因此，每个闭环都要回答：测量什么、怎样测量、证据是否可信、允许修改多少、多久必须停止，以及失败是否回到同一个检查重新验证。SCOPE-V 的 $P \Rightarrow E$ 快内环采用“最小修复—原检查重跑”，正是为了限制纠正增益并保持证据连续性。

3.2.5 闭环自治何时必须让人接管（HITL）

闭环自治必须有退出条件。以下情况不适合让 Agent 继续自我修复：

- 业务目标或规则发生冲突；
- 需要修改已经签署的契约；
- 需要扩大数据、工具或生产权限；
- MUST 约束失败且无法在授权范围内恢复；
- 证据互相矛盾或缺少独立来源；
- 超过时间、Token 或重试预算；
- 涉及不可逆操作；
- 法律、伦理或商业责任需要人承担。

控制论带来的核心启示不是“让机器无限循环”，而是：**自治能力取决于系统能否根据可信反馈收敛，并在不该继续时停下来**。第三部分将把这一原则实现为 SCOPE-V 六个控制面、 $P \Rightarrow E$ 快内环、Verify 独立裁决、Telemetry 慢外环与机械门禁。

闭环控制解释了系统怎样沿时间持续纠偏，但复杂研发还面临结构性问题：业务、架构、安全、数据和运维包含不同种类的变化，单一主体未必拥有足够的响应方式。怎样用受治理的多样性应对这种复杂性，是系统论视角进一步回答的问题。

3.3 系统论：以受治理的多样性应对复杂性

本节所说的系统论视角，关注的不是一次反馈怎样发生，而是多个角色、模块、工具和环境怎样共同构成一个整体。控制论更强调时间上的“观测—比较—纠正”，系统论则进一步追问：面对种类繁多的变化和扰动，治理系统是否拥有足够的响应方式，又怎样防止这些响应方式彼此冲突。

复杂环境要求治理系统具备与之匹配的响应多样性。多 Agent 扩展了系统的响应方式，也带来了新的协调债。为使这种多样性真正服务于整体目标，需要用 Work Graph 将协作关系组织成可执行结构，用意图图谱、全局约束和跨模块协议校准局部执行，并通过韧性机制应对治理之后仍无法避免的失败。

3.3.1 必要多样性：治理能力必须匹配环境复杂性

阿什比的必要多样性法则常被概括为“只有多样性才能吸收多样性”。它并不是说治理系统必须比被治理对象更加复杂，也不是说增加 Agent 就能消除复杂性。它强调的是：**环境能够以多种方式偏离目标，治理系统就必须具备相应的识别和响应能力**。如果系统无法区分不同状态，只能把它们当成同一种情况处理；如果系统缺少相应的响应方式，即使识别出差异，也无法采取有效行动。

组织通常通过两类策略建立这种匹配。一类是**削减需要处理的多样性**，例如缩小任务范围、隔离环境、标准化接口、限制权限、拆分系统，以及拒绝当前无法安全承接的高风险变化。另一类是**增加有效的响应多样性**，例如引入领域、安全、测试和运维等不同视角，配置不同的验证方法、处置路径与恢复策略。前者减少系统必须面对的变化，后者增加系统能够处理的变化；成熟治理通常会同时使用两者。

急诊分诊可以说明这种关系。急诊面对的患者在病因、严重程度和资源需求上差异很大，既不能按到达顺序一律处理，也不可能为每位患者临时设计一套流程。医院首先通过标准化问诊、生命体征检查和分诊分级，把无结构的个体差异转化为可识别、可排序、可分流的类别；再根据类别调用不同专科、检查手段、抢救流程和医疗资源。传染风险需要隔离，非急症可以转诊，危重患者则立即进入抢救通道。分诊并没有消除病情本身的复杂性，而是降低了处置系统需要同时面对的无序程度；专业能力和处置路径则提供了与不同病情相匹配的响应方式。

映射到 AI 研发，需求裁剪、标准化接口、最小权限和变更边界（Change Envelope）用于减少任务暴露出的无序变化；领域 Agent、测试 Agent、安全 Agent，以及多种验证与恢复工具用于扩展系统的有效响应方式。关键不在于追求最多的 Agent、规则或工具，而在于让治理能力与任务的复杂度和风险相匹配。

3.3.2 多 Agent 提供多样性，但不会自动形成涌现

现代软件系统同时包含产品、架构、开发、测试、安全、数据、运维和合规视角。单个模型即使能力很强，也很难在一次推理中稳定兼顾所有目标。多 Agent 的价值首先来自职责差异和视角互补：

- Domain-Agent 识别业务规则和例外；
- Dev-Agent 关注最小实现及代码影响；
- QA-Agent 从 AC、不变量和状态模型生成反例；
- Security-Agent 检查攻击面、数据流和权限；
- Ops-Agent 关注运行、恢复和可观测性；
- Critic-Agent 挑战假设、证据与局部最优。

多个执行单元可以并行探索互不依赖的方案，也可以通过相互独立的证据减少单一视角盲区。但这只提供了产生更好结果的条件，不构成结果必然正确的证明。“涌现”应理解为系统整体表现出单一执行单元不容易稳定获得的能力，而不是 Agent 数量增加后自然出现超越个体的智慧。

判断多 Agent 是否真正有价值，应看它是否增加了有效覆盖、独立证据或恢复能力，而不是看调用了多少模型、生成了多少消息。

然而，一旦这些能力单元真正进入同一条交付链，问题就会从“有没有足够视角”转向“这些视角怎样协同而不互相干扰”。多样性解决覆盖不足，也会产生新的协调成本。

3.3.3 协调债：Agent 多样性也会制造新的复杂度

增加 Agent、工具和验证方法的同时，也会增加：

- Handoff 信息损耗；
- 对目标和规则的不同理解；

- 文件、环境和计算资源争用；
- 重复工作与相互覆盖；
- 循环依赖和无法收敛的互评；
- 多个 Agent 共享同一错误假设形成的错误共识；
- Token、时延与运行成本；
- 最终责任归属的模糊。

十个没有共同契约的 Agent，可能比一个受约束的 Agent 更危险。因此，必要多样性不等于最大多样性。有效多样性必须与任务风险相称，并由共同意图、明确边界、可执行协作关系和统一证据标准约束。

这些新增成本可以统称为**协调债**：能力单元越多，交接、依赖、状态、资源和责任越需要显式治理。仅靠组织图或“大家自行协作”无法偿还协调债，下一步必须把真实协作关系表达成运行时可以执行和检查的结构。

3.3.4 Work Graph：把协调关系变成可执行结构

Work Graph 直接回应协调债。它不是组织图或待办列表，而是一张能够被人和Agent运行时共同读取的任务关系图：节点表达输入、输出、执行主体与完成判据，边表达依赖、交接和控制条件。它至少要让系统知道哪些工作可以并行、哪些资源会冲突、什么证据阻断下游，以及何时重试、回退或升级。第十二章将展开其节点、边、状态和控制条件。

Work Graph 主要回答“工作怎样协同执行”，却不能保证“执行方向符合整体价值”。一张结构正确、运行顺畅的图，仍然可能高效地完成错误目标，或者让每个节点局部通过却损害系统整体。协作关系理顺之后，还要解决局部与整体的对齐问题。

3.3.5 系统对齐：防止局部最优破坏整体目标

多 Agent 协作中，节点通常按照自己的输入、目标和完成判据优化。如果缺少更高层的共同目标，局部合理并不会自然汇聚为系统合理。Agent 可能在局部任务上表现出色，却损害整体系统。例如：

- 为提高接口速度，引入破坏数据一致性的缓存；
- 为让测试通过，绕过真实依赖；
- 为减少 Token，省略关键上下文；
- 为提高自动化率，把应该由人批准的操作也自动化；
- 为优化单模块指标，破坏跨模块兼容。

系统对齐要求局部任务继承共同意图和约束，并在集成时接受系统级证据检验。它不是压制局部优化，而是要求局部改进能够证明没有牺牲更高层目标。意图图谱、跨模块协议和证据聚合是后续章节对此的工程实现。

系统对齐可以减少目标冲突和局部优化带来的损害，却无法消除未知依赖、环境故障、模型波动和真实世界变化。复杂系统也不可能通过一次设计获得“永不失败”的保证。因此，治理还必须回答一个更现实的问题：**当失败发生时，系统能否及时发现、限制影响并恢复运行？**

3.3.6 韧性闭环：为无法消除的失败设计恢复能力

韧性是前面几层治理的自然终点：必要多样性提供响应能力，Work Graph 管理协作关系，系统对齐守住整体目标，而韧性机制负责承接它们仍然覆盖不到的剩余不确定性。复杂系统无法消除所有失败，Agent 和工具的加入也不会改变这一事实。更现实的治理目标是：

- 失败能够被发现和隔离；
- 影响范围能够被限制；
- 关键状态能够被观测；
- 系统能够降级、补偿或回退；
- 未知风险能够被标记而不是伪装成通过；
- 人类能够在必要时接管；

- 失败能够沉淀为新的约束、测试、知识或记忆。

系统论为 Agentic Agile 提供的核心启示是：用足够而适当的响应多样性应对复杂性，同时用契约、约束、Work Graph、证据和责任控制多样性自身带来的复杂度。

至此，三种视角形成递进：认知科学让关键信息得以外部化和裁剪；控制论让执行能够观测、纠偏和停止；系统论让多个能力单元在复杂环境中形成受治理的协作。任何一项单独存在，都不足以构成可信自治。

3.4 三大理论的融合：从理论启示到工程机制

三大理论在这里是三副相互补充的设计镜头：它们分别暴露不同类型的治理缺口，并导出可以被工程实现和验证的机制。

理论视角	暴露的核心问题	设计原则	四维主要落点	在 3-4-3 中的主要映射	理论边界
认知科学	人脑与模型上下文都有限，信息会丢失、过载或过期	外部化、结构化、分层与任务裁剪	人员、工具	意图图谱、意图契约、Context Cropping、Knowledge Engineering	结构化信息不能替代价值判断，也不能保证执行正确
控制论	高速执行会放大偏差，晚反馈导致大范围返工	真实观测、目标比较、适度纠正、停止与恢复	流程、工具	SCOPE-V、P $\rightleftharpoons$ E、Verify、五道门、Telemetry	闭环可能受噪声、延迟和错误目标影响，不能无限自治
系统论	单一主体的响应方式不足，多主体又会增加协调复杂度	增加受治理的多样性，兼顾局部与整体	组织、流程	IO/OA/AS、多 Agent、Work Graph、Protocol、Evidence Aggregation	Agent 数量不等于有效多样性，多数共识不等于事实

三者可以连成一条工程逻辑：

认知外部化：让系统知道当前目标、边界与必要上下文

↓

闭环控制：让系统根据真实结果纠偏，并在越界时停止

↓

受治理的多样性：让不同能力单元协作处理复杂问题

↓

证据与责任：让人和系统能够对下一步行动作出裁决

借用信息论的语言，契约、约束和验证提高了研发协作的信噪比，遥测与反馈则持续减少未知和偏差。最终支撑 Agentic Agile 是否有效的，仍然是可执行工件、真实证据、可重复验证和实际业务结果。

三大理论共同形成 Agentic Agile 的基本立场：

人类负责价值与责任，机器负责被授权的执行，规则负责守住边界，证据负责支持决策，遥测负责推动系统进化。

这一立场也解释了 Agentic Agile 与敏捷、DevOps 和现有工程体系的关系。已有方法继续负责产品探索、团队协作、生命周期和交付基础设施；3-4-3 与 SCOPE-V 重点补充概率性 Agent 进入执行路径后所需的意图、权限、反馈、证据和责任控制。二者可以叠加和裁剪，而不需要被描述为互相替代的两套阵营。

3.5 本章实践：用三副镜子诊断一个 AI workflow

选择一条正在使用 AI 的真实研发价值流，分别回答：

认知视角：信息是否散落？Agent 拿到的上下文是否过多、过少或过期？

控制视角：哪里产生真实反馈？失败如何修复？何时停止？

系统视角：任务怎样分解？依赖怎样表达？局部结果怎样汇聚为整体证据？

完成后，为每个缺口标记它主要发生在人员、组织、流程还是工具维度。不急着增加更多 Agent，先选择一个会阻断端到端价值、且能在本轮获得验证的缺口。

3.6 本章小结

认知科学解释为何需要把意图、边界和知识外部化；控制论解释为何必须建立反馈、停止与恢复；系统论解释为何既要利用多 Agent 的多样性，又要控制协作复杂度。它们共同回答了四维为什么必须协同重构，以及 3-4-3 为什么采用这样的运行设计。

本章最小实践：用认知、控制、系统三种视角诊断一条现有 AI 研发价值流，并把问题映射到人员、组织、流程和工具。

完成证据：三个主要系统缺口、对应的四维位置和一个优先改进项。

延伸实践：推荐学习《Agentic Agile 认知课：AI 转型为什么不是工具升级》，用认识论、控制论和系统论重新审视 AI 时代的研发协作。

第四章 | Agentic Agile 五条宣言：先有价值取舍，再有方法

方法只能落实选择，不能代替组织作出选择。同一套 Agent、测试和流水线，既可以用于制造一场短期的效率展示，也可以用于建立长期可信的交付能力。两者的差别不在工具清单，而在速度、质量、成本、风险和责任发生冲突时，组织究竟优先保护什么。

Agentic Agile 用五条宣言明确这种优先级：

意图锚定，胜过详尽规约
多模态涌现协作，胜过仅依赖跨职能人类团队
算法规则与架构韧性，胜过仅依赖流程纪律与仪式
实时遥测与闭环对抗，胜过仅依赖阶段性复盘
基于验证的系统工程，胜过随性提示与氛围编程

“胜过”沿用敏捷宣言的表达。右侧仍然有价值，但当两者发生冲突时，左侧应优先得到保护。五条宣言不是五个可以分别安装的模块，而是贯穿人员、组织、流程和工具的五条取舍原则。

4.1 意图锚定，胜过详尽规约

AI 擅长执行明确指令，也会把不完整的指令执行得看似无懈可击。一个写满细节，却没有说明目的、边界和取舍的需求，只会让 Agent 更快地放大错误。

意图锚定不是用愿景代替规格，也不是鼓励少写需求，而是先明确规格赖以成立的判断：要改善什么结果，本次不做什么，哪些约束不能被牺牲，以及依据什么事实判定完成。规格提供执行精度，意图则在规格出现缺口、异常或冲突时提供取舍依据。

以“超过 5000 元增加总监审批”为例，仅有这条规则还不够。组织还需要说明它要降低什么风险，5000 元是否包含边界值，哪些币种和报销类型属于本次范围，以及普通报销的时效不能受到怎样的影响。否则，实现、测试和验收可能各自看似正确，最终却共同偏离业务目标。

这条宣言要求每个正式任务至少具备：

- 可观察的目标与完成条件；
- 明确的范围和非目标；
- 可执行的规则与例外归属；
- 对高影响取舍负责的人类签署者。

当管理层把“工具激活率”或“生成代码占比”设为转型目标时，意图锚定首先追问：这项投入究竟要改善哪条价值流，哪些质量、合规和客户结果不得变差？采用率可以作为过程信号，却不能替代价值目标。

4.2 多模态涌现协作，胜过仅依赖跨职能人类团队

跨职能团队仍然重要，但人类岗位不再是组织能力的唯一载体。需求文本、代码、日志、测试、运行数据和设计图像等不同模态的信息，可以由人和不同 Agent 共同处理。一个任务因此能够在同一意图下获得多种专业视角，而不必完全依赖岗位之间的顺序交接。

“涌现”不是让更多 Agent 自由对话，更不是把多数意见当成真相。它指的是把互补能力组织成受约束的工作网络，使系统获得单一角色难以稳定提供的覆盖能力：谁分析规则，谁寻找反例，谁实现最小改动，谁检查安全，谁汇总证据；每个角色输出什么，发生冲突时又由谁裁决。

例如，一项报销规则变更可以并行展开：领域分析识别边界和币种歧义，测试从验收条件构造反例，安全检查数据与权限，实施者只在确认后的范围内修改。并行的价值在于增加独立发现问题的机会，而不是让更多 Agent 同时修改同一段代码。

因此，组织要稳定两件事：

- 动态组合能力：任务按依赖、写入范围和风险拆分，Agent 获得最小必要上下文与权限；

- 稳定承担责任：业务意图、例外处理、发布授权和风险后果必须有明确的人类责任节点。

执行组合可以动态变化，责任却不能随之漂移。是否新增一个 Agent，也应有明确标准：它必须带来新的专业视角、独立证据或可验证的吞吐改善；如果只是增加消息和对话轮次，就没有形成有效协作。

4.3 算法规则与架构韧性，胜过仅依赖流程纪律与仪式

“发布前要小心”、“不能碰生产数据”、“必须先回归测试”这类要求，如果只存在于培训、会议纪要和个人记忆中，就很难在持续加速和高压环境下稳定执行。Agent 不会因为读过制度就始终遵守，人也不应反复充当机器本可完成的检查器。

算法规则是能够被运行环境自动检查和执行的约束，架构韧性则是系统在错误发生后限制影响、降级运行、恢复状态和升级处理的能力。前者回答“什么不能做、越界后如何停止”，后者回答“出错后如何控损和恢复”。两者共同界定系统的安全自治边界。

一条可运行的规则至少应明确适用范围、检查方式、失败动作、证据位置、例外批准人和失效时间。对应的工程能力包括权限边界、策略检查、测试门禁、超时与重试、幂等处理、回滚以及人工接管条件。但并非所有判断都能自动化：价值冲突、例外授权和不可逆行动仍必须由有权的人裁决。

这条宣言反对两种极端：一种是在引入 Agent 后叠加更多人工签字，把机器速度重新变成流程等待；另一种是把所有风险都交给自动化，误以为门禁可以替代责任。合理的分工是，让重复、明确、可检查的控制进入运行环境，把涉及价值冲突、重大例外和不可逆后果的判断留给人。

4.4 实时遥测与闭环对抗，胜过仅依赖阶段性复盘

阶段性复盘仍有价值，但它发生在行动之后，无法独自控制正在高速运行的系统。Agent 可以在一项任务中连续生成、失败、重试、绕开限制并消耗资源；如果组织只在月报中看到“最终完成”，就无法知道偏差何时出现、修复发生了多少次、成本又被转移到了哪里。

实时遥测持续记录运行事实，包括目标达成情况、首次成功率、约束失败、自动修复、人类介入原因、耗时与成本，以及上线后的质量和价值。闭环对抗则让不同角色、证据和反指标相互校验，防止单一指标制造成功假象。自动化率上升，但返工、等待或事故同时增加，不是进步；人类介入减少，但例外和风险被隐藏，也不是自治。

遥测的对象是由人、Agent、上下文、流程和工具共同构成的生产系统，而不是用于员工排名的监控装置。遥测只有能够触发行动才形成闭环：某类任务反复失败，系统应补充意图、修正规则、调整编排，还是降低授权等级？某项试点表现良好，组织应扩大范围，还是先检查成本和风险是否转移到了下游？

阶段性复盘因此不会消失，而是从“凭记忆讲故事”转变为基于任务证据、趋势和反指标的组织学习。

4.5 基于验证的系统工程，胜过随性提示与氛围编程

氛围编程（Vibe Coding）适合探索，可以帮助团队快速制作原型、验证想法和理解陌生代码库。问题不在探索本身，而在于探索产物被直接当成正式交付，或者生成者既定义标准、又检查结果，最后再自行宣布完成。模型给出的自信解释不能代替证据。

正式工程需要建立独立于生成者的判定机制。验收条件、约束、测试、运行结果和发布结论必须能够相互追溯；失败不能通过降低标准或补写说明而消失；高影响行动必须由有权的人基于证据裁决。

验证强度不应一刀切。低风险、可丢弃的探索可以保持轻量；进入正式价值流的改动应建立契约、约束和 Evidence Bundle；涉及资金、数据、客户权益或不可逆发布的任务，则需要更严格的职责隔离和人工授权。这些机制不是单纯给速度增加负担，而是让快速交付能够被信任、复查和重复使用。

对于“超过 5000 元”的规则，把代码写成 `amount >= 5000` 并不等于完成。验证必须回到已确认的业务口径，独立覆盖 4999.99、5000.00、5000.01 等边界，并保留真实运行记录。即使没有参与生成过程，裁决者也应能够仅凭契约、工件和证据判断结果是否满足约定。

4.6 五条宣言如何共同工作

五条宣言共同形成一个从价值定义到证据裁决、再回到系统改进的闭环：

意图锚定：先确认要创造什么价值、承担什么约束

↓

涌现协作：组织人和 Agent 的互补能力

↓

规则与韧性：把可重复控制和恢复能力放入执行环境

↓

遥测与闭环：在运行中识别偏差、成本与风险转移

↓

验证工程：用独立证据支持通过、调整、回退或停止

这不是只能顺序执行一次的流程。执行中的遥测和验证结果会持续反馈到意图、约束、协作与规则，使五条宣言构成闭环，并推动系统在保持责任边界的同时逐步演化。

五条宣言也不能互相替代。没有意图，系统可能高质量地完成错误任务；没有涌现协作，关键问题容易被单一视角遗漏；没有规则与韧性，多主体执行会更快越界，也更难恢复；没有遥测，组织无法判断局部速度是否损害整体价值；没有验证，所谓“完成”仍然只是执行者的声明。

4.7 把宣言用于一项真实转型决策

当组织提出“本季度统一接入 Agent 平台，并把 AI 生成代码占比提升到 50%”时，五条宣言不会先讨论这个数字是否足够激进，而是要求把工具推广目标重写为一项可验证的转型决策：

1. 要改善的业务结果、价值流和不可退让的风险边界是什么？
2. 哪些人机能力需要并行组合，谁对意图、例外和发布负责？
3. 哪些控制应变成运行时规则，哪些行动必须保留人类裁决？
4. 用哪些相互制衡的信号证明效率提升没有转化为返工、等待或风险转移？
5. 第一批真实任务需要什么证据，才能决定扩大、调整或停止？

只有回答这五个问题，组织才拥有一项可以执行、验证和调整的转型假设，而不只是一份工具推广计划。

4.8 本章小结

认知科学、控制论和系统论解释了 Agentic Agile 为什么需要外部化认知、建立反馈闭环并治理复杂协作。五条宣言在此基础上进一步明确：当速度、质量、成本、风险和责任发生冲突时，组织应优先保护什么。

宣言提供的是取舍原则，而不是现成流程。后续的 3-4-3、SCOPE-V、Harness、Evidence 和 Telemetry，将把这些原则转化为可以执行、验证、停止、恢复和持续演化的日常运行能力。

第二部分 | Agentic Agile：人机协同研发操作系统架构

第一部分回答为什么要重构，并建立四维模型、理论基石和价值取舍。第二部分开始回答“换成什么”：人员、组织、流程与工具怎样通过一套共同架构连接起来，而不是各自推进 AI 项目。

本部分先给出人机协同研发操作系统的总装图，说明四维转型、3-4-3、SCOPE-V、Harness、Evidence 与 Telemetry 分别解决什么问题；随后展开三个超级角色、四大动态工件与三大自治机制。第三部分把这些结构放入真实任务与交付流水线运行；第五部分再处理既有系统、知识记忆和多主体协作。

本部分阅读路径

1. **第五章 | 系统总览**：先看四维转型、3-4-3、SCOPE-V、Harness、Evidence 与 Telemetry 如何组成一套系统，而不是一串术语。
2. **第六章 | 三个超级角色**：明确谁定义价值、谁设计控制系统、谁在授权边界内执行。
3. **第七章 | 四大动态工件**：让意图图谱、意图契约、约束矩阵与 Evidence Bundle 拥有机器可读取、可追溯的共同语言。
4. **第八章 | 三大自治机制与 Harness**：回答 Agent 怎样获得执行权、怎样被纠偏，以及何时必须把决定交还人类。

读完这一部分，你应能画出一项 AI 研发任务的责任、工件与自治边界；第三部分将把这张图放进真实任务和交付系统中运行。

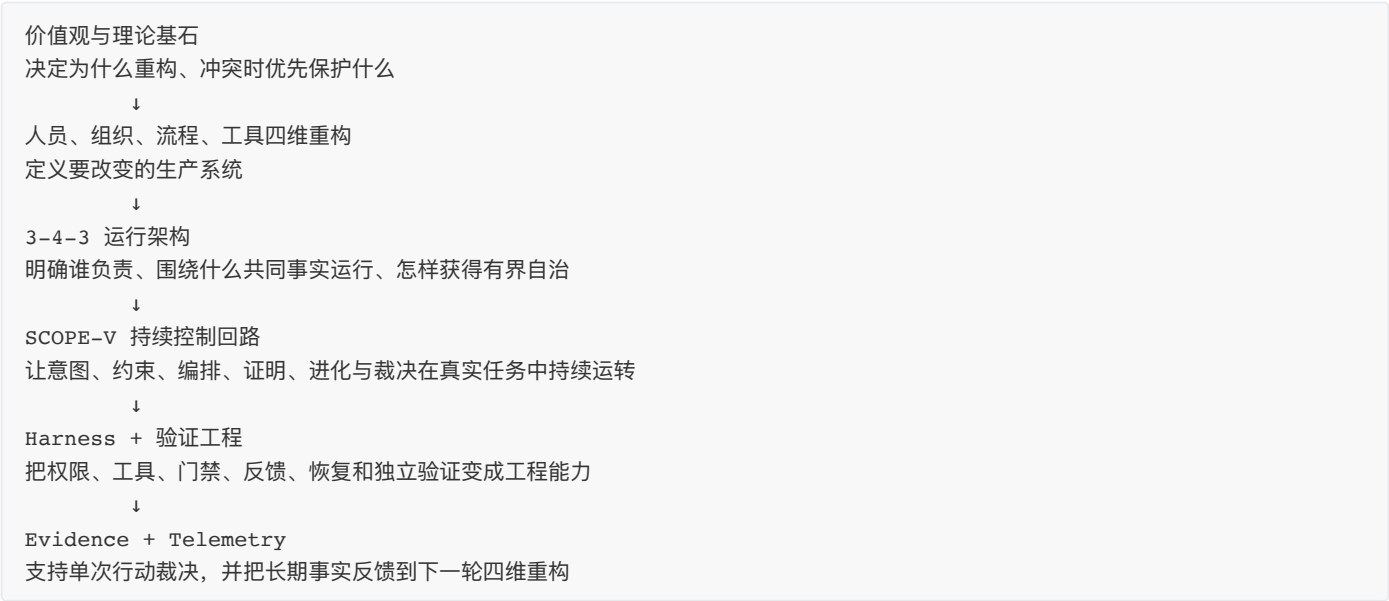
第五章 | 系统总览：四维转型、3-4-3、SCOPE-V 与遥测如何组成整体

第一部分说明了为什么人员、组织、流程和工具必须围绕同一条价值流协同重构。接下来的问题是：这四个维度靠什么连接，又怎样在一项真实任务中运行起来？

Agentic Agile 用 3-4-3 建立责任、工件与自治架构，用 SCOPE-V 驱动任务持续运行，再由 Harness、验证工程、Evidence Bundle 和 Telemetry 承载控制、裁决与长期反馈。本章先给出整套系统的总装图，回答一个核心问题：**这些要素怎样共同作用，把 AI 的执行速度转化为可信、可持续的组织效能？**

5.1 总装图与职责分层：从价值意图到组织进化

从整体看，这套操作系统由六个相互衔接的层级组成：



这六层不是一次性从上向下执行的流程。任务完成后，业务结果和运行事实还会通过 Evidence 与 Telemetry 返回组织，用来调整角色能力、授权范围、流程节点、工具配置、约束规则、知识资产和投资决策。正是这条反馈路径，让操作系统持续运转，而不只是一份项目启动时的设计方案。

下表给出各层最主要的职责。它们彼此配合，但不能互相替代。

层级或机制	核心问题	主要产出	不应被理解为
四维重构	人机生产系统的什么必须改变？	人员责任、组织授权、价值流与工程能力的目标状态	四个独立转型项目
3-4-3	谁负责，系统围绕什么运行，自治怎样受控？	三个超级角色、四大动态工件、三大自治机制	新岗位编制或文档清单
SCOPE-V	一次真实任务怎样持续校准、执行、纠偏和裁决？	六个持续控制面及其状态转换	六步瀑布流程
Harness	规则和控制怎样进入 Agent 的实际运行环境？	上下文、工具权限、门禁、执行、恢复与观测能力	某个模型或单一供应商平台
验证工程	候选结果怎样变成可以信任的完成事实？	与风险相称的独立验证和证明链	测试数量竞赛或末端审批
Evidence Bundle	这一次任务是否有足够事实支持下一步行动？	AC、约束、测试、风险、差异、恢复与裁决证据	发布后补写的总结报告
Telemetry	这套人机系统长期是否更准确、自主、经济并持续学习？	可比较的价值、能力、效率与进化趋势	AI 使用量或个人绩效排名

具体实践可以根据风险和规模裁剪，但边界不能混淆：SCOPE-V 解决不了责任缺位，Dashboard 代替不了单次任务的证据，工具平台无法替业务负责人定义意图，签署也不能代替真实运行中的验证。

5.2 从任务闭环到企业扩展：三个运行范围

同一套控制关系会出现在三个不同范围。任务运行层关注一项可签署、可验证的工作；价值网络层处理多个责任单元之间的依赖和证据聚合；战略意图层决定投资方向、风险红线以及是否继续扩大。战略约束向下转化为权限、门禁和升级条件，任务证据则在保留来源、口径与缺口的前提下逐层向上汇总。

三个范围都围绕意图、约束、证据、裁决和反馈运行，但责任人、信息粒度和决策节奏不同。任务层的一次 PASS，只能说明当前行动得到了哪些证据支持；要判断一条价值流是否值得扩展，还必须观察跨任务、跨责任单元的长期结果。

5.3 3-4-3：相对稳定的运行架构



3-4-3 用三个问题概括人机协同运行中最关键的关系。

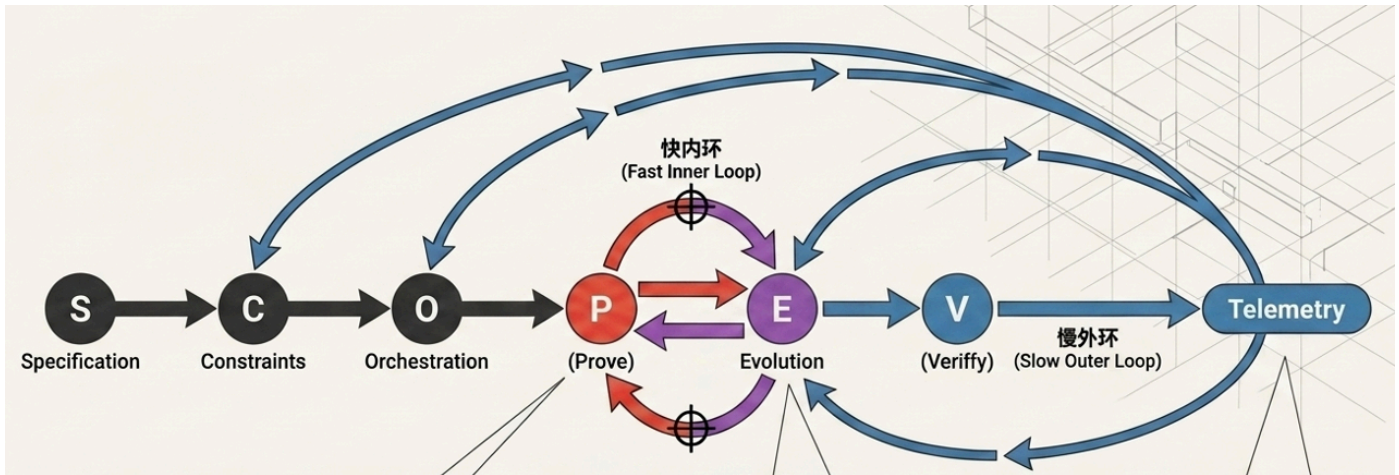
第一个“3”回答谁负责。 意图主理人 (IO) 对价值意图和风险取舍负责, 编排架构师 (OA) 对控制系统与任务编排负责, 自治蜂群 (AS) 负责在约束内执行并产出候选结果。三者代表的是责任, 不是固定职位。同一个人可以在不同任务中承担不同角色, 但任何一项关键责任都不能悬空。

中间的“4”回答大家依据什么协作。 意图图谱连接长期方向与当前任务, 意图契约写明本次工作的目标和完成标准, 约束矩阵划定可执行边界, 证据包 (Evidence Bundle) 汇集足以支持裁决的事实。原本散落在人脑、会议和系统中的认知, 由此变成人与 Agent 都能读取、执行和复查的共同依据。

最后一个“3”回答自治怎样形成。 意图注入把开放探索收敛为可签署的任务, 高频对抗与自净化推动 Agent 根据反馈持续纠偏, HITL 与证据验收则把机器无权决定的事项交还给有权的人。

3-4-3 描述的是相对稳定的责任结构、共同事实和自治逻辑, 并不规定每项任务必须遵循怎样的时间顺序。让这些要素在真实任务中持续运转的, 是 SCOPE-V。

5.4 SCOPE-V: 把架构变成持续控制回路



SCOPE-V 把 3-4-3 从架构带入真实任务。它关注的不是六个阶段依次走完, 而是六个控制面能否在执行过程中形成清晰的状态、门禁和回流:

控制面	控制的问题	与 3-4-3 的连接
Specify	当前意图是否足够明确、可判定并已签署？	IO、OA、意图图谱与意图契约
Constrain	哪些边界不可突破，例外和恢复怎样处理？	OA、约束矩阵、工具权限与 HITL
Orchestrate	哪些能力以什么依赖、权限和停止条件协作？	OA、AS、Work Graph 与 Handoff
Prove	什么证据能够证明实现满足意图和约束？	AS、验收标准、测试与机械门
Evolve	失败怎样诊断、最小修复并回到原检查？	高频对抗、自我修正、知识与记忆晋升
Verify	现有证据是否足以支持发布、补证、返工或停止？	Evidence Bundle、IO/OA 责任与独立裁决

六个控制面并非单向推进。新事实出现后，Specify、Constrain 和 Orchestrate 都可能重新打开；Prove 与 Evolve 之间会反复纠缠；Verify 一旦发现证据不足或相互矛盾，也可以要求补证、返工或退回。图上可以把它画成一条主线，实际运行却是一组有状态、有门禁、能够回流的控制回路。

5.5 三种时间尺度：机器快内环、任务控制闭环、组织慢外环

机器可以在几秒内完成一次尝试，组织决策却往往以天、周甚至季度为单位。要让两种节奏共存，系统必须同时处理三种时间尺度：

机器快内环：Prove ⇌ Evolve
秒、分钟或小时级执行、测试、诊断、最小修复和重跑

任务控制闭环：Specify → Constrain → Orchestrate → Prove ⇌ Evolve → Verify
围绕一份已签署契约完成授权、执行、证据聚合和责任裁决

组织慢外环：Verify → Telemetry → 人员/组织/流程/工具调整
跨任务观察价值、自治、效率和学习，决定扩大、收缩、改进或停止

三个尺度各有边界。机器快内环可以修正实现，却无权重新定义业务目标；一次 Verify 通过，只能说明当前任务具备行动条件，不能据此宣称组织转型已经成功；组织慢外环观察的是长期趋势，也不能用平均值掩盖某个高风险任务的证据缺口。

人不需要跟随机器内环的每一步执行，但不能退出最终责任链。涉及意图与底线的确认、目标与证据冲突的裁决、权限扩展与不可逆操作的批准，以及剩余风险的接受，都必须由人承担；系统的长期运行事实，则应成为收紧、维持或扩大授权的依据。

5.6 Evidence 与 Telemetry：一次裁决和长期进化

Evidence Bundle 与 Telemetry 都建立在事实之上，但回答的问题不同。Evidence 支持当前任务的行动裁决，Telemetry 则帮助组织判断跨任务的长期趋势。

维度	Evidence Bundle	Telemetry
核心问题	这一次交付是否足以支持某项行动？	这套人机系统长期是否真正提效并变得更健康？
观察粒度	单契约、单任务、单次发布	多任务、项目、价值流或可比组织单元
时间范围	当前证据窗口	带分母、样本量和风险分层的时间序列
典型内容	AC、约束、测试、差异、风险、恢复、签署与裁决	目标准确率、首次通过、自愈、HITL、上下文、Token、执行效率与知识进化
主要动作	发布、灰度、补证、返工、缩小范围或停止	扩大自治、收紧权限、修改流程、补工程基础或停止试点
主要误用	由 Agent 自己写结论，缺少原始证据	用调用量和生成量排名个人，忽略成本转移与反指标

Evidence 是 Telemetry 的重要来源，却不是唯一来源。生产结果、用户采用、故障、成本和组织等待同样需要进入长期观察。反过来，Dashboard 上的平均成功率再高，也补不上某个具体任务缺失的约束证据和发布签署。

5.7 四维重构怎样落到运行系统

四维与 3-4-3 并不是简单的一一对应。三个角色会改变组织授权和流程责任，四大工件也不仅仅是工具中的几种数据结构。每个运行要素都会同时影响多个维度：

四维	3-4-3 的主要承载	SCOPE-V / Harness 的主要承载	要证明的改变
人员	IO、OA、AS 的责任分配与制衡	HITL、签署、异常升级和独立裁决	人从重复执行转向价值、边界、系统设计与异常判断
组织	意图图谱、责任节点、共享约束与 Evidence 接口	Work Graph、跨模块协议、授权和证据聚合	决策权与能力能够围绕意图组合，责任不漂移
流程	三大自治机制与工件生命周期	SCOPE-V、机械门、回流、停止与恢复	等待和交接减少，机器内环与人类外环各自高效运行
工具	工件结构、约束协议和证据格式	Context、Orchestration、Harness、Evidence、Telemetry、Knowledge & Memory	不同模型和 AI 宿主能够在共同协议下执行、验证和交接

这张表也解释了为什么 Agentic Agile 不能只交给平台团队。平台团队可以建设 Context、Harness、Evidence 和 Telemetry 能力，却不能代替业务负责人定义价值，不能自行重画组织授权，更不能决定风险最终由谁承担。

治理把人类责任、组织授权、流程边界和工具权限连接起来，使 Agent 能够在明确范围内提速。验证工程把候选结果变成可复查的事实，避免生成者自行宣布完成。Dashboard 则把分散的运行数据转化为管理者可以采取行动的信号，帮助判断提效是否真实、自治是否健康、成本是否发生转移、系统是否仍在学习。

三者承担不同职责，却指向同一个结果：让 AI 的速度穿过组织系统，最终形成可持续的业务价值。失去这个目标，治理会滑向官僚，验证会沦为测试堆积，Dashboard 也会变成数字表演。

5.8 一个报销规则如何穿过整套操作系统

沿用“超过 5000 元增加总监审批”的案例，可以看到这些层级怎样围绕同一项改变协同工作：

1. **价值与四维**：目标是降低高额报销风险，同时不增加普通报销摩擦；业务、财务、合规和技术责任被明确，审批流程和工程工具只改本轮必要部分。
2. **3-4-3**：IO 签署金额、币种、范围和成功标准，OA 设计上下文、约束、验证和异常路由，AS 执行规则分析、测试与最小实现；契约、约束和 Evidence 形成共同事实。
3. **SCOPE-V**：需求先澄清“超过”是否包含 5000，生产数据和权限被限制，任务按依赖编排，边界测试先红后绿，失败进入最小修复，最终由证据支持发布或补证。
4. **Harness 与验证**：工具只能访问允许的仓库与测试环境，机械门检查范围、测试和约束，真实运行结果进入 Evidence Bundle，发布和回退仍由有权的人控制。
5. **Telemetry**：上线后观察普通报销周期、高额风险命中、首次通过、返工、HITL、Token 和人工验证成本，判断这次改变是否真的提效，以及是否值得扩展到外币和其他报销类型。

少了任何一层，闭环都会断开：意图不清，团队可能高效实现错误目标；责任不明，例外发生时无人裁决；缺少运行约束，风险会以机器速度扩散；没有证据，“全绿”也无法令人信任；没有遥测，组织就不知道一次成功能否稳定复制。

5.9 最小可运行操作系统

企业不需要先建成完整平台才能开始，但第一次真实闭环至少要有：

- 一个必须改善的业务结果和一条目标价值流；
- 一位能够签署意图与承担结果责任的人；
- 明确的执行范围、非目标、验收标准、权限和停止条件；
- 一个能在约束内执行、测试、失败并留下记录的 Agent 路径；
- 与风险相称的独立验证、Evidence Bundle 和发布裁决；
- 少量具有决策价值的遥测，以及扩大、调整或停止条件。

“最小”指的是只保留本轮裁决真正需要的责任、边界、验证、恢复和反馈，而不是一味减少字段。低风险任务可以采用轻量契约和自动验证；风险越高，职责隔离、行业控制和独立证据就越不能省略。

判断最小系统是否成立，不能只看“3-4-3 文件是否齐全”，而要看一项真实任务能否从意图进入执行，错误能否被发现和阻断，结果能否交给有权者裁决，运行事实能否进入下一轮改进。

5.10 总览阶段最常见的三种误判

- **模型混层**：把四维、3-4-3、SCOPE-V、Harness 与 Dashboard 画在同一层，读者不知道哪个负责决策、哪个负责运行、哪个负责判断长期效果。
- **能力悬空**：契约、约束和 Evidence 只停留在文档中，平台建设却先于价值流、责任人和真实任务。
- **一次成功外推**：用单次演示、生成量或零 HITL 宣称转型成功，忽略证据独立性、成本转移和长期遥测。

面对任何一张架构图，都可以追问五件事：它要改善哪个业务结果，改变人员、组织、流程、工具中的哪些关系，由谁负责，通过什么机制生效，又由什么事实证明。只要其中一项答不出来，这张图就还没有成为可运行的系统。

5.11 本章小结：从模型集合到运行系统

至此，各个概念的关系可以归结为一条完整链路：四维明确要改变怎样的生产系统，3-4-3 建立责任、共同事实和自治架构，SCOPE-V 驱动任务控制回路，Harness 与验证工程提供可信执行能力，Evidence 支持单次裁决，Telemetry 再把跨任务事实带回组织，推动下一轮改进。

这条链路同时运行在三种时间尺度上：机器在快内环中执行与纠偏，任务在控制闭环中被签署和裁决，组织在慢外环中根据长期事实调整人员、组织、流程和工具。只有三种节奏彼此衔接，AI 的局部速度才可能转化为组织层面的真实效能。

本章最小实践：选择一个真实 AI 研发任务，分别写出它的四维改变、3-4-3 责任与工件、SCOPE-V 控制路径、Harness 能力、Evidence 裁决和 Telemetry 反馈。

完成证据：一张没有模型混层、能够从业务意图走到组织反馈的系统总装图。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，亲手完成从意图、约束到证据裁决的第一条任务闭环。

第六章 | 三个超级角色：从岗位分工到人机责任网络

四维重构首先改变人员与组织中的一个基本问题：工作不再只能分配给岗位，也可以分配给 Agent；但价值判断、组织承诺和风险责任仍然必须由人承担。旧岗位名称无法直接说明一次人机任务中谁定义目标、谁设计控制、谁执行、谁有权裁决。

IO、OA、AS 构成了一组最小责任内核。它们把价值、治理和执行分开，使组织可以动态组合人员与 Agent 能力，同时保持关键责任可指认、可制衡、可追溯。三者可以由现有岗位承担，却不能靠岗位名称自动获得。

6.1 从岗位名称转向任务责任

Agentic Agile 的三个角色描述的是责任，而不是固定职位。一个人在不同任务中可以承担不同责任，但同一项关键决策必须明确由谁负责。

IO：决定什么值得做，并对价值和风险取舍负责

OA：把意图变成可治理的执行系统

AS：在授权范围内产生并验证候选结果

划分这三个角色，是为了避免价值判断、治理设计和执行行为混在一起，而不是在组织中增加一层新的职务。

传统岗位可以承担这些责任，但不存在固定的一一映射：

现有岗位或专业	可能承担的责任	不能由岗位名称自动推定的事项
业务负责人、产品负责人	通常承担 IO，也可提供领域约束	是否真正拥有价值取舍、风险接受和发布签署权
架构师、技术负责人	通常承担 OA，也可能对技术意图承担 IO	是否具备业务目标最终解释权，是否独立于实现结果
工程经理、项目经理、敏捷教练	可以承担 OA 的协调与系统运营责任	是否具备 Context、约束、Harness、验证和恢复设计能力
开发、测试、安全、运维专家	可以设计专业 Agent、承担 OA 子责任或参与 AS	是否有权改变业务目标、批准例外或签署发布
研发效能与平台团队	提供 Harness、Evidence、Telemetry 和共享治理资产	不能因为拥有平台就成为所有任务的 IO 或中央审批者

低风险、小范围任务中，一个人可以同时承担 IO 与 OA，但两项责任仍需分别表达；高影响任务则应根据行业要求、利益冲突和证据独立性进行职责分离。灵活分配角色，不等于把责任混在一起。

高风险任务中的独立验证，不构成第四个超级角色。它是一项职责隔离要求：可由未参与实现的人、团队或受隔离的验证能力承担。IO 仍对价值与风险裁决负责，OA 仍对控制设计与证据完整性负责，AS 仍只提供受约束的候选结果；独立性只保证没有同一主体既定义标准、又实现、再批准自己。

6.2 IO：意图主理人

IO 拥有目标的最终解释权和关键取舍权。它可能由产品负责人、业务负责人、技术负责人或其他对结果负责的人承担。

IO 的核心职责包括：

- 明确业务目标和成功标准；
- 选择当前最有价值的范围；
- 确认明确的非目标；
- 决定风险偏好和不可牺牲底线；

- 处理相互冲突的业务规则；
- 审阅完整意图契约并显式签署；
- 基于 Evidence Bundle 作出上线、灰度、返工、缩小范围或停止决策。

IO 不需要亲自编写所有技术细节，但不能把价值判断交给模型。

成熟 IO 不只是“把需求说清楚”，还要能够用业务结果说明为什么值得做，用 Non-goals 主动控制机会成本，用证据与反指标防止局部优化，并在目标、成本、风险和时间冲突时作出可追溯取舍。IO 的价值不由提出多少需求衡量，而由意图是否准确、责任是否明确、最终结果是否值得交付衡量。

6.2.1 IO 的判断能力如何被增强

IO 不需要独自完成所有分析，但必须能够判断 Agent 提供的分析是否足以支持一次取舍。一个高质量的 IO 工作节奏是：

1. 先写出价值结果、成功指标和明确的 Non-goals；
2. 要求 OA 或 Agent 列出关键假设、未知项、影响范围和至少一个替代方案；
3. 针对不可逆、高风险或高成本决定，主动要求反例和失败后果；
4. 只在目标、边界、授权和验收标准足够清楚时签署契约；
5. 根据 Evidence 判断继续、缩小范围、补证、回退或停止，而不是根据 Agent 的表达流畅度做决定。

IO 的效率来自减少低价值的信息搬运，而不是减少关键判断。Agent 可以整理事实、比较方案和模拟边界；IO 必须决定什么值得做、什么不能做、哪些风险可以接受，以及什么结果才算完成。

6.3 OA：编排架构师

OA 是治理系统的设计者和运行维护者。OA 的价值不在于写出更花哨的 Prompt，而在于让 Agent 获得正确上下文、工具、约束、反馈和停止条件。

OA 的核心职责包括：

- 对模糊或可疑需求进行批判性校准；
- 引导 IO 逐项作出关键决策；
- 起草意图契约，但不得代替 IO 签署；
- 维护约束矩阵和 AI Coding Guide；
- 设计 Work Graph、权限和工具白名单；
- 选择验证策略并维护机械门禁；
- 识别需要 HITL 的异常；
- 聚合 Evidence Bundle 和遥测；
- 把新教训反馈到意图图谱和治理资产。

OA 可以由架构师、研发效能负责人、工程经理或具备治理能力的高级工程师承担。

OA 不是换了名称的项目经理，也不是所有任务都要经过的“AI 审批员”。项目协调、会议组织和状态跟踪可以是 OA 工作的一部分，但核心责任是设计一套能够自行运行、在边界处停止并产生可信证据的控制系统。成熟 OA 应持续减少重复人工协调，把一次裁决沉淀为契约字段、约束、路由、验证器、恢复策略或共享 Skill。

OA 也不能为了让系统看起来完整而过度治理。对于低风险、可丢弃的探索任务，简单范围、沙箱和预算限制可能已经足够；对于资金、隐私、生产写入等高影响任务，OA 则必须增加职责隔离、独立验证和恢复证明。治理重量由风险与可证明性决定，不由 OA 的流程偏好决定。

6.4 AS：自治蜂群

AS 是一个或多个执行 Agent 的集合。它们可以承担代码实现、测试、安全扫描、数据处理、文档更新、部署准备等任务。

AS 的“自治”始终是有界自治：

- 只接收当前任务所需的上下文；
- 只能使用 Tools Manifest 中允许的工具；
- 只能修改授权范围；
- 必须遵守 MUST 约束；
- 失败修复不得改变已签署目标；
- 超出预算、权限或风险边界时必须停止；
- 自己宣布完成不构成完成证据。

模型输出具有概率性。确定性来自它外部的契约、约束、工具权限、测试、门禁和证据，而不是来自对模型的想象。

“自治蜂群”也不等于一群被拟人化的“数字员工”。AS 可以是一个 Agent、多个专业 Agent、确定性工具与模型的组合，也可以在不同任务中更换模型和宿主。组织真正依赖的应是稳定的输入、权限、协议、完成判据和证据接口，而不是某个 Agent 名称或人格设定。

AS 可以提出方案、发现矛盾和请求扩权，但不能自行批准自己的请求。它的成熟度不以自主运行时间衡量，而以能否稳定遵守最小权限、保留失败事实、生成可验证 Handoff、在边界处停止并接受外部裁决衡量。

6.5 分开执行、判断与责任

“这项任务交给 人 还是 AI”，这样的划分过于简单粗暴。同一项工作可以由 Agent 执行、由人判断，并由另一位具备授权资格的人承担最终责任。分工时至少要拆开三件事：谁执行，谁判断结果是否成立，谁对后果和组织承诺负责。

工作性质	Agent 可以做什么	人必须做什么	责任设计
高频、可表达、可验证、可恢复	检索、生成、测试、重复修复、证据收集	设定目标、边界与抽查策略	可预授权 AS 执行，OA 对控制有效性负责
复杂但可提供候选	比较方案、识别风险、生成反例、辅助诊断	处理语义冲突、选择方案、接受剩余不确定性	AS 提供候选与来源，IO/OA 在各自权限内判断
高影响或不可逆	分析影响、准备脚本、模拟和预演	批准或执行生产写入、资金操作、外部发布和高风险例外	有权的人签署，职责隔离与恢复证据按风险增加
当前禁止自治	识别缺失信息并停止	完成法律、伦理、资质或组织规定不可委托的判断	工具层阻断，不允许通过提示词临时绕过

判断标准不是“让人介入多少”，而是每类工作是否交给最适合的执行主体，判断是否发生在正确位置，最终责任是否真实存在。减少无价值的人工接力是提效，删除必要的人类责任不是提效。

6.6 显式签署契约为什么重要

逐项澄清只是决策形成过程，不等于契约签署。签署意味着 IO 已经看到完整契约，并愿意对目标、非目标、规则和验收标准承担责任。

因此：

- OA 可以起草契约；
- AS 可以建议 AC；
- 系统可以检测缺失字段；
- 只有 IO 可以完成签署；
- 签署前不得进入业务编码。

这条规则不是形式主义。它防止“大家似乎都同意”被误当成真实授权。

6.7 三个角色之间的制衡

三个角色的价值不仅在于分工，还在于形成制衡。

IO 可能过度追求业务速度，OA 需要指出需求中的歧义、权限扩张和不可验证目标；OA 可能为了治理完整而引入过多流程，IO 需要判断治理成本是否与业务风险匹配；AS 可能提出实现便利的替代方案，但不能自行改变业务目标；Evidence Bundle 可能显示技术上可行，却仍需要 IO 判断商业风险是否值得承担。

IO 不能绕过约束直接命令 AS

OA 不能代替 IO 决定价值和签署契约

AS 不能修改标准证明自己正确

任何角色都不能用口头判断替代 MUST 门禁

这种制衡使“人类在环”从随机介入变成清晰责任设计。

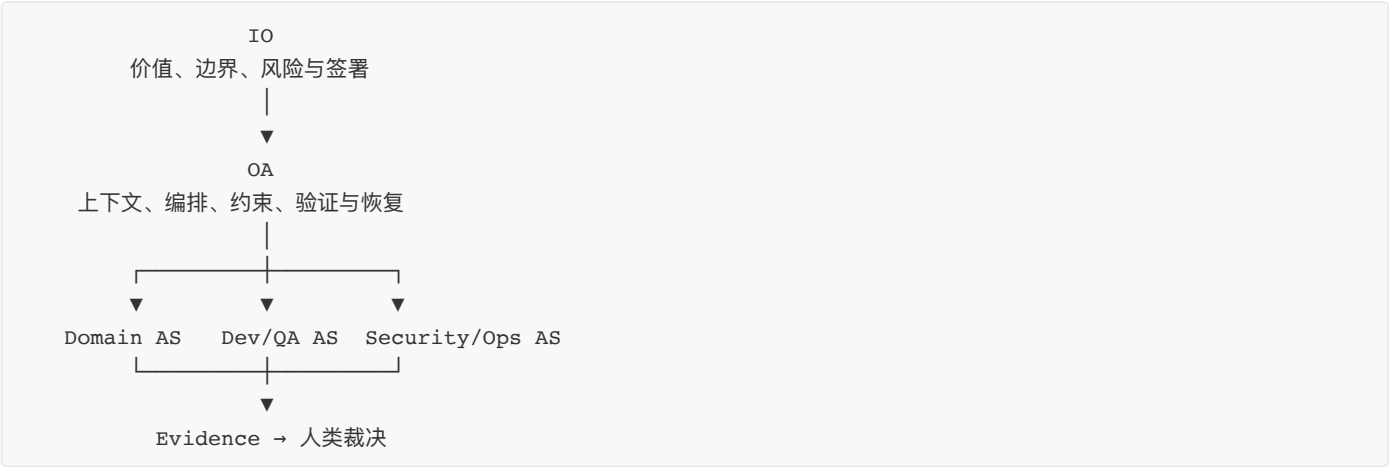
6.8 授权原则：运行时的最小分界

在一项任务中，IO 决定并签署目标、非目标、关键业务规则、风险取舍与不可逆行动；OA 把这些决定转成上下文、约束、Work Graph、门禁、异常路由和证据要求；AS 只在已授权范围内选择执行路径、产生候选结果并报告失败。

因此，业务规则变化必须重新签署，工具或数据权限扩大必须经过相应责任人批准，自动修复只能在 OA 预设的包络内发生，AS 不得自行发布生产或修改标准来证明自己正确。这条分界确保价值判断、控制设计和受约束执行彼此独立：授权可以随任务风险调整，但业务规则、权限扩大和不可逆行动不能被执行者静默改变。

6.9 最小人机责任单元

三个角色进入组织后，不应变成新的纵向部门。更可行的结构是围绕一份真实意图形成最小人机责任单元：一组承担 IO、OA 责任的人类节点，加上一组可以按任务组合和替换的 AS 能力。



这张图表示责任与能力关系，不是汇报关系。IO 不直接逐步指挥每个 Agent，OA 也不成为所有专业判断的唯一中心。领域专家可以把知识转成上下文、约束、测试和验证器，稳定团队继续承载长期产品责任，平台团队提供共享能力；责任单元只组合完成当前意图所需的部分。

责任单元的规模应由任务复杂度和风险决定：

- 低风险单仓任务可以由一人承担 IO/OA，并调用一个通用 Agent；
- 跨模块交付可以设置全局 OA 与模块 OA，保持单一 IO 或明确的意图层级；
- 高风险任务应分离价值签署、控制设计、执行与独立验证，不能由同一主体自定义标准并批准自己；
- 无论规模大小，异常升级只能进入预先指认的人类责任节点，不能在群聊中等待“谁有空谁决定”。

人机责任网络的组织价值，是让执行能力可以动态扩展，而价值与风险责任不会随 Agent 数量增加而稀释。

6.10 一个任务中的角色协作

以费用报销 T-001 为例：

1. IO 提出高额报销增加审批的业务意图；
2. OA 发现“超过”“金额口径”“报销类型”存在歧义；
3. OA 与 IO 逐项确认关键决定；
4. OA 起草契约，IO 审阅后签署；
5. OA 从图谱和约束中裁剪上下文，生成 Work Graph；
6. AS 分别完成测试、影响分析和最小实现；
7. 约束失败在授权内自动修复，业务冲突升级给 IO；
8. OA 聚合 Evidence Bundle；
9. IO 根据证据批准限定范围发布；
10. OA 采集遥测并回写图谱。

整个过程中，人并没有逐行指挥 Agent，Agent 也没有替人承担业务责任。

6.11 角色反模式

IO 只提需求，不承担签署

结果是所有人都可以说“我以为不是这个意思”。解决办法是让签署成为独立、可审计动作。

OA 退化为 Prompt 工程师或流程协调者

OA 花大量时间润色提示词、组织会议和追踪状态，却没有建立契约、约束、工具权限和证据标准。提示词与协调活动可以改进局部执行，但不能替代可运行的治理系统。

AS 被描述为确定性机器

模型输出仍然具有概率性。AS 的可靠性来自外部 Harness，而不是角色名称。

人类只在出事后介入

HITL 既包括异常升级，也包括事前价值签署和不可逆操作批准。介入点需要预先设计。

用自治率评价角色成功

如果减少 HITL 是因为 Agent 绕过了应有审批，自治率越高反而越危险。应同时观察目标、约束、质量和责任。

6.12 角色能力如何成长

IO、OA、AS 的成熟不取决于职级或工具熟练度，而取决于能否在真实任务中稳定履行责任并产生可复查结果。

角色	起步能力	成熟能力	能力证据
IO	说明要做什么，确认基本 AC	定义业务结果与 Non-goals，识别激励冲突，权衡风险并根据证据裁决	意图准确率、返工原因、业务结果、例外与发布签署质量
OA	编写任务说明，配置 Agent 和检查清单	设计 Context、Work Graph、约束、Harness、验证、恢复与遥测，并持续降低无效协调	门禁有效性、异常路由、验证成本、自愈与恢复、治理资产复用
AS	在明确任务中生成候选结果	遵守最小权限，生成可验证 Handoff，保留失败证据，在边界处停止并跨工具稳定执行	目标与约束通过、首次成功、自愈质量、证据完整性、越界与停止记录

传统专业经验不会失效，而会改变承载方式。测试人员对可判定性和独立证据的敏感度，架构师对边界和韧性的判断，产品人员对价值与非目标的取舍，安全人员对威胁与权限的理解，都可以被转化为更高杠杆的契约、约束、验证器、知识和 Skill。

这些能力不会在一天内由新岗位替代旧岗位。更现实的路径是在一个真实任务中明确责任、观察缺口，再通过连续任务积累可复用资产。角色评估也不能只看 AI 生成量、Token、自治率或 HITL 数量，否则会奖励隐藏失败和责任逃逸。

6.13 本章小结

三个超级角色不是新岗位，而是人机协作的最小责任内核。IO 保护价值与责任，OA 设计可运行的控制系统，AS 在授权范围内提供可替换、可验证的执行能力。三者把执行权与最终责任分开，又通过签署、授权、证据和制衡重新连接。

人类没有退出系统，而是从重复执行转向意图、边界、系统设计、异常和裁决；Agent 获得更大执行空间，但不能自批权限、自改标准或自证完成。最小人机责任单元允许能力动态组合，同时确保责任不会在多 Agent 网络中漂移。

本章最小实践：选择一个真实任务，分别指定 IO、OA 与 AS，拆开执行、判断和最终责任，并画出异常升级路径。

完成证据：一张人机责任网络图、一份授权矩阵和三项不可转移的人类责任。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，在一次真实任务中区分执行、编排和最终责任。

第七章 | 四大动态工件：让组织共同语言可以被机器执行

人机协作真正稀缺的，是人和 Agent 对目标、授权、边界与完成事实拥有同一套可执行理解。四大动态工件把这套共同语言保存下来：不同岗位可以据此讨论和签署，不同模型、工具与运行环境也可以读取、检查和交接。

本章导读：三层架构中的工件分辨率

四大工件并不只服务于单个任务，也不意味着每个组织层级都要复制一套同样的文件。它们在三层架构中拥有不同的分辨率：战略意图层维护方向、价值网络层维护能力与责任关系，任务运行层冻结一次可执行授权。

层级	主要承载	变化方式
战略意图层	组织级 Intent Graph、价值假设与长期红线	由战略变化、经营结果和重大风险触发
价值网络层	价值流/产品域 Intent Graph、跨团队 Constraint Matrix 与协议	由依赖、交接、资源和组织责任变化触发
任务运行层	Intent Contract、任务级 Constraint Matrix、Evidence Bundle	由一次授权、执行事件、验证和裁决触发

上层约束向下继承，下层证据向上聚合；下层不得放宽上层红线，但可以为具体任务增加更严格的限制。任务 Evidence 只能证明该任务，不能自动证明价值流或组织成功；只有经过相同口径的聚合和人工解释，才可以进入更高层的遥测与转型决策。这样既避免把治理工件复制成文档森林，也避免用一个“任务完成”冒充“组织转型完成”。

7.1 四大动态工件的闭环关系

在传统软件研发中，需求通常沿着“战略—规划—需求—任务—代码”逐层分解。产品路线图、PRD、Product Backlog、用户故事和验收标准分别承担不同层次的信息表达。这套方式并没有因为 AI 出现就失效，但它有一个默认前提：人类团队会通过会议、追问、经验和组织关系，主动补齐文档之间的空白。

AI Coding 改变了这个前提。Agent 不会天然知道某条需求与公司北极星的关系，不会因为“大家都知道”就理解隐含规则，也不会自动区分会议中的讨论意见和最终决定。更重要的是，它可以在模糊目标下立即行动，并把一种猜测快速传播到代码、测试、文档和多个模块。

四大动态工件把过去依赖人脑和组织默契完成的四种关键功能显式化：

- 用意图图谱保存全局方向与关系；
- 用意图契约冻结一次行动的明确授权；
- 用约束矩阵把跨任务红线变成可执行门禁；
- 用 Evidence Bundle 把执行结果变成可裁决证据。

动态工件	组织共同回答的问题	机器怎样使用	主要责任
意图图谱	我们为什么做，目标、能力、规则、风险与历史怎样关联？	导航和裁剪任务上下文，追溯来源与影响	IO 负责价值方向，OA 负责结构和新鲜度，领域专家确认专业事实
意图契约	这一次被授权做什么、不做什么、怎样算完成？	作为任务执行、测试、变更围栏和门禁的基准	OA 挑战并起草，IO 完整审阅后签署
约束矩阵	无论多急，哪些组织边界仍不能突破？	配置权限、检查、阻断、告警、升级和恢复动作	OA 维护，安全、架构、合规和领域责任人批准各自规则
Evidence Bundle	当前事实足以支持发布、补证、返工还是停止？	聚合 AC、约束、测试、差异、风险与恢复证据供 Verify 使用	AS 和工具产生原始证据，OA 聚合，独立验证者与有权者裁决

机器可执行不等于所有内容都要写成 YAML。价值意图、风险取舍和裁决理由仍要用人能够理解的语言表达；稳定 ID、状态、版本、来源、验收标准、约束和证据引用则应尽可能结构化。需要统一的是语义、责任和协议，不是某一种文件格式或 AI 宿主。

四大动态工件之所以是“动态”的，是因为它们都会随着真实事件改变状态：图谱因战略、任务和事故更新，契约因关键意图变化而重新签署，约束因风险与教训版本化，Evidence 随执行、失败、修复和裁决持续积累。没有所有者、状态、版本和触发事件的文件，只是静态材料，不是运行工件。



四件工件沿着同一条责任链流动：意图契约要能追溯到图谱，执行要同时满足契约和约束，证据包要映射 AC 与约束，最终结果和教训则要回写图谱。

可以把它们理解成一次航行中的地图、航次任务书、航行禁区与航行记录。地图很重要，但不能代替某一次航行的目的和批准；任务书很明确，但不能因此允许船只穿越禁区；船到了目的地，也需要航行记录证明它走了哪条路线、经历了什么风险、是否值得继续扩大航程。

7.2 意图图谱：全局导航与组织记忆

为什么传统需求列表在 AI Coding 时代还不够

传统 Backlog 擅长表达“接下来可能做什么”，通常以 Epic、Feature、Story 或任务的线性列表呈现。它对优先级和计划非常有用，但不一定显式保存需求背后的战略原因、业务规则之间的依赖、历史事故、长期约束和任务完成后形成的新知识。

人类团队可以依赖产品经理、架构师和老员工把这些关系串起来。Agent 却需要机器可检索的连接：这个需求服务哪个目标，涉及哪些用户和能力，依赖哪些模块，违反过什么规则，过去发生过什么事故，哪些知识已经失效。

意图图谱把需求背后的关系网络显式化。列表回答“有哪些事项”，图谱还要回答“它们为什么存在、怎样关联、依据来自哪里”。

传统需求列表常见视角	意图图谱补充的视角
一条需求的标题和优先级	它连接的战略目标与价值假设
当前待办事项	已完成任务、证据和失败教训
上下级拆分关系	用户、能力、模块、规则、风险的多向关系
由人理解背景	可被 Agent 检索和裁剪的上下文来源
完成后关闭	完成后回写知识并影响后续任务

这并不意味着要抛弃 Backlog。Backlog 仍可管理承诺与顺序；意图图谱提供 Backlog 往往没有承担的全局语义和知识导航。二者可以通过任务 ID 相互链接。

为什么 AI 越强，越需要意图图谱

模型能力增强，意味着 Agent 能处理更大范围的任务，也意味着错误意图会影响更大范围。一个只补全函数的工具，误解可能局限在十几行代码；一个能搜索仓库、调用工具、修改多个模块的 Agent，可能把错误假设扩散到整个交付链。

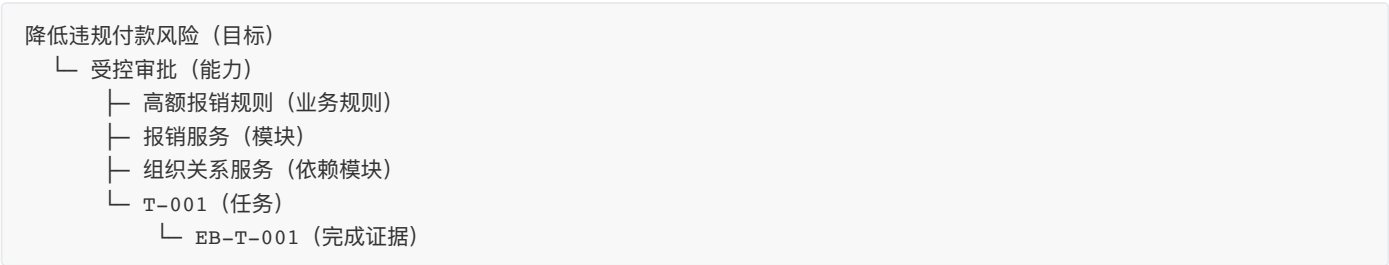
长上下文窗口也不能替代图谱。把所有 PRD、会议纪要和代码一次性塞给模型，会引入过期信息、重复规则、矛盾版本和敏感数据。图谱的作用不是存储一切，而是建立导航：先找到与当前任务相关的节点，再裁剪最小必要上下文。

意图图谱包含什么

意图图谱保存项目长期存在的方向和关系，包括：

- 北极星目标；
- 用户和利益相关者；
- 核心能力与业务域；
- 关键依赖；
- 长期约束；
- 已完成任务；
- 失败教训与模式；
- 后续演进方向。

图谱中的“节点”可以是目标、用户、能力、业务规则、模块、任务、风险或证据；“边”说明它们的关系，例如“支持”“依赖”“受约束于”“由某证据验证”“替代旧版本”。



意图图谱怎样产生

它不是由 IO 一次性画完，也不应完全由模型自动生成后直接当作事实。一个可行过程是：

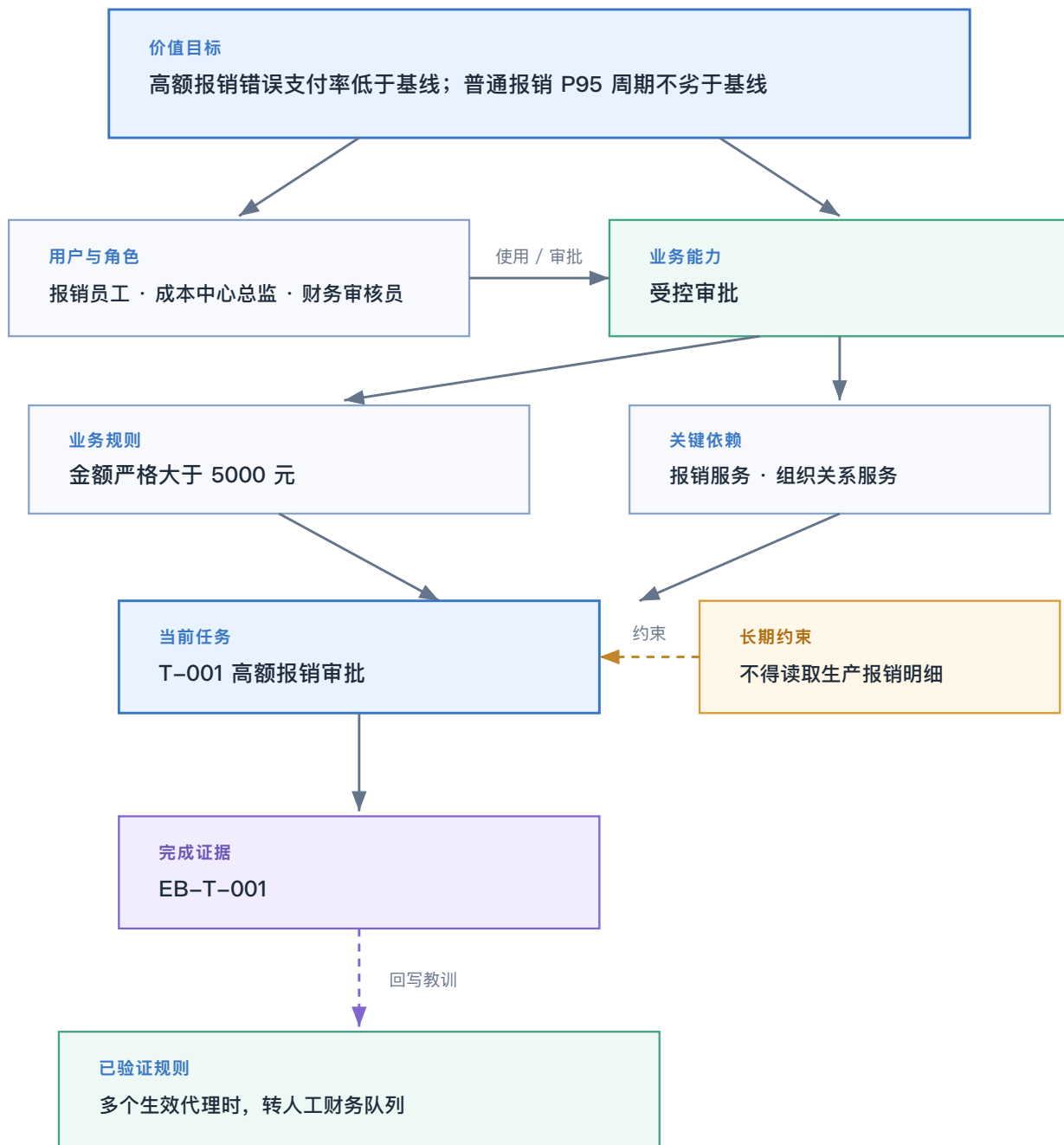
1. IO 提供战略目标、用户问题、现行业务和原始材料；
2. OA 从 PRD、会议纪要、代码、事故记录中提取候选节点与关系；
3. 核对目标冲突、过期规则和缺失来源；
4. 与责任人逐条确认北极星、关键规则、用户范围和硬边界；
5. 把确认内容写入图谱，未确认内容标记为假设或待验证；
6. 每次任务完成后，从 Evidence Bundle 回写新能力、教训、风险与后续方向。

模型可以加速提取，不能自行决定哪些信息是组织事实。IO 对价值方向负责，OA 对结构、来源和新鲜度负责，领域专家对专业规则提供权威输入。

一张最小图谱不需要覆盖整个企业，但目标必须带有结果口径，用户也必须进入关系链。下面的例子展示“可度量目标—用户与角色—能力—规则—任务—证据”如何连成可追溯结构：

费用报销意图图谱

从可度量价值目标到可追溯证据



一个费用报销图谱是怎样长出来的

第一版可能只有“降低违规付款风险”这一北极星和“费用报销”业务域。T-001 启动后，OA 增加高额审批能力、成本中心总监、报销服务和组织关系服务等节点。任务中发现代理总监冲突，完成后又把“多个生效代理时转人工财务队列”作为经验证规则回写。

图谱不是先完整再使用，而是在真实任务中逐步生长。每次生长都应有来源和证据，不能因为 Agent 推测合理就自动成为长期知识。

图谱的常见失效

- 只画战略概念，不连接代码、任务和证据；

- 把所有文档全文塞进图谱，失去导航功能；
- 自动抽取后无人确认，假设与事实混杂；
- 只增加节点，从不淘汰旧规则；
- 没有责任人、版本和最后验证时间；
- 完成任务后不回写，图谱永远停留在项目启动时。

意图图谱不必一开始就是图数据库。一份结构清晰、可维护的 Markdown 就能起步。判断它是否有价值，只需问：当一个新 Agent 接手任务时，能否从任务追溯到目标、规则、依赖和历史教训，并裁剪出可信上下文？

意图图谱与企业本体不是同一个东西：意图图谱描述组织希望改变什么、为什么改变以及目标之间如何关联；本体描述企业中有哪些业务对象、它们处于什么状态、可以发生哪些受控动作。前者回答“组织想去哪里”，后者回答“组织世界中有什么、如何变化”。当本体进入平台建设阶段时，仍然必须由意图图谱提供方向，并由意图契约、约束矩阵和 Evidence 限定每一次具体行动。企业本体的来龙去脉及其与平台化转型的关系，见第二十二章 22.6。

7.3 意图契约：单次任务的执行基准

为什么有了图谱，还需要契约

这是理解四大动态工件的关键。图谱是持续演进的全局知识空间，内容广、关系多，并且可能同时包含事实、假设、候选方向和多个任务。它适合导航，却不适合直接作为一次执行的唯一依据。

如果 Agent 直接依据图谱行动，会遇到三个问题：第一，不知道当前任务到底选中了哪些目标和范围；第二，图谱在任务执行中仍可能更新，导致执行基准移动；第三，图谱没有天然表达“由谁在何时批准了这一次行动”。

意图契约就是从动态全局图谱中裁剪出来的一张“运行态切片”。它把这一次做什么、不做什么、遵循什么规则、如何验收以及由谁授权冻结下来。图谱回答“整个系统知道什么和长期要去哪里”，契约回答“本次被允许做什么”。

契约怎样从图谱产生

契约不是几个图谱节点的复制，而是一次从模糊走向确定的决策过程：

```
IO 发起本次意图
→ OA 从图谱检索相关目标、用户、规则、模块和历史教训
→ 核对歧义、冲突、不可验证项和过宽权限
→ 与 IO 逐条确认关键决定
→ Example Mapping 将抽象规则展开为示例、反例和问题
→ OA 形成 Goal、Non-goals、Rules、AC、风险与边界
→ IO 审阅完整契约并显式签署
```

这里有两个不能混淆的动作：逐项澄清用于形成决策，签署意味着 IO 对完整契约承担最终责任。回答了若干问题，不等于已经批准一份可能还有遗漏的完整契约。

契约至少包含什么

意图契约至少包括：

```
task_id: T-001
goal: 本次真正要改变的结果
non_goals:
  - 明确不做的事项
rules:
  - 必须成立的业务规则
acceptance_criteria:
  - id: AC-01
    description: 可判定的验收标准
    verify: assert/http/db/shell
sign_off:
  intent_owner: 由 IO 本人填写
```

除了这些字段，复杂任务还需要输入输出、边界条件、未决事项、假设、回退要求、相关模块和数据权限。字段多少可以裁剪，但“目标、非目标、可判定 AC、授权”不能被省略。

运行态切片：意图契约 (Intent Contract) – 示例

Intent_Contract_T-001.md

意图契约: T-001 – 项目脚手架 + 设计系统 + 首页 MVP

契约 ID: T-001

版本: 1.0

状态: SIGNED

创建日期: 2025-07-22

关联图谱: governance/Intent_Graph.md §2.4 BRAND

关联约束: governance/Constraint_Matrix.md

§1 背景与目标

背景

门户网站需要从零搭建。需要先建立项目脚手架、设计系

目标

1. 初始化 React 19 + TypeScript + Vite 项目 (基于 m

2. 配置 Tailwind CSS 深色/亮色双主题 (深色科技风 +

3. 创建布局组件 (Header/Navigation/Footer/ThemeT

4. 实现首页核心内容:

• Hero 区域: 价值主张 + CTA

§2 非目标 + 约束引用

约束 ID	约束内容
H-01	遵循 V1.10 Skill 规范
H-02	基于白皮书 v1.3 内容
H-03	用户隐私保护
H-04	不偏离 3-4-3 术语

§3 输入输出契约

输入

• 意图图谱 (Intent_Graph.md)

• 约束矩阵 (Constraint_Matrix.md)

• modern-webapp 模板 (/root/.codebuddy/plugins/.

• dashboard.html 设计参考

• agentic.iloveagile.me 宣言格式参考

输出 (文件清单)

文件

§4 业务规则

设计系统规则

1. 深色主题: 背景 #0a0e17, 卡片 #111827, 边框 #1e293b

2. 亮色主题: 背景 #ffffff, 卡片 #f8fafc, 边框 #e2e8f0

3. 主品牌色: 青色 #06b6d4 (深色/亮色统一)

4. 字体: Inter (正文) + JetBrains Mono (代码/数据)

5. 宣言展示: 左右对比格式 (左侧=我们写信, 右侧=我们承认其价值)

响应式规则

• 桌面 (>1024px): 多列布局

• 平板 (768-1024px): 双列布局

• 手机 (<768px): 单列堆叠

§5 验收标准 (AC)

AC ID	验收标准	验证方式
AC-01	npm run dev 启动成功, 无编译错误	shell: `npm run de
AC-02	首页在 / 路由可访问	http: GET / expe
AC-03	深色/亮色主题切换正常	assert: 点击切换按
AC-04	Hero 区域包含标题、副标题、CTA 按钮	assert: DOM 元素?

契约如何防止两个方向的失控

- 防缺漏：把关键目标、规则和成功标准说清楚；
- 防发散：通过 Non-goals 限制顺手扩展和过度设计。

费用报销契约中的 Non-goals：不迁移历史草稿，会阻止 Agent 因为发现旧数据结构而“顺手完成迁移”； AC：5000.00 元保持原路径，会阻止执行者用模糊的“高额报销测试通过”掩盖边界错误。

契约与传统 PRD、用户故事有什么不同

意图契约不是要取代 PRD 或用户故事。PRD 可以解释产品背景和完整方案，用户故事适合沟通用户价值；契约重点处理一次 Agent 执行的授权与判定。

常见需求材料	主要用途	意图契约额外强调
PRD	产品背景、方案和整体体验	本次执行范围、非目标、机器可验证 AC
用户故事	从用户视角表达价值	规则边界、权限、签署和证据映射
任务卡	安排具体工作	与全局意图和约束的可追溯关系
Prompt	引导模型生成	组织授权、版本、责任和稳定执行基准

Prompt 可以从契约生成，但 Prompt 不是契约。Prompt 面向模型交互，契约面向组织责任和后续验证。

契约什么时候需要重新签署

实现技术选型改变但目标和 AC 不变，通常只需记录技术决定；若目标、非目标、业务规则、关键 AC、风险承担或权限范围发生变化，就必须形成新版本并重新签署。签署后的契约不能为了适配错误实现而被悄悄修改。

读者可以用一句话检验契约：**如果换一个没有参加过会议的 Agent，它能否在不猜测关键业务决定的情况下执行，并让第三方客观判断完成与否？**

契约也是人的判断增幅器

意图契约的价值不只是给 Agent 下达任务，也是在帮助 IO 检查自己的判断。签署前，契约应迫使人回答四个问题：

判断问题	契约中的落点
我们真正要改变什么？	Goal、价值指标和用户结果
哪些事情明确不做？	Non-goals、Change Envelope
什么情况说明方案不成立？	反例、UNKNOWN、退出条件
凭什么接受结果？	可判定 AC、Evidence 映射和责任人

Agent 可以帮助补全歧义、生成反例和比较方案，但不能因为把契约写得完整，就自动获得价值判断权。契约越清楚，人的判断越容易被审阅、被挑战和被复用；这正是把个人经验转化为组织能力的第一步。

7.4 约束矩阵：可执行的安全边界

为什么契约明确了，还需要约束矩阵

契约定义单次任务的目标和授权，但组织中还有大量跨任务长期有效的底线：不能泄露密钥、不能未经批准写生产数据、必须先签后码、关键接口必须向后兼容、资金金额必须使用精确类型。这些规则如果复制到每份契约，不仅冗长，而且容易版本不一致。

约束矩阵回答的是另一个问题：**无论当前任务多急、模型多强，哪些边界都不能被突破？** 契约规定“本次要做什么”，约束矩阵规定“做任何事情都必须怎样”。

还有一个更重要的原因：契约可能本身有错。IO 可能为了赶进度要求直接修改生产数据，但法律、安全和组织红线不能因为某份任务契约被签署就失效。因此需要明确优先级：

法律与安全约束 > 约束矩阵 > 已签署意图契约 > 实现便利性

约束怎样产生

约束不是 OA 凭经验写一套“最佳实践大全”。来源至少包括：

- 法律、监管与行业标准；

- 企业安全、数据和发布政策；
- 系统架构中的不变量与兼容要求；
- Intent Graph 中的长期业务边界；
- 当前契约暴露出的任务特定风险；
- 历史事故、Bug 回溯和混沌实验教训；
- 工程基线，如测试、性能和可观测要求。

OA 负责把自然语言原则转成可执行候选约束，安全、架构、领域和 IO 按责任范围确认。每条约束都需要唯一 ID、适用范围、验证方式、失败动作、Owner 和证据位置。

从一句原则到一条可执行约束

原则：“注意保护财务数据。”这句话没有检查对象，也没有失败动作。

转成约束后可以是：

```
id: C-DATA-07
level: MUST
rule: Agent 不得读取生产报销明细；测试仅可使用脱敏数据集
check: audit_tools --deny production-expense-db
enforcement: block
owner: data-security-owner
recovery: revoke_session_and_escalate
evidence: tool-audit-T-001.json
```

现在系统知道检查什么、失败时做什么、由谁负责、证据在哪里。约束才从“提醒”变成 Harness。

六个核心域

核心六域包括：

域	关注点
STRUCT	项目结构与工件完整性
DATA	数据范围、隐私和完整性
BEHAVE	业务行为和不变量
QUAL	测试、构建、性能和质量门槛
PROC	签署、TDD、证据和流程合规
STYLE	AI Coding Guide 与代码规范

Web 服务可按风险扩展 SEC、REL、OBS 等非功能域。

约束应包含：

- 唯一 ID；
- MUST / SHOULD / MAY 等级；
- 适用范围；
- 可执行检查；
- 所属门禁；

- 失败动作：Block、Warn、Log 或 Escalate；
- 例外批准人和有效期；
- 恢复方式。

“注意安全”不能被执行；“禁止提交密钥，检测到即阻断”才能成为护栏。

Block、Warn、Escalate、Log 有什么区别

- `block`：违反后立即停止，适用于不可接受的 MUST 风险；
- `warn`：保留风险并允许继续，适用于可接受但需要关注的 SHOULD；
- `escalate`：系统无权决定，暂停并提交人类裁决；
- `log`：记录信息，用于趋势观察和未来改进。

不是所有规则都写成 Block 才安全。过多硬门禁会制造约束疲劳和绕过行为；过多 Warn 又会让红线失去意义。选择执行方式时，应考虑影响、可逆性、判断确定性和责任归属。

约束矩阵的生命周期

项目初始化时先建立少量高价值约束，真实任务和事故再驱动增加。约束需要观察命中率、误报、漏报、执行成本和例外使用情况。长期不命中且无法说明风险的规则应复核；经常被申请例外的规则可能与现实脱节，也可能揭示团队在系统性规避底线。

约束矩阵不是越厚越成熟。成熟的标志是关键红线能够被机器稳定执行，例外有负责人和有效期，失败能够阻断或恢复。

7.5 Evidence Bundle：支持裁决的证据包

为什么测试报告还不够

代码完成、测试全绿，并不自动等于可以上线。测试可能没有覆盖关键 AC，约束可能未执行，测试环境可能与当前提交不一致，Agent 可能越权访问工具，回退也可能从未验证。更重要的是，测试报告通常面向工程师，而发布决定还涉及业务价值、残余风险、运行准备和责任承担。

Evidence Bundle 不是项目总结，也不是把日志打包下载。它把“我们做了什么、依据什么相信、还有什么不知道、当前证据支持哪种行动”组织成面向裁决的产品。

Evidence Bundle 怎样产生

证据包从任务启动时就开始积累，而不是等开发结束后再由 OA 回忆补写：

1. 契约签署时，记录任务、版本和意图证据；
2. 编码门保存可信 Red，证明测试确实能发现缺失行为；
3. 实现过程中记录代码差异、工具调用、失败和修复；
4. Prove 阶段汇总 AC、测试、构建、性能和安全证据；
5. Verify 阶段检查约束覆盖、未知项、回退和证据独立性；
6. 采集时长、Token、首次成功率、HITL 和价值信号；单任务摘要可附入证据包，可比较的时间序列再进入 Telemetry；
7. 人类裁决及其条件被写入；
8. 发布后发现 Bug 时，原证据包被追加修正，而不是保持虚假全绿。

AS 负责产生原始运行证据，自动化工具负责采集与校验，OA 负责映射和完整性，IO 或授权责任人负责最终裁决。任何一个角色都不能只用自己的总结证明自己正确。

五类证据

一份成熟证据包至少考虑五类：

证据类型	回答的问题	示例
意图证据	做的是不是已确认的事	契约版本、签署、规则来源
变更证据	实际改了什么、没改什么	Git diff、文件清单、排除范围
行为证据	功能是否满足 AC	单测、接口测试、数据库断言、演示记录
约束证据	红线是否真实执行	权限审计、扫描、门禁与批准记录
运行与价值证据	成本和真实效果如何	Token、时长、失败率、用户与业务指标

证据不是越多越好，而是要映射到判断

一千行日志如果不能说明它支持哪项 AC，就只是材料。证据包需要一张映射表，把目标或约束连接到证据来源、结果、独立性和缺口。例如 AC-02 的“5000.00 元不触发总监审批”对应边界接口测试；C-DATA-07 对应工具审计；回退可用性对应一次实际演练。

高风险结论还要注意证据独立性。同一个 Agent 根据自己的实现生成测试、执行测试并总结通过，可能形成同源自证。关键场景可以使用独立流水线、第二模型、人工审查、外部基准或受控真实数据交叉验证。

最低内容

Evidence Bundle 最低内容包括：

- 契约和约束版本；
- 实际变更范围；
- AC 逐条验证结果；
- MUST 约束检查结果；
- 测试、构建、安全、性能等原始证据；
- 失败与修复历史；
- 未知项和已知风险；
- 回退与恢复方案；
- 单任务遥测摘要；
- 建议裁决及适用条件。

常见裁决包括：

1. Approve；
2. Approve with conditions；
3. Return；
4. Reduce scope；
5. Stop；
6. Escalate。

实际模板还可以支持 `request_clarification` 等裁决。只要任一 MUST 为 FAIL 或 UNKNOWN，建议就不能是无条件 Approve。

HITL裁决卡与证据包示例

人类 Gate 4：裁决卡

检查项	通过条件	结果
意图合法	Contract 状态为 APPROVED，无未决关键项	PASS/FAIL
约束完整	所有 MUST 都有 Enforcement 和 Evidence	PASS/FAIL
行为正确	全量测试通过	PASS/FAIL
覆盖充分	coverage >= 90%	PASS/FAIL
性能充分	专用 benchmark p95 <= 2 ms	PASS/FAIL
故障可检出	两轮注入都先红后绿	PASS/FAIL
风险可接受	无未处理阻断风险	PASS/FAIL
权限正确	最终批准由人类填写	PASS/FAIL

裁决规则

- **APPROVE**: 所有检查项 PASS，且无 MUST 为 FAIL/UNKNOWN。
- **REJECT**: 任一 MUST 为 FAIL，或证据证明行为不满足契约。
- **CONDITIONAL APPROVE**: 仅用于非 MUST 风险，必须记录条件、责任人和截止日期；不得绕过 MUST。

一个证据包怎样改变发布决定

报销任务所有单元测试都通过，但 Evidence Bundle 显示外币字段未验证、功能开关回退已通过、当前只影响两个试点部门。IO 因此作出“仅人民币场景灰度、外币入口阻断、24 小时后复核”的条件批准。

如果只看测试报告，团队可能全量上线；如果因为存在未知项一律拒绝，又会失去合理的渐进交付。证据包的价值，是让人能够在看清已知、未知和边界后作出更精细的裁决。

证据包如何被阅读

裁决不应从日志开始，而应沿着决策顺序审阅：

1. 这次目标和非目标是什么？
2. 实际修改范围是否一致？
3. AC 和 MUST 是否全部通过？
4. 发生过哪些失败，怎样修复？
5. 哪些事项仍然未知？
6. 回退是否可执行？
7. 当前建议是什么，适用什么条件？

如果证据包不能让决策者快速回答这些问题，它就只是材料堆积，而不是支持裁决的产品。

证据包的常见失效

- 只有成功截图，没有原始命令、环境和退出码；
- 测试很多，但没有与 AC 和约束映射；
- 只记录最终全绿，删除失败与修复历史；
- 使用上一提交的测试结果证明当前代码；
- Agent 的自然语言总结被当成运行证据；
- 未验证事项被留空，而不是标记 UNKNOWN；
- 证据包完成后发现 Bug，却不修正首次成功率和原裁决。

证据包不替人决策。它让人的决策可解释、可追溯，也允许人基于真实证据选择“不上线”。

📄 Evidence_Bundle.md artifacts

Evidence_Bundle.md 已创建。文档结构：

章节	内容
§1 构建标识	Python 3.11.1, Git master (0 commits), pytest 9.1.1, PyYAML 6.0.3
§2 目标与非目标	15 项 AC 全部 ✅
§3 约束符合性表	40 条 CON，逐条 PASS/FAIL + 检查方式 + 证据
§4 测试结果	42/42 passed, exit 0
§5 覆盖率	100% (146/146)
§6 性能结果	1000 warmup / 10000 measure / p95=0.0067ms
§7 故障注入	两轮演练：float 污染 + 阈值篡改，含注入/失败/修复/重验完整链路
§8 已知风险	8 项 UNKNOWN（含环境、规模、配置、未决事项）
§9 回退方案	触发条件 + 具体操作
§10 人工介入	5 次 IO + 2 次自动修复
§11 建议裁决	READY_FOR_HITL （全部 MUST PASS）
§12 人类裁决区	留空

工件创建出来，只是第一步。它们还要相互追溯、保持版本一致，并以合适的粒度服务下一次任务，才能成为真正可用的运行资产。这需要两类维护：先保证彼此一致，再管理各自的生命周期。

7.6 工件一致性：从意图到证据

工件一致性检查关注的不是工件数量，而是每一步是否可追溯、是否仍然适用、是否由当前证据支持。

检查关系	必须回答的问题	典型证据
Intent Graph → Intent Contract	本次目标、范围和来源是否来自已确认意图？	目标节点、来源、契约版本
Intent Contract → Constraint Matrix	契约是否叠加了适用的硬边界，而不是自行放宽？	约束 ID、适用范围、例外记录
Contract + Constraints → Implementation	实现是否只改变授权范围，并满足规则与 AC？	差异、测试、静态检查
Implementation → Evidence Bundle	证据是否来自当前提交、当前环境，并覆盖关键判定？	命令、退出码、环境、结果、UNKNOWN

任何一条链断裂，都可能出现“测试通过但任务错误”。

一致性解决“能否对上”的问题；一旦对齐关系成立，工件还必须随真实事件创建、更新、冻结、撤回和回写，才能持续保持可信。

7.7 工件维护链：生命周期、版本时效与粒度

四大动态工件各自有不同生命周期：

工件	创建与所有者	主要触发事件	变更规则	完成后的状态
意图图谱	项目或价值流启动；IO 负责方向、OA 负责维护	战略调整、任务完成、事故、知识验证或失效	保留来源、状态和关系变化，不把候选假设直接晋升为事实	持续存在并为后续任务提供导航
意图契约	单任务 Specify；IO 对签署负责	目标、非目标、业务规则、关键 AC、风险或权限变化	形成新版本并重新签署，不能静默覆盖	完成后冻结并保留
约束矩阵	项目或稳定模块初始化；OA 与领域责任人共同维护	架构、法规、风险、事故、例外和验证教训变化	明确适用范围、级别、检查、失败动作、例外和有效期	持续执行，过期规则必须撤销或重签
Evidence Bundle	从任务开始持续积累；OA 负责完整性	每次执行、验证、失败、修复、提交和裁决	只引用当前版本与新鲜证据，保留失败历史和 UNKNOWN	完成后成为裁决与审计记录

工件不是在最后一次性补写。契约和约束必须在编码前存在，证据应随执行产生，图谱必须在收尾前回写。

版本与时效

常见失败不是工件不存在，而是版本不一致：

- Agent 使用了旧契约；
- 约束已经改变，Evidence Bundle 仍引用旧结果；
- 代码提交发生变化，测试证据来自上一提交；

- 图谱中的业务规则已经失效；
- 例外已经过期仍被继续使用。

因此工件需要任务 ID、版本、时间、代码提交或产物标识。证据新鲜度检查应确认：当前证据仍对应当前提交、当前环境与当前约束版本。

跨 Codex、Claude、WorkBuddy 或其他宿主协作时，稳定 ID、状态枚举、签署语义、证据引用和版本关系尤其重要。宿主可以使用不同界面和模型，但不能各自解释“SIGNED / CLOSED”“PASS”“UNKNOWN”或“当前版本”。跨工具能力来自共享协议，不来自所有工具长得一样。

粒度

工件过粗会让一次任务携带整个项目的复杂度；过细则会制造大量维护成本。

合理粒度通常满足：

- 一个契约对应一个可独立判断完成的任务；
- 一个 Evidence Bundle 对应一个契约；
- 约束矩阵以项目或稳定模块为边界；
- 图谱维护长期稳定关系，不记录每个临时执行步骤；
- 跨模块任务由模块契约和 Release Evidence Bundle 聚合。

当工件既能相互对齐、又能在正确时间以正确粒度更新时，它们才真正能够被跨角色、跨 Agent 和跨工具复用；否则只会形成难以维护的文档森林。

7.8 工件反模式

- 图谱只增加节点，从不删除过期知识；
- 契约写成需求散文，没有 Non-goals 和可执行 AC；
- 约束矩阵复制模板后全部保留，包含大量不适用检查；
- Evidence Bundle 只有“全部通过”，没有原始命令和失败历史；
- 用一个项目级证据包覆盖几十个无法追溯的任务；
- 完成后才补签契约；
- 将模型总结当成运行证据。

7.9 本章小结

四大动态工件把全局方向、单次授权、执行边界和完成事实连接成一套人机共同语言。它们都有所有者、状态、版本和触发事件：图谱负责导航，契约负责授权，约束负责守界，Evidence 负责支持裁决。

格式可以按风险与工具裁剪，但目标、责任、状态和证据语义必须稳定，才能支持不同人员、Agent 和宿主持续交接。

本章最小实践：为同一个真实任务创建四大动态工件的最小版本，标明所有者、状态、版本、触发事件和下游机器用途。

完成证据：四份可相互追溯的工件、统一 ID 映射，以及至少一个能够读取契约或约束并产生真实证据的机械检查。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把意图、契约、约束和证据变成机器可读、可检查的共同语言。

第八章 | 三大自治机制与 Harness：让 Agent 获得有边界的执行权

8.1 三大自治机制

本章所说的“自治”，是让 Agent 在已签署意图、明确权限和可验证边界内获得执行权，而不是把责任整体交给 AI。三大机制回答三个连续问题：**任务从哪里获得确定目标，执行偏差怎样被纠正，机器不能决定时由谁接管**。缺少其中任何一个，Agent 的能力越强，失控时造成的影响就越大。

机制一：意图注入与对话到契约——给自治执行一个稳定起点

探索阶段充满讨论、假设和变化，执行阶段却需要一份稳定、可判定、经过授权的任务基准。意图注入负责完成这次转换，而不只是把一段 Prompt 发给 Agent。两者如果混在一起，Agent 可能把一句试探性意见当成最终决定，也可能在对话后半段遗失前面的关键限制。

意图注入通常经过四步：

- 1. IO 提出目标或问题，不要求第一句话已经完整；
- 2. OA 通过结构化澄清揭示歧义、冲突与风险；
- 3. OA 把已经确认的决定整理成 Intent Contract；
- 4. IO 审阅完整契约并显式签署，系统再把任务所需上下文、约束和 AC 注入 AS。

以“超过 5000 元的报销增加总监审批”为例，注入 Agent 的内容还应包括金额口径、边界值、适用组织、非目标、异常路径、权限限制以及可以证明完成的 AC。这样，Agent 面对的是一项获得授权的工程任务，而不是一句需要自行猜测的愿望。

意图注入的核心价值是建立“探索—承诺”的分界线。契约签署前可以充分讨论；签署后，AS 不能静默改变目标。如果执行中出现新事实，需要判断它属于实现细节，还是已经改变契约。后者必须暂停并重新签署。

意图注入常在三个地方失效：把全部会议纪要塞进上下文，导致噪声大于信息；把 IO 的第一句话直接视为最终意图，跳过校准；或者由 OA、Agent 自动写上“已签署”，让授权变成形式。进入执行的信息必须经过筛选、确认和授权，数量并非越多越好。

机制二：高频对抗与自净化——让偏差在小范围内尽早暴露

高频对抗用测试、规则、独立检查和反例不断挑战当前结果，与多个 Agent 互相争论无关。传统交付常在开发末尾才集中测试；Agent 的执行速度更快，如果反馈仍然滞后，错误会在几分钟内跨越多个文件和模块。把反馈嵌入执行过程，才能在偏差尚未扩散时发现它。

一次最小对抗循环包括：

提出候选结果

- 用 AC、测试和约束挑战
- 记录失败类型与原始证据
- 判断是否在授权范围内可修复
- 最小修复并重跑原检查
- 收敛或停止升级

例如测试发现 5000.00 元也进入总监审批。系统先保存输入、期望、实际结果和规则版本，再判断这是普通实现错误、规格冲突、环境问题还是约束违规。如果只是比较符号写错，AS 可以在限定次数内修复；如果财务制度与已签署规则冲突，AS 就没有权力“选择更合理的一边”，必须升级。

自净化必须有终点。每次循环都需要预算、最大次数、允许修改范围和停止条件。连续三次出现不同失败、修复需要降低测试强度、必须扩大权限，或错误根因无法解释时，都应停止。无限自修复只会掩盖系统没有收敛的事实，并消耗更多时间与 Token。

高频对抗的结果不只是一份全绿报告。失败分类、修复次数、被触发的约束、人工介入点和最终残余风险，首先应进入当前任务的 Evidence Bundle，支持本次裁决；其中具有跨任务分析价值的运行事实，再按 Measurement Contract 汇入 Telemetry。这样组织才能判断 Agent 是一次做对，还是经过大量试错勉强通过。

机制三：人类异常裁决与证据包验收——把机器不该决定的事情交还责任主体

HITL（Human in the Loop）常被翻译为“人在回路中”。很多团队据此在每一步增加人工审批，最后既没有自治，也没有效率。另一种极端是只在线上让人点一下同意，使人类成为无法真正审查的橡皮图章。

Agentic Agile 中的人类介入按任务性质触发，而不是按固定流程节点机械出现。至少四类事项应交给人类：

- **不可逆操作**：删除生产数据、对外发布、资金支付、扩大生产权限；
- **价值判断**：范围取舍、风险偏好、用户体验与成本冲突；
- **不确定验证**：证据互相矛盾、语义依赖业务背景、自动测试无法覆盖；
- **例外授权**：临时放宽约束、接受已知风险、改变签署目标。

异常升级时，系统不应只抛出一句“请决定”。OA 要把问题整理为可裁决包：发生了什么、违反了哪条规则、有哪些原始证据、继续或停止各有什么影响、推荐选项和理由是什么。人类作出决定后，决定本身也要进入契约、例外记录或证据包，形成可追溯责任链。

有效的 HITL 依靠的不是更多人，而是**正确的人在充分信息的基础上，对机器不应决定的事情负责**。人类若经常被迫处理可自动化的重复问题，说明 Harness 不成熟；重大价值和不可逆操作若从不触发人类，说明自治已经越界。

三大机制如何连成一条运行链

意图注入提供目标与授权，高频对抗持续检查执行是否偏离，人类异常裁决处理机器边界之外的决定。三者不是三个孤立功能，而是一条从“可以开始”到“可以继续”再到“必须停下”的控制链：



读者可以用一个问题检查自己的流程：**Agent 在什么时候获得执行授权，什么失败允许它自行修复，出现什么信号时它必须停下？**如果三个答案都不清楚，就还没有形成自治机制。

8.2 从三大自治机制到 Harness 工程承载

三大自治机制规定运行逻辑：意图注入让执行从已签署的目标和边界开始；高频对抗与自净化让偏差在有限范围内暴露、修复或停止；人类异常裁决处理价值取舍、例外与不可逆行动。

这些逻辑如果只写在制度、培训或 Prompt 里，仍可能被错误上下文、过大权限、状态丢失或可跳过的门禁破坏。广义 Harness 为它们提供可运行的工程环境：意图注入依赖可信上下文、状态和约束；高频对抗依赖执行编排、评估、预算和停止条件；人类裁决依赖工具权限、证据、状态与恢复。三种机制共同使用这些能力，并非各自对应一套独立模块。

四类工程方法提供建设路径；SCOPE-V 则将这些能力组织为一次任务中的控制回路。

8.3 Harness 的六大能力支柱

Harness 原意是“挽具”或“安全带”：它不替代动力和驾驶者，却把力量约束在可控制的方向上。在 AI Coding 中，广义 Harness 是包裹 Agent 的完整工程控制环境，可以由现有仓库、CI/CD、策略、脚本和平台共同构成。六大支柱分别解决信息、能力、顺序、连续性、判断和安全恢复问题，不需要分别采购六套平台。

支柱一：上下文管理——让 Agent 看到“足够且正确”的信息

模型并不会因为读到更多文件就必然理解得更好。无关材料会挤占注意力，过期文档会提供错误规则，敏感信息会扩大泄露风险。上下文管理因此不是简单扩充上下文窗口，而是持续回答：当前任务必须知道什么、可以不知道什么、哪些信息不能看到、信息是否仍然有效。

3-4-3 使用分层注入：意图图谱提供全局导航，约束和编码规范提供跨任务边界，当前契约和代码切片提供任务上下文。费用报销 Agent 需要看到金额规则、审批接口和相关测试，不需要读取整套薪酬数据、所有历史会议记录和生产凭证。

上下文支柱失效时，常见表现包括 Agent 反复询问已存在信息、引用旧接口、修改无关代码、不同 Agent 对同一术语理解不一致。改进它时，重点不是继续添加文档，而是建立来源、新鲜度、裁剪规则和任务完成后的回写。

支柱二：工具系统——区分“知道什么”与“能够做什么”

Agent 知道数据库结构，不等于它应该获得生产数据库写权限；它知道发布命令，也不等于可以自行发布。Tools Manifest 像一份机器员工的岗位授权书，声明允许调用哪些工具、访问哪些目录和网络域、执行哪些操作，以及哪些动作必须取得人工批准。

权限应遵循最小必要原则，并同时限制“工具”和“作用范围”。只允许 Git 并不够，还要区分只读、创建分支、提交、推送和合并；允许数据库工具也要区分测试环境查询、生产只读与生产写入。工具调用应留下审计记录，越权请求应被阻断而不是仅告警。

工具系统的典型失败，是为了减少中断给 Agent 一个过大的万能权限。短期看执行更顺畅，长期却让一次 Prompt 注入、错误路径或模型误判获得真实破坏能力。

支柱三：执行编排——把复杂任务从聊天顺序变成工作关系

一个复杂任务往往包含需求澄清、测试、实现、接口适配、安全检查和证据汇总。仅靠 Agent 在对话中自行决定下一步，依赖关系很容易丢失。Work Graph 用 DAG（有向无环图）表达“什么必须先发生、什么可以并行、什么结果是下一步的输入”。

DAG 可以先理解成一张带箭头的任务图：箭头表示依赖，节点表示可验证工作。测试节点完成可信 Red 后，实现节点才能开始；实现和回退准备可以在边界清晰时并行；所有检查结束后，证据汇总节点才有资格运行。

成熟编排不仅描述成功路径，还描述超时、失败、重试、回滚和升级。它追求的不是让最多 Agent 同时工作，而是减少等待、冲突和无效交接，让系统始终知道“现在在哪、为什么停、接下来由谁做”。

支柱四：状态与记忆——让跨步骤、跨会话的执行保持连续

模型会话结束后，临时推理不会自然变成组织知识。状态用于记录当前任务进行到哪里，记忆用于保存经过验证、未来可能复用的事实与教训。两者混在一起，会把临时猜测永久化；两者都不保存，则每次执行都从头开始。

Loop Memory 可以记录节点状态、关键决定、失败与恢复；Intent Graph 保存稳定的业务关系和经过验证的知识；反思日志记录某次任务值得复用的模式。每条记忆都应有来源、适用范围、版本、责任人和失效条件。

例如“外币报销必须先折算为本位币”来自一次真实缺陷，可以进入业务规则；“某接口可能很慢”只是一次观察，需要更多证据，不应直接升级为全局事实。记忆支柱的质量，不看保存了多少，而看下次是否能在正确场景被正确使用。

支柱五：评估与观测——既判断结果，也看见过程

评估回答“做得对不对”，观测回答“系统正在怎样做”。只有最终测试结果，团队看不到 Agent 是否越权、重试多少次、花费多少 Token、在哪个环节偏离；只有过程日志，又无法证明业务目标完成。

因此这一支柱同时包含 AC 验证、测试、证据审计、图谱—契约—约束一致性、NFR 检查和遥测。它要把目标命中、质量、成本、异常、人类介入与学习联系起来。

“测试全绿”只是一个信号，不是完整结论。若测试没有覆盖关键 AC，或者测试与错误实现来自同一误解，全绿仍然可能无效。评估必须能追溯到签署意图，观测必须保留真实来源，二者共同支撑发布裁决。

支柱六：约束与恢复——给高速执行装上护栏、刹车和逃生通道

约束说明 Agent 不能做什么、做到什么程度必须停止；恢复说明失败以后怎样回到安全状态。只有约束没有恢复，系统一出错就只能人工救火；只有恢复没有约束，错误会反复发生并不断扩大影响。

可执行约束可以限制文件范围、依赖、权限、性能、安全、Token 预算和发布条件。恢复策略则定义自动修复次数、功能降级、事务回滚、数据补偿、人工接管和事故记录。

以生产发布为例，约束可以规定 Agent 无权直接持有生产凭证；恢复要求功能开关已经验证、回退命令可用、监控阈值和负责人明确。安全不是“希望不出问题”，而是即使出问题，影响也能被限制并且系统能恢复。

六大支柱的木桶效应

六大支柱并非各自打分后求平均。某一根明显缺失，就可能成为整个自治系统的失效点：上下文正确但工具权限过大，Agent 会准确地做出未经授权的操作；约束严格但不可观测，团队不知道门禁是否真正运行；证据充分但没有恢复，发布后仍无法控制事故。

因此建设 Harness 时应先寻找“最危险的短板”，而不是平均铺开六个平台。读者可以逐项回答：信息是否可信、权限是否最小、依赖是否显式、状态是否连续、结果是否可证、失败是否可恢复。回答不出的地方，就是下一项建设重点。

8.4 四类工程方法：实现手段，不是第四套架构

四类工程方法是把 Harness 能力装配起来的实现手段，分别处理认知输入、协作拓扑、反馈收敛和行动边界。

工程方法	解决的问题	主要产物	本书展开位置
Context Engineering	Agent 此刻应看到什么，哪些信息可信且有权限	上下文切片、来源/版本、隔离检查	第九至第十二章；第二十五、二十六章
Graph Engineering	谁在何时依赖谁完成什么，哪些工作可以并行	Org Graph、Work Graph、节点契约	第十二章、第二十七章
Loop Engineering	偏差如何分类、修复、复验并停止	失败分类、重试预算、反馈记录	第十四章及第十五章
Harness Engineering（狭义）	哪些动作获准，怎样阻断、证明和恢复	Policy as Code、Tools Manifest、Gate、Recovery	本章六大支柱与第十七章

层级关系可以概括为：**3-4-3 是架构**，三大自治机制是运行原则，**Harness 是执行环境**，**SCOPE-V 是任务控制回路**，**四类工程方法是建设手段**。四类工程不是新的自治机制。

8.5 轻量模式与完整模式：按风险裁剪

不是所有项目都需要相同治理重量。

单人单模块项目也应保留四大动态工件的语义，但不必部署复杂平台：用一份最小 Intent Graph 或项目上下文索引提供方向，以意图契约、约束矩阵和 Evidence Bundle 管理当前任务。多人、多工具、多模块项目再增加 Protocol、模块治理、跨模块契约、集成 DAG 和证据聚合。

治理复杂度应与以下因素匹配：

- 变更影响范围；
- 数据敏感度；
- 操作可逆性；
- 业务关键性；
- 参与者和模块数量；

- 自动化验证能力；
- 当前团队成熟度。

8.6 风险驱动的最小建设路径：先闭环，再扩大自治

团队不应同时建设六大支柱的所有高级能力。推荐顺序：

契约与核心约束

- 可执行 AC 和测试
- Evidence Bundle
- 上下文与工具权限
- Work Graph 和多 Agent
- 遥测与组织级价值效能视图

先建立完成闭环，再扩大自治范围。没有证据基础时引入更多 Agent，只会增加不可见复杂度。

上面的顺序是通用起点，不是所有团队必须照搬的成熟度阶梯。实际优先级应由能力缺口和风险决定：

不同团队的第一短板并不相同。可以用四个问题判断优先级：

1. 团队最常在哪个阶段发现错误？越晚发现，越应优先前移相应门禁。
2. 哪类错误代价最高？高代价风险优先于高频但低影响的摩擦。
3. 哪项证据最难获得？缺失证据往往揭示系统不可观测区。
4. 哪个环节最依赖某个“知道所有事情的人”？这通常是尚未资产化的隐性治理能力。

例如，一个需求经常反复变更的团队，应先强化 Specify 和签署，而不是急于增加 Agent 并发；一个发布后事故多、测试却常常全绿的团队，应优先检查 AC、测试独立性与 Evidence Bundle；一个多人协作频繁冲突的团队，应先建设 Work Graph、所有权和 Handoff 协议。

8.7 常见概念混淆与机制反模式

实施中更常见的问题不是工具不足，而是概念层级混用：

- **六大支柱不是六套平台。** 它们是六种能力域，可以由同一仓库、现有 CI/CD、YAML 与少量脚本共同实现。
- **四类工程不是四根新支柱。** 它们是建设和组合六大能力的方法。
- **狭义 Harness Engineering 不等于广义 Harness。** 前者重点建设约束、工具、门禁与恢复，后者是六大支柱形成的完整环境。
- **证据和遥测不是第二套支柱。** 它们主要属于评估与观测能力，并贯穿其他能力域。
- **Graph 不是额外支柱。** Work Graph 是执行编排的核心工程载体，Intent Graph 则服务于意图与系统对齐。
- **SCOPE-V 不是第五套架构。** 它是三大机制、六大能力和四类工程在任务运行时的控制回路。

常见实施反模式包括：

- 把意图注入理解为复制一段超长 Prompt；
- 允许 Agent 无限自我修复；
- 失败只记录最终结果，不保留原始信号与修复过程；
- HITL 没有预设触发条件，完全依赖人临时观察；
- 六大支柱各自建设，彼此没有任务 ID 和证据映射；
- 只有治理文档，没有机器门禁、权限隔离和恢复演练；
- 为展示多 Agent 而增加不必要节点，导致成本和协调债上升；
- 单人小项目照搬多模块企业治理，使治理成本高于任务风险。

识别这些反模式的共同问题：系统讲得出流程，却无法用真实工件和机器输出证明流程确实运行。

8.8 管理者如何判断 Harness 是否有效

管理者不必审阅每条 Prompt，却应要求系统回答五个问题：

- 本次任务的价值目标和非目标是什么？
- 哪些风险被机器门禁阻断，哪些仍由人承担？
- 当前绿色状态由哪些独立证据支持？
- 发生异常时是否可停止、可恢复、可追责？
- 这次执行给下一次留下了什么可复用资产？

如果系统只能展示生成代码量、调用次数和节省工时，就还没有证明治理有效。Harness 的价值不是让 Agent 看起来更忙，而是让团队能够在更大自治范围内保持判断力。

8.9 本章小结：自治必须被环境承载

三大自治机制规定 Agent 怎样获得授权、怎样接受挑战、何时把决定交还人类；六大 Harness 支柱说明这些机制需要什么能力支撑。Context、Graph、Loop 与狭义 Harness Engineering 是建设这些能力的工程方法。

本章最小实践：选择一个真实任务，写出三大机制的触发条件，并找出六大支柱中最危险的一项短板。下一章再把这些原则放入 SCOPE-V 的任务控制回路。

第三部分 | 验证工程与 3-4-3 落地：让操作系统真正运行

如果说 3-4-3 定义了治理系统的角色、工件和运行机制，那么 SCOPE-V 解决的是这些要素怎样在一次真实任务中持续运转。它把“目标要明确、风险要受控、结果要验证”从原则变成控制回路，使 Agent 的每一次行动都能找到方向、边界、执行关系、证明方式、修复路径和最终裁决依据。

SCOPE-V 由六个持续运行的控制面组成：Specify 控制方向，Constrain 控制可行动空间，Orchestrate 控制协作关系，Prove 证明实现满足契约，Evolve 根据失败诊断、修复和重证，Verify 判断证据支持什么行动。六者会随任务状态反复运行，而不是按顺序走完便结束。Verify 之后，Telemetry 构成组织级慢外环；流水线则把这些能力承载到交付系统。

本部分聚焦一次任务的核心运行闭环：规格、约束、编排、证明、演进、裁决、长期反馈和交付承载。新项目与既有系统、知识与记忆工程、多人多工具多模块协作，则在第五部分进一步展开。

读完本部分，你应当能够为一个任务建立可签署契约、可执行约束、可失败的验证、可收敛的修复回路、可审理的 Evidence Bundle 和可接入流水线的发布路径；也能判断测试全绿、扫描通过或模型自评良好究竟证明了什么，还缺少什么。

本部分阅读路径

1. **第九章 | SCOPE-V**：先理解整条控制回路，以及快内环、独立裁决与慢外环各自承担什么。
2. **第十至十二章 | Specify、Constrain、Orchestrate**：把任务变成已签署的意图、可执行的边界和可调度的工作图。
3. **第十三至十五章 | Prove、Evolve、Verify**：用可信 Red、有限修复和独立证据裁决，回答“是否真的可以采取行动”。
4. **第十六章 | 价值与效能遥测**：从单项任务的事实，观察长期的有效交付、成本、能力与学习。
5. **第十七章 | AI 原生交付流水线**：把契约、证据、不可变制品、灰度与回滚连成可追溯的交付谱系。
6. **第十八章 | 端到端案例**：把前面所有控制面放进一项真实规则变更，完整走完从需求到发布后的反馈闭环。

读完这一部分，你应能为一项真实任务设计从签署到发布、从失败到回流的可信执行闭环；第五部分再讨论既有系统、组织记忆与规模化协作如何支撑这种闭环。

第九章 | SCOPE-V：人机协同研发的持续控制回路

三大理论解释了 3-4-3 为什么需要外部化认知、建立反馈并治理复杂协作；三大自治机制说明系统怎样获得意图、纠正偏差，并把机器不该决定的事情交还人类。要让这些原则进入日常研发，还需要一套持续运行的控制架构。SCOPE-V 就是这套架构。

9.1 六个持续运行的控制面

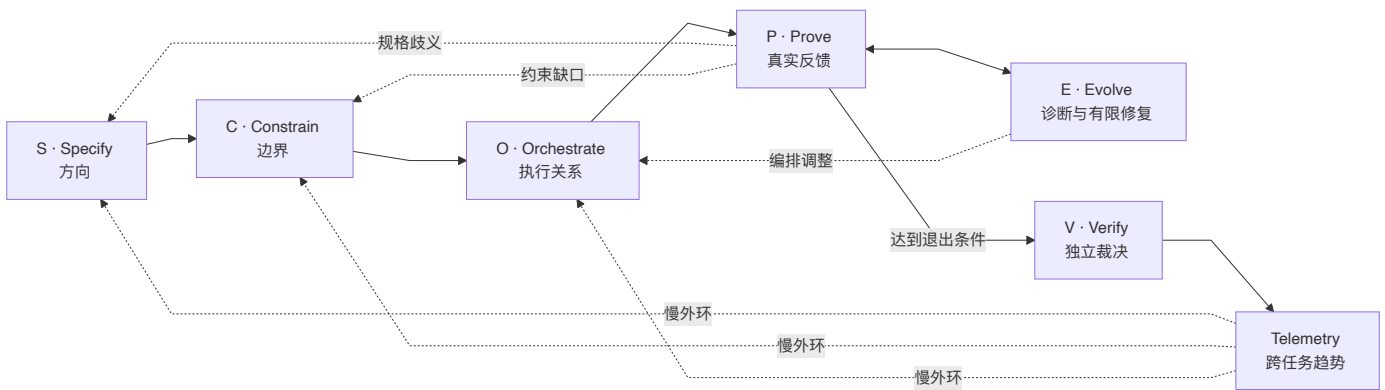
SCOPE-V 的名称看起来像六个依次发生的阶段：Specify、Constrain、Orchestrate、Prove、Evolve、Verify。实际运行中，它们是六个贯穿任务生命周期的控制面：

- **Specify** 持续回答目标是否仍然明确、规格是否发生漂移；
- **Constrain** 持续检查权限、数据、质量和过程边界；
- **Orchestrate** 根据依赖、失败和资源状态动态调整执行关系；
- **Prove** 不断用测试与可执行证据挑战候选结果；
- **Evolve** 对失败进行诊断、修复、重构与知识回写；
- **Verify** 保持独立视角，决定现有证据支持哪一种行动。

Telemetry 汇聚任务执行中产生并经 Verify 确认的信号，在跨任务尺度形成慢速外环，把趋势反馈到下一轮 S、C、O。于是 SCOPE-V 的真实结构不是：



而更接近：



在整个过程中，S、C、O 的结果仍然约束 P 与 E；P 暴露的失败可能要求回到 S 重新澄清、回到 C 增加边界，或回到 O 调整 Work Graph。它是一个可回流、可停止、可重新签署的控制系统，而不是把所有变化都塞进当前步骤继续向前。

9.2 SCOPE-V 与传统 SDLC 的分工

SDLC（Software Development Life Cycle，软件开发生命周期）回答的是软件从构想到运行通常经历哪些生命周期活动。不同组织可以采用瀑布、迭代、敏捷、DevOps 或持续交付，但仍然需要需求、设计、实现、测试、部署和运维。

SCOPE-V 不否定这些活动，也不试图再发明一套项目管理生命周期。它解决的是另一个问题：当执行主体中出现能够高速生成、调用工具、并行协作和自主修复的 Agent 时，怎样在运行时持续控制它的方向、权限、反馈和责任。

比较视角	传统 SDLC 主要关注	SCOPE-V 额外关注
核心对象	软件产品的生命周期活动	Agent 行为及其治理状态
执行主体	以人类团队和确定性工具为主	人类责任主体 + 概率性 Agent
需求入口	文档、Backlog、评审和沟通	可签署意图契约与规格漂移控制
流程推进	阶段、迭代或流水线状态	证据、门禁、停止和回流条件
质量反馈	测试、评审、CI/CD、运行监控	P⇌E 快内环、独立 V、证据映射
权限治理	角色权限和流程审批	任务级工具能力、最小权限、越权阻断
完成判断	阶段完成、故事完成、发布成功	契约、约束与 Evidence 支持单任务裁决；Telemetry 支持跨任务调整
学习方式	复盘、事故分析、流程改进	Telemetry 外环回写图谱、约束与编排

二者可以叠加。一个采用 Scrum 的团队，可以在每条高风险用户故事内部运行 SCOPE-V；一个 DevOps 团队，可以把五道门嵌入现有 CI/CD；一个传统阶段式项目，也可以用意图契约、约束矩阵和 Evidence Bundle 治理 Agent 执行。

可以把两者的分工概括为：**SDLC 描述软件怎样走过生命周期，SCOPE-V 描述智能体在每次行动中怎样被持续控制**。前者给出交付路线，后者负责运行中的方向、边界、反馈、停止和恢复。

9.3 为什么人机协作时代需要这样的控制回路

人类开发者与 Agent 的差异，不只是速度。Agent 还具有四个会改变治理设计的特征。

第一，**执行速度快于人类审查速度**。一个人在准备评审时，Agent 可能已经修改几十个文件。如果反馈仍然只发生在阶段末尾，方向性错误会快速扩散。

第二，**输出具有概率性**。同一个 Prompt 在不同上下文、模型或时间可能产生不同结果。仅靠“流程执行过”不能证明行为正确，必须要求真实测试、约束结果和可复查证据。

第三，**Agent 能够行动，而不只是建议**。当它能读写仓库、调用数据库、访问网络和执行发布命令时，错误不再停留在文本层面。Constrain 与 Tools Manifest 必须成为运行时边界。

第四，**多 Agent 会放大协调复杂度**。每个 Agent 都可能局部合理，但 Handoff、共享文件、上下文版本和目标理解可能互相冲突。Orchestrate 必须把协作关系变成可执行 Graph。

于是传统“人先理解—人再编码—人最后评审”的隐性控制，必须转变为显式的机器可执行控制：目标写进契约，风险写进约束，协作写进图，成功写进测试，判断依据写进证据，系统表现写进遥测。

9.4 六个控制面分别控制什么

9.4.1 S：Specification——控制方向

S 不只是“写需求”。它把模糊意图转成 Goal、Non-goals、Rules 和可执行 AC，并由 IO 显式签署。它控制的是系统朝哪个目标前进，以及什么变化已经构成规格漂移。

S 的典型工件是 Intent Contract、Example Mapping 和签署记录。没有这些工件，Agent 很容易把讨论、建议或历史规则误当成当前授权。

9.4.2 C：Constraints——控制可行动空间

C 把法律、安全、数据、架构、质量、工具和过程边界转成可执行规则。它不告诉 Agent 如何实现，而是限制哪些实现路径不可接受。

风险只有转换为规则、测试、权限或恢复动作，才真正进入控制系统。“注意隐私”不是控制；“Agent 不得读取生产报销明细，触发即阻断并升级”才是控制。

9.4.3 O：Orchestration——控制执行关系

O 用 Work Graph 定义节点、依赖、并行、超时、重试、Handoff 和停止条件。它控制的不只是先后顺序，还包括失败后返回哪里、哪个结果可以被下游信任、什么时候需要人类。

传统任务列表告诉团队“要做什么”；Orchestration 还要让运行时知道“当前谁能做、拿什么输入做、做到什么程度算完成”。

9.4.4 P：Proof——控制反馈真实性

P 的目标不是生成“看起来正确”的实现，而是让候选结果接受挑战。AC、TDD Red、运行时断言、边界测试、安全检查和性能基准都属于证明机制。

证据先于结论。单一 LLM-as-Judge 或 Agent 自述不能作为成功证明，因为同一个误解可能同时影响实现和评价。关键结论需要可复现运行证据，必要时还需要独立来源。

9.4.5 E：Evolution——控制修复与学习

E 接收 P 暴露的失败，判断它属于实现错误、规格冲突、约束缺失、编排问题还是证据不足。只有在授权边界内，系统才允许进行有限修复；超出范围则回流到 S、C、O 或触发 HITL。

E 不只是把测试改绿，还要保留失败、修复和重验记录。尚未裁决的发现先进入 Evidence Bundle 与 Loop Memory，经过 V 确认且具有复用价值后，才能晋升为图谱、约束或测试资产，避免把一次猜测写成组织事实。

9.4.6 V：Verification——控制最终裁决

V 站在执行与自愈内环之外，独立检查契约、约束、实现、证据和时效是否一致。它不生产用于证明自己的证据，而是审查证据并决定：批准、条件批准、补证、缩小范围、拒绝或升级。

V 的独立性非常重要。如果负责修改结果的同一 Agent 同时决定证据是否充分，就可能出现自我证明。Verify 可以使用自动化工具，但价值取舍、不可逆操作和残余风险仍由人类责任主体裁决。

9.5 为什么 E 必须在 V 之前

Evolve 位于 Verify 之前，是为了把受控修复与最终裁决分开。

因为 P 已经承担了执行内环中的失败发现。P 暴露测试失败、约束拦截或证据缺口后，E 负责在当前授权范围内诊断和修复，再回到 P 重新取证。这个 $P \rightleftharpoons E$ 回路可以快速运行多次：

P：边界测试失败

- E：判断为实现错误，完成最小修复
- P：重跑同一测试并取得新证据
- E：检查是否需要反思与知识回写
- 满足退出条件后提交 V

V 的职责不是参与每一次试错，而是在内环达到退出条件后进行独立审查。如果把 V 放在 E 前面并让其参与修复，裁判就进入了比赛：验证者既发现问题、又修改结果、再宣布通过，独立性会被削弱。

E 在 V 之前还带来三个好处：

- 1. **减少人类审查噪声**：可在边界内自动修复的普通错误先被收敛；
- 2. **保留失败历史**：V 看到的不只是最终绿灯，还包括系统怎样从红到绿；
- 3. **明确退出条件**：只有测试、约束、证据和修复记录达到要求，才进入裁决。

但 E 不能无限运行。超过重试、Token 或时间预算，必须修改契约、降低约束，或出现证据矛盾和不可逆风险时，内环立即停止并交给人类。

9.6 快内环：P ⇌ E 的诊断与自愈

快内环针对当前契约、当前上下文和当前执行结果运行。它的时间尺度可以是秒到分钟，目标是让局部偏差尽早收敛。

快内环必须具备：

- 明确失败信号，而不是模型觉得“不太好”；
- 失败分类与路由；
- 最小修复范围；
- 原检查重跑；
- 最大轮次与资源预算；
- 失败和修复证据；
- 超界后的 HITL 条件。

费用报销任务中，5000.00 元边界测试失败。E 判断为比较符号错误，修改一行实现，再由 P 重跑相同边界测试。如果 E 发现财务制度与签署契约冲突，它不能选择一边修复，而应停止内环，返回 S 请求 IO 重新决策。

9.7 独立裁决：V 的审计与放行

当快内环达到退出条件，V 执行三角验证：

Intent Contract：做的是不是被批准的事？

Constraint Matrix：执行是否守住边界？

Evidence Bundle：结论是否由真实、当前、足够独立的证据支持？

V 还检查工件版本、代码提交、UNKNOWN 项、回退准备和人工批准。它可以得出“测试通过但暂不发布”，也可以给出带灰度、时间和监控条件的批准。

9.8 慢外环：Telemetry → S/C/O 的组织学习

V 之后的 Telemetry 形成慢速外环。它不是修复当前一行代码，而是观察多个契约后的累积趋势：Token 是否持续上升、首次成功率是否下降、HITL 是否集中在某类问题、约束自愈是否改善、知识沉淀是否减少重复失败。

这些趋势反过来改变下一轮控制面：

- 目标准确率下降，回到 S 检查意图和契约质量；
- MUST 失败增加，回到 C 检查约束表达与工具权限；
- Token、轮次和 Handoff 摩擦上升，回到 O 调整上下文和编排；
- 重复出现同类失败，回到 E 改进诊断与修复策略，并把经验证的模式更新到图谱和组织知识。

快内环保证当前任务收敛，慢外环保证整个系统不会在长期中退化。前者像即时纠偏，后者像根据连续航行数据改进航线、规则与训练。

9.9 每个控制面都必须产生可检查工件

控制面不能只停留在概念层。每一面至少要有可读取、可版本化、可审计的产物：

控制面	关键工件或证据
S	Intent Contract、Example Mapping、IO 签署
C	Constraint Matrix、constraints.yaml、Tools Manifest
O	Work Graph、节点契约、Handoff 记录
P	TDD Red/Green、AC 运行结果、NFR 检查、Evidence Bundle 原始证据
E	失败分类、修复与重验记录、Loop Memory、Evidence Bundle 更新
V	三方一致性审计、Evidence Bundle 裁决状态、发布或退回决定
Telemetry	单任务事件与度量、聚合 Dashboard、跨任务趋势

这就是“每节必产工件”的含义：没有工件，控制状态无法被机器检查；没有原始证据，人类只能相信 Agent 的叙述。

9.10 五道机械门把控制状态变成不可跳过的事实

五道机械门进一步防止系统跳过关键状态。它们是状态转换的机械裁决点，不是把 SCOPE-V 重新切成五个线性阶段：

前置门：契约、IO 签署、约束和 AC 是否存在？
编码门：测试是否先写并产生可信 Red？
验证门：测试、AC、构建和证据是否通过？
收尾门：Evidence Bundle 是否可裁决，本次真实信号是否已进入单任务遥测与 Dashboard，合格知识是否回写？
Bug 回溯门：完成后发现问题，是否如实修正证据和组织记忆？

Agent 不能仅凭自然语言声明“我已经完成前置检查”。门禁需要读取真实工件、时间、版本和运行结果。

收尾门检查遥测入口是否完整，不负责人一次任务解释长期趋势。组织层结论必须等待多个可比任务按 Measurement Contract 聚合后，才能进入 Telemetry 慢外环。

9.11 一个完整控制回路示例

仍以“超过 5000 元的报销增加总监审批”为例：

1. S 把“超过”、金额口径、适用范围和异常路径写成签署契约；

2. C 禁止访问生产明细、禁止修改历史已审批单，并规定回退开关；

3. O 安排边界测试先于实现，组织关系检查与回退准备可并行；

4. P 先证明 5000.00 与 5000.01 的测试在旧实现上失败；

5. E 完成最小实现，处理代理总监冲突，并把候选规则及来源写入 Evidence Bundle；

6. P 重跑原测试和约束，生成当前提交的证据；

7. V 独立检查契约、约束、证据、UNKNOWN 与回退，批准两个部门灰度，并将已确认规则晋升为组织知识；

8. Telemetry 记录首次成功、修复轮次、HITL、Token 和知识沉淀；

9. 多个契约后发现代理关系类 HITL 持续偏高，外环驱动 S 增加前置问题、C 增加组织关系约束、O 提前安排领域检查。

这个例子说明 SCOPE-V 的完成点不是“代码写完”，而是当前任务已经由证据支持裁决，并且执行经验已经进入下一轮控制。

9.12 SCOPE-V 设计的四条底线

SCOPE-V 的控制论立场可以归结为四条底线：

1. 证据先于结论：不能以“看起来正确”或单一 LLM Judge 作为成功证明；

2. 风险转化为控制：热点名词必须落为规则、测试、权限、门禁或恢复动作；

3. **价值判断不可让渡**：AI 可以有限自治，但目标、公平、伤害取舍和不可逆操作由人负责；
4. **每个控制面必须可检查**：不堆砌概念，每一面都要产生工件、矩阵、模型、策略或证据。

这六个英文单词只是索引。SCOPE-V 真正提供的，是一套让概率性、高速度、可行动的 Agent 持续校准、受控修复、独立裁决并长期学习的运行模型。

9.13 与三大自治机制的关系

三大自治机制不是另一套流程，而是贯穿 SCOPE-V 的三条运行原则：意图注入主要落在 S、C、O，使任务在签署契约与可治理上下文中开始；高频对抗与自净化主要运行在 $P \rightleftharpoons E$ ，用原判据重验并在超出预算、范围或授权时停止；人类异常裁决贯穿全程，并在 V 集中处理价值取舍、约束例外、不可逆操作和残余风险。

分工保持简单：**3-4-3 规定责任和工件，三大机制规定自治原则，SCOPE-V 组织一次任务的控制回路，Harness 把这些控制执行出来**。Telemetry 是跨任务的慢外环，流水线则承载制品、门禁和发布，不再另设一套概念映射。

9.14 本章小结

SCOPE-V 为概率性 Agent 提供运行时控制，并与 Scrum、DevOps 或其他 SDLC 实践共同工作。S、C、O 建立方向、边界和执行关系， $P \rightleftharpoons E$ 用真实反馈形成快速纠偏内环，V 保持独立裁决，Telemetry 再把多次任务的运行信号反馈给下一轮治理。三大自治机制由此进入一个可执行、可停止、可审计和可进化的控制系统。

本章最小实践：选择一项真实 AI Coding 任务，画出它的 SCOPE-V 控制回路，并标出三大自治机制、 $P \rightleftharpoons E$ 快内环、HITL 异常出口、五道门和 Telemetry 慢外环。

完成证据：一张带工件、门禁、回流与责任人的 SCOPE-V 运行图。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，完整跑一遍 SCOPE-V，并观察每个状态如何产生下一步证据。

第十章 | Specify：从模糊需求到可签署意图

10.1 需求本身就是第一类治理对象

很多 AI 项目从收到需求后才开始治理：规定编码规范、限制工具权限、要求测试覆盖。但如果需求本身模糊、冲突或不可验证，后续执行得越严格，越可能精确地产生错误结果。

Specify 要发现字面需求背后的真实意图、假设和风险，再把探索性对话转化为组织愿意承担责任的执行授权。它关心的不是措辞是否更正式，而是人机系统这一次究竟被允许改变什么。

常见可疑信号包括：

- 目标只有动作，没有业务结果；
- 成功标准使用“优化一下”“更智能”等不可判定词语；
- 需求同时包含过多激励，模型可能通过钻空子刷指标；
- 范围看似很小，却要求修改高风险核心模块；
- 权限要求远大于任务所需；
- 多个业务规则互相冲突；
- 要求跳过验证、审计或人工批准；
- 所谓“完成”无法被独立验证。

OA 的职责是先校准，再进入执行。IO 的第一句话可以启动讨论，却不能直接成为执行规格。

10.2 批判性校准：只追问会改变结果的问题

批判性校准围绕四类会改变结果的风险展开：

检查	要问的核心问题
目标	这项变化要带来什么业务结果，而不只是做什么动作？
边界	哪些对象、场景、数据和操作明确不在范围内？
判定	什么输入、输出和边界例子可以独立证明完成？
责任	谁承担价值取舍、例外授权和不可逆后果？

结构化追问（Grill-Me）可以把这些风险转成一组可选择的决定。它不是一份长问卷：每轮只问一个会显著改变价值、范围、风险、验收或权限的问题，并给出推荐默认、理由和备选项。推荐答案用于降低 IO 的分析负担，最终选择仍由 IO 作出。

例如，面对“报销超过 5000 元自动交总监审批，明天上线”，先只追问三个关键点：

1. “超过”是否意味着 5000.00 不触发、5000.01 触发？
2. 金额按单张发票、报销单总额，还是含税本位币计算？
3. 找不到审批人时拒绝、转人工，还是暂缓发布？

每个答案都必须落入 Goal、Non-goals、Rules、AC、约束或例外记录。技术细节如果不改变接口、风险和验收，不应打断 IO；如果改变不可逆行动或责任承担，就不能由 Agent 自行补全。

这些回答仍然只是局部决定。OA 必须把它们汇总为完整契约，由 IO 审阅并显式签署；逐条回答、OA 总结或工具生成 `SIGNED` 都不能代替签署。

最小协议：

发现风险 → 给出推荐与理由 → IO 选择或改写
→ 写入契约/约束/AC → 汇总审阅 → IO 签署

有了这些决定，Example Mapping 才能把它们推进为规则、示例、反例和可执行 AC；只有 AC 与约束都达到可审阅状态，任务才具备签署条件。

10.3 从事实与意图图谱裁剪任务上下文

新项目可以从战略材料、业务规则和初始架构建立最小 Intent Graph；既有系统则必须先做只读 Recon，识别 Baseline、Preserve、Unknown、依赖和候选 Change Envelope。代码现状不等于业务意图，但也不能被新需求的语言覆盖。只有把“组织想去哪里”和“系统现在真实是什么”同时放入上下文，Agent 才不会在错误基线上精确执行。

OA 随后从可信事实与意图图谱中选择与当前任务有关的：

- 业务目标；
- 用户角色；
- 现行规则；
- 相关模块；
- 历史事故；
- 长期约束；
- 已知依赖。

裁剪的目标是让当前任务拥有足够信息，同时避免无关信息、敏感信息和旧规则污染上下文。

与第八章的 Harness 定义一致，上下文按四层注入：

L1：任务意图层——目标、用户、价值、签署契约
L2：组织约束层——Constraint Matrix、Tools Manifest、合规边界
L2+：工程惯例层——代码结构、接口约定、测试方式、仓库级 AI Coding Guide
L3：节点切片层——当前 Work Graph 节点所需文件、AC、测试与上游产物

L1 和 L2 保持方向与底线稳定，L2+ 保存团队工程常识，L3 随执行节点变化。不同 AI 工具可以采用不同注入方式，但必须消费同一契约版本、约束语义和来源标识；宿主差异不能改变授权边界。

10.4 意图契约：把决定冻结为执行基准

意图契约不是开工前的汇报文档，而是后续测试、验证和证据的来源。

费用报销案例的最小契约可以是：

```
task_id: T-001
contract_version: 1
status: DRAFT
goal: 将超过 5000 元的境内差旅报销转交部门总监审批
non_goals:
  - 不改变 5000 元及以下审批路径
  - 不处理外币报销
rules:
  - 以报销单含税总额判断
  - 金额严格大于 5000 元时触发
acceptance_criteria:
  - id: AC-01
    description: 5000.01 元进入总监审批
    verify:
      type: command
```

```
command:
  argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_01]
  timeout_seconds: 60
- id: AC-02
description: 5000.00 元保持原审批路径
verify:
  type: command
  command:
    argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_00]
    timeout_seconds: 60
sign_off:
  intent_owner: "<由 IO 本人填写>"
  signed_at: null
```

这仍是一份待签署草案。只有 IO 审阅完整内容、亲自填写签署区并明确确认后，状态才能从 `DRAFT` 进入 `SIGNED`。OA 或宿主工具不得代签，也不得仅凭前面的逐条问答自动改变状态。

契约格式可以是 Markdown、YAML 或由平台管理的结构化记录，但跨工具语义必须稳定：任务 ID、版本、状态、Goal、Non-goals、Rules、AC 和签署主体不能因为换了 AI 工具而丢失或被重新解释。Prompt 可以从契约生成，不能反过来取代契约。

好的 AC 必须可观察、可判定，并尽量使用运行时验证。仅检查某个字符串是否出现在文件中，通常无法证明业务行为正确。

10.5 Example Mapping 与 AC：把规则推进到可签署的验证协议

Example Mapping 可以直译为“示例映射”。它是一种用具体例子澄清抽象规则的方法。读者不需要先掌握复杂的需求工程理论，只需要记住一个直觉：一句规则往往允许多种解释，而一个恰当的例子能够迫使大家暴露各自不同的理解。

例如业务说：“高额报销需要总监审批。”团队如果只讨论这句话，所有人都可能觉得自己听懂了。直到分别写出 5000.00 元、5000.01 元、外币报销和总监缺席的例子，隐藏分歧才会出现。

Example Mapping 的四种基本元素

通常使用四类卡片或条目：

- **Story（目标）**：这次希望实现的业务结果；
- **Rule（规则）**：系统必须遵守的一般判断；
- **Example（示例）**：规则在某个具体输入下应产生什么结果；
- **Question（问题）**：团队尚不能决定、必须向 IO 或领域专家确认的事项。

在 Agentic Agile 中还建议显式增加 **Counterexample（反例）**，用于说明哪些相似情况绝对不应触发该规则。

类型	内容
Story	对高额境内差旅报销增加总监审批，同时不改变普通报销路径
Rule	金额严格大于 5000 元进入总监审批
Example	5000.01 元进入总监审批
Counterexample	5000.00 元不能进入总监审批
Question	外币是否按申请日汇率折算？

怎样开展一次最小 Example Mapping

第一步，写出一个业务目标，不急着手写解决方案。第二步，把 IO 已经明确的规则列出来，每条规则尽量只有一个判断。第三步，为每条规则至少写一个典型示例和一个边界或反例。第四步，把无法从现有信息推出答案的事项转为问题，而不是让 Agent 猜测。第五步，将已经确认的示例转成 AC 和测试候选。

费用报销案例可能经历这样的对话：

规则：超过 5000 元需要总监审批。
示例：5000.01 元人民币差旅报销 → 总监审批。
反例：5000.00 元人民币差旅报销 → 保持原路径。
问题：5000 美元是否直接按数字 5000 判断？
决定：否，先按提交日财务汇率折算为人民币含税总额。
新示例：1000 美元，汇率 7.2 → 7200 元 → 总监审批。
异常问题：提交时缺少汇率怎么办？
决定：进入人工财务队列，不得自动通过。

可以看到，示例不是为了“举例说明已经确定的规则”，而是用来发现规则还没有确定的部分。每出现一个问题，团队要么找到现有制度依据，要么请 IO 决策；不能把问号直接留给执行 Agent。

从示例走向可执行规格

经过确认的示例可以自然转为 AC，再由测试证明。AC 不是需求复述，也不等于某一条测试用例：Goal 给出方向，Rule 给出业务判断，AC 规定可观察的结果，测试则用具体数据和步骤验证该结果。

内容层次	回答的问题	报销示例
Goal	为什么做、要产生什么价值	高额报销进入更高等级审批，降低财务风险
Rule	业务怎样判断	含税总额严格大于 5000 元时需要总监审批
AC	什么可观察结果算满足规则	5000.01 元进入 <code>director</code> ，5000.00 元保持原审批路径
Test	用什么数据、步骤和断言证明	调用提交接口，检查响应、审批轨迹和规则版本

一条关键 AC 要说清前置状态、触发动作、关键输入与边界、预期行为、取证位置和通过口径。Given—When—Then 是常用表达：

Given 一张中国区差旅报销单
And 含税本位币总额为 5000.01 元
When 员工提交报销
Then 审批链应增加成本中心总监节点
And 应记录规则版本 `expense-v2`

这一步把业务语言连接到测试，但不能让测试语法反过来绑架业务讨论。IO 不必会写 Gherkin，OA 负责把业务决定转成可执行形式，再交由 IO 检查语义。每条关键 AC 都应有稳定 ID，能回链到规则，向前映射到测试与 Evidence Bundle；缺少证据的 AC 只能处于“未证明”状态。

什么时候需要更强的规格建模

Example Mapping 擅长澄清规则和边界。如果系统涉及大量状态变化，例如报销单从草稿、提交、审批、退回、撤销到支付，不同状态允许的操作不同，仅靠例子可能遗漏组合。此时可以引入状态机、决策表、不变量或属性测试。

- **决策表**适合多个条件组合决定一个结果；
- **状态机**适合事件驱动的状态转换；
- **不变量**描述任何时候都不能被破坏的性质，如“同一报销单不得重复付款”；
- **属性测试**自动生成大量输入，检查某个性质始终成立。

规格不是越复杂越专业。只有当某种模型确实减少歧义、覆盖重要风险并能驱动验证时，才值得使用。对于简单文案修改，写清 Goal、Non-goals 和两个 AC 可能已经足够。

Example Mapping 的常见误区

- 只有正常示例，没有边界和反例；
- 由 OA 或 Agent 替业务决定所有问号；
- 写了大量例子，却没有归纳它们属于哪条规则；
- 示例确认后没有转入契约、AC 和测试；
- 需求变化后只改测试，不同步更新规则与签署；
- 把工具形式看得比业务讨论更重要。

读者可以拿一条当前需求做练习：写出一条规则、两个示例、一个反例和一个问题。如果完全写不出问题，往往不是需求已经完美，而是团队还没有触碰边界。

10.6 显式签署与前置门：确认 AC 已可执行

完成批判性检查、必要的 Example Mapping 和 AC 覆盖设计后，OA 把所有确认结果组织为完整契约，交给 IO 审阅。签署前不要要求每一条 AC 都已经自动化，但必须已经明确验证协议、边界和责任人；否则后续执行仍会在“怎样才算完成”上重新猜测。只有 IO 本人明确填写签署区或回复“签署”，契约才生效；逐条决策确认、OA 总结或工具生成 `SIGNED` 都不能替代这个动作。

前置门至少机械检查：

- 契约文件存在；
- IO 已真实签署；
- 没有 OA 代签或自动签署；
- 约束矩阵存在；
- AC 可执行性达到要求；
- 文本 grep 类验收没有占据主要比例。

这些检查应由团队接入现有门禁工具，并输出可追溯的检查结果；命令形式取决于项目技术栈，不能用一条示例命令替代真实门禁。

前置门没有通过，不得进入业务编码。

10.7 规格漂移

规格漂移是指执行过程中，团队真正实现和验证的目标逐渐偏离已签署规格。它可能来自合理的新发现，也可能来自为了迁就实现而偷偷改变标准。危险之处在于：最终代码、测试和证据可能彼此一致，却已经不再对应 IO 原本批准的目标。

漂移是怎样发生的

规格漂移很少以“我们决定改变需求”的形式明确出现，更多发生在看似微小的动作里：

- Agent 发现原规则难以实现，于是选择更容易的解释；
- 测试失败后，执行者修改期望值而不是修复实现；
- 新接口不支持某个边界，团队将其悄悄列为“不处理”；
- 为兼容历史行为，把“严格大于”改成“大于等于”；
- IO 在聊天中提出新想法，但没有更新完整契约；
- 多个 Agent 使用了不同版本的契约和 AC；
- 截止时间临近，团队先上线“接近的版本”，却仍沿用原批准结论。

这些动作单独看都可能很合理，累积后却造成“验证通过了另一个需求”。

先区分发现与漂移

执行中出现新事实并不等于发生错误。软件开发本来就包含发现。关键是判断新事实影响哪一层：

新事实类型	示例	处理方式
实现细节	当前模块使用既有规则引擎而不是手写判断	在契约边界内调整，记录技术决定
估算变化	接口适配比预计多半天	更新计划和遥测，不改变业务规格
新风险	旧草稿可能受到新规则影响	暂停相关路径，补充风险与验证
业务规则变化	“超过 5000”改为“5000 及以上”	新契约版本，IO 重新签署
范围变化	增加外币报销和历史数据迁移	重新评估目标、约束、DAG 和证据
权限变化	实现需要生产数据写权限	停止，走权限与人工裁决流程

判断标准不是“代码改动大不大”，而是它是否改变目标、非目标、业务规则、关键 AC、风险承担或授权边界。一行比较符号变化，也可能是重大的规格变化。

规格漂移的治理流程

当 AS 或 OA 怀疑发生漂移时，应执行以下步骤：

1. **停止受影响节点**，避免其他 Agent 继续基于旧假设扩散修改；
2. **保留现场**，记录触发输入、现规格、当前实现、失败证据和新事实来源；
3. **评估影响**，识别哪些 AC、约束、模块、测试和发布计划会变化；
4. **提出选项**，例如保持原规格并增加实现成本，或改变范围并接受相应影响；
5. **由 IO 裁决**，涉及安全、合规或架构责任时加入相应责任人；
6. **生成新版本并重新签署**，旧版本保持可追溯，不直接覆盖；
7. **重新裁剪 Context、更新 Graph 与测试**，确保所有节点使用同一版本；
8. **在 Evidence Bundle 中记录漂移及其处理。**

例如实现过程中发现历史接口一直把 5000.00 元视为高额报销。团队有两个选择：保持新契约的“严格大于”，处理兼容差异；或者让 IO 决定延续历史语义，把规则改为“大于等于”。无论选择哪一种，都不能由 Agent 静默决定，更不能只修改测试使其变绿。

如何发现漂移

人工审查固然有用，但高速 Agent 工作流更需要机械信号：

- 契约内容摘要与签署摘要不一致；
- AC 或测试在签署后被修改；
- 不同节点引用不同契约版本；
- Evidence Bundle 中的规则文本无法回链到契约；
- 新增文件或模块超出原影响范围；
- Non-goals 中的事项出现在代码差异；
- 测试从 Red 变 Green 的过程中断言被削弱或删除；
- 约束例外没有批准记录。

这些信号可以进入前置门、验证门和证据审计。机械检查不能理解全部业务语义，却能发现“签署后发生了什么变化”，把最值得人审查的地方暴露出来。

规格版本不是文档管理小事

每个有效版本至少需要版本号或内容摘要、签署人、签署时间、变更原因和替代关系。新的签署不是抹掉旧决定，而是说明组织在什么证据基础上改变了什么。这样发生事故时，团队才能判断问题来自错误执行、错误规格，还是正确规格后来发生变化。

规格漂移管理的目的也不是冻结需求。Agentic Agile 接受变化，但要求变化从暗处走到明处：**可以改变目标，不能伪装成目标从未改变；可以接受新风险，不能让 Agent 替责任人接受。**

无论契约保持原样还是经过重新签署，关键 AC 都必须锁定在当前版本上。进入执行后，它们还要持续经受测试、证据与裁决链的检验。

10.8 签署后的 AC：保持受控并进入证据链

关键 AC 已在签署前形成验证基线；进入执行后，重点转为防止它被静默改写，并让每条 AC 能够稳定进入测试、证据和裁决链。实现者不能在失败后自行降低 AC、删除断言或把边界移出范围；发生这种变化时，应按规格漂移处理并重新签署。AC 可以采用人工判定、半自动验证或自动运行三种协议。重要业务规则应尽量进入自动验证，同时保留确实需要人类判断的价值与体验问题。

从“可读”到“可执行”的三个层级

“可执行 AC”不意味着每条文字都必须直接变成自动化代码，而是每条 AC 都有明确、可重复的验证协议。可以按以下层级推进：

层级	表现	适用情况
L1 人工可判定	写明观察对象、步骤、预期和责任人，可重复人工验收	审美、价值取舍、探索性体验
L2 半自动验证	脚本采集结果，由人对照判据裁决	复杂报表、跨系统 UAT、语义判断
L3 自动可执行	通过安全 <code>predicate:</code> 、 <code>http:</code> 、 <code>db:</code> 、测试命令或策略检查自动给出结果	接口、规则、状态、不变量、性能与安全门槛

对 AI Coding 任务，应尽可能让关键业务行为进入 L3，并保留确实需要人的价值判断。仅用 `grep` 检查某个字符串、文件或函数是否存在，通常只能证明代码长得像预期，不能证明运行行为满足业务意图。按照 3-4-3 的机械门禁，使用 `shell:grep` 的 AC 不应超过总数的一半，至少一半应通过安全 `predicate:`、`http:`、`db:` 或真实测试命令等运行时方式验证。任意 Python `assert:` 与 `setup` 形式已经因执行风险停用；自定义业务验证应进入受控测试脚本，而不是把可执行代码藏进契约。

例如，可以在 Intent Contract 中直接声明验证方式：

```
acceptance_criteria:
- id: AC-EXP-001
  rule: 金额严格大于 5000 元时进入总监审批
  verify:
    type: command
    command:
      argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_01]
      timeout_seconds: 60
    evidence: evidence/ac/AC-EXP-001.json
- id: AC-EXP-002
  rule: 金额等于 5000 元时保持原审批路径
  verify:
    type: command
    command:
      argv: [python, -m, pytest, tests/test_expense_approval.py, -k, amount_5000_00]
```

```
timeout_seconds: 60
evidence: evidence/ac/AC-EXP-002.json
```

具体语法可以因工具链而异，关键是验证命令、输入、断言和证据位置都可追溯，任何人或 Agent 在同一版本上重跑，都应得到可解释结果。

AC 要形成覆盖组合，而不是只写“快乐路径”

单条成功示例不足以定义规则边界。AC 组合至少应按风险考虑：

- **典型成功路径**：最常见输入产生正确结果；
- **边界值**：阈值前、阈值本身和阈值后，例如 4999.99、5000.00、5000.01；
- **明确反例**：看起来相似但不应触发规则的情况；
- **权限拒绝**：无权限主体不能通过另一入口完成同一动作；
- **异常与恢复**：依赖失败、超时或重复提交后，状态是否一致并可恢复；
- **关键不变量**：无论路径怎样变化都不能被破坏的规则；
- **必要 NFR**：对该意图真正关键的性能、安全、隐私和可用性门槛；
- **可观测性**：不仅结果正确，还能识别使用了哪个规则版本、发生了什么裁决。

并非每个任务都要机械凑齐八类 AC。选择标准是风险：哪一种误解最可能发生，哪一种失败代价最高，哪一项若没有证据便无法上线裁决。简单文案可以只有两三条 AC；资金、权限、隐私和跨系统状态变化则需要更完整的组合。

AC 如何贯穿 SCOPE-V 与证据链

AC 一旦签署，就不再只是 Specify 阶段的一段文字：

自然语言意图

- Example Mapping / 结构化追问消除歧义
- Intent Contract 中形成带 ID 的 AC
- Work Graph 把 AC 分配到测试、实现和验证节点
- Prove 先产生可信 Red，再挑战候选实现
- Evidence Bundle 按 AC ID 收集命令、结果、时间与来源
- Verify 逐条裁决通过、失败、豁免或证据不足
- 生产遥测继续观察 AC 所代表的业务效果

每条关键 AC 都应拥有稳定 ID，并能回链到规则和目标，向前映射到测试与证据。Evidence Bundle 不能只写“所有测试通过”，而要说明 `AC-EXP-001` 由哪项检查证明、检查针对哪个契约和提交、何时运行、结果是否新鲜、是否存在例外。若某条 AC 没有证据，正确状态不是“默认通过”，而是“未证明”。

同样，测试全绿也不代表所有 AC 都满足：测试可能漏掉某条 AC，可能运行在旧提交上，也可能与实现共享同一个错误假设。因此 Verify 必须检查“AC—测试—证据”的映射完整性，而不能只看测试总数和绿色比例。

防止 AC 被 Agent 反向操纵

在 $P \Leftrightarrow E$ 快内环中，AS 可以修改实现并重跑测试，但不能自行降低 AC、删除失败断言或把边界值移出范围。建议把以下内容纳入 Harness 门禁：

- 签署后的 AC 使用版本号或摘要锁定；
- AC、关键测试和断言发生变化时触发漂移检查；
- 实现 Agent 不拥有批准关键 AC 变更的权限；
- Red 到 Green 过程中保存测试差异和原始失败证据；
- AC 豁免必须记录原因、风险承担人和有效期限；

- Verify 由独立角色或独立上下文审查证据，而不是由实现者自行宣布完成。

判断一条 AC 是否合格，可以做最后的“五问检查”：它是否回到某个已确认的目标或规则？不同的人是否会作出相同判定？机器或责任人是否知道到哪里取证？它是否覆盖最危险的边界或反例？实现失败时，Agent 能否通过修改 AC 逃避失败？如果其中任何一问答不上来，AC 仍然只是验收愿望，还没有成为可执行规约。

10.9 Spec 不是越详细越好

规格过少会导致猜测，规格过多会带来维护和漂移成本。可以按风险选择层级：

场景	推荐规格
低风险界面调整	Goal、Non-goals、少量 AC
规则密集业务	Rules、Example Mapping、边界 AC
状态机或协议	Model-Based Specification、状态不变量
高风险跨模块	契约、XC、回滚和审计要求

10.10 Specify 常见失败

- 把 IO 第一句话直接写进 Goal；
- 一次提出十几个问题，让 IO 无法决策；
- 推荐默认值没有理由；
- Non-goals 为空；
- 业务术语没有统一口径；
- AC 只检查文件或字符串；
- 签署区由 OA 填写；
- 签署后仍允许 Agent 静默修改。

10.11 本章小结

Specify 把一句自然语言需求变成可判定、可签署、可追溯的组织承诺。批判性校准发现关键问题，结构化追问帮助 IO 作出决定，Example Mapping 与 AC 把决定转成验证协议，显式签署则建立授权边界。这是人机系统取得执行权之前的责任形成过程。

本章最小实践：对一个模糊需求完成必要的关键追问，生成并由 IO 签署 T-001。

完成证据：带 Goal、Non-goals、Rules、AC 和签署区的契约。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把模糊需求改写为可以被人签署、被 Agent 执行的意图契约。

第十一章 | Constrain：把组织原则变成可执行边界

11.1 从原则提醒到运行边界

“不要破坏旧功能”“确保数据安全”“尽量少花 Token”表达了组织原则，却无法直接驱动系统行为。Constrain 要把它们转成机器能够检查、阻断和升级的条件。

```
- id: C-DATA-01
  domain: DATA
  level: MUST
  description: 测试不得读取生产报销数据
  check_type: predicate
  check: "not is_file('.env.production')"
  gate: G2
  on_failure: block
  owner: Data Owner
  evidence: evidence/constraints/C-DATA-01.json
```

一条有效约束至少回答：适用于哪里、如何检查、失败后怎么办、谁能批准例外、例外何时失效。

11.2 六大核心约束域：从分类目录到工程控制面

STRUCT、DATA、BEHAVE、QUAL、PROC、STYLE 六个核心域，分别覆盖 Agent 执行中的六种典型失控：工件缺失、数据越界、行为走样、质量退化、过程绕过和代码失序。对每个域都要回答三个工程问题：它保护什么风险，怎样变成机器检查，违反后留下什么证据并触发什么动作。

STRUCT：确保治理骨架没有被拆掉

STRUCT 保护的是项目和治理制品的**结构完整性与可寻址性**。Agent 不仅可能写错代码，也可能把契约放错目录、删除关键测试、改变证据命名、制造两个任务共用同一 ID，最终让自动化找不到应检查的对象。

典型 STRUCT 约束包括：必需目录和文件存在；契约、约束矩阵、Evidence Bundle 使用同一任务 ID；关键模块入口不能被绕过；生成文件不得覆盖源文件；测试和回退脚本位于约定位置；图谱节点引用的产物真实存在。

它适合用目录扫描、Schema 校验、引用完整性检查和内容摘要验证来执行。例如：

```
- id: C-STRUCT-003
  level: MUST
  rule: 每个已签署任务必须拥有同 ID 的约束矩阵与证据目录
  check: "执行工件引用完整性检查"
  gate: preflight
  on_failure: block
  evidence: "evidence/structure/artifact-links.json"
```

STRUCT 通过只能证明“治理骨架存在且能连接起来”，不能证明流程真实执行过。这正是它与 PROC 的区别：前者检查有没有正确的轨道，后者检查列车是否真的按规则运行。

DATA：限制 Agent 可以接触和改变的数据世界

DATA 保护数据的**范围、机密性、完整性、用途和生命周期**。它不仅回答“能不能读生产数据库”，还要回答哪些字段可以进入上下文、是否需要脱敏、能否写回、保存多久、是否允许跨客户或跨地域流动，以及失败后怎样恢复。

典型 DATA 约束包括：生产数据不得进入开发上下文；PII 必须脱敏；测试数据必须可追溯到合成或批准来源；Agent 只能访问当前租户；数据库写操作必须限定环境与表；数据导出必须经过出域扫描；临时上下文和日志按期限删除。

DATA 不能只靠扫描最终代码，因为越界可能已经发生在提示词、工具调用或日志输出中。有效控制通常需要数据分级标签、上下文注入过滤、工具网关、读写权限、DLP 扫描和审计日志共同完成。违反生产数据、密钥或客户隔离等 MUST 时，应立即阻断、撤销会话能力并升级，而不是让 Agent“注意下次不要再犯”。

DATA 与安全高度相关，但并不等于 SEC。DATA 关注被保护对象及其允许流动方式；SEC 扩展域还会覆盖身份认证、依赖漏洞、攻击面和威胁响应。行业风险、供应商与数据治理的组织级安排，见第二十四章。

BEHAVE：把业务语义和系统不变量锁进运行行为

BEHAVE 保护的是系统做出的业务决定和状态变化，防止 Agent 生成了结构合理、测试不少，却在关键语义上偏离意图的实现。它通常直接来自 Intent Contract 的 Rules、边界 AC、状态模型和历史兼容承诺。

以报销规则为例，“金额严格大于 5000 元才进入总监审批”属于 BEHAVE；“同一报销单重复提交不得重复生成审批实例”也是 BEHAVE；“已支付单据不能退回草稿状态”则是状态不变量。它们应通过接口测试、数据库断言、属性测试、状态机测试或契约测试证明，而不是通过 `grep` 判断代码里是否出现了某个比较符号。

BEHAVE 最危险的失败不是程序崩溃，而是程序稳定地做出错误业务决定。因此关键行为约束通常是 MUST，必须覆盖典型例、边界和反例，并在 Evidence Bundle 中按 AC 或 Rule ID 建立映射。如果实现需要改变行为约束，应触发规格漂移和 IO 重新签署，AS 无权为了兼容现有代码自行改写。

QUAL：定义“做到了”之外的工程可接受程度

QUAL 保护的是交付结果的工程质量底线。BEHAVE 回答业务行为是否正确，QUAL 回答结果是否足够可靠、可维护、可构建和可持续运行。一个接口可能正确地把 5000.01 元送入总监审批，却因为响应需要十秒、存在高危漏洞或一并破坏了其他模块，仍然不能交付。

典型 QUAL 约束包括：编译和构建必须通过；增量与回归测试满足要求；静态分析不得出现新增阻断问题；关键接口达到明确的性能基准；依赖风险在允许等级内；复杂度、重复度或覆盖率不得发生不可接受退化；关键可观测信号能够产生。

QUAL 要防止“代理指标替代真实目标”。覆盖率高不等于关键行为得到验证，测试数量多不等于测试有效，平均响应时间也可能掩盖长尾。质量约束应明确环境、样本、阈值和比较基线，并尽量组合结果指标与防护指标。

PROC：证明关键治理动作真实发生，而非只留下结果

PROC 保护的是治理过程的真实性和责任链。当 Agent 能在几分钟内完成多步工作时，最终产物存在并不代表签署、TDD、独立验证、人工裁决和遥测回写真的发生过。PROC 约束负责阻止“跳过过程但伪造完成状态”。

典型 PROC 约束包括：编码前必须存在 IO 真实签署；测试必须先产生可信 Red；实现者不得独自批准关键 AC 变更；Verify 前必须生成 Evidence Bundle；高风险例外必须由授权角色批准且带失效时间；任务结束必须写入契约级与全局遥测；重大决策必须能够追溯到责任人。

PROC 需要时间戳、签署摘要、Git 历史、流水线事件、角色身份、命令退出码和审计记录等过程证据。只检查最终文件里写着“已完成 TDD”没有意义。比如 TDD 门禁要证明测试文件和失败记录先于实现变化出现，而不仅是仓库最后同时存在测试和代码。

这也解释了 PROC 与 STRUCT 的边界：Evidence Bundle 文件存在属于 STRUCT；其中包含与当前提交匹配的真实运行记录、并经过独立 Verify，属于 PROC。两域常同时命中，但证明的问题不同。

STYLE：限制高概率生成下的代码熵增

STYLE 保护的是代码库内部可持续的表达方式和架构惯例。AI Coding 可以快速产生大量“能运行但不像这个系统”的代码：命名体系不一致、错误处理各自发明、绕过现有抽象、重复创建工具函数，或把本应进入领域层的逻辑塞进控制器。单次看似无害，长期会迅速放大维护成本和上下文噪声。

STYLE 不应变成一本几百页的审美手册。真正适合约束矩阵的是少量高价值规则：必须复用的框架和公共组件；禁止绕过的分层边界；错误、日志、配置和依赖注入方式；禁止引入的模式；语言格式化与 Lint 基线。其余偏好可以进入 AI Coding Guide，按语言、模块和任务裁剪注入。

STYLE 约束可以通过 formatter、linter、架构依赖测试、AST 规则和代码评审执行。格式错误通常可自动修复；破坏架构边界可能需要阻断；带有合理原因的新模式则应升级人工评审并更新 Coding Guide。STYLE 的目的不是让所有代码看起来完全相同，而是防止 Agent 每次执行都发明一套新的局部世界。

六域如何协同：一项变更往往同时穿过多个控制面

约束的分类不是互斥标签。同一风险可能涉及多个域，但应指定一个主域作为 Owner 和主要门禁，再用标签关联其他域，避免重复维护两条含义相同的规则。

例如“禁止 Agent 删除金额边界测试”主要保护业务不变量，可以归入 BEHAVE；测试文件必须存在由 STRUCT 检查；测试必须先 Red 再 Green 由 PROC 证明；完整套件必须通过属于 QUAL；若测试数据来自脱敏数据集，则又受到 DATA 控制。六域共同作用，才构成一条完整约束链。

可以用下面的诊断顺序为新约束确定主域：

工件或引用是否完整？	→ STRUCT
数据是否被不当读取、流动或修改？	→ DATA
业务决定或状态是否错误？	→ BEHAVE
工程品质是否低于交付底线？	→ QUAL
关键治理动作是否被绕过？	→ PROC
代码表达或架构惯例是否失序？	→ STYLE

每项任务不必平均配置六类规则。低风险文案修改可能只需要少量 STRUCT、PROC 和 STYLE；涉及资金、权限与生产数据的任务则必须强化 DATA、BEHAVE 和 QUAL。成熟的约束矩阵追求的不是规则数量，而是高风险路径没有无主、不可检查或无法恢复的空白。

11.3 按风险扩展 NFR

NFR（非功能需求）描述系统以什么质量和运行条件完成任务，例如安全、可靠性、性能、可观测性和恢复能力。它们跨越单个功能，但必须落到当前任务：资金审批接口需要幂等、审计、性能与恢复证据；一次性内部脚本只需满足与其风险相称的错误处理和日志要求。

每条 NFR 都要带环境、负载或触发条件、指标和阈值。“性能要好”不能进入门禁；“在指定数据规模与 200 并发下，审批接口 p95 不超过 300ms”才可以。不要把 Web 服务的清单机械套给 CLI 或纯库项目。

11.4 MUST、SHOULD、MAY

MUST、SHOULD、MAY 用来表达规则强度。它们不是“大、中、小”三个优先级，而是在规则未满足时决定系统还能不能继续。

- **MUST**：不可妥协的要求，失败即阻断。适合安全红线、核心业务不变量和完成定义；
- **SHOULD**：强烈建议满足，但在说明理由、责任人和有效期后可以例外；
- **MAY**：可选能力，本次不满足不会影响当前裁决。

例如“不得把生产密钥写入仓库”是 MUST；“新增核心模块应保持不低于现有覆盖率”可能是 SHOULD，特殊重构期间经批准暂时例外；“本次顺便优化仪表板配色”则是 MAY。

选择等级时可以问三个问题：违反后影响是否严重，结果是否不可逆，是否存在明确责任人能够接受风险。不要为了表示重视把所有规则都写成 MUST。硬门禁过多会造成频繁阻断和绕过；规则全部是 SHOULD，又会让真正的底线消失。

MUST 也不是绝对永恒。法律、安全等最高层约束通常不可由项目角色豁免；组织工程约束若确有例外，则必须由授权人书面批准，说明范围和失效时间，不能由 AS 自行降级。

优先级遵循：

法律与安全	>	约束矩阵	>	已签署意图契约	>	实现便利性
-------	---	------	---	---------	---	-------

11.5 Tools Manifest 与最小权限

Agent 的能力不仅来自模型，也来自工具。Tools Manifest 应定义：

- 允许调用的工具；
- 允许读写的路径；
- 允许访问的域名和 API；
- 数据库操作边界；
- 命令预算和超时；
- 哪些操作需要人类批准。

不可逆操作不应因为“自动化更方便”而默认开放。

11.6 上下文安全

外部网页、Issue、日志、文档和用户输入都可能包含恶意或过期指令。Agent 必须区分：

- 系统和项目约束；
- 已签署契约；
- 任务上下文；
- 外部不可信内容。

Prompt Injection 和 Context Poisoning 的防御重点不是让模型“更聪明”，而是：降低不可信内容的指令权限、隔离敏感上下文、限制工具能力、对高风险动作设置机械批准。

11.7 数据安全：在任务上执行，在组织层治理

任务边界包括数据分级、最小必要上下文、Tools Manifest、脱敏、出域阻断和证据留存。密钥、口令、生产凭证和严格受限数据不得进入模型上下文、日志或 Evidence Bundle；敏感任务必须在已批准环境中运行，并用合成探针验证阻断真的生效。

供应商训练与留存、区域与子处理者、企业租户配置、删除演练、DLP 与事件响应属于组织级风险治理。它们决定一项任务能否使用某个模型或工具，但不应在每个任务里重新审计；组织应维护可复用的服务目录和证据包，任务只需证明自己落在已批准边界内。完整的风险裁剪与数据治理见第二十四章。

11.8 例外、降级与恢复

约束失败后不只有“阻断”或“放行”两个选择。团队需要区分三种动作：

- **例外 (Exception)**：某条约束暂时不满足，但责任人明确接受残余风险；
- **降级 (Degradation)**：不突破底线，通过缩小范围、降低权限或关闭能力继续运行；
- **恢复 (Recovery)**：执行失败后，把代码、数据、权限和任务状态带回已知安全状态。

优先顺序应是：**能恢复先恢复，能安全降级就不申请例外；只有确实需要承担残余风险时才创建例外。** 法律、安全、生产数据隔离等不可豁免约束，不应通过项目级例外放行。

情况	推荐动作	示例
结果错误且可撤销	恢复后重新执行	回滚提交、恢复测试数据、撤销临时权限
完整能力不可用但核心路径仍安全	降级运行	关闭自动发布，改为人工发布；只开放只读工具
约束暂时无法满足且风险可承担	限时例外	MVP 暂无性能环境，由责任人批准仅限内测
风险不可逆、责任人不明或证据不足	阻断并升级	生产数据可能出域、回退脚本从未验证

一份有效例外至少要记录：约束 ID、业务原因、影响范围、残余风险、补偿控制、批准人、到期时间和退出条件。只写“项目紧急”不构成批准依据。

```
exceptions:
  - id: EX-001
    constraint_id: C-QUAL-03
    reason: MVP 阶段暂时没有正式性能环境
    scope: internal-pilot
    residual_risk: 高并发容量尚未被证明
    compensating_controls:
      - 限制为 30 名内部用户
      - 关闭自动扩量
      - p95 超过 500ms 自动告警并停止试点
    approved_by: Product-Owner
    valid_until: 2026-09-01
    exit_condition: 完成正式性能基准并更新 Evidence Bundle
    on_expiry: block
```

降级策略应预先写入约束，而不是故障发生后临时发挥，例如 `full-auto → human-approval → read-only → stop`。每一级都要明确触发条件、允许能力和恢复入口，避免系统虽然“降级”，却仍持有原来的高风险权限。

恢复也不能只写“支持回滚”。至少要回答：恢复到哪个已知版本，代码、数据和权限分别怎样处理，由什么信号触发，恢复后运行哪些验证，以及谁确认业务重新可用。关键恢复路径需要定期演练，演练记录进入 Evidence Bundle。

Harness 应在例外到期时自动撤销授权并恢复原约束；无法自动撤销时必须阻断后续门禁。Dashboard 还应持续显示未关闭例外、剩余期限、重复申请次数和恢复演练状态。例外反复续期通常意味着约束不合理或技术债长期未还，不能继续伪装成“临时情况”。

11.9 约束冲突

约束可能互相冲突。例如性能要求鼓励缓存，而数据一致性要求禁止使用过期结果；成本约束要求减少模型调用，而安全约束要求双模型复核。

冲突不能由 AS 根据实现便利自行解决。OA 应识别冲突，按法律与安全、数据、业务、质量、效率的优先顺序提出方案，由拥有责任的人批准。

冲突处理需要记录：

- 涉及哪些约束；
- 为什么冲突；
- 选择了什么优先级；
- 是否产生临时例外；
- 何时重新评估。

11.10 约束的经济性

每条约束都有执行成本。团队应观察：

- 命中次数；
- 真正阻止的风险；
- 误报率；
- 执行耗时；
- 例外数量；
- 是否已经被其他门禁覆盖。

没有命中、没有风险依据、持续产生误报的约束应被审查，而不是因为“安全第一”永久保留。

11.11 Constrain 常见失败

- 从模板复制几十条约束却不裁剪；
- 将 SHOULD 全部升级为 MUST；
- 检查命令只验证文件存在；
- 工具白名单允许任意 shell 和任意路径；
- 生产权限与开发权限混在一起；
- 只凭“企业版”“零留存”或“不训练”标签批准敏感数据进入模型；
- Prompt 做了脱敏，但日志、缓存、Memory、MCP 或 Evidence 继续保存原文；
- 私有化部署没有检查遥测、依赖更新、向量库、备份和模型制品来源；
- 例外没有批准人或期限；
- 约束更新后没有重新验证相关 Evidence Bundle。

11.12 本章小结

Constrain 把组织的隐性经验变成机器可执行的护栏。Agent 获得的不是无限自治，而是在明确数据、权限、质量、预算和恢复边界内的行动能力。

本章最小实践：为 T-001 建立六域约束矩阵、Tools Manifest 和四级数据路由矩阵，并用合成密钥或测试 PII 验证一次阻断。

完成证据：至少五条 MUST 约束、模型与工具路由、DLP 负向测试和数据责任人记录。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，为真实任务配置约束矩阵、失败动作和人工升级边界。

第十二章 | Orchestrate：从团队协作到可计算的人机能力网络

12.1 从任务列表到 Work Graph

Work Graph 可以直译为“工作图”。它回答一个直接的问题：为了完成这次交付，哪些工作必须发生，它们彼此依赖什么，满足什么条件才可以继续？

任务清单只能列出“澄清规则、生成测试、修改实现、完成验证”；工作图还要说明：规则没有签署，测试节点就不能启动；测试没有产生可信 Red，实现节点就不能开工；当前提交没有通过集成验证，证据汇总就不能宣告完成。箭头表达的不是展示顺序，而是下游必须满足的真实启动条件。

AI Coding 中也一样。复杂任务不是一串可以任意领取的待办事项。多个 Agent 执行得很快，如果它们不知道依赖和边界，就可能同时修改同一文件、在测试尚未可信时开始实现、拿过期产物继续工作，或者在集成验证之前宣布完成。Work Graph 把这些原本藏在人脑、会议和聊天记录里的协作关系，变成可以检查、调度和留证的结构。

12.1.1 一张工作图由什么组成

最小 Work Graph 包含四类元素：

- 节点 (Node)**：一项可独立执行和验证的工作，例如“生成边界测试”或“完成接口契约验证”。节点不是一句模糊任务名称，还要有输入、执行者、允许范围、输出和完成判据。
- 边 (Edge)**：两个节点之间的关系。它可能表示先后依赖、数据交接、审批放行、资源互斥或失败路由，而不只是“A 做完再做 B”。
- 状态 (State)**：节点当前是等待、可运行、执行中、通过、失败、阻塞还是需要人工裁决。状态必须来自真实执行结果，不能只靠 Agent 自报。
- 控制条件 (Control Condition)**：什么时候启动、停止、重试、回退、汇合或升级 HITL。例如“只有测试产生可信 Red，且影响分析没有发现范围外依赖，实现节点才可启动”。

一张可执行的工作图应能回答：

- 现在有哪些节点已经具备启动条件？
- 哪些工作可以并行，哪些只是表面上可以同时开始？
- 下游正在等待谁的什么产物或证据？
- 某个节点失败后，应该重试、返回上游、回退还是交给人类？
- 哪些节点全部通过后，才允许汇合、发布或生成 Evidence Bundle？

12.1.2 它与任务清单、流程图和项目计划有什么不同

形式	主要回答	通常缺少什么
任务清单	有哪些事情要做	依赖、输入输出、失败路线和汇合条件
流程图	大致经过哪些步骤	节点执行契约、真实状态、证据和调度能力
项目计划	谁在什么时间完成什么	面向 Agent 的上下文、工具权限、停止与重试条件
CI/CD Pipeline	代码如何构建、测试和发布	从意图澄清到实现、裁决的完整工作关系
Work Graph	当前交付如何被分解、依赖、执行、验证和恢复	它不是某种固定模板，需要按契约与风险生成

Work Graph 可以吸收项目计划和流水线中的节点，但不等于二者。项目计划可能写“周三完成测试”，工作图则要说明测试使用哪个契约版本、消费什么输入、通过判据是什么，以及测试失败后哪个节点失效。CI/CD Pipeline 通常从代码提交开始，Work Graph 可以更早地从意图澄清、约束确认和测试设计开始，并把人工裁决也纳入图中。

12.1.3 为什么经常使用 DAG，但又不能只理解成 DAG

DAG 是 Directed Acyclic Graph，即“有向无环图”。“有向”表示边有方向，例如测试结果流向实现节点；“无环”表示在同一次调度里不能出现 A 等 B、B 又等 A 的死锁依赖。DAG 很适合表达一次交付的主干依赖、并行组和汇合点。

但现实中的研发会返工：验证失败后可能回到测试、约束或规格重新处理。为避免概念混乱，可以把一次运行中的可调度依赖保持为 DAG，把“失败后生成修复节点、重新签署并启动新一轮图”看作控制回路。这样既允许系统学习和返工，又不会把图变成无法判断何时结束的无限循环。

12.1.4 它与 Intent Graph 的区别

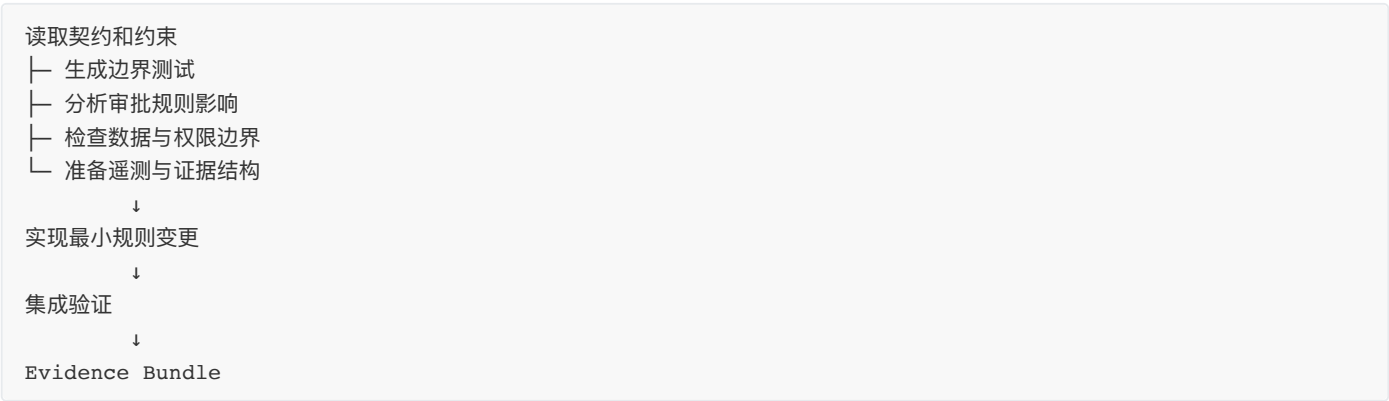
这两张图最容易混淆：

- **Intent Graph（意图图谱）** 组织“为什么做、价值如何分解、哪些意图彼此关联”；
- **Work Graph（工作图）** 组织“这一次怎样做、谁先谁后、产物如何交接、失败如何处理”。

例如，“降低虚假报销并保持员工体验”属于 Intent Graph；为了修改单笔金额规则而安排“澄清边界—生成测试—影响分析—实现—SIT—证据裁决”，属于 Work Graph。前者相对稳定，可以服务多个契约；后者面向一次具体交付，会随着契约、风险和执行状态变化。

12.1.5 费用报销案例：从清单变成可执行关系

费用报销案例可以拆成：



这张简图还只是骨架。要成为可执行 Work Graph，每个节点还要绑定契约与 AC、输入产物、负责人或 Agent、工具权限、允许修改范围、完成证据、预算和失败路线。例如：

- “生成边界测试”读取已签署的金额规则和 Example Mapping，输出处于可信 Red 状态的测试及运行记录；
- “实现最小规则变更”必须等待可信 Red 和影响分析完成，只允许修改金额策略与指定适配层；
- “集成验证”必须消费当前提交的制品，不能拿旧分支的 Green 结果冒充；
- “Evidence Bundle”只负责汇聚与核验既有证据，不能自己补造缺失证据；
- 若影响分析发现未知调用方，图进入阻塞状态并升级 OA/IO，而不是静默扩大修改范围。

只有无前置依赖、无写冲突、上下文边界明确、失败互不污染的任务才适合并行。**Work Graph** 的价值不是让更多 Agent 同时开工，而是让每个 Agent 只在条件成立时开工，并让任何“已经完成”的声明都能沿图追溯到输入、规则和证据。

12.2 每个 Agent 节点的执行契约

每个 Agent 节点至少需要：

- 当前目标；
- 非目标；
- 输入和预期输出；
- 相关 AC 和约束；
- 允许修改的文件；
- 禁止触碰的模块；
- 可用工具；
- 时间、Token 和重试预算；
- 完成条件；
- 停止和升级条件。

12.3 Handoff 与跨 Agent 协作

Handoff 即“工作交接”。在人类团队中，接收者会追问不清楚的地方，并凭共同背景判断上游是否遗漏；Agent 之间缺少这种默契，一句“测试已经完成，可以继续”很容易把错误假设传到下游。

Handoff 是一份小型、可验证的交付契约，而不是聊天总结。它应传递结构化事实：

- 已完成什么；
- 产生了哪些文件；
- 哪些检查已运行；
- 哪些失败仍未解决；
- 哪些假设尚未验证；
- 下一个 Agent 需要什么输入。

还应带上任务与契约版本、产物路径、运行命令和退出码。下游收到后先做低成本验证：文件是否存在，测试是否确实处于预期 Red 或 Green，产物是否来自当前提交，上游声明的未知项是否会阻塞自己。

例如测试 Agent 的 Handoff 不能只写“边界测试已完成”，而应说明测试文件、对应 AC、运行命令、退出码为 1 且失败原因是功能尚未实现。这样实现 Agent 才能确认这是可信 Red，而不是测试环境损坏。

Handoff 的价值是隔离错误。如果下游验证失败，系统可以把问题路由回具体上游节点，而不是让整个任务在错误基础上继续。接收方应验证输入，而不是盲信上游 Agent 的“已完成”；上游也不应传递自己的全部上下文，只传递下游完成工作所需的事实、产物和未知项。

一份可验证的 Handoff 示例

```
task: WG-TEST-01
contract_version: T-001@sha256:8d31...
source_commit: a81f5c2
status: completed
outputs:
  - path: tests/expense-threshold.test.ts
    sha256: 6a72...
checks:
  - command: npm test -- expense-threshold
    exit_code: 1
    meaning: expected_red
    evidence: evidence/red/WG-TEST-01.txt
assumptions:
  - currency is CNY
unknowns:
```

```
- historical drafts migration not evaluated
next:
  allowed_node: WG-IMPLEMENT-01
  precondition: coding_gate_passed
```

接收方至少校验契约版本、来源提交、产物摘要和证据文件，再判断 `exit_code: 1` 是否真是预期 Red，而非依赖缺失或环境故障。验证失败时，图应把任务退回 `WG-TEST-01`，不能让实现节点带着不可信输入继续执行。

12.4 并行是否真的带来收益

并行收益取决于可独立任务比例、协调成本、共享资源和最终集成成本。团队可以记录：

- 串行关键路径长度；
- 可并行节点数量；
- Handoff 失败次数；
- 写冲突和重做；
- 并行带来的真实时间节省；
- 增加的 Token 和验证成本。

如果两个 Agent 需要反复同步同一文件，并行可能比串行更慢。

并行只是三类编排选择中的一类。OA 还必须预先确定失败时是重试还是升级，以及哪些上下文可以共享、哪些必须隔离；这些决定共同决定一张 Work Graph 是否真的可运行。

12.5 编排中的三个关键决策

决策一：串行还是并行

只有当任务边界稳定、共享写入较少、验收接口明确时，并行才有意义。若两个 Agent 不断修改同一核心对象，表面的并发会转化为合并冲突、上下文重载和重复验证。OA 应比较关键路径缩短量与协调成本，而不是把 Agent 数量当作生产力指标。

决策二：自动重试还是升级人类

网络抖动、临时资源不可用、已知幂等操作可以有限重试；目标冲突、权限不足、数据不可逆、证据自相矛盾则必须停止。重试次数、退避策略和升级条件都应预先写入 Work Graph，不能让执行 Agent 在失败后临场决定自己还可以尝试多少次。

决策三：共享上下文还是隔离上下文

共享上下文有利于一致性，却增加信息泄漏、锚定偏差和上下文污染；隔离上下文减少干扰，却可能产生局部最优。合理做法是共享稳定的契约、约束和接口，隔离与本节点无关的实现细节，并通过 Handoff 传递已经验证的产物。

这三项决策如果没有被写进节点条件、预算和失败路由，Work Graph 很容易退化为一张只描述正常路径的任务清单。下面列出最常见的退化方式，帮助团队在首次运行前检查图是否缺少控制。

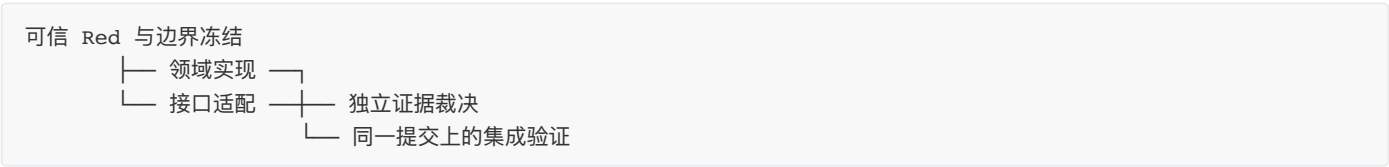
12.6 Orchestrate 常见失败

- 把任务切得太细，Handoff 成本超过收益；
- 多个 Agent 同时修改共享核心文件；
- 下游不检查上游产物；
- DAG 只描述正常路径，没有失败分支；
- 自动重试没有上限；
- 把短暂资源异常当成可以无限重试的理由；
- 用历史成功率掩盖任务类型差异。

避开这些失败后，团队不需要一开始构建复杂的多 Agent 平台。先建立一张能承载签署、交接、验证和失败回流的最小 Work Graph，就足以让协作开始受控运行。

12.7 一个最小 Work Graph

以金额阈值规则为例，先由规则测试冻结 5000.00 与 5000.01 的边界；随后领域实现与接口适配只在写集合隔离、接口语义稳定时并行；最后由不修改实现与测试的节点汇总 Evidence Bundle。每条边都应是启动条件：输入来自哪个契约版本、哪些检查已运行、失败后回到哪里。



如果规则发生冲突，回到 OA/IO 重新裁决；如果是实现错误，只在当前授权范围内进入 $P \rightleftharpoons E$ ；如果证据来自不同提交，重新在汇合制品上验证。跨模块协作还需要处理集成 DAG、共享资源、分支隔离与发布顺序，使局部并行不会破坏整体交付。

12.8 本章小结

Orchestrate 的目标不是让更多 Agent 同时工作，而是把人类责任节点、专业 Agent、工具和依赖组织成可计算的能力网络，让合适的任务在明确上下文、权限、启动条件和停止条件下安全流动。

本章最小实践：为 T-001 创建 Work Graph，并写明每个节点的输入、输出、启动条件和停止条件。

完成证据：DAG、节点上下文、依赖和停止条件。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把角色、工具和交接关系组织成可验证的 Work Graph。

第十三章 | Prove：证明实现满足契约

Prove 只回答一个问题：当前实现是否满足已经签署的 Intent Contract、AC 和 MUST 约束？

是否值得发布、残余业务风险能否接受，由 Verify 处理；组织长期是否提效，由 Telemetry 判断。三个问题分开之后，测试全绿才不会被自动解释为任务完成。

13.1 AC 不是事后检查清单

AC 是驱动实现的可执行规约。Prove 不是代码完成后再浏览一遍清单，而是把签署后的 AC 转成能够否定错误实现的挑战，再让实现围绕挑战收敛。

签署 AC → 生成测试与断言 → 运行并确认可信 Red
→ Green：最小实现使测试通过
→ Refactor：不改变行为地改善结构

以“严格大于 5000 元进入总监审批”为例，至少要固定 5000.00 不触发、5000.01 触发两个相邻边界。只测试 6000 元，无法区分“大于”和“大于等于”。测试数量增加了，业务歧义仍然存在。

13.2 编码门：先证明 Red 可信

编码门检查的不是“有没有红色结果”，而是这个失败是否真能驱动正确实现：

- 测试真实存在并实际执行；
- 测试对应当前契约版本和明确 AC；
- 失败来自业务断言，而不是语法、依赖或环境初始化；
- 当前代码缺少目标行为时，测试确实会失败；
- Red 记录包含命令、退出码、时间、提交和原始输出；
- 关键断言已经锁定，后续修改会触发漂移检查。

编码门应读取真实的测试变更、失败记录、契约版本与基线提交，并保留可追溯结果；具体检查由项目现有工具链承载。

Red 至少分为三类：expected_red 才能进入实现；environment_red 应先修复环境；spec_red 表示 AC 冲突或无法判定，必须返回 Specify。把后两类当成 TDD 起点，只会让 Agent 围绕错误信号高速试错。

13.3 最小实现：让证据收敛

最小实现不是最少代码行，而是当前签署意图所需的最小语义变化和最小影响半径。AS 应先说明 Red 的原因、计划修改的位置和预期影响，再开始 Green。

实现过程中：

- 不顺手重构无关模块或升级无关依赖；
- 不添加尚未签署的顺便功能；
- 不扩大数据、网络、目录和工具权限；
- 不修改 AC 或关键测试来适配错误代码；
- 不用 Mock、skip、降低阈值绕过真实失败。

如果 Green 必须跨出允许文件范围，应暂停并回到影响分析或契约裁决。Green 后的 Refactor 也要与新增行为分开观察：测试保持通过、对外行为不变、差异可解释。

13.4 选择证明组合

不是每个任务都需要运行所有测试。验证组合应由 AC、MUST、风险和影响范围决定，并在任务开始时写入 Intent Contract 或 Work Graph。

AC/约束	风险	证明方式	主要执行者	通过判据
金额边界	高	单元 + 接口	测试 Agent	5000.00 与 5000.01 行为相反且正确
审批链路	高	集成测试	集成 Agent	总监节点只在超额时出现
响应时间	中	性能测试	性能 Agent	固定负载下达到 NFR 阈值
页面提示	中	UI + 业务场景	AI 执行、业务确认	用户能完成并理解操作

证明组合至少要回答：

- 1. 哪条 AC 由什么证据证明？
- 2. 一个测试层是否足够，是否需要交叉证明？
- 3. 哪些结果可由 AI 自动执行，哪些需要独立上下文或人工确认？
- 4. 证据对应什么环境、版本、时间和数据？
- 5. 哪些内容属于价值判断，不是实现正确性？

最后一个问题很重要。技术测试可以证明审批节点出现，不能证明审批政策公平；接口测试可以证明请求被拒绝，不能替业务负责人接受这个拒绝逻辑。

13.5 多层证明的职责

单元与组件测试

验证局部规则、不变量和边界，适合进入 $P \Leftrightarrow E$ 快内环。金额规则至少覆盖 4999.99、5000.00、5000.01、零值、非法精度和缺失值。

接口与集成测试

验证服务契约、鉴权、持久化、事件和跨系统状态，不能只断言 HTTP 200。证据应包含请求、响应、关键业务状态、数据副作用和参与系统版本。

UI 与业务场景

验证用户旅程、提示和可访问性。AI 可以大量准备和执行场景，但业务代表仍需确认规则是否可理解、操作是否可接受。

性能、安全与数据验证

性能证据必须说明环境、数据规模、负载、持续时间、指标和阈值。安全和数据验证要覆盖越权、敏感数据、Schema、迁移、精度和状态一致性。不能把“普通测试很快”当成性能证明，也不能把扫描全绿当成安全完成。

生产前验证

Prove 可以为灰度、回退和生产观察准备可执行检查，但它只证明交付候选的实现事实。是否批准灰度、接受什么残余风险，由 Verify 决定；上线后的长期结果，由 Telemetry 观察。

13.6 防止同源误判

如果 Agent 先把“超过 5000 元”理解成“大于等于”，再按照这个理解写实现、补测试和总结报告，所有检查都可能绿色，但错误假设已经贯穿全链路。

降低同源误判的方法是：

1. 测试首先从 IO 签署的契约和 AC 生成；
2. 关键规则使用不同测试层、不同技术路径或独立上下文交叉证明；
3. 测试文件和关键断言对实现节点保持只读；
4. 高风险任务增加真实样本、第二验证主体或人工领域审查；
5. 保存 Red、Green 和断言差异，检查判据是否被移动。

独立性不是简单更换 Agent 名称，而是让判据来源、执行上下文或验证技术与实现拉开距离。

13.7 证明门的状态

Prove 门的结果不能只有 PASS 或 FAIL：

- `passed`：判据满足，证据完整并对应当前版本；
- `failed`：已有证据明确证明实现不满足；
- `unproven`：缺少证据、环境不可用或映射不完整；
- `exception`：存在有效的人类批准、补偿控制和到期时间。

`unproven` 不是通过。没有性能环境不能默认性能满足，测试报告来自旧提交也不能进入当前证据包。`exception` 也不等于 Prove 通过，它只是把事实和例外交给后续责任裁决。

13.8 什么是无效全绿

绿色只能证明“被执行的检查没有报告失败”，不能自动证明契约已经实现。常见无效全绿包括：

- 测试从未出现可信 Red；
- 关键用例被 skip、only 或条件排除；
- 为适应实现修改了期望值、边界或 Mock；
- 只检查字符串、函数名或文件存在，没有验证运行行为；
- 测试数据绕开了关键边界和失败路径；
- 测试、制品和证据来自不同提交或环境。

绿色审计可以比较 Red 到 Green 的测试差异，统计 skip/only，核对测试数量是否异常下降，检查提交和环境指纹，并随机重跑一条关键 AC。若绿色无法回答“证明了哪条 AC、使用了什么输入、在什么版本上运行”，它只是界面颜色。

13.9 证明强度，而不是测试数量

证明强度至少取决于五个维度：

1. **相关性**：是否直接对应 AC、约束或关键风险；
2. **区分力**：是否能让常见错误实现失败；
3. **独立性**：是否避免与实现共享同一错误假设；
4. **真实性**：环境、数据和依赖是否接近目标；
5. **可追溯性**：结果是否可回到命令、版本、输入和责任人。

覆盖率只能说明哪些代码被执行，不能说明业务语义正确。对金额规则，三个相邻边界和一条真实接口路径，往往比大量正常值测试更有证明力。

13.10 Prove 与后续控制面的边界

Prove、Verify、Telemetry 和流水线沿用同一份任务事实，却分别作出不同判断：

控制面	核心问题	输出
Prove	实现是否满足已签署契约？	Red/Green、测试、构建和实现证据
Verify	当前证据支持采取什么行动？	发布、灰度、补证、拒绝或升级裁决
Telemetry	多次运行后组织是否形成能力？	长期趋势、净收益、复发模式和改进建议
流水线	这些能力如何进入交付系统？	版本谱系、机械门、制品晋级和回退通道

Prove 可以产生 Evidence Bundle 的原始材料，但不独占 Evidence Bundle，也不替 Verify 作发布决定。Prove 可以准备生产检查，但不把上线后的观察写成当前实现已经正确。

13.11 本章小结

Prove 让实现围绕可否定错误的证据收敛。它要求测试来自契约，Red 真实可信，Green 对应最小实现，证据绑定当前版本，并明确区分失败、未知和例外。

本章最小实践：为 T-001 从 AC 生成测试，记录 Red，再完成最小实现和 Green。

完成证据：Red 与 Green 两次真实运行结果，以及 AC—测试—提交的映射。

下一章：Evolve 处理失败、修复和知识回写；Verify 再独立判断这些证据支持哪种行动。

第十四章 | Evolve：让失败进入系统能力，而不是只进入复盘

14.1 失败不是例外，而是反馈

概率性模型、复杂系统和不完整上下文决定了失败不可避免。模型可能误解上下文，工具可能暂时不可用，多个 Agent 可能产生冲突，原本看似完整的规格也可能在真实数据面前暴露遗漏。

这并不意味着降低质量要求。既然失败不可完全避免，就要把发现失败、限制影响、恢复状态和保留证据设计成系统能力。一个从未报告失败的 Agent 系统未必更成熟，也可能只是测试过宽、日志缺失或失败被自动重试掩盖。

Evolve 关注两个层次：当前任务怎样从失败中恢复；组织怎样避免同类失败反复发生。前者可能是一处代码修复或任务重调度，后者则可能需要增加 AC、更新约束、修正图谱、改进工具权限或改变 Work Graph。

面对失败，第一步不应是立即重试，而要先判断：这是什么类型的失败，影响是否仍在授权范围内，现有证据是否可信，继续尝试会不会扩大风险？成熟系统并非从不失败，而是能够正确分类失败、在边界内修复，并留下可复用的教训。

14.2 失败分类与回流路由

失败发生后，第一步不是修代码，而是形成一张最小诊断卡：**哪项检查失败、首次出现在哪个节点、原始信号是什么、影响范围多大、是否仍在授权边界内**。没有这些信息，“自动修复”很容易把规格、环境和权限问题都误诊为实现缺陷。

失败类型	识别信号	应回到哪里
目标或规则不清	AC 无法判定、示例相互矛盾	Specify, 重新确认并签署
约束缺失或冲突	两条 MUST 无法同时满足、风险无人负责	Constrain, 由 OA/责任人决策
依赖或编排错误	输入过期、写冲突、错误节点提前启动	Orchestrate, 调整 Work Graph
实现缺陷	规格稳定、测试可信、候选行为错误	$P \rightleftharpoons E$, 最小修复并重跑原检查
测试或环境故障	语法、依赖、夹具或环境导致假 Red	Prove, 先修复验证基础设施
证据不足或不一致	缺少原始输出、提交与报告不匹配	补取证后再进入 Verify
权限、法律、伦理风险	需要越权、不可逆动作或责任裁决	立即停止, 升级 HITL

路由应由失败分类器或 OA 根据证据决定，不能由当前实现 Agent 按“最容易继续”的路径选择。分类不确定时，状态应是 `needs_diagnosis`，而不是默认进入重试。正确返回上游看似变慢，却能避免在错误目标或坏环境上高速迭代。

14.3 可自动修复的条件

AS 只有在失败落入预先定义的“修复包络”时才适合自治修复：根因可定位，修改范围仍在授权内，不改变 Goal、Non-goals、Rules 和关键 AC，不降低 MUST，不需要新权限，并且修复后可以重跑同一项检查。

修复包络至少包含：允许修改的文件或资源、允许的失败类型、最大轮次、时间与 Token 预算、必须重跑的检查，以及立即停止的信号。例如：

```
auto_repair:
  allowed_failures: [implementation_defect, transient_tool_error]
  write_scope: [src/domain/expense/**]
  max_rounds: 2
  must_rerun: [AC-EXP-001, AC-EXP-002, domain-regression]
  stop_on: [rule_conflict, permission_expansion, new_must_failure]
```

网络抖动可以退避重试；比较符号错误可以在限定文件内修复；数据语义冲突、权限不足和不可逆操作则不能靠“再试一次”解决。每轮修复后如果失败类型变化、影响范围扩大或新增错误多于被消除错误，都应退出自动循环并升级。

14.4 修复证据：证明问题真的被消除

修复完成不能只保存最后一次 Green。每轮至少形成一条可比较记录：失败的 AC 或约束、原始错误、根因分类、修改差异、是否扩大范围、重跑命令、结果和下一动作。

一条可信修复链应当能够回答：

原检查为什么失败

→ 哪个最小变化针对该根因

→ 同一检查是否从失败变为通过

→ 相邻回归和约束是否仍然成立

→ 是否引入新风险、例外或人工决定

“修改后改跑另一套更容易通过的测试”不能证明恢复；只保留最终日志也无法判断 Agent 是否移动过判据。Evidence Bundle 应保存修复前后的命令、提交摘要和关键输出，并记录测试或 AC 是否发生变化。若判据确需变化，应先走规格漂移，而不是把它混进修复差异。

对报销边界错误，证据不仅要显示 5000.01 已通过，还应重跑 5000.00 和领域回归，证明修复没有把另一侧边界或既有审批路径破坏。修复证明的是“原失败已消除且影响受控”，不只是“当前屏幕变绿”。

14.5 反思与 Failure Mining

Failure Mining 可以译为“失败挖掘”：从多次失败记录中寻找可复用模式，而不是只修复眼前的一处代码。它与普通复盘的区别在于，输入必须来自真实证据，输出必须能够改变后续系统行为。

反思不应只是模型写一段“下次会更努力”。有效反思要从证据中提取：

- 哪个假设错误；
- 哪条约束缺失；
- 哪类上下文造成误导；
- 哪个检查最早发现问题；
- 哪个模式值得复用；
- 哪些知识需要进入图谱；
- 哪些记忆需要失效或衰减。

例如三个任务都因“业务阈值等号边界未确认”返工，Failure Mining 得到的不是三个独立 Bug，而是一个通用模式：含“超过、不少于、以内”等自然语言阈值时，Specify 必须生成等于、略低、略高三个边界例子。这个模式可以进入结构化追问、Example Mapping 清单或约束检查。

一次失败适合修复局部实现；重复失败才可能说明流程、约束或知识资产有系统缺口。团队应保留失败类型、发现阶段、修复成本和影响范围，再判断是否值得提升为全局规则。否则每个偶然故障都新增门禁，会快速积累规则债务。

Failure Mining 的产出至少要回答“以后什么会不同”：下次会多问哪个问题、注入哪条上下文、运行哪项检查、限制哪个工具，或淘汰哪条过期记忆。没有改变治理系统的反思，只是文字总结。

14.6 规则债务与约束疲劳

失败不断转化为新规则，会形成另一种技术债：同义规则重复、旧规则与新架构冲突、门禁越来越慢、上下文被规则淹没，最终团队习惯性申请例外或直接关闭检查。规则多不等于治理成熟。

每条长期约束至少要有 Owner、来源事故或风险、创建版本、最近命中时间、误报率、执行成本和复核日期。团队可以定期把规则分成四类：保留、合并、降级为告警、删除。经常命中且真正阻止高损失风险的规则应保留；长期零命中但风险仍然存在的安全底线不能仅凭数据删除；频繁误报或持续申请例外的规则则必须重新设计。

清理时也要防止“为提高通过率而删门禁”。任何删除或降级都应说明风险是否已被其他控制覆盖，并在有限范围验证。Dashboard 可观察门禁耗时、误报、例外率和重复规则数，让规则债务像代码债务一样可见，而不是等开发者集体绕过时才发现。

14.7 Champion-Challenger 与有界演进

Champion-Challenger 是一种“现行方案与候选方案并行比较”的方法。Champion 是当前经过验证、正在使用的稳定方案；Challenger 是希望表现更好的候选方案。它解决的问题是：系统需要学习，却不能每出现一个新想法就直接替换生产规则。

例如当前上下文裁剪策略平均向 Agent 传入 30 个文件，这是 Champion。团队提出一种基于图谱关系的新策略，只传入 12 个文件，这是 Challenger。不能仅因为 Token 更少就立即替换，还需要比较目标准确率、依赖遗漏、首次成功率、人工介入和安全风险。

受控比较通常经过六步：

1. 明确 Challenger 想改善的指标和不能退化的底线；
2. 选择沙箱、历史回放、影子流量或低风险任务；
3. 保证两种方案面对可比较输入；
4. 保存结果、成本、失败和异常证据；
5. 达到预设门槛后由责任人决定晋升、继续观察或淘汰；
6. 晋升后仍保留回退到旧 Champion 的能力。

“有界演进”强调候选方案只能在批准范围内学习。Agent 不得自行修改全局约束、业务目标或发布阈值来提高分数，也不能用单一指标取胜。Challenger 即使降低了 Token，只要关键依赖遗漏率上升，就不能宣布成功。

Champion-Challenger 不只适用于模型，也适用于 Prompt、上下文裁剪、路由、测试生成和恢复策略。它让组织在保持稳定基线的同时持续试验，是“可以进化”与“不能失控”之间的桥梁。系统可以演进，但不能自主修改它用来证明自己正确的标准。

14.8 停止条件：自治系统也必须知道何时认输

没有停止条件的自我修复会持续消耗 Token、污染更多文件，并把一个局部错误演化成系统性风险。停止条件应在执行前写入节点契约，而不是由 Agent 失败后自行判断“还值得再试一次”。

常见停止信号包括：达到最大轮次、Token 或时间预算；同一失败重复出现；新增失败持续增加；根因从实现问题变成规格或约束冲突；需要扩大权限或修改关键 AC；证据可信度下降；回滚安全无法证明；出现不可逆数据风险。

触发停止后，系统要执行四个动作：冻结受影响节点和输出、保留现场证据、撤销临时权限或锁、生成面向 OA/HITL 的裁决包。状态应明确区分 `stopped_by_budget`、`blocked_by_spec`、`blocked_by_constraint` 和 `unsafe_to_continue`，方便正确路由。

停止不是把任务标成失败后结束。它是系统主动防止风险继续扩散，并把“机器为什么无法继续”转换为人类可以裁决的信息。

14.9 Evolve 常见失败

Evolve 最常见的失败，是复盘写了很多，系统却没有发生改变：

- **错误路由**：所有失败都自动重试，规格、环境和权限问题也被当作代码缺陷；
- **虚假恢复**：修复后换了一项检查，或只保存最终绿色结果；
- **空洞反思**：只写“下次更仔细”，没有新增问题、规则、测试或路由；
- **污染记忆**：把未经验证的猜测写入下一轮上下文；
- **失控演进**：系统自行修改 MUST、AC 或评价标准；
- **伪实验**：Champion 与 Challenger 使用不同数据集，或只比较对候选方案有利的指标；

- **数据美化**：事后 Bug 被当成新任务，原任务首次成功率仍保持 100%。

检查一次 Evolve 是否有效，可以只问一句：**下一次相似任务具体会在哪个环节表现得不同？** 如果回答不出新增的上下文、测试、约束、路由、恢复动作或失效记忆，这次演进仍停留在文字层面。

14.10 Bug 回溯门：完成之后发现错误也必须回流

任务关闭后发现 Bug，不能只创建一个新故事并保持原任务“全绿”。Bug 是对原完成结论的新反证，必须回到原契约、原 Evidence Bundle 和原遥测，修正组织对真实交付质量的认识。

最小回溯动作包括：确认 Bug 归属的契约与 AC；保存复现输入和影响范围；在原 Evidence Bundle 追加事后记录；把首次成功状态修正为失败或自愈；重新采集测试、修复轮次和成本；把漏测原因回写测试、约束、图谱或记忆。

回溯门应在缺陷流程中机械检查原契约、原证据、修复证据与度量修正是否建立；检查方式由项目现有的缺陷与交付工具承载。

例如上线后发现 5000.00 元在移动端接口仍进入总监审批，不能只给移动端开一个修复任务。团队还要回到 T-001，标记接口证据组合存在缺口，补充跨入口契约测试，并修正“首次通过”的 Dashboard 数据。否则组织会不断制造看似成功的旧任务和彼此无关的新 Bug，永远学不到验证体系哪里失效。

如果 Bug 暴露的是普遍模式，例如所有入口都可能各自实现阈值规则，还应启动 Failure Mining 和影响扫描。Bug 回溯门的目的不是追责某个人，而是让完成事实、度量和组织学习重新与现实一致。

14.11 演进的长期资产

失败产生的候选规则、记忆、图谱关系与实验策略，都必须带来源、适用范围、Owner、证据和失效条件；未经验证的结论不能自动改变下一项任务。第二十六章将进一步说明这些经验如何写入、晋升和撤回，第二十七章再处理它们如何跨任务、跨团队复用。

在受控环境主动注入工具不可用、上下文过期、权限拒绝或遥测丢失，能够验证系统是否会发现、停止、恢复并正确回流。这样的实验应是规模化治理能力的一部分，不是单任务关闭前的附加仪式。

14.12 本章小结

Evolve 把修复、反思和知识更新连接起来。自治不是永不找人，而是知道哪些失败可以自己修、哪些必须停止，以及如何用证据证明已经恢复。未经 V 确认的反思只能作为候选记忆，不能直接晋升为长期规则或组织事实。

本章最小实践：故意注入一个边界错误，完成失败分类、修复、重跑和图谱回写。

完成证据：先红后绿的运行记录和一条可复用教训。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把失败、修复和知识回写变成下一轮执行可以使用的资产。

第十五章 | Verify：从全绿到基于证据的责任裁决

Prove 与 Evolve 已经让候选结果经历测试、失败、修复和重新取证，但“内环已经收敛”仍不等于“组织应当上线”。Verify 必须站在执行和自愈之外，检查目标、边界、证据、时效、残余风险与恢复条件，再由拥有责任的人作出行动裁决。

15.1 测试通过不等于上线批准

测试通过只能证明“在给定环境、给定输入和给定断言下，已经覆盖的行为符合预期”。它不能自动证明需求本身正确，不能证明没有遗漏场景，也不能替组织接受发布风险。

上线还涉及测试之外的问题：目标是否仍有价值，生产环境是否准备好，数据迁移能否回退，监控和告警是否存在，权限是否经过批准，未验证事项是否可接受，事故发生时谁负责接管。Verify 的任务就是把分散证据组织成可裁决结论。

读者可以把 Verify 理解为法庭上的“证据审理”，而不是流水线最后一个绿色按钮。测试报告是一组证物，约束结果、代码差异、回退演练、运行记录与人工审查也是证物。Evidence Bundle 负责组织本次任务事实，OA 检查证据链，IO 或授权责任人作出发布、灰度、补证、拒绝等裁决；Telemetry 在这些任务事实进入事件账本并聚合后，才用于跨任务趋势判断。

Verify 可以得出“测试全绿但暂缓上线”，也可以在明确残余风险、灰度范围和回退条件后作出有边界的批准。关键在于决定有证据支持，责任主体清楚。

15.2 六维验证

“六维验证”不是把测试重复六遍，而是从六种不同失败来源检查同一交付。单元测试只能覆盖其中一部分，任何一个维度缺失都可能产生一种特定的假成功。

验证体系包括：

1. **契约 AC 验证**：证明本次承诺的行为是否实现；
2. **Evidence Bundle 完整性审计**：检查关键结论是否都有证据；
3. **图谱、契约、约束一致性**：防止高质量完成了错误或过期目标；
4. **NFR 验证**：检查安全、可靠性、性能和可观测性；
5. **回滚安全验证**：确认失败时能够恢复，且不会破坏下游状态；
6. **工件时效与版本一致性**：确认测试、证据和签署对应当前代码与规则。

未知项必须标记为 UNKNOWN，不能被默认当作通过。

费用报销任务可能在第一维全部通过，却在第三维发现测试依据旧契约；也可能功能正确，却在第五维发现数据迁移不可回滚。六维验证让团队看见“功能正确”之外的发布条件。不同风险可以调整六维强度，但不能用“暂不检查”伪装成通过。低风险文案变更的回滚验证可能只是版本恢复；高风险资金规则则需要受控环境演练。

15.3 Evidence Bundle 的证据映射

理想证据包能够回答：

每条 AC → 哪项测试或运行证据
每条 MUST → 哪个约束检查
每个变更 → 属于哪个授权范围
每次失败 → 如何修复并重新验证
每项风险 → 由谁接受、怎样回退

证据来源优先级通常是：真实运行与机器结果高于模型自述，经过校准的人工判断高于未经解释的自动评分。

Evidence Bundle 不是把日志压缩成一个附件，而是把“事实—结论—决定”连接起来。最低限度应保留证据来源、产生时间、对应提交或制品、执行者、结果状态和对裁决的影响。例如 T-001 可以用下面的方式组织：

待证明事项	证据	状态	对裁决的影响
5000.00 元不触发总监审批	边界测试原始输出	PASS	支持人民币场景发布
5000.01 元触发成本中心总监审批	集成测试与审批链记录	PASS	支持人民币场景发布
无法找到总监时不得自动通过	异常路由测试	PASS	支持人工队列兜底
历史草稿不会被静默改写	迁移检查与样本演练	UNKNOWN	不批准历史草稿迁移
功能开关关闭后恢复旧路径	回退演练记录	PASS	支持灰度发布

这张表还要能追溯到具体文件、命令和版本。只有“测试通过”四个字，没有原始输出和对应版本，不能称为可审计证据。相反，UNKNOWN 也不是失败：它代表当前事实不足，并明确限制批准范围或触发补证动作。

15.4 收尾门：没有完整证据就不能宣告完成

收尾门机械检查：

- Evidence Bundle 存在；
- 单任务遥测存在；
- 项目遥测包含本任务；
- 仪表板已刷新；
- 意图图谱包含本任务和教训。

收尾门应机械核对 Evidence Bundle、裁决状态、任务级遥测入口和已批准回写是否齐全，并保留可追溯的检查结果。

未通过收尾门，任务不能标记完成。

收尾门要求遥测文件与 Dashboard 已生成，不是因为 Verify 负责分析长期趋势，而是因为一次裁决必须确认本次任务的真实信号已经进入外环。Verify 检查“遥测是否完整可信”，下一章再讨论“这些信号怎样被度量和解释”。

15.5 人类发布裁决

人类发布裁决不是让负责人在几十页日志末尾点击“同意”。有效裁决必须把技术事实翻译成可理解的选择：已经证明什么、尚未证明什么、残余风险是什么、影响范围多大、继续与停止分别有什么后果。

常见裁决至少包括六类：

- **批准**：证据充分，残余风险在授权范围内；
- **条件批准**：只允许灰度、限定时间或限定用户，并绑定监控与回退条件；
- **补充证据**：目标不变，但关键验证缺失；
- **拒绝**：当前结果不满足目标或风险不可接受；
- **撤回**：先前批准所依据的证据已经失效；
- **升级**：超出当前责任人的授权，需要安全、法务、财务或更高层级决定。

OA 可以把 Evidence Bundle 转成决策摘要，并给出推荐方案及理由，但不能替代责任人签署。IO 也不能只看 Agent 的自然语言总结，应能访问真正影响决定的原始证据。

裁决还应写明生效范围、有效期、责任人和撤回条件。例如“批准两个部门灰度 24 小时；错误率超过 0.5% 自动关闭功能开关；次日上午由财务负责人复核”。这样的决定比单一的“同意上线”更可执行，也更容易审计。

Verify 与其他控制面的边界也必须清楚：Prove 负责在实现前后建立证明路径，Evolve 负责根据失败修复并回写系统，Verify 负责独立审理本次交付能否被组织接受，Telemetry 负责把已裁决的运行事实用于跨任务分析。它们可以共享证据，但不能由同一个“全能 Agent”从生成实现一路自我验收。

15.6 发布裁决示例

T-001 的 Evidence Bundle 显示：

- 两条核心 AC 通过；
- 全量回归通过；
- 未触碰生产数据；
- 发生一次边界值失败并已修复；
- 外币报销未验证，已列为 Non-goal；
- 回退开关可用；
- 单任务遥测已采集。

合理裁决不是“功能完全没有风险”，而是：批准在境内人民币差旅范围发布，保持外币路径不变，并在发布后观察审批路由异常率。

15.7 证据如何提高人的判断效率

证据的作用不是替人做决定，而是减少人做决定时必须重新寻找和核对的成本。一个有效的 Verify 摘要应把信息压缩成可比较的判断单元：

已证明什么

+ 尚未证明什么

+ 哪些风险仍然存在

+ 风险影响谁、影响多久

+ 继续、补证、回退和停止分别意味着什么

这样，IO 不必阅读所有执行日志，也不会只接受 Agent 的结论。人把注意力放在价值与风险取舍上，Agent 和流水线负责整理事实、关联原始证据并指出矛盾。关键证据缺失时，最有效的做法不是加快批准，而是拒绝作出超出证据范围的判断。

15.8 Verify 常见失败

- 把单元测试全绿等同发布批准；
- UNKNOWN 被默认视为通过；
- 证据与当前提交、制品或契约版本不一致；
- 只展示最终结果，不展示失败与修复历史；
- 人类批准没有范围、条件、有效期和撤回条件；
- 由产生实现的同一 Agent 生成证据并自行宣布通过；
- 缺少回退、监控或责任人，却因为时间压力直接上线。

15.9 本章小结

Verify 不重复执行者的自检，也不替 Evolve 修复结果。它以独立视角审查契约、约束、证据、版本、UNKNOWN、回退与责任链，并把“批准、条件批准、补证、拒绝、撤回或升级”变成明确裁决。测试全绿只是重要证据之一，不是自动上线权。

本章最小实践：为 T-001 汇聚六维证据，运行收尾门，并形成一份带范围、期限、监控、回退和撤回条件的发布裁决。

完成证据：Evidence Bundle、收尾门输出与人类签署的裁决记录。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，用 Evidence Bundle 和人类裁决判断什么是真正完成。

第十六章 | 价值与效能遥测：证明 AI 转型真正兑现了提效

Verify 解决一次交付能否被批准，Telemetry 判断人机协同研发系统能否在多次交付中形成可重复的组织能力。二者共享契约 ID 与 Evidence 引用，但时间尺度和责任不同：前者面向单次行动，后者面向趋势、异常、净收益与组织改进。四层九维 Dashboard 用来判断长期提效是否真实、健康并可持续，而不只是展示 AI 使用量。

遥测不重新审理某一次发布，也不替代交付流水线。单次任务的事实由 Evidence Bundle 保存；遥测负责跨任务观察变化、识别反效果，并把信号转成组织动作。

本章导读：遥测的三层聚合边界

四层九维模型要与三层人机协同架构对齐。任务运行层采集执行、验证、Token、上下文和人工接管事实；价值网络层观察价值流吞吐、等待、返工、协作和能力变化；战略意图层只讨论跨周期的业务结果、风险暴露和是否继续投资。任何一层都必须保留来源、口径和失效状态，不能把任务层的 output 直接抬升为组织层的 outcome。

Dashboard 应支持下钻和裁决：先从组织趋势发现异常，再回到价值流和任务 Evidence 查明原因，而不以 Agent 数量、工具激活率或 Token 消耗给个人和团队排名。只有在相同任务类型与风险边界下，有效交付、等待和返工连续改善，同时质量、责任和恢复能力没有恶化，才能把结果归因于转型。

本章依次回答四个问题：遥测、证据、指标和看板分别服务什么判断；哪些指标能共同解释真实价值；怎样把单任务事实聚合成项目视图；发现异常后由谁采取什么行动。

16.1 从度量到遥测：Agentic Agile 的反馈控制系统

传统研发效能领域早已积累大量周期、吞吐、质量、稳定性和组织健康指标。AI Coding 并没有让这些度量失效，反而增加了一组过去很少被系统观察的问题：Agent 是否理解了正确目标？它经历多少轮尝试才完成？哪些错误被自主修复，哪些风险升级给人？消耗的 Token 对应什么价值？一次失败是否改善了下一次执行？

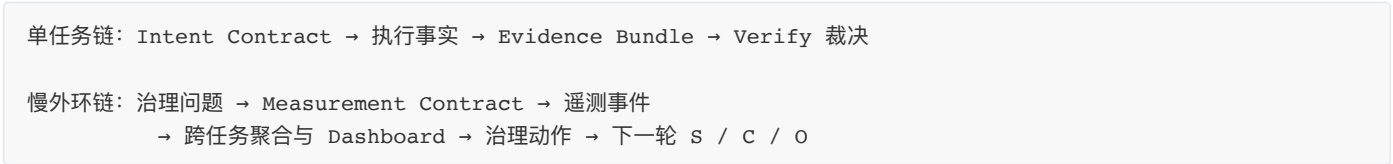
因此，Agentic Agile 需要的不是再造一套孤立指标，而是把传统研发效能与 AI 原生运行信号连接起来。前者观察研发系统交付得怎么样，后者解释 Agent 为什么得到这个结果、付出多少算力、触发什么边界，以及自治范围是否值得扩大。

16.1.1 度量纪律：从治理问题到可比较信号

这五个概念经常和管理讨论中被混成“数据”，但它们承担完全不同的治理责任。

概念	核心问题	例子
Measurement（度量）	我们想判断什么变化？	判断 Agent 是否在相同风险下提高有效交付能力
Metric（指标）	用什么口径表达这种变化？	首次成功率、逃逸缺陷率、每有效任务 Token
Telemetry（遥测）	如何持续、自动、低干扰地获得信号？	从测试、门禁、Agent 日志和客户端用量自动采集事件
Evidence（证据）	哪些事实足以支持本次裁决？	T-101 的测试、约束、回退、人工审查和生产结果
Dashboard（仪表板）	如何让人发现异常、下钻原因并采取行动？	单契约体检页和项目价值效能趋势视图

可以简化为两条共享任务事实、但服务不同时间尺度的链：



两条链通过 Contract ID、Evidence ID 和 Release ID 相互追溯，却不能互相替代。度量定义判断框架，遥测提供连续信号，Evidence 支持单次裁决，Dashboard 支持趋势判断。缺少任何一环，数据都可能沦为装饰。

从“产出多少”转向“有效交付能力”

代码行数、提交数、故事点、模型调用次数和生成代码占比都容易获得，却未必代表价值。AI 可以在几分钟内生成大量代码，也可以同时制造更多测试负担、架构耦合与维护成本。衡量 AI 研发不能只看产出量，而应沿着一条因果链观察：



Input 和 Output 容易测，Outcome 和 Impact 才接近价值。好的度量体系不会跳过中间因果关系，也不会把“生成更多”直接解释为“创造更多”。

把传统研发效能与 AI 原生信号放在同一坐标中

传统研发效能通常关注流动、质量、稳定性、体验和价值。Agentic Agile 在此基础上增加意图、自治、算力、证据与学习信号：

观察面	传统研发效能问题	AI 原生补充问题
流动	交付周期、等待、吞吐如何	Agent 重试、Graph 阻塞、Handoff 是否制造新等待
质量	缺陷、返工、稳定性如何	首次是否真正成功，Bug 是否回溯原契约
价值	是否交付客户和业务结果	Agent 是否正确理解了签署意图
成本	人力、平台和机会成本如何	Token、模型、验证 Agent 与治理成本如何归因
风险	安全、合规、发布风险如何	工具越权、MUST 失败和自治边界是否被发现
体验	工程师能否顺畅完成工作	人类是否被低价值 HITL、核对和提示词返工淹没
学习	团队是否减少重复错误	记忆、知识、约束、测试和 Skill 是否有效复用

3-4-3 不替代现有研发效能体系。例如，DORA 主要从部署频率、变更前置时间、变更失败和恢复能力观察软件交付表现；SPACE 从满意度、绩效、活动、沟通协作和效率流动等多个维度理解开发者生产力。3-4-3 提供的是一层契约级解释坐标：把这些结果连接到具体意图、Agent 行为、约束、证据和算力，而不是用一个新的“AI 总分”覆盖原有体系。

指标不能直接变成绩效目标

古德哈特法则常被概括为：**当一项度量成为目标，它就不再是一项好的度量。**人和系统会围绕指标优化，直到数字与原本想观察的现实脱钩。

与之相关的坎贝尔定律提醒我们：定量指标越被用于重要奖惩和决策，它越容易受到操纵压力，也越可能反过来腐蚀原本想改善的过程。

AI 研发尤其容易发生指标游戏：

- 为了降低 Token，过度裁剪上下文，导致返工和遗漏增加；
- 为了提高首次成功率，隐藏上线后的 Bug，或另开任务不回溯原契约；
- 为了降低 HITL，Agent 遇到价值冲突和不可逆操作也不再上报；
- 为了提高自愈率，降低约束、删除失败测试或无限重试；
- 为了制造“学习很多”的假象，把每次偶然问题都写成规则；
- 为了缩短交付周期，把验证、UAT 和运维成本推给下游；
- 为了证明 AI 覆盖率，优先挑选大量简单任务，回避真正有价值的复杂问题。

所以，指标的第一用途不是奖励，而是学习；第一评价对象是治理系统，而不是个人。只要指标直接绑定个人排名、奖金或强制目标，就必须预先分析它会诱发什么行为。

每个主指标都需要反指标

单一指标只展示一个切面。主指标描述希望改善的方向，反指标负责暴露这种改善是否通过转移成本或隐藏风险获得。

主指标	可能被怎样游戏	必须联读的反指标
首次成功率	隐藏 Bug、只选简单任务	逃逸缺陷、Bug 回溯率、任务复杂度
Token 效率	过度裁剪上下文	目标准确率、返工、执行轮次
自愈率	弱化门禁、无限重试	MUST 通过率、修复轮次、约束变更记录
HITL 升级率	风险不再上报	越权事件、事后人工返工、残余风险
交付周期	验证成本后移	UAT 否决、回滚、QA 和业务人工小时
知识沉淀与复用	制造规则垃圾	有效复用率、冲突率、过期率
复合 ROI	夸大节省工时	真实人工投入、失败折现、治理建设成本

当一个指标改善而反指标恶化时，不能宣布成功，而应检查系统是否发生局部最优。

为重要指标建立度量契约（Measurement Contract）

意图需要契约，重要指标同样需要。Measurement Contract（度量契约）不是第五个动态工件，而是指标目录中的结构化定义，用来防止不同团队用同一个名称表达不同含义。

```
metric_id: METRIC-FIRST-PASS-01
name: first_pass_success_rate
purpose: 判断任务是否第一次完整执行就达到签署契约
formula: 首次正确完成任务数 / 完成任务总数
unit: percentage
scope:
  task_types: [feature, bugfix]
  risk_levels: [low, medium]
window: 最近10个同类任务或30天
sources:
  - Intent Contract
  - Evidence Bundle
```

```
- Bug Backtrace
owner: OA
counter_metrics:
  - escaped_defect_rate
  - task_complexity
gaming_risks:
  - 删除失败任务
  - 将事后Bug另开任务
  - 只选择简单样本
data_quality:
  freshness: 24h
  estimated_allowed: false
action_rules:
  warning: 低于历史基线15个百分点
  response: 检查契约、上下文、测试和任务组合
```

一份合格度量契约至少说明：目的、定义、公式、分母、单位、范围、窗口、来源、责任人、反指标、数据质量、操纵风险和触发动作。没有这些字段，同一个“成功率”可能在不同会议中被解释成完全不同的东西。

基线、分群与分布比平均值更重要

“Token 降低 30%”只有在比较对象一致时才有意义。度量至少要控制：

- 任务类型与复杂度；
- 风险等级和验证强度；
- 项目、模块与技术栈；
- 使用的模型、工具和价格口径；
- 新项目还是既有大型系统；
- 时间窗口、样本量和是否包含失败任务。

不要只展示平均数。十个任务平均消耗 20 万 Token，可能是每个都接近 20 万，也可能是九个只用 5 万、一个失控到 155 万。中位数、分位数、最大值、离群任务和趋势往往比单一均值更有治理价值。

没有历史基线时，第一阶段的目标不是宣称提升，而是建立可信基线。没有分母时不谈比例，没有任务分群时不做横向排名，没有样本量和置信说明时不把偶然波动解释成趋势。

领先、滞后与护栏指标要形成组合

- **领先指标**提示过程可能走向哪里，例如契约歧义、上下文缺失、门禁失败和重试轮次；
- **滞后指标**说明最终发生了什么，例如逃逸缺陷、回滚、业务命中和实际成本；
- **护栏指标**规定即使主目标改善也不能突破的底线，例如高危安全事件、MUST 失败和 UAT 否决。

只看滞后指标，反馈太慢；只看领先指标，可能优化了过程代理而没有改善结果；没有护栏指标，速度和成本目标会吞噬质量与安全。三类指标需要共同进入遥测和裁决。

为什么证据包（Evidence Bundle）之后还需要遥测

Evidence Bundle 回答的是“这一次交付是否有足够证据支持某个行动”；Telemetry 回答的是“这套交付系统是否在多个任务中变得更准确、更自主、更经济、更会学习”。前者主要面向一次裁决，后者主要面向趋势判断和治理改进。

如果只有 Evidence Bundle，团队能判断 T-001 是否应该发布，却很难回答：最近十个契约的首次成功率是否下降，人工介入是否越来越多，Token 成本为什么突然翻倍，知识回写是否真实减少了重复错误。Telemetry 把单点证据连接成时间序列。

在 3-4-3 中，遥测不是项目结束后的统计，而是每个意图契约完成 Verify 后的强制动作。每份契约形成一次独立运行记录，再聚合到项目级数据中。这使度量与真实任务、真实契约、真实证据一一对应，而不是月底由管理者追问团队“这个月 AI 大概节省了多少时间”。

度量纪律建立后，组织才适合选择一组稳定而不过度膨胀的核心观察面。四层九维不是“AI 总分”，而是让价值、自治、效率和学习能够同时被解释的最小模型。

16.1.2 四层九维：观察人机系统是否形成能力

4 层模型从价值、能力、效率和进化四个角度观察 Agentic 交付。九个核心维度之外，Dashboard 还展示 HITL 升级率、MUST 通过率、失败折现率、门禁状态、测试和性能等支撑指标。这样既保持核心模型稳定，又不丢失任务裁决所需的工程细节。

Layer 1 价值层：AI 是否完成了值得完成的事情？

1. 目标准确率

2. 首次成功率

3. 复合 ROI

Layer 2 能力层：Agent 能在多大范围内可靠自治？

4. 约束自愈率

5. 自治能力画像

支撑：HITL 升级率、MUST 约束通过率

Layer 3 效率层：人、上下文、模型与编排是否高效？

6. 上下文压缩比

7. Token 效率

8. 执行效率

Layer 4 进化层：系统是否把经验变成下一次能力？

9. 知识沉淀与复用

第一层：价值层——AI 创造了多少真实价值

1. 目标准确率

目标准确率衡量被正确完成的任务占总分配任务的比例：

$$\text{目标准确率} = \text{正确完成任务数} \div \text{总分配任务数} \times 100\%$$

它不同于测试通过率。测试可能全部通过，但如果实现偏离签署目标，目标准确率仍应判低。这个指标要求每个“正确完成”都能回到 Intent Contract 和 Evidence Bundle，而不是由 Agent 自报。

2. 首次成功率

首次成功率衡量任务是否在第一次完整执行中就满足契约，而不是经过返工、Bug 回溯或多轮人工修正后才通过：

$$\text{首次成功率} = \text{首次即正确完成任务数} \div \text{总完成任务数} \times 100\%$$

它是连接质量与成本的重要指标。两个团队最终都完成十个任务，一个团队八个任务一次通过，另一个团队每个任务平均修复三轮，两者的全绿结果相同，算力、人力和交付风险却完全不同。

任务完成后发现 Bug，必须把该契约的首次成功状态修正为失败，并重新采集遥测。否则 Dashboard 会把发布后返工隐藏掉，变成宣传工具。

3. 复合 ROI

ROI 不能只用“节省工时 × 人力单价”减去模型账单。失败、返工、人工复核和治理建设都属于成本。3-4-3 引入失败折现，强调只有有效完成才能产生价值：

$$\begin{aligned} \text{复合 ROI} = & (\text{节省人力价值} \times (1 - \text{失败率}) - \text{AI 与治理成本}) \\ & \div \text{AI 与治理成本} \times 100\% \end{aligned}$$

对于无法可靠估算节省工时的任务，应显示 N/A，而不是编一个漂亮 ROI。ROI 适合观察相同口径下的趋势，不适合跨团队直接排名。

第二层：能力层——Agent 有多自主，而不是看起来有多自动

4. 约束自愈率

约束自愈率衡量发生约束失败后，Agent 在授权范围内自主修复并通过原检查的比例：

$$\text{约束自愈率} = \text{自主修复的约束失败数} \div \text{约束失败总数} \times 100\%$$

自愈率高不必然更好。如果 Agent 为了提高自愈率降低门禁、删除测试或无限重试，指标已经失真。只有保留原约束、原检查和修复证据的自愈才计入。

5. 自治能力画像

自主性不是“人类出现得越少越好”，也不适合压缩成一个通用分数。组织应把自愈率、MUST 通过率、HITL 原因、授权范围、重试次数与回退情况并列呈现，形成可解释的能力画像。

资金支付、生产发布或价值冲突触发 HITL，可能恰恰说明系统正确识别了责任边界。若重复性、低风险事项持续升级人类，才需要检查规则、上下文或工具边界；不可逆事项的人工批准不应被视为能力不足。

第三层：效率层——人机协作与算力使用是否合理

6. 上下文压缩比

上下文压缩比衡量从候选上下文到实际注入上下文的裁剪效果：

$$\text{上下文压缩比} = \text{裁剪前 Token} \div \text{裁剪后 Token}$$

压缩比越大不一定越好。极端裁剪可能遗漏关键依赖。它必须与目标准确率、首次成功率共同阅读：Token 下降而返工上升，说明裁剪过度；Token 下降且准确率稳定，才说明 Context Engineering 真正有效。

7. Token 效率

Token 效率通常表示每个完成任务消耗的 Token：

$$\text{Token 效率} = \text{Token 总消耗} \div \text{完成任务数}$$

它可以进一步拆分输入、输出、缓存、工具调用阶段或不同 Agent。单看总 Token 没有意义：复杂任务理应比文案修改消耗更多；增加独立验证也可能提高 Token，却显著降低风险。正确做法是在相似任务类型、相同质量门槛下比较趋势。

8. 执行效率

执行效率衡量每轮 Agent 执行完成多少有效任务：

$$\text{执行效率} = \text{正确完成任务数} \div \text{执行轮次}$$

它用来发现重复重试、编排阻塞和 Handoff 摩擦。执行轮次上升但完成任务不变，可能意味着上下文不清、测试反馈太晚、Graph 拆分不合理，或工具环境不稳定。

第四层：进化层——系统是否真的学会了

9. 知识沉淀与复用

不要用“本期新增模式 ÷ 累积模式”评价学习，那会奖励堆积条目。更可信的信号是：经过验证的模式是否被后续相似任务实际使用，是否减少重复失败或返工，以及过期规则是否得到合并、修正和撤回。

每条被计入的模式都必须能回到来源 Evidence、适用范围和 Owner。模型自动写的一段反思不是组织知识；只有进入测试、约束、图谱、记忆或 Playbook，并改变后续行动的经验才算沉淀。

四层九维提供的是一组解释坐标，不是九项彼此独立的 KPI。任何一项变化都必须与相关的质量、风险和成本信号一起阅读。

16.1.3 指标联读与反指标：避免局部最优

遥测最危险的用法是把某个指标变成单一目标。追求最低 Token 可能造成上下文不足；追求零 HITL 可能让 Agent 越过人类责任；追求最多知识条目可能制造规则垃圾；追求首次成功率可能诱导团队隐藏 Bug。

真正有价值的是指标之间的关系：

组合信号	可能解释	下一步检查
Token 下降，首次成功率稳定	上下文裁剪或编排改善	确认任务复杂度可比
Token 上升，HITL 与返工同时上升	意图、上下文或工具环境恶化	检查最近契约和失败路由
自愈率上升，MUST 通过率下降	可能通过弱化约束获得“自愈”	审查测试和约束变更
准确率稳定，复用模式始终缺席	当前完成但组织没有学习	检查回写、撤回与 Failure Mining
HITL 上升，事故下降	可能是系统更早识别高风险	区分必要与重复介入

这些指标不是用来评价个人，而是判断治理系统是否越来越准确、稳定、经济和会学习。

四层九维模型回答的是“持续观察什么、怎样联动解释”。要让这些观察进入运行系统，下一步需要把每次任务的真实事实固定为可下钻的治理快照。

16.2 一份契约一张仪表盘（Dashboard）：任务级治理快照

3-4-3 遥测的基本单位是意图契约。每个完成 Verify 的契约都应形成独立运行记录，并能从管理视图下钻到 Evidence Bundle；呈现方式可以是静态页面、BI 系统或现有工程平台，不应被某种文件结构绑死。

单契约 Dashboard 可以理解为一张“任务治理体检报告”。它围绕同一个任务 ID 展示：

- 契约目标与采集时间；
- 目标准确率和首次成功情况；
- 约束自愈、HITL 与 MUST 通过率；
- 上下文、Token 与执行轮次；
- 知识回写与进化结果；
- 测试、门禁、性能与成本；
- Evidence Bundle 和裁决状态。

Evidence Bundle 与单契约 Dashboard 不是重复。Evidence Bundle 保留细粒度证据和裁决依据；Dashboard 把关键指标与状态可视化，让 IO、OA 和管理者在几分钟内理解任务的价值、代价和治理质量，并能继续下钻到证据。

例如 T-002 显示目标准确率 100%、首次成功率 100%，但自主性只有 35/100、HITL 升级率 100%、每任务 Token 超过 80 万。结论不应只是“成功”，而是：结果正确，但自治能力和算力效率仍处于基线，需要检查为什么两轮执行都需要人工介入，以及上下文是否过大。

任务快照解决的是“这一份契约付出了什么代价、形成了什么结果”；只有把同类快照按统一口径聚合，组织才能判断这不是一次偶然成功，而是一种正在形成或退化的能力。

16.3 全局仪表盘（Dashboard）：从任务快照到项目价值与效能视图

项目级 Dashboard 聚合所有契约运行记录，并保留任务级下钻路径。

它不是把所有任务数字简单相加，而是提供三个层次的管理视图：

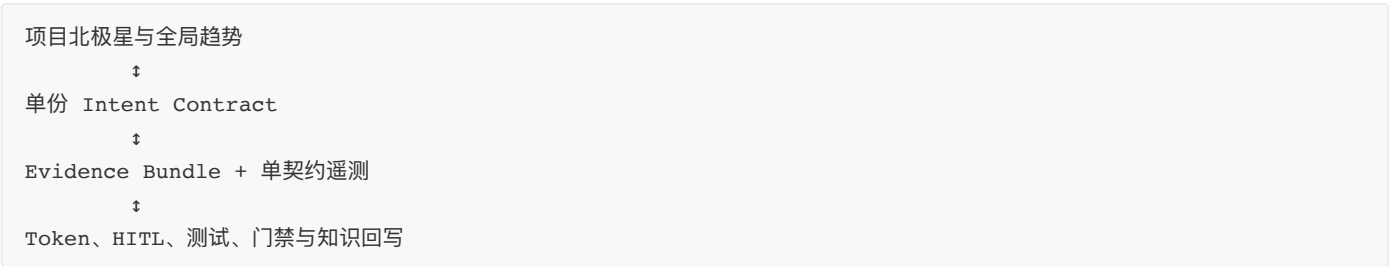
1. **项目全局状态**：当前治理成熟度、总体准确率、首次成功率、自主性、门禁与成本；
2. **历史契约列表**：每个 T-XXX 的采集时间和核心指标，可点击下钻；
3. **趋势与异常**：准确率、首次成功率、Token、HITL、知识沉淀等随时间如何变化。

双向导航让管理者从趋势进入现场，也让执行者从单次任务回到项目基线。静态站点、受控 BI 与工程平台都可以承载这种视图；关键是数据从真实执行和证据自动汇集，而不是为了汇报手工填数。

为什么双层视图不可缺少

传统研发看板往往以 Sprint、团队或项目为中心，任务完成后只留下状态和工时；AI 工具自身的用量页面又通常只展示 Token、请求次数和费用。两类看板彼此分离：一边不知道 AI 怎样工作，另一边不知道工作产生什么价值。

双层 Dashboard 通过契约 ID 把价值、行为、约束、算力、证据和学习连接起来：



这使组织第一次能够回答：“哪个业务意图消耗了多少算力，经过多少轮修复，触发哪些人工决策，最终产生了什么证据和价值？”

在这些聚合信号中，Token 与算力最容易被误读成单纯账单。要判断一笔算力投入是否值得，必须把它放回契约、风险、验证和有效结果的共同坐标中。

16.4 Token、算力成本与价值归因

Token 为什么是治理指标

Token 是模型处理文本和部分多模态信息时的基本计量单位。输入上下文、模型输出、缓存读写和不同模型的单价共同决定调用成本。对个人偶发使用，Token 可能只是账单数字；对多 Agent、长上下文和高频验证系统，它将成为真实的生产资源。

Agentic Agile 不主张一味压低 Token，而是要求 Token 可测、可归因、可解释。每次契约消耗多少 Token，应当能够关联到具体目标、执行轮次、失败修复和最终价值。

Token 成本怎样计算

最基本的成本模型是：

任务模型成本 =
输入 Token × 输入单价
+ 输出 Token × 输出单价
+ 缓存读写、工具或其他计费成本

实际计算必须使用对应模型和计费时点的价格。不同模型、输入输出、缓存和供应商价格差异很大，因此 Dashboard 应保存模型、Token 来源和价格口径，不能只保存一个美元结果。

实测优先，估算必须标记

Token 应优先来自模型网关、企业账单或本地客户端的可验证记录，并保存来源、时间与采集口径。无法获得实测数据时才允许估算，并明确标记 `estimated`；实测与估算不能在没有说明的情况下拼成一条趋势。

怎样读每任务 Token

“每任务 10 万 Token”本身既不代表好，也不代表坏。需要同时看：

- 任务复杂度和风险等级；
- 输入、输出与缓存 Token 构成；
- 执行轮次和失败修复次数；
- 上下文压缩比；
- 是否使用多个 Agent 或独立验证；
- 目标准确率、首次成功率和残余风险；
- 相似任务的历史基线。

高风险支付任务为了独立验证多消耗 Token 可能合理；低风险文案任务因反复读取整个仓库而消耗大量 Token，则暴露上下文和编排问题。

从 Token 成本走向价值归因

真正的管理问题不是“本月花了多少 Token”，而是“这些算力支持了哪些契约，产生了多少有效结果”。可以从三个层次观察：

1. **任务级**：T-001 的 Token、轮次、HITL、成功与证据；
2. **能力级**：哪类任务或模块的单位有效交付成本最高；
3. **项目级**：算力投入变化是否带来准确率、速度、质量或业务价值改善。

这也是“算力与价值遥测”的意义：算力不是孤立成本，价值也不能脱离失败和治理成本被夸大。两者必须在同一契约坐标系中比较。

但无论指标设计得多完整，若采集依靠事后回忆或手工美化，Dashboard 只会制造错觉。只有回到真实、可追溯的采集闭环，指标才具有解释力。

16.5 遥测真实性与采集闭环

遥测不能靠手填“漂亮数字”。

- 测试数来自真实测试运行；
- Token 用量优先使用本地客户端数据实测；
- 无法实测时必须标记为 estimated；
- 单任务遥测文件必须存在；
- 项目累计 run_count 必须更新；
- 仪表板必须刷新；
- 已知缺口必须进入 Evidence Bundle。

采集入口应由团队现有平台或脚本统一执行，并保留任务、提交、环境、采集时间和原始来源。工具链可以不同，但不能用手工补数替代可追溯采集。

可信数据不是终点。遥测只有在触发具体的检查、调整与复核时，才会从记录系统变成反馈控制系统。

16.6 遥测如何驱动动作

信号	可能原因	治理动作
目标准确率下降	意图或上下文漂移	检查图谱与契约质量
首次成功率下降	AC、架构或技术债问题	加强前置规格和关键模块治理
MUST 失败增加	约束不清或工具越权	检查权限和约束注入
HITL 激增	自治范围过大或规则冲突	收缩权限、改善决策入口
Token 上升	重试、上下文或编排低效	裁剪上下文、优化 Graph
知识沉淀停滞	反思没有回写	检查 Evolve 和 closing gate

这张表不是自动处方：同一信号可能由任务复杂度、风险提高或控制不足造成。动作前仍要回到任务分群、Evidence 和反指标下钻；否则看似合理的动作，可能恰好放大问题。

16.7 遥测常见失败

- 遥测手工估算却未标记，或测试、Token 和耗时没有真实来源；
- 只有项目总览，没有单契约遥测，无法定位异常来源；
- 只有 Token 账单，没有对应的契约、结果、证据和价值；
- 实测与估算 Token 混用，却没有标记口径和生效时间；
- Dashboard 为了汇报被手工美化，与原始遥测文件不一致；
- 把 HITL 越少、Token 越低直接等同治理越成熟；
- 完成后 Bug 未回溯原契约，导致首次成功率长期虚高；
- 指标只有红绿颜色，没有 Owner、阈值、诊断路径和治理动作。

这些失败共同说明：遥测既需要工程上的真实来源，也需要治理上的解释纪律。否则数字会反过来替代判断。

16.8 遥测治理：让数字保持可解释

指标需要度量契约、基线、反指标、真实来源和行动规则；其中仍有三条治理纪律不能被 Dashboard 自动替代。

第一，遥测首先服务学习，不服务个人排名。默认评价对象是契约质量、上下文、约束、编排、验证、工具和组织环境；Token、代码量、Agent 使用率或最低人工介入率都不能单独给个人贴标签。否则成员会拆小任务、延迟上报缺陷或压低必要 HITL，数字更漂亮，系统反而更脆弱。

第二，指标也有生命周期：

提出治理问题

→ 定义度量契约

→ 试采集与校准

→ 正式发布

→ 观察使用与操纵

→ 修订、合并、停用或归档

业务目标、模型、任务组合或工具链变化时，指标口径可能失效。修改公式、数据源或价格口径必须保留版本和生效时间；数据质量不足时，Dashboard 应显示 UNKNOWN、STALE 或 ESTIMATED，而不是补成绿色。

第三，评审只处理少量异常和改进假设：发生了什么，数据可靠吗，先下钻哪些契约与证据，要改变哪个治理部件，何时复核是否有效。治理委员会关注跨任务趋势、重大例外和自治范围；OA 关注契约、编排、验证与成本；IO 关注价值命中和风险取舍；AS 执行被批准的调整。组织也应定期检查度量体系本身：长期无人查看的指标、无行动的告警、频繁手工修正的数据和无法追溯的趋势，都应被修订或淘汰。

16.9 本章小结

度量定义组织想判断什么，Telemetry 持续采集真实信号，双层 Dashboard 让管理者从项目趋势下钻到单份契约、证据与算力现场。古德哈特法则提醒团队：核心指标必须具有度量契约、比较基线、任务分群、反指标、护栏、数据质量和行动责任。遥测的价值不是证明 AI 很成功，而是让治理假设能够被数据证伪并持续修正。

本章最小实践：为一个核心指标建立 Measurement Contract，采集 T-001 单契约遥测并更新全局 Dashboard，再基于一个异常信号提出可验证的治理动作。

完成证据：度量契约、单任务遥测、全局仪表板、解释记录和带验证窗口的改进行动。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，学习如何区分 Token、活动量、产出与真实结果。

第十七章 | AI 原生交付流水线：让每次发布携带意图、证据与回滚能力

意图已经签署，约束已经建立，Agent 也完成了测试和证据汇聚，仍然不能直接推导出“可以发布”。门禁在哪里执行？多个 Agent 的修改如何隔离？通过测试的提交与最终制品是不是同一个？契约、代码、证据和版本怎样对应？Agent 是否有权合并主干或触达生产？这些问题最终都要落到 CI/CD、DevOps、分支和版本管理中。

核心判断是：

CI/CD 负责把代码安全地送到生产；3-4-3 负责证明这份代码为什么有资格被送到生产。

17.1 DevOps 底座与 3-4-3 治理

DevOps 解决开发、交付、运行与反馈之间的协作和自动化；CI 持续集成代码并执行构建与验证；CD 让合格制品持续交付或部署；Harness 把权限、约束、门禁、观测和恢复变成 Agent 无法随意绕过的运行环境；SCOPE-V 则控制一项意图如何从规格走向证据和裁决。

它们处在不同层次，不能互相替代：

Intent Contract：为什么做、做到什么程度
Constraint Matrix：什么边界不可突破
Work Graph：任务、测试与集成怎样编排
CI Pipeline：构建、测试、检查并采集证据
Evidence Bundle：为什么可以作出当前裁决
CD Pipeline：制品怎样晋级、灰度、部署和回退
Telemetry：上线后是否产生真实价值和异常

没有 DevOps 底座，3-4-3 的可执行门禁很难规模化；只有 DevOps 而没有 3-4-3，流水线可以高效发布代码，却未必知道代码对应哪项业务意图、风险由谁接受、证据是否足以支持上线。

Agentic Agile 要把契约、约束、证据和遥测嵌入企业已有的 Git、CI/CD、制品库、发布平台和可观测系统，而不是在现有流水线之外再搭建一条平行流程。

17.2 为什么 AI 流水线必须前移到 Commit 之前

传统流水线通常从代码提交开始：

Commit → Build → Test → Scan → Deploy

但 AI Coding 的许多风险发生在 Commit 之前：Agent 是否读取了正确版本的契约，是否获得了过大的数据和工具权限，是否修改了未授权文件，是否擅自增加依赖，测试是否在实现前产生，多个 Agent 是否正在覆盖彼此的工作，以及 Token、时间与重试是否已经超过预算。

AI 原生交付流水线需要向前延伸：

意图签署
→ 上下文、权限与基线校验
→ 分支或 Worktree 隔离
→ TDD Red
→ Agent 实现与增量约束检查
→ 多层验证与集成
→ Evidence Bundle 与独立 Verify
→ 人类发布裁决
→ 不可变制品晋级与部署
→ 生产遥测和 Bug 回溯

这里的“流水线”不再只是 CI 服务器中的几个 Job，而是从 Agent 获得任务到生产反馈回流的完整受控通道。Commit 只是其中一个事件，不是治理起点。

17.3 隔离与汇合：发布前先保证证据仍然有效

每个独立执行节点都要绑定任务、契约、起始基线、允许写入范围、上游版本和对应证据。提交进入稳定基线前，必须在汇合后的当前制品上重新验证；Git 没有文本冲突，不能证明业务语义没有冲突。

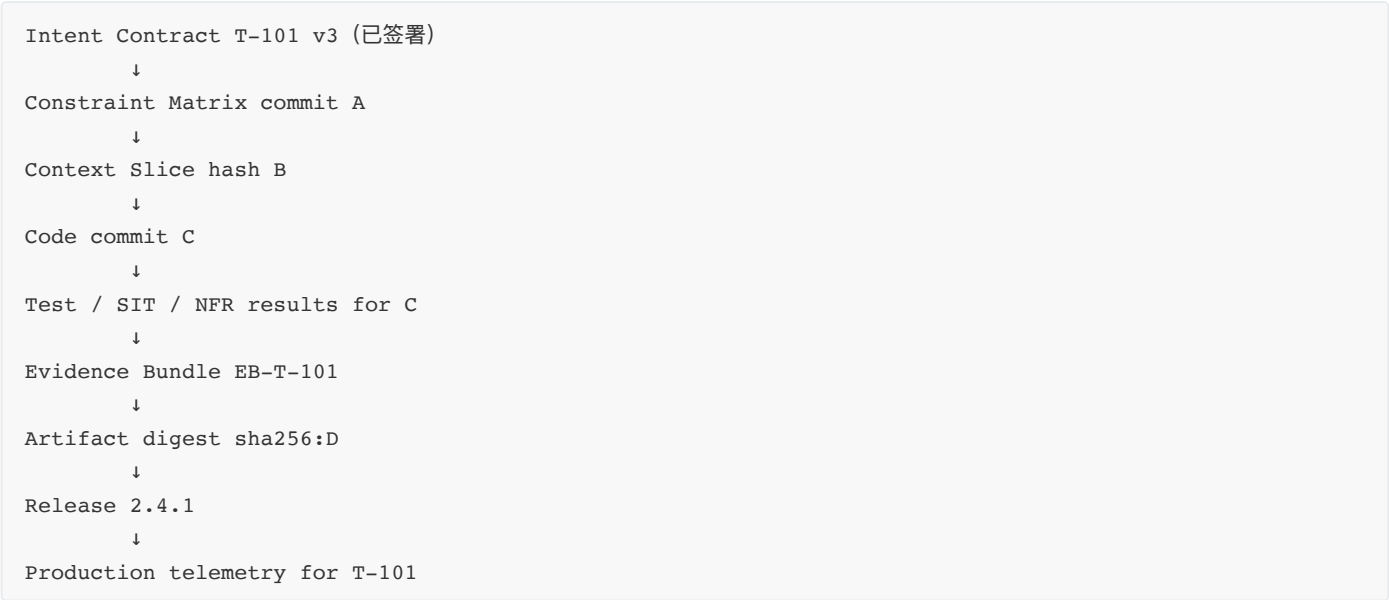
多数团队可采用受保护主干、短生命周期分支或 Worktree、自动门禁与 Merge Queue。跨模块所有权、共享资源、分支策略和集成 DAG 属于规模化协作问题，见第二十七章。

17.4 从代码版本升级为交付谱系

传统版本管理主要关注 Commit、Tag、软件版本号和发布制品。AI Coding 时代还要管理影响结果的认知与治理资产：

- Intent Graph 的版本；
- Intent Contract 的版本和签署状态；
- Constraint Matrix 与 Tools Manifest 的版本；
- Work Graph、Context Slice 和 Handoff 版本；
- Agent、模型、Prompt 和 Skill 版本；
- 测试、Golden Set、数据集和 Schema 版本；
- Evidence Bundle 与发布裁决版本；
- Artifact Digest、配置和部署环境版本；
- 契约级遥测及生产事件。

这些版本共同形成 **Delivery Lineage**（交付谱系）：



交付谱系回答的不是“当前版本号是多少”，而是：生产环境中的这份制品，依据哪份意图，由哪些 Agent 在什么约束下生成，经过哪些真实验证，由谁接受风险，并在上线后表现如何。

如果测试报告对应 Commit C，Evidence Bundle 却没有记录版本；如果发布平台从最新主干重新构建了 Commit D；或者契约签署后约束矩阵已经发生重大变化，证据链就已经断裂。

17.5 建立 Release Manifest

Evidence Bundle 面向人类解释“为什么可以裁决”，还可以增加一份机器可读的 Release Manifest，固定发布对象的谱系关系：

```
release_id: R-2.4.1
task_id: T-101
contract:
  path: governance/contracts/Intent_Contract_T-101.md
  version: 3
  signed_at: 2026-08-07T10:30:00+08:00
governance:
  constraints_commit: a1b2c3d
  work_graph_hash: sha256:...
source:
  repository: expense-service
  commit: c4d5e6f
artifact:
  image: registry.example.com/expense:2.4.1
  digest: sha256:...
evidence:
  bundle: governance/evidence/EB-T-101.md
  verdict: CONDITIONALLY_APPROVED
  approved_by: Finance-IO
deployment:
  strategy: canary
  initial_traffic: 10%
  rollback_to: sha256:previous...
```

这份清单不一定采用相同字段，但至少应绑定契约、源码、治理基线、证据裁决、不可变制品、部署策略和回退对象。CI 可以自动生成大部分字段，人类只负责不能自动推导的风险接受和发布条件。

17.6 CI 如何承载五道机械门

五道门不是要求所有检查都放在同一个 CI Job，而是说明不同时间点必须有什么事实。它们可以映射到本地 Harness、Pull Request、Merge Queue、发布流水线和事后回溯：

门	典型执行位置	CI/CD 要证明什么
Pre 前置门	Agent 启动前、本地或任务编排器	契约真实签署、约束与 AC 已准备
Coding 编码门	分支首次实现前	测试先写并产生可信 Red
Prove 验证门	PR 与候选合并流水线	测试、AC、构建、NFR 与证据通过
Closing 收尾门	发布候选生成前	Evidence Bundle、遥测、Dashboard 和图谱回写完整
Bug 回溯门	生产缺陷处理流水线	原契约、证据、首次成功率和知识已知实修正

Pre 与 Coding 发生在 Commit 之前，因此不能只依赖远程 CI；它们可以由本地 Hook、Agent Harness 或任务平台机械执行。Prove 适合在 PR 和 Merge Queue 中运行。Closing 连接验证结果、发布裁决与制品。Bug 门则由事故或缺陷事件触发。

门禁应满足三条工程要求：

- 1. **不可静默绕过**：跳过、允许失败或管理员强制合并必须形成例外记录；
- 2. **结果可消费**：使用结构化输出、稳定退出码和原始日志，而不是只显示一个绿色图标；
- 3. **结论可追溯**：每项结果绑定 Task ID、Commit、环境和时间，并进入 Evidence Bundle。

17.7 Build Once, Verify Once, Promote Many

传统 DevOps 强调 Build Once, Deploy Many：代码构建一次，产生不可变制品，同一制品逐级部署，避免不同环境重复构建出不同结果。在 3-4-3 中还要再强调一步：被验证和裁决的必须是最终晋级的那一份制品。

```
Source Commit C
    ↓ 一次构建
Artifact Digest D
    ↓
测试 / 扫描 / SIT / 性能
    ↓
Evidence Bundle 绑定 D
    ↓
Verify 与发布裁决
    ↓
D 晋级 UAT / 预生产 / 生产
```

不能出现“Commit C 通过测试，Agent 随后又提交 D，发布平台从最新代码重新构建”的情况；也不能在 UAT 环境测试镜像 A，却在生产重新构建镜像 B。版本号或镜像标签可能被覆盖，Artifact Digest 才是更可靠的不可变身份。

因此应遵循：

- CI 只从明确 Commit 构建；
- 制品进入受控 Registry，禁止覆盖同一 Digest；
- Evidence Bundle 和 Release Manifest 记录 Digest；
- 环境晋级使用同一制品，只改变经过治理的环境配置；
- 部署后再次校验运行中的 Digest；
- 若源码、依赖或制品变化，原裁决自动失效并重新验证。

这条规则可以概括为：**验证过的就是发布的，发布的就是被裁决的。**

17.8 配置与非代码资产同样属于交付物

Tag 和版本号不是证据。配置、Feature Flag、Schema、基础设施以及模型、Prompt、Skill、工具和 Golden Set 都可能改变生产行为，必须与源码和制品一起进入 Delivery Lineage；破坏性接口变更还要进入跨模块契约与迁移治理。

17.9 CD 中的人机权限边界

AI 可以在交付流水线中承担大量工作：创建任务分支和 Worktree，提交代码，创建 Pull Request，运行 CI，分析失败，在预算内自动修复，生成发布说明，部署测试环境，准备灰度计划，观察指标并汇总证据。

但自动化能力不能等同于授权。以下行为通常需要明确批准：

- 合并到受保护主干；
- 使用或扩大生产凭证；
- 修改生产配置和安全策略；
- 执行破坏性数据库迁移；
- 跳过或降低 MUST 门禁；
- 扩大灰度流量或全量发布；
- 删除生产数据或执行不可逆补偿；
- 接受高风险例外；
- 撤回或覆盖已有发布裁决。

一个实用原则是：

AI 可以准备发布
AI 可以证明发布条件
AI 可以执行已授权、可恢复的步骤
人类负责价值判断、风险接受和不可逆裁决

并非每次部署都要让高管点击按钮。团队可以通过风险分级进行预授权：低风险、可逆、证据完整的变更由 CD 自动晋级；资金、隐私、权限和不可逆数据变更触发 HITL；任何 MUST 失败都不能进入自动发布。

17.10 灰度、Feature Flag 与条件批准

发布不是只有“上线”和“不上线”两个选项。Evidence Bundle 可能支持条件批准：允许指定用户、地区、部门、时间或流量范围内发布，并绑定观察指标和撤回条件。

Feature Flag、金丝雀、蓝绿和影子流量是实现这种治理决定的工程手段：

- Feature Flag 将代码部署与功能启用分离；
- 金丝雀逐步扩大真实流量；
- 蓝绿发布保留可快速切换的旧环境；
- 影子流量在不影响用户的情况下比较新旧行为。

这些手段并不会自动降低风险。Feature Flag 如果没有负责人和失效日期，会变成长期配置债务；金丝雀如果没有明确成功指标，只是把事故延迟发生；蓝绿如果数据库迁移不可逆，也无法真正回切。

条件批准至少要写明：范围、持续时间、观察指标、阈值、负责人、扩大条件、停止条件和回退目标。CD 把这些条件转成自动步骤，Dashboard 显示实时状态，人类不必在每个正常步骤中介入，只在边界被触发时裁决。

17.11 回滚不只是重新部署旧版本

应用制品回滚通常容易，数据、消息、外部副作用和跨模块状态却可能已经向前推进。一个可靠回退计划至少要分析：

- 下游节点是否已经消费新事件；
- 数据库迁移是否可逆或需要补偿；
- 新旧版本能否同时读取当前 Schema；
- 缓存、队列和搜索索引是否需要恢复；
- Feature Flag 关闭后是否停止新行为；
- 已发生的资金、通知和外部调用如何处理；
- 回退是否影响同一并行组的其他任务。

因此，CD 的回滚动作应受 Work Graph 和 `verify_rollback_safety` 一类检查约束。Agent 可以分析依赖、生成命令和在沙箱演练，但高风险生产回退仍需要授权。Evidence Bundle 中的“可回滚”应来自真实演练或可执行验证，不应只是一句计划。

17.12 从发布版本回到契约级遥测

发布完成后，生产观测必须能够从 Artifact Digest 或 Release ID 回到 Contract ID。否则错误率上升时，团队只能看到“2.4.1 有问题”，却不知道它改变了哪项业务意图、由谁裁决、当时接受了什么风险。

理想链路是：

生产事件 / 指标
→ Release ID 与 Artifact Digest
→ Source Commit
→ Contract ID 与 Evidence Bundle
→ 对应 AC、约束和发布条件
→ Bug 回溯、遥测修正和图谱学习

单契约 Dashboard 可以显示该任务发布后的错误率、性能、业务命中和异常审批比例；全局 Dashboard 则观察不同版本、团队、模型或分支策略对首次成功率、HITL、Token、部署失败和恢复时间的影响。

这里仍需避免传统 DevOps 指标崇拜。部署频率更高不一定代表价值更高，变更前置时间更短也不代表风险更低。DORA 类工程指标可以与 3-4-3 的目标准确率、首次成功率、证据完整度、Token 效率和经验复用信号联动阅读，而不是单独追求某个数字。

17.13 最小交付底线

无论团队规模如何，最终代码必须对应签署意图，验证结果必须对应唯一版本，发布制品必须与被裁决制品一致。团队规模扩大后再增加分支隔离、跨模块契约、集成 DAG、签名制品和多级审批；不要先复制大企业流程，再寻找实际风险。

17.14 常见失败

- Agent 直接在共享主干写入，多个任务的修改和证据混在一起；
- 分支存在很久，契约和主干已变化却没有重新裁剪上下文；
- PR 单独全绿，合并后不在最新基线重新验证；
- AI 自动解决语义冲突，没有重新运行组合 AC；
- 测试报告没有 Commit、环境和时间；
- Evidence Bundle 记录源码版本，却没有记录 Artifact Digest；
- UAT 测试的是制品 A，生产重新构建并部署制品 B；
- 使用可覆盖的 `latest` 标签作为发布身份；
- Agent 可以关闭分支保护或将门禁设为允许失败；
- Release Note 由模型生成，却与真实契约和变更不一致；
- Feature Flag 没有负责人、到期时间和清理计划；
- 把“重新部署旧镜像”误认为完整数据回滚；
- 发布指标无法关联 Contract ID，生产缺陷成为无归属事件；
- 为追求部署频率，把必要证据和 HITL 视为效率障碍。

17.15 本章小结

AI 原生交付流水线把契约、分支、Commit、测试、Evidence Bundle、Artifact Digest、发布裁决和生产遥测连接成完整交付谱系。CI/CD 负责把合格制品安全送向生产，3-4-3 负责证明该制品为什么有资格被交付。受保护主干、隔离工作区、机械门、Build Once、条件批准、灰度和可验证回退，共同确保 Agent 不能绕过责任与证据直接发布。

本章最小实践：把一个真实契约接入现有 CI/CD，生成绑定 Commit 与 Artifact Digest 的发布证据。

完成证据：Release Manifest、Evidence Bundle、门禁记录、发布裁决、灰度与回退验证。

延伸实践：推荐学习《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》，把意图、证据和回滚能力带进 AI 原生交付流水线。

第十八章 | 端到端案例：从一句话需求到可验证发布

前文分别介绍了 SCOPE-V 的各个控制面。本章把它们放进同一个真实任务，展示一条责任链如何从澄清开始，经过受控执行、证据裁决和灰度发布，最终回流为下一轮能力。

案例是费用报销规则变更：**中国区差旅报销单笔含税人民币金额超过 5000 元时，需要增加成本中心总监审批。**业务负责人希望第二天上线。

报销业务只是载体。这里要观察的是一项任务如何经历澄清、授权、执行、证明、裁决、发布和回流；换成库存、客服、支付或内部平台任务，业务规则会变化，控制结构仍然适用。

18.1 任务入口：一句话不等于意图

原始需求至少存在以下歧义：

- “超过”是否意味着严格大于，5000.00 元如何处理？
- 金额按原币、含税金额还是人民币本位币判断？
- 总监是直属总监、成本中心总监还是财务总监？
- 历史草稿、审批中单据是否重新计算？
- 总监缺席或存在多个代理时怎么办？
- 明天上线是全量发布，还是限定部门的灰度发布？

如果 Agent 直接开始修改代码，它必然会替某一种解释作出选择。速度只会让未经签署的选择更快进入系统。

18.2 Specify：形成可签署契约

IO 与 OA 经过澄清后形成以下意图：

项目	本次决定
适用范围	中国区差旅报销
金额口径	含税人民币本位币总额
边界行为	5000.00 元不触发，5000.01 元触发
审批人	申请人所属成本中心总监
历史数据	不改写规则启用前已有草稿和已提交单据
异常处理	无法解析总监或代理冲突时进入人工财务队列，不得自动通过
发布方式	首日只对两个试点部门启用功能开关

关键 AC：

- 5000.00 元提交后不增加总监审批节点；
- 5000.01 元提交后增加成本中心总监节点；
- 无法找到总监时进入人工财务队列；
- 旧草稿再次保存时审批链不被静默改写；
- 关闭功能开关后，新提交单据恢复旧路径。

IO 签署目标、非目标、风险和 AC。OA 可以起草和追问，但不能代替 IO 签署。

18.3 Constrain：把底线写进执行环境

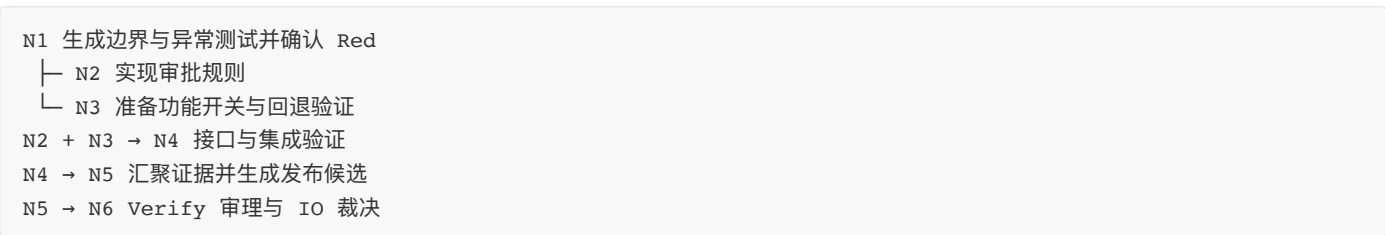
本次任务的约束包括：

- 禁止修改已审批单据；
- 禁止读取或输出生产个人报销明细；
- 禁止 Agent 获取生产发布凭证；
- 原则上不得引入数据库迁移；
- 单次自动修复最多三轮；
- 任一关键 AC 失败时不得生成可发布结论；
- 功能开关、监控、回退步骤和负责人必须在发布前验证；
- Evidence Bundle 必须绑定当前提交、制品和契约版本。

能机械执行的规则进入 Harness、CI 或 Policy as Code；不能机械判断的规则绑定责任人和升级路径。约束不是提醒语，而是 Agent 行动空间的一部分。

18.4 Orchestrate：组织有依赖的工作

OA 建立最小 Work Graph：



每个节点只获得完成任务所需的 Context Slice。实现节点不能修改契约，证据节点不能只读取执行 Agent 的自然语言总结。若发现旧系统使用 `>=`，节点必须停止并报告规则冲突，不能自行把旧行为伪装成新契约。

18.5 Prove：先证明实现满足契约

测试 Agent 先提交 5000.00、5000.01、缺失总监和历史草稿等测试，并在新规则尚未实现时运行。预期 Red 必须来自缺少目标行为，而不是环境损坏或断言错误。

实现 Agent 只完成使这些测试收敛的最小改动。随后运行领域单元测试、审批链集成测试、功能开关回退测试、权限检查和相关全量回归。Prove 在这里回答一个明确问题：

当前实现是否满足已签署的契约和约束？

Prove 的输出是可复查的实现证据：测试原始输出、代码差异、构建结果、失败修复历史和版本标识。它不负责决定是否上线，也不负责证明长期业务价值。

18.6 Evolve：把失败变成下一轮能力

第一次集成测试发现代理总监配置可能返回两个候选人。AS 没有权限决定优先级，于是停止并提交候选人列表、输入数据、当前规则和两个方案。

IO 决定优先使用当前生效代理；多个代理并存时转人工财务队列。契约和 AC 更新后重新签署，AS 再完成实现和验证。这个教训被写回约束矩阵和意图图谱，下一次涉及代理关系的任务会主动提出同类问题。

Evolve 不等于“自动重试直到绿色”。它负责分类失败、控制影响、修复根因，并决定哪些经验有资格进入组织记忆。

18.7 Verify：判断当前证据支持什么行动

Verify 审查本次 Evidence Bundle：

事实	状态	裁决影响
边界 AC 与异常路由通过	PASS	支持人民币场景
功能开关关闭路径通过	PASS	支持灰度发布
未触碰生产数据和凭证	PASS	风险在授权范围内
历史草稿迁移行为未验证	UNKNOWN	不批准历史草稿迁移
代理冲突规则已重新签署	PASS	支持当前异常路由

Verify 不是再次执行所有测试，而是判断证据是否足以支持某个行动。IO 最终裁决：批准两个部门灰度 24 小时；异常率超过阈值自动关闭功能开关；次日由财务负责人复核后，再决定是否扩大范围。

18.8 流水线：把已经证明的能力送到交付系统

流水线不重新发明 SCOPE-V，而是承载它：

契约与约束版本

→ 隔离分支与工作区

→ Prove 验证门

→ 不可变制品

→ Evidence Bundle

→ Verify 裁决

→ 灰度发布

→ 生产遥测与回退

被 Verify 审理的是最终要发布的 Artifact Digest，而不是某个后来又被修改的分支。流水线可以自动执行构建、测试、证据收集和低风险晋级，但不能用 CI 全绿替代 IO 的风险接受。

18.9 发布后：Telemetry 让组织继续学习

灰度期间，一张外币报销单被错误地按原币数值判断。系统随即关闭功能开关，记录错误率、影响范围、发布版本和回退动作。

Telemetry 在这里不回答“这次能不能发布”，而回答：

- 规则错误是否重复出现？
- 灰度后的异常率是否高于基线？
- 这类任务的返工成本是否下降？
- 哪些约束、测试或上下文应该被改进？

根因不只是测试样本全部为人民币。更关键的是，Verify 批准的是“人民币场景灰度”，流水线却只按部门限制流量，没有把币种条件机械地写进 Feature Flag 或发布门禁。换言之，裁决条件没有完整变成执行条件。修复因此不能只增加外币测试，还要补充外币 AC、强化字段类型、更新约束和灰度规则，并修正原 Evidence Bundle 与 Dashboard 中“首次成功”的记录。

18.10 案例结论

这次任务的成功，不是 Agent 在一天内改完了代码，而是团队能够说明：

- 目标由谁签署；
- Agent 被允许做什么、禁止做什么；
- 实现如何满足契约；

- 当前证据支持发布到哪里；
- 什么仍然未知；
- 出现问题时谁能停止和回退；
- 失败如何进入下一轮组织能力。

这就是一个可信闭环。它既适用于新项目，也适用于既有大型系统中的局部变更；区别只在于既有系统需要额外进行 Recon、Baseline、Preserve 和 Change Envelope，而不是改变闭环本身。

这个案例如何增强人的判断

如果只看代码变更，Agent 似乎完成了大部分工作；如果看完整责任链，人和 Agent 的分工更加清楚：

环节	Agent 提供的增幅	人承担的判断
意图澄清	枚举歧义、边界、依赖和反例	选择业务口径和本次范围
方案执行	定位代码、修改实现、运行测试	判断方案是否值得采用
结果验证	汇总失败、修复、测试和运行信号	判断证据是否足以支持行动
发布反馈	监测异常并触发回退	决定扩大、收缩或重新定义任务

本案例的效率提升，不在于“让人少看一次结果”，而在于更早暴露真正需要判断的分歧，并让每个判断都带有范围、证据和后果。人负责意图与判断，Agent 负责被授权的执行与反馈，组织负责把一次判断沉淀为下一次可以复用的能力。

本章最小实践：选择一个真实任务，用同一张表记录意图、约束、Work Graph、Prove 证据、Verify 裁决和发布后遥测。

完成证据：一份完整任务记录，能够从需求追溯到制品、裁决和生产反馈。

第四部分 | 四维组织转型：把工程能力变成企业运行方式

第三部分解决“一项任务怎样可信地运行”；第四部分继续追问：企业怎样把这种能力变成自己的运行方式？讨论从任务运行层开始，逐步进入价值网络、管理制度、战略意图与投资组合，回答工程能力如何穿过组织边界并形成可持续的企业能力。

本部分阅读路径

- 1. 第十九章 | 人员与组织重构：人与 Agent 分别承担什么责任，决策权怎样配置？
- 2. 第二十章 | 流程与工具重构：价值流怎样从岗位交接改为受控状态流？
- 3. 第二十一章 | 三层四维协同落地：怎样用一条价值流，在 90 天内证明或证伪转型？
- 4. 第二十二章 | 企业转型路线：怎样从任务运行闭环扩展到价值网络，再进入战略意图与投资组合？
- 5. 第二十三章 | 管理与评价体系：管理者如何运营授权、能力、资产、激励与信任？
- 6. 第二十四章 | 风险与行业裁剪：不同行业 and 风险等级，哪些边界与证据必须加重？

读完后，你应能把一个工程闭环翻译成企业决策：先在哪条价值流试点，怎样组成价值网络，哪些约束向下继承，哪些证据向上聚合，何时扩展，何时收缩，以及谁为每一层决定负责。

第十九章 | 人员与组织重构：从岗位团队到人机责任网络

AI 转型首先改变的不是组织架构图，而是价值流中的任务分配、判断权和责任归属。把 Copilot 发给所有人，仍按原岗位、原交接和原审批运行，只会得到局部提速；真正的重构要回答：哪些执行交给 Agent，哪些判断必须由人承担，谁可以授权，谁必须停止系统，谁对最终结果负责。

19.1 重构单位是任务组合，不是岗位名称

岗位由多种任务组成：信息收集、方案生成、编码、验证、协调、价值判断、风险承担和对外承诺。AI 对这些任务的适配度不同，因此“某岗位是否会消失”不是一个可执行问题。组织应逐项判断：

任务类型	默认责任安排
高频、可逆、可自动验证的执行	优先交给 AS，在 Harness 内自治
规则明确但影响较高的执行	Agent 执行，独立证据与人类裁决
目标、优先级和价值取舍	IO 负责，Agent 只能提供分析与选项
权限例外、不可逆操作、法律伦理判断	具备资格的人类责任人批准
失败分类、编排和控制系统设计	OA 负责，专业人员与 Agent 共同参与

提效来自任务重新组合：机器承担更多高速执行，人类集中在意图、边界、异常和责任判断。若只是要求同一批人在原流程中多使用一个工具，组织没有更换操作系统，只是增加了一个输入设备。

任务重新组合之后，新的问题随即出现：这些责任由谁承担？IO、OA、AS 正是对三类责任的抽象，而不是组织里新增的三个岗位。

19.2 IO、OA、AS 不是三个新职位

IO、OA、AS 是三种不可混淆的责任，而不是强制新增三个岗位：

- **IO** 对价值意图、非目标、关键 AC、风险偏好和最终签署负责；
- **OA** 对 Context、Graph、Harness、验证策略、异常路由和证据完整性负责；
- **AS** 由专业 Agent、自动化工具和参与执行的工程人员共同构成，在授权边界内交付可验证结果。

小团队可以由业务负责人兼任 IO、技术负责人兼任 OA；高风险场景则应拆开签署、执行和验证责任。判断角色设计是否有效，不看头衔是否齐全，而看价值责任、治理责任和执行责任能否被独立指认。

这三类责任可以由不同岗位兼任，却不能彼此混淆。尤其是价值取舍、风险接受和最终承诺，不能因为 Agent 参与而被转移。

19.3 人类不可转移的五类责任

无论模型能力多强，以下责任不能因为“AI 建议如此”而消失：

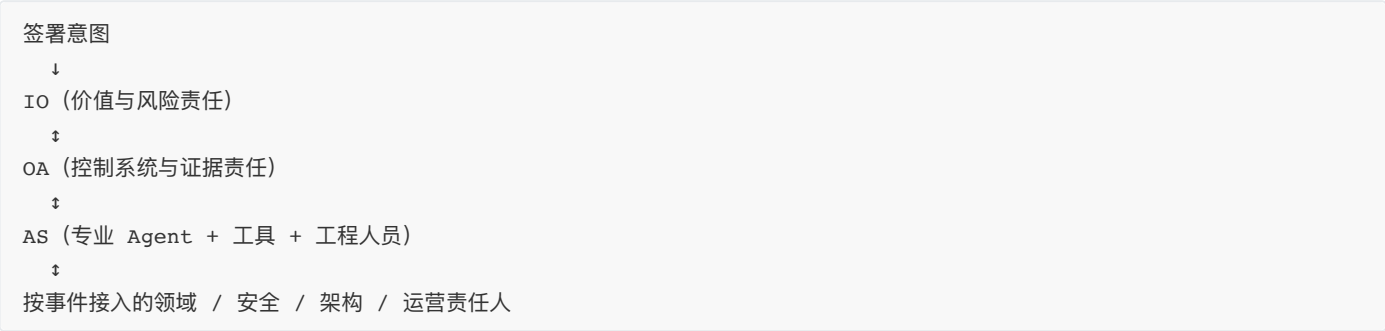
1. 决定什么价值值得追求，以及明确不做什么；
2. 在速度、成本、公平、安全和体验冲突时作出取舍；
3. 批准权限扩大、约束例外和不可逆操作；
4. 对证据不足、UNKNOWN 和残余风险作出行动裁决；
5. 对外部客户、监管者、员工和社会后果承担责任。

Agent 可以生成方案、模拟影响和整理证据，但不能成为责任逃逸装置。组织授权给 AI 的是执行权，不是责任主体资格。

责任不能转移，能力却可以重新组合。组织不必再让任务沿岗位逐级排队，而可以围绕一项已签署意图，把完成任务所需的人、Agent 和专业能力组织起来。

19.4 从固定团队到意图单元和动态能力网络

传统项目常围绕岗位和部门组队，再通过会议与交接协调。Agentic Agile 以一个签署意图为中心形成最小责任单元：IO 提供价值方向，OA 装配所需能力，AS 按 Work Graph 执行，领域、安全、架构和运营专家在关键节点加入。



这种结构不是取消长期团队。稳定团队仍保存领域知识、工程资产和共同责任；动态意图单元则减少跨部门排队，让能力围绕真实价值流临时组合，任务结束后把证据和知识带回组织资产。

动态组合减少了交接，却不会自动缩短决策时间。只要授权仍停留在原有层级，Agent 节省下来的执行时间就会重新耗在审批队列里。

19.5 决策权必须与信息 and 风险一起前移

如果 Agent 可以在几分钟内执行，而所有例外仍等待周会、层层汇报或委员会逐项审批，自动化只会制造更长队列。组织需要预先定义授权矩阵：什么由规则自动放行，什么由 IO/OA 当场裁决，什么必须升级到安全、合规或治理委员会。

RACI-D 在传统 RACI 上增加 Decision Owner：谁执行、谁负责、咨询谁、通知谁，以及谁拥有最终决定权。每个高风险节点只能有一个清晰的 Decision Owner；多人“共同负责”通常等于无人能及时决定。

决策权前移之后，人的能力要求也随之变化：重点不再是亲自完成所有步骤，而是设计一套可验证、可接管的人机系统。

19.6 能力成长：从亲自执行到设计人机系统

传统角色	需要增强的能力
产品与业务负责人	意图建模、Non-goals、AC、价值证据、风险签署
架构与技术负责人	Context、Graph、Harness、恢复和跨模块协议
开发工程师	可执行规格、测试、Agent 工作流和工具设计
测试工程师	Eval、属性测试、证据独立性和门禁完整性
安全与合规	Policy as Code、工具权限、例外与审计证据
工程经理与研发效能	人机系统运营、遥测解释、能力建设和资产产品化

专业经验不会因 AI 失效，而会从“亲自完成每一步”迁移到“把判断转成上下文、约束、验证和恢复能力”。

这种能力迁移只有进入真实工作才有意义。下面以一个八人团队为例，看责任网络如何从原有分工中长出来。

19.7 一个团队如何完成第一次重组

以一个负责费用报销系统的八人团队为例。旧分工是产品经理收集需求，开发工程师编码，测试工程师验收，运维工程师发布，经理在每个环节排队审批。引入 Agent 后，团队不应简单地给八个人各配一个 Copilot，而应围绕一条真实价值流重新分配责任：

原有活动	新的责任安排	保留的人类判断
规则澄清与优先级排序	产品负责人担任 IO，Agent 协助列出歧义和影响	价值、范围、非目标、风险偏好
方案拆分、上下文准备、任务编排	技术负责人担任 OA，维护 Work Graph 与 Harness	架构边界、权限、停止条件、证据要求
编码、测试样例、静态检查、文档草拟	AS 由工程师与多个受控 Agent 组成	例外处理、复杂取舍、结果复核
发布、灰度、回退	IO 与运维责任人共同执行授权矩阵	发布范围、残余风险、事故接管

第一次重组的目标不是减少岗位，而是让一条任务从“人找人”变成“意图找能力”。团队先选择低风险且可回退的规则变更，连续积累十至二十个 Evidence Bundle，再决定哪些授权可以扩大。没有这些样本，组织无法区分真实提效和把复杂工作转移给少数专家。

第一批样本的用途不是证明团队已经可以扩大自治，而是帮助组织把默认允许、必须升级和明确禁止的边界固化下来。授权矩阵由此成为责任网络的可执行接口。

19.8 授权矩阵：让执行速度不再等待层级速度

组织可以先用四个问题定义授权，而不是先画新的组织架构图：操作是否可逆，结果是否可自动验证，影响范围是否可控，是否涉及法律、资金、隐私或对外承诺。一个简化矩阵如下：

操作	Agent 默认权限	必须升级的条件	Decision Owner
生成测试、文档和本地代码变更	可在隔离环境执行	触及受保护分支或敏感数据	OA
修改审批规则并运行集成测试	可提交候选变更	AC 失败、契约冲突或证据缺失	OA / IO
对试点部门开启功能开关	不可自行开启	需要灰度、监控和回退条件	IO
修改生产数据、扩大权限或关闭审计	禁止自治	任何例外都需专业责任人审批	对应领域责任人

矩阵必须与工具权限、流水线门禁和审计记录一致。若制度写着“Agent 不得发布”，但凭证实际注入了 Agent 运行环境，组织拥有的只是口头边界。反过来，工具限制过严却没有升级路径，也会把所有工作重新推回人工排队。有效授权既给出默认允许，也明确停止条件和接管路径。

授权矩阵解决了“谁可以做什么”，却也可能被错误的管理方式抵消：如果组织按 AI 调用量、代码量或最低人工介入率评价个人，人们就会主动绕过升级和停止机制。

19.9 激励与反监控边界

组织重构必须从一开始守住反监控边界：不把 AI 调用、Token、代码量或最低人工介入率当作个人排名依据。这里的重点是责任网络是否清楚；授权经营、绩效评价、遥测使用和团队信任由第二十三章系统展开。

第十九章完成的是人员、责任和授权的重构；但责任网络如果仍被旧的需求、开发、测试和发布交接包围，机器速度依然无法穿过整条价值流。下一章转向流程与工具，说明这些责任如何进入持续运行的控制系统。

19.10 本章小结

人员与组织重构不是岗位消亡计划，而是任务、执行权、决策权与责任的重新分配。IO/OA/AS 建立最小责任网络，动态意图单元减少无效交接，授权矩阵让决策跟上机器速度，人类则继续承担价值、例外、不可逆操作和最终后果。

本章最小实践：选择一条真实价值流，把十项关键任务标记为 Agent 执行、人机共决或人类专属，并为每个高风险节点指定 Decision Owner。

完成证据：人类责任节点与 Agent 能力网络图、RACI-D、授权与升级矩阵。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把岗位分工重构为人机责任网络和可签署的授权关系。

第二十章 | 流程与工具重构：从旧流程加 AI 到 AI 原生价值流

人员和组织责任改变后，若需求、开发、测试、审批和发布仍按原交接链运行，提效会被等待时间吞噬；若工具仍停留在个人 Copilot，又无法让意图、权限、证据和反馈跨步骤连续。流程与工具必须共同重构：流程定义控制怎样流动，工具保证关键状态不能被静默跳过。

20.1 旧流程加 AI 为什么只会局部提速

典型做法是在需求完成后让 AI 写代码，在测试阶段让 AI 补用例，在发布前继续人工汇总证据。编码时间缩短了，但需求等待、环境准备、跨团队交接、重复审批和证据整理没有减少，甚至因生成速度提高而增加返工。

判断流程是否真正重构，可以看三个信号：决策是否前移到执行前，反馈是否进入执行内环，完成是否由证据而非状态汇报决定。

20.1.1 先测价值流，不要先画未来蓝图

流程重构应从一条真实价值流的事实开始。选择一种重复出现的任务，例如“从业务规则变化到生产灰度”，沿途记录：

观察项	要采集的事实	常见问题
触达时间	从提出意图到产生业务结果的总时长	只统计编码时间，忽略排队和上线等待
活动时间	真正分析、实现、验证和裁决的时间	把会议时长全部当作价值活动
等待时间	等需求、权限、环境、评审、测试和发布窗口	AI 加速后瓶颈转移到审批和环境
交接次数	任务、上下文和责任主体发生多少次切换	每次交接都重新解释目标并丢失约束
返工与失败	因规格、实现、环境、权限和证据不足造成的重做	最终全绿掩盖多轮失败
决策时延	异常出现到有权者作出决定的时间	Agent 数分钟执行，例外等待数天
证明成本	为确认完成投入的测试、人工核对和证据整理	编码节省被 QA、业务和运维成本抵消

流程图只显示“经过哪些部门”还不够。必须把工作、等待、决策、证据和返工放在同一张价值流图上。否则组织容易优化最显眼的编码节点，却看不见真正限制端到端提效的约束。

基线暴露瓶颈之后，不能把每项旧活动原样自动化。先要分清哪些活动仍承担价值和责任，哪些只是传递状态、等待日历或重复解释上下文。

20.1.2 识别三类被 AI 放大的流程债

第一类是**批量债**：需求、测试和发布按周或按 Sprint 批量处理，机器速度无法变成更短反馈。第二类是**交接债**：每个岗位只交付文档或状态，下游必须重新理解意图。第三类是**审批债**：低风险事项也依赖人工逐项放行，而高风险事项又缺少真正有资格的 Decision Owner。

AI 不会自动消除这些债，反而会让它们更明显：上游几分钟生成更多工作，下游队列更长；更多 Agent 并行产生更多交接；为了控制失速，组织继续增加审批。流程重构必须减少批量、消除不必要交接，并把审批改造成“预授权规则 + 事件触发例外”。

20.2 哪些敏捷与工程活动保留，哪些应被替代

Agentic Agile 不以“删除所有会议”为先进。活动是否保留，取决于它是否承担不可自动化的价值和责任：

传统活动	处理原则
目标、优先级和风险取舍	保留人类决策，用 Intent Contract 固化结果
站会中的纯状态播报	由 Work Graph、事件和 Dashboard 自动替代
需求澄清	从大批量评审转为批判性检查、Grill-Me 和 Example Mapping
开发后集中测试	前移为 AC、TDD Red 与 P⇌E 快内环
每一步人工审批	改为约束、门禁和事件触发 HITL
发布前手工拼报告	由 Evidence Bundle 和 Release Manifest 汇聚
复盘	保留价值与系统判断，输入来自真实失败和 Telemetry 趋势

流程重构的目标不是少开会，而是让机器处理可机械判断的状态，让人的时间集中在目标冲突、异常和责任裁决。

保留下来的活动也不应继续靠岗位交接和固定会议串联，而应由契约、事件、证据和人工裁决共同驱动。

20.2.1 用事件替代状态汇报，用证据替代完成声明

旧流程围绕日历运行：到时间开会、汇报状态、请求批准。AI 原生流程更多围绕事件运行：契约已签署、可信 RED 已形成、MUST 被阻断、重试预算耗尽、Evidence 已具备裁决资格、生产事实推翻原结论。事件发生时，系统自动推进、停止或通知正确责任人。

这并不取消人的节奏。战略优先级、跨团队冲突、重大例外和遥测趋势仍需要定期讨论；但会议输入应是事件、证据和选项，而不是逐人询问“做得怎么样”。

20.2.2 一项活动是否保留的四问

- 1. 它是否作出机器不能承担的价值或风险判断？
- 2. 它是否产生下游真正使用的决策或证据？
- 3. 它是否能被事件触发，而不是固定等待日历？
- 4. 若删除它，哪个门禁、责任人或恢复机制接替其控制功能？

不能回答第四问时，直接删流程会造成控制真空；前两问都答“否”时，继续保留通常只是组织惯性。

20.3 AI 原生价值流：从任务传递到持续控制

AI 原生价值流以可验证状态而不是岗位交接推进：意图未签署时只能探索；契约、约束和 AC 成立后才允许准备与执行；证据收敛后才进入独立裁决；裁决完成后才形成跨任务遥测。第三部分已经给出 SCOPE-V 的运行细节，本章关心的是这些状态如何替代排队、反复汇报和事后拼接材料。

HITL 也随事件路由：规格冲突找 IO，权限例外找相应责任人，跨模块冲突找 Decision Owner，证据不足找独立验证者。通知必须带问题、影响、证据、选项和决策时限。大量低风险问题反复升级，说明规则或工具尚未产品化；重大风险从不触发升级，更可能是漏报而不是成熟。

要让这些状态真实运行，工具必须能够承载上下文、编排、权限、验证和证据；否则所谓状态流仍只是画在流程图上的理想路径。

20.4 工具从 Copilot 孤岛走向最小工程系统

AI 原生工具体系至少要形成五种相互连接的能力：

- 1. **Context**：从图谱、契约、约束、代码和证据裁剪可信上下文；
- 2. **Orchestration**：用 Work Graph 管理节点、依赖、并行、Handoff 和停止；
- 3. **Harness**：落实权限、沙箱、门禁、预算、观测和恢复；

- 4. **Evidence**：按任务和 AC 聚合真实、当前、可追溯的完成事实；
- 5. **Telemetry**：把多个任务信号聚合为价值与效能慢外环。

它们不要求采购一套庞大平台。仓库中的 Markdown/YAML、现有 Git 和 CI/CD、结构化脚本与静态 Dashboard 就能跑通最小系统；只有当真实使用暴露复用、权限、规模或运营瓶颈时，才把能力产品化。

20.4.1 六层工具架构：能力分层，事实贯通

工具层	主要职责	不应承担的责任
AI 交互与宿主层	对话、搜索、编辑、Agent 执行与人工交互	自行定义签署、约束和完成语义
组织语义层	Intent Graph、Contract、Constraint、Evidence 的稳定 ID、状态与 Schema	绑定某个模型的专有 Prompt 格式
运行控制层	Context、Work Graph、Tools Manifest、Harness、五道门和恢复	替人作价值取舍和例外批准
验证交付层	测试、Verification Plan、CI/CD、Release Manifest、制品与回滚	用流水线绿色代替责任裁决
事件与遥测层	追加式事实、Measurement Contract、单任务与聚合 Dashboard	把 UNKNOWN 填成 0 或直接排名个人
企业集成层	身份、权限、代码托管、工单、知识、制品、安全与审计系统	建设另一套无法追溯的平行事实源

分层的价值不是增加平台数量，而是防止职责混乱。AI 宿主可以替换，组织语义不能随之丢失；CI 可以执行检查，不能签署业务风险；Dashboard 可以暴露趋势，不能自动扩大自治范围。

20.4.2 先连接现有系统，再决定是否建平台

大多数企业已经拥有代码仓库、CI/CD、身份权限、测试平台、制品库、工单和监控。第一步应把 Contract ID、Evidence ID、Commit、Artifact Digest 和 Release ID 穿过这些系统，而不是复制一套“AI 专用研发平台”。

只有出现以下瓶颈时，平台化才有明确理由：多个团队无法复用约束和验证器；权限与审计无法统一；跨工具状态频繁丢失；Evidence 聚合成本过高；指标口径无法治理；治理资产缺少 Owner、版本和灰度。平台建设必须对应已观测问题，并有停用旧路径的迁移计划。

20.4.3 Build、Buy 与复用的决策原则

- **复用现有系统**：身份、Git、CI/CD、制品、安全扫描和监控通常优先复用；
- **购买通用能力**：模型访问、IDE Agent、可观测和安全产品可按企业要求采购，但必须接入稳定协议；
- **自建差异能力**：组织特有的意图语义、约束策略、证据映射和度量契约往往需要内部所有权；
- **开源与可替换**：对关键运行路径保留数据导出、Schema 和替换能力，避免治理事实被供应商锁定。

决策标准不是功能清单最长，而是能否在目标风险下提供可验证控制、可持续运营和合理总成本。

工具分层之后，还要保证更换模型、IDE 或 Agent 宿主不会改变责任、边界和证据语义。这就需要一层独立于具体工具的稳定协议。

20.5 跨 AI 工具是架构要求，不是兼容性加分项

企业不会永久只使用一个模型、IDE 或 Agent 宿主。组织资产必须独立于具体工具：契约 ID、状态、约束语义、Evidence Schema、事件账本和门禁退出码保持稳定，Codex、Claude、WorkBuddy、IDE Agent 或内部平台只负责适配这些协议。

跨工具不等于最低公分母。不同宿主可以发挥各自搜索、执行、并行和交互能力，但不能拥有不同的签署规则、成功定义或证据真相。宿主工具与 Token 数据客户端也应解耦，避免从哪个工具启动就只能度量哪个工具。

20.5.1 稳定协议至少包含什么

跨工具交接至少要保留：任务与契约版本、签署状态、Change Envelope、允许工具、节点输入输出、AC 与约束结果、Evidence 引用、UNKNOWN、裁决状态和追加式事件。自然语言总结可以附加，不能成为唯一交接载体。

20.5.2 能力探测与降级必须诚实

不同工具拥有不同的终端、浏览器、多 Agent、MCP、上下文窗口和用量数据能力。适配器应先探测能力，再选择路径；缺失功能时保留 UNKNOWN、改用受控替代或升级人工，不能为了展示“全工具兼容”而静默跳过验证。跨工具的最低要求不是功能完全相同，而是责任、边界和证据不降级。

稳定协议保证共同事实不随工具漂移，并不意味着所有任务都要使用同样重的控制。风险、影响范围和可逆性不同，流程与工具也要同步裁剪。

20.6 流程与工具共同裁剪

轻量任务可以使用最小图谱索引、单契约、核心约束和一份 Evidence；高风险跨系统任务需要 Verification Plan、职责分离、多层验证、Release Manifest 和生产观察。裁剪依据是影响、可逆性、数据敏感度和验证能力，而不是团队是否愿意填写更多文件。

任何轻量化都不能删除四件事：目标有人签署、边界可以执行、完成能够证明、异常有人裁决。任何平台化也不能把 UNKNOWN 自动补成绿色。

风险 Profile	流程形态	最小工具控制	人类责任点
轻量、可逆	单责任单元，短闭环，少量门禁	最小契约、核心约束、真实测试、Evidence	意图签署与异常裁决
标准业务变更	Work Graph、P⇌E、独立 V、灰度	四大动态工件、权限围栏、多层验证、Telemetry	IO、OA、发布责任人
高风险或不可逆	职责分离、双人复核、独立环境和演练	Verification Plan、强约束、完整谱系、Release Manifest、恢复验证	领域、安全、合规及有资格批准者

裁剪必须同时改变流程与工具。流程要求双人复核，工具却允许单一 Agent 持有生产凭证，控制并不存在；工具设置大量门禁，流程却没有异常责任人，失败只会堆积成队列。

裁剪后的系统也不是一次性工程。模板、权限、验证器、遥测和退出路径都会随业务与技术变化，必须由明确的 Owner 持续运营。

20.7 谁运营这套流程和工具

AI 原生价值流不是安装后自动运行的系统，需要清晰的产品与运营责任：

运营对象	主要 Owner	持续职责
意图与任务类型模板	IO 社群 / 产品负责人	校准业务语义、Non-goals 和价值证据
Context 与知识来源	OA / 领域知识 Owner	来源、新鲜度、权限、失效和回写
约束、工具权限与门禁	OA + 安全/架构/数据责任人	规则版本、例外、误报、恢复与审计
Work Graph 与 Agent 路由	OA / 平台工程	节点能力、并发、预算、Handoff 和停止条件
验证器与 Evidence Schema	测试/质量工程	独立性、覆盖、证据新鲜度和兼容性
Telemetry 与 Dashboard	研发效能 + 数据 Owner	Measurement Contract、数据质量、访问和行动闭环
AI 宿主与模型组合	平台/采购/安全	能力、成本、数据边界、替换和退出策略

Owner 不是审批所有任务，而是维护默认路径，使大多数低风险任务无需临时协调。治理资产应像内部产品一样拥有版本、使用数据、问题反馈、灰度和淘汰机制。

职责和工具架构是否有效，最终仍要回到一条真实价值流中检验。以费用报销规则变更为例，可以看到流程、工具、责任和证据怎样同时变化。

20.8 费用报销规则：一条价值流怎样被真正重构

旧流程中，“超过 5000 元增加总监审批”可能经历产品写需求、架构评审、开发排期、编码、QA 补测试、安全检查、发布审批和人工汇报。每个环节都完成自己的文档，但金额口径、外币、代理总监和回退条件可能直到测试或生产才暴露。

重构后的流程不是把八个岗位替换成八个 Agent：

1. IO 与 OA 在意图进入队列时完成 Grill-Me 和 Example Mapping，关键决定进入完整契约并由 IO 签署；
2. Constraint Matrix 和 Tools Manifest 预先限定数据、文件、权限、预算和生产禁区；
3. Work Graph 释放边界测试、影响分析和回退准备，只有可信 RED 后才释放实现节点；
4. AS 在 $P \rightleftharpoons E$ 中完成有限修复，代理总监规则冲突触发领域 HITL，而不是继续猜测；
5. Evidence Bundle 按 AC、约束、提交和回退聚合事实，V 独立给出灰度、补证或拒绝建议；
6. 有权者决定两个部门灰度，Release Manifest 绑定制品和恢复路径；
7. 单任务信号进入 Telemetry，多个同类任务后再决定是否调整 Context、约束、编排或自治范围。

流程变化的价值体现在：关键决策前移，测试和恢复准备并行，普通偏差在快内环收敛，人工只处理真正的业务冲突，完成与发布都能追溯到同一意图。工具变化的价值则是让这些规则跨 AI 宿主、会话和流水线持续成立。

这条价值流也揭示了改造中最常见的陷阱：表面上增加了 Agent，责任与等待结构却没有改变，局部提速便被误报成了组织能力。

20.9 常见反模式

- **旧流程自动化**：把原来的工单、评审和审批全部做成机器人，却不减少批量与交接；
- **Agent 数量崇拜**：用更多 Agent 模拟更多部门，协调成本反而上升；
- **平台先行**：没有真实价值流和基线，先建设统一门户、知识库和指标中心；
- **宿主锁定**：契约、记忆、证据和用量只能存在某个 AI 工具的私有空间；
- **工具即治理**：购买安全或观测产品后，仍没有 Decision Owner 和异常路由；
- **门禁堆叠**：所有任务使用同样重的检查，导致团队绕过正式路径；
- **数据假闭环**：Dashboard 指标丰富，却不能下钻到契约和 Evidence，也不触发行动；
- **影子流程并存**：新旧流程长期双轨，正式状态与实际执行逐渐分离；
- **自动化例外**：为了不中断 Agent，把权限扩大、UNKNOWN 压平或让模型自行批准风险。

这些反模式的共同点是保留旧系统的责任与等待结构，只在表面增加 AI 和自动化。

是否真正完成重构，不能靠主观感受判断，而要回到同类任务的基线、护栏和连续证据。

20.10 如何判断流程工具重构是否成功

不要只统计 AI 生成代码占比。更有意义的信号包括：端到端等待是否减少，首次成功是否提高，证据整理是否自动化，HITL 是否集中在真正需要人的事项，跨工具交接是否保持语义，失败是否可停止和恢复，以及单位有效交付的总成本是否下降。

结果维度	建议信号	必须联读的护栏
流动	端到端周期、等待占比、交接数、决策时延	逃逸缺陷与下游人工不能恶化
质量	目标准确率、首次成功率、证据完整度	不得通过弱化 AC 和测试获得改善
自治	自动收敛轮次、HITL 类型、停止与恢复成功率	必要 HITL 不应被压低
工具	跨宿主成功率、上下文新鲜度、门禁可用性	失败必须显式，不得静默降级
成本	人工时间、Token、平台和证明成本	观察总成本，不只看编码成本
学习	同类失败复发率、资产复用率、规则淘汰率	防止知识和约束无限膨胀

改造前后必须比较相同任务类型和风险等级。一个低风险演示任务不能证明核心交易流程提效；一个成功样本也不能证明组织能力。流程与工具重构只有在连续任务中减少等待和返工，同时保持质量、责任与恢复能力，才算完成第一轮验证。

20.11 本章小结

流程与工具重构不是把 AI 插入旧节点，也不是把旧流程逐项机器人化，而是把价值流改造成意图驱动、边界可执行、事件触发、反馈高频、证据裁决和遥测学习的持续控制系统。流程决定责任、状态和回流，工具把这些决定变成跨宿主、跨会话、跨流水线仍不能被静默跳过的运行事实。

本章最小实践：选择一条真实研发价值流，测量活动、等待、交接、决策和证明成本；删除一项纯状态交接，把一项审批改为“预授权规则 + 事件触发 HITL”，并画出六层最小工具架构。

完成证据：当前/目标价值流图、控制状态与事件清单、Owner 矩阵、六层工具架构、风险 Profile，以及带护栏的提效假设。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把旧流程、工具和审批链重构为 AI 原生价值流。

第二十一章 | 三层四维协同落地：转型画布、成熟度与 90 天首轮验证

人员、组织、流程和工具必须围绕同一个业务结果协同改变。只改人员能力，决策权可能仍被旧组织卡住；只改流程，工具可能无法执行新的边界；只上工具，等待、返工和责任真空仍会吞噬效率。本章用“三层 × 四维”建立共同坐标，再用成熟度和 90 天首轮验证判断这些改变能否重复、迁移和扩展。

本书所说的四维转型，不是人员、组织、流程和工具四个并行项目，而是围绕同一个业务结果共同改变四件事：**人员**回答谁完成工作、谁需要具备什么能力；**组织**回答谁拥有决策权、谁承担最终责任；**流程**回答工作如何流动、验证、升级和回退；**工具**回答什么能力支撑上下文、编排、执行、验证、证据和遥测。四个维度缺一，局部提效都可能被等待、返工、权限或责任真空抵消。

21.1 三层四维协同转型

三层不是组织架构，也不是先后执行的三个阶段，而是观察同一场转型的三个分辨率：

分辨率	核心问题	主要对象
战略意图层	为什么转型，要创造什么价值，哪些风险不能接受？	战略结果、投资假设、风险红线与价值组合
价值网络层	哪些价值流和责任单元需要围绕共同结果协同重构？	端到端价值流、责任单元、依赖、协议与联合回退
任务运行层	一项真实工作如何由人和 Agent 分工、协同、验证、回退并完成责任闭环？	意图契约、约束、执行、证据与任务裁决

四维贯穿每一层，回答“具体改什么”：

	人员	组织	流程	工具
战略意图层	配置能力	投资取舍	组合评审	结果与风险观测
价值网络层	跨单元协同	责任边界	依赖与回退	共享协议与证据
任务运行层	人机分工	授权责任	执行与验证	Context/Harness/Telemetry

因此，三层回答“在哪一层发生、判断什么”，四维回答“改变什么、由谁改变”。同一项转型必须在两条轴上都能落到明确对象、责任人和证据，不能只写“引入 AI”或“加强协同”。

90 天由四个连续的证据窗口组成：首周跑通一个真实任务闭环，30 天在同一价值流重复并修正，60 天迁移到相邻场景或责任单元，90 天根据遥测和业务结果决定扩展、收缩或停止。首轮验证从任务运行层开始；价值网络层和战略意图层的聚合与投资裁决将在第二十二章展开。若首周连任务证据都没有，后面的组织规模化就没有依据。

21.2 三层四维转型画布：在每一层检查四个改变对象

分辨率	人员	组织	流程	工具
战略意图层	需要什么领导、业务和治理能力	谁拥有投资取舍、风险红线和停止权	如何形成组合评审、预算调整和退出机制	如何观察业务结果、风险、成本与能力趋势
价值网络层	哪些角色和责任单元共同交付端到端结果	谁是网络 Owner，如何处理跨单元依赖和联合责任	如何编排依赖、集成、发布和联合回退	如何共享上下文、协议、权限、Evidence 与 Telemetry
任务运行层	人和 Agent 如何分工，哪些判断必须由人完成	谁签署意图、授予权限、处理异常并接受结果	任务如何经过契约、执行、验证、升级和回退	需要哪些 Context、Orchestration、Harness、Evidence 与 Telemetry 能力

画布还要单独记录三类跨层输入与输出：

- **目标与基线**：客户或经营结果是什么，当前周期、等待、返工、缺陷、人工投入和成本在哪里；
- **提效假设与护栏**：希望改善什么，哪些质量、安全、合规、成本和责任条件不能恶化；
- **阶段证据与裁决**：首周、30 天、60 天和 90 天分别用什么事实决定继续、受限继续、先补基础或停止。

画布必须从真实价值流和基线开始，不能先讨论“采购什么 Agent 平台”。四个维度必须共享同一个提效假设，并在三层之间保持可追溯：战略意图层的价值假设约束价值网络，价值网络的责任和依赖约束任务运行，任务证据再向上支持网络和战略裁决。

配套工件：可以下载[《四维转型画布总览模板》](#)，把同一条价值流的基线、四维重构、责任授权、运行底座、证据窗口和管理裁决放在一张图上。画布用于统摄各阶段工件，而不是替代它们；价值网络和战略意图层的扩展关系见第二十二章。

21.3 证据驱动的成熟度

成熟度不是给整个企业贴一个单一等级，也不只看“自动化率”或“人介入多少”。它应放回“三层 × 四维”坐标，按任务类型、风险等级和证据窗口判断：同一组织可以在任务运行层达到 L3，却在价值网络层仍处于 L2；也可以在工具维度领先，但在组织责任维度没有升级。

建议结合：

- 目标准确率；
- 首次成功率；
- MUST 通过率；
- 约束自愈率；
- HITL 是否发生在正确位置；
- Evidence Bundle 完整度；
- Bug 回溯真实性；
- 经验复用与撤回是否真实发生；
- 多模块一致性；
- 价值和成本表现。

21.4 四级演进路径

没有企业能一键切换到完全自治。四级演进路径描述的不是采购了哪种工具，也不是有多少代码由 AI 生成，而是**执行权在什么边界内交给 Agent、组织用什么证据证明这种授权仍然安全有效。**

等级	名称	AI 的组织身份	主要治理方式	自治范围
L1	工具依附期（Copilot Agile）	个人外骨骼	人工目标、人工检查	单点辅助
L2	人机编队期（Hybrid Agentic Scrum）	可派发任务的虚拟同事	任务说明、检查清单、人工验收	边界清晰的低风险任务
L3	受控自治期（Constrained Autonomy）	在 Harness 内运行的自治执行单元	3-4-3、SCOPE-V、机械门禁、证据裁决	可验证、可恢复、风险受控的工作流
L4	硅基自治期（Silicon Autonomy）	意图驱动系统中的自治执行网络	遥测反馈、治理资产运营、持续审计	执行闭环默认自治，价值与不可逆风险仍由人负责

四级不是给整个企业贴一个标签。同一组织可能在测试生成上达到 L3，在生产发布上维持 L2，在业务价值判断上始终保留人类裁决。成熟度应按任务类型、风险等级、三层分辨率和四个维度分别评估；升级一个任务的自治范围，不等于升级整个价值网络，更不等于完成战略转型。

21.4.1 L1：工具依附期——AI 是个人外骨骼

L1 保持现有 Scrum、Kanban、瀑布或 DevOps 流程基本不变。AI 被用于代码补全、问答、文档草稿、测试草稿和局部重构，但工作仍以“人提出—人判断—人验收”为主。

典型运行画像：

- 提示词和个人经验决定输出质量；
- 同一个问题在不同工程师手中得到不同结果；
- AI 生成内容与任务、约束和验收证据之间缺少追溯；
- 安全、成本、质量数据散落在个人工具中；
- AI 出错后主要依靠人工 Review 和返工。

L1 并不是失败状态。对于低频、探索性或难以自动验证的工作，L1 可能就是合理终点。它的风险在于把个人效率提升误报成组织交付能力。

进入 L2 的证据不是“安装了 Agent 工具”，而是团队能够把一类边界清晰的任务显式派给 AI，定义期望结果，并稳定回收、验证和记录结果。

21.4.2 L2：人机编队期——AI 成为可管理的虚拟同事

L2 中，Agent 开始承担完整任务，而不只是生成片段。人类仍主导目标、优先级、风险判断和对外承诺，Agent 可以完成单元测试、简单 CRUD、报表、文档整理、代码迁移或重复性修复。

最小治理能力：

- 任务有明确目标、非目标和验收条件；
- 可说明哪些工具、数据和代码区域允许 Agent 使用；
- 结果进入现有 CI、测试和 Review 流程；
- 能记录任务耗时、失败、人工往返和实际 Token 成本；

- 发生越界、证据不足或目标冲突时，Agent 会停止并请求人类裁决。

此时可以弱化站会中的纯状态同步，但不能删除障碍暴露、价值判断和责任确认。工具能够汇总“做到了哪一步”，却不能替组织决定“这件事是否值得做、风险是否可以接受”。

L2 的常见假成熟是给 Agent 分配任务，却没有统一约束和独立证据；产出看似更快，QA、架构师和业务人员却承担了更多隐性验证成本。

进入 L3 的证据是连续一组真实任务已经把 IO/OA/AS 责任、四大动态工件、SCOPE-V 门禁和任务级遥测变成默认交付方式，而不是演示时才使用。

21.4.3 L3：受控自治期——3-4-3 接管可验证 workflow

L3 的核心变化不是“AI 写更多代码”，而是执行权从会议和个人提醒转移到**契约、约束、编排、证明与异常升级组成的控制系统**。

在这一阶段：

- IO 签署意图契约，对价值、非目标和风险取舍负责；
- OA 设计 Context、Work Graph、Harness、验证策略和恢复边界；
- AS 在最小权限下执行、测试、自检、自愈并生成 Handoff；
- 意图图谱、意图契约、约束矩阵和 Evidence Bundle 可相互追溯；
- CI/CD、接口测试、UI 测试、SIT、性能测试和 UAT 被纳入证据链；
- MUST 失败会机械阻断，无法收敛或不可逆操作会触发 HITL；
- 每份契约和全局 Dashboard 都能看见质量、成本、自治与价值趋势。

L3 不要求废弃 Scrum 或看板。只要关键执行和发布裁决已经由可验证治理闭环承担，原有节奏可以按需要保留或裁剪。

建议观察窗口：至少连续 10 个已完成任务，且包含正常任务、失败自愈、人工升级和回退样本。组织应按任务类型、风险等级和基线判断以下信号，而不是套用统一分数线：

- 目标准确率和首次成功率趋势稳定；
- MUST 约束无未闭环失败，任何例外都有签署和时效；
- Evidence Bundle 能覆盖关键 AC、约束、测试和发布结论；
- 自治能力画像显示：低风险重复工作能够在原边界内收敛，HITL 没有被隐藏；
- 无未关闭的高危安全项；
- 相较基线，验证总成本没有被转移给下游人员。

进入 L4 的证据不是“连续十次没有人介入”，而是组织已经能够根据真实遥测调整授权范围、路由、约束和治理资产，并能证明这些调整改善了结果。

21.4.4 L4：硅基自治期——执行闭环自治，价值与责任仍归人

L4 的重点是硅基自治：Agent 不再只是等待人类逐项派发任务，而是在已签署意图、可执行约束、授权范围和停止条件内，持续完成规划、执行、验证、恢复和经批准的进化。它向“意图即软件”演进，意图图谱与运行态、业务结果、发布证据、算力成本和风险信号持续关联；系统可以提出路由、约束、测试、知识或 Skill 的改进建议，并在批准后的范围内执行。

L4 的关键能力包括：

- 组织级价值与效能视图能够按契约、模块、项目和风险等级观察运行状态；
- 经验通过 Memory Write Gate 和 Evidence Bundle 晋升为知识、约束、测试或 Skill；
- 自治权限可以按遥测结果扩大，也可以在风险上升时自动收缩；
- 多团队共享治理资产具有所有者、版本、灰度、回退和淘汰机制；
- 人类 HITL 主要集中于商业策略、价值冲突、法律伦理和不可逆风险，而非默认逐任务返工；

- 发布和回滚路径均可证明，生产写权限仍由明确授权策略控制。

建议在包含生产相关或较高风险样本的连续窗口中观察：目标准确率、约束自愈、HITL 原因、回退、逃逸缺陷和总成本是否共同改善。低 HITL 连续段必须能够证明来自能力提升，而非漏报或越权。

L4 硅基自治不是“人类退出”，更不是系统可以自行修改目标和安全底线。它改变的是默认执行主体，而不是最终责任主体：Agent 可以自治执行，价值定义权、不可逆决策权、例外批准权和责任承担仍掌握在人类手中。

21.4.5 等级跃迁必须同时满足四类门槛

任何升级自治范围的决定，都应同时检查：

1. **能力门槛**：Agent 在目标任务上是否稳定完成，而非偶尔成功？
2. **验证门槛**：错误能否被独立测试、门禁和证据发现？
3. **恢复门槛**：失败能否停止、回退、重试或升级，不造成不可接受损失？
4. **责任门槛**：意图、约束、例外和发布分别由谁签署，是否可追责？

四者缺一，不应升级。能力强但无法验证，属于高风险黑箱；验证充分但无法回退，不适合扩大生产自治；责任模糊时，再好的技术指标也不能替代治理授权。

21.5 90 天首轮验证：不是等 90 天才跑通闭环

90 天不是交付一个项目所需的时间，更不是到第 60 天才跑通闭环。线下工作坊第二天就能完成一个真实仓库的可信增量；企业试点也应在首周完成同等闭环。90 天用于收集样本、验证重复性、迁移到第二场景并作出规模化决策。其目标是回答：**3-4-3 在本组织、此类任务和当前工程基础上，是否持续带来可证明的净收益？**

21.5.1 先写试点契约，而不是先安装工具

试点本身也要有一份 Intent Contract：

- **目标**：验证一种明确任务类型能否在不降低质量与安全的前提下，提高首次成功率、缩短交付时间或降低总验证成本；
- **非目标**：不证明所有项目都适合，不以裁员、代码生成量或“零 HITL”为成功标准；
- **范围**：一个团队、一个代码域、1—3 类重复出现且可验证的任务；
- **基线**：选取试点前 4—8 周同类任务，记录周期、缺陷、返工、人工投入、回滚和成本；
- **对照**：条件允许时保留同类常规交付样本；不能并行对照时，使用历史基线并标注差异；
- **假设**：例如“首次成功率提高 15 个百分点，同时逃逸缺陷不增加”；
- **护栏指标**：MUST 失败、高危安全事件、回滚率和 UAT 否决不得恶化；
- **退出条件**：出现重大越权、连续证据失真、总成本明显上升或业务 Owner 无法持续参与时暂停试点。

没有基线和退出条件，90 天结束时几乎任何结果都能被解释成“初步成功”。这不是实验，而是自我证明。

21.5.2 试点准入：选择能被证明的场景

首个试点宜同时满足：

- 任务边界相对清晰且有稳定业务 Owner；
- 至少具备基础 CI、自动化测试、静态检查、隔离环境和回滚路径；
- 仓库具备最基本的模块说明、依赖关系和权限边界；
- 能获得真实的质量、周期、人工投入和 Token 数据；
- 风险可以隔离，不直接从核心交易、生产数据库写入或高度敏感数据开始。

优先场景可以是内部工具、BFF、报表、自动化脚本、存量测试补强、可回退的小型重构。若选择既有大型系统，应从一个治理岛和下一次真实变更开始，而不是要求先理解百万行代码。

21.5.3 最小角色与工件

最小团队不必新设岗位，但三种责任必须可指认：IO 对目标和签署负责，OA 对控制系统和证据完整性负责，AS 负责受约束执行。

试点至少建立四大动态工件：

1. **意图图谱**：说明试点目标、任务关系、影响范围和回写结果；
2. **意图契约**：每个任务的目标、非目标、AC、风险与 IO 签署；
3. **约束矩阵**：权限、质量、安全、成本、环境和升级条件；
4. **Evidence Bundle**：汇聚 AC、约束、测试、风险、回退和裁决证据。

Work Graph、Execution Log、Loop Memory 和 Telemetry Dashboard 是支撑运行与反馈的基础设施。工件数量可以轻量，关键字段和责任链不能省略。

21.5.4 第1—7 天：完成真实闭环并在企业场景复现

线下课堂使用标准仓库或合格的企业自带仓库，在一天内完成角色重组、契约签署、可信 RED、真实实现、Evidence 与裁决。使用标准项目的小组必须在课后 7 天内把同一闭环迁移到企业场景；使用企业项目的小组则在 7 天内完成第二次真实任务。

关键动作：

- 锁定一条价值流、一个可回退切片和基线数据；
- 明确 IO/OA/AS、Decision Owner 与升级条件；
- 初始化四大动态工件和最小 Harness；
- 接入现有测试、CI 和沙箱，不重建一套平行流水线；
- 运行一次完整 SCOPE-V，形成真实代码、测试、Evidence、Telemetry 和 Dashboard；
- 主动设计至少一次失败注入，检查权限越界、MUST 失败和证据不足能否阻断；
- 在企业场景复现，记录与课堂标准项目的环境、数据和责任差异。

证据闸门：必须已有真实增量、真实测试、签署契约、Evidence Bundle 和 Telemetry Dashboard。无法形成这些事实时，不进入“稳定运行”阶段，而是先补工程基础。

21.5.5 第 8—30 天：稳定运行并验证可重复提效

第二阶段不是重新设计方法，而是在同一类价值流中连续运行，验证首周结果不是一次演示。样本量按任务频率决定，不机械追求数量，但必须覆盖正常成功、失败修复和至少一次 HITL。

关键动作：

- 稳定 Work Graph、Handoff、重试上限和 HITL 路由；
- 让接口、UI、SIT、性能和必要 UAT 证据回填同一契约；
- 记录首次成功、自愈、人工升级、返工、Token 与等待时间；
- 将试点组与历史基线或对照任务比较，识别成本是否被转移给 QA 和业务；
- 对高频失败做一次 LOOP-3 晋升，将稳定经验转为约束、测试、知识或 Skill；
- 审查至少一次回退、暂停或人工接管是否真实有效。

证据闸门：30 天时必须看到相同任务类型下可重复的提效，且质量、安全、下游人工和总成本没有恶化。速度提升不能抵消逃逸缺陷或成本转移。

21.5.6 第 31—60 天：跨场景扩展并验证方法可迁移

第三阶段把已经稳定的方法迁移到第二类任务、第二个模块或第二个责任单元，验证它能否跨场景复用。迁移不是复制全部模板，而是在保持责任、状态和证据语义的前提下重新裁剪参数。

关键动作：

- 选择风险或技术条件不同、但价值边界清晰的第二场景；
- 培养第二组 IO/OA/AS，避免成功依赖单一专家；
- 复用经过验证的约束、测试、Skill 和 Dashboard 口径，同时记录必要差异；
- 检查跨工具、跨模块 Handoff 与 Evidence 聚合是否仍可追溯；
- 比较两个场景的净收益、失败模式和治理维护成本；
- 对不能迁移的部分区分“场景差异”与“方法缺陷”。

证据闸门：60 天时必须证明方法至少在第二场景可迁移，或明确证明它只适用于原场景。不能迁移不是自动失败，但不能再宣称具有组织级可复用性。

21.5.7 第 61—90 天：形成组织化与规模化决策

最后阶段不继续堆功能，而是把已经验证的做法转成组织决策：哪些任务扩大自治，哪些保持受限，哪些先补基础，哪些停止。治理委员会查看原始证据和分群趋势，确定资产 Owner、授权边界、支持机制和下一轮投资。

建议以“效果指标 + 护栏指标 + 成本指标 + 学习指标”组成平衡记分卡：

类别	核心指标	要回答的问题
效果	目标准确率、首次成功率、端到端周期、吞吐	是否更快地交付了正确结果
护栏	MUST 失败、逃逸缺陷、回滚、高危安全事件、UAT 否决	是否把风险转移到了下游或生产
能力	自愈率、HITL 类型、验证覆盖、证据完整度	自治是否来自真实能力
成本	人工小时、Token、工具成本、等待和返工	总成本是否下降，而非只减少编码时间
学习	知识/约束/测试/Skill 晋升数及复用效果	一次经验能否改善下一次任务
价值	业务结果、采用率、风险损失避免	交付是否产生了值得支付的价值

每项指标都应同时保留任务类型、风险级别、样本数和时间窗。没有分母的“提升 80%”和没有风险分层的平均值都不适合作为自治升级依据。

证据闸门：90 天结束时决定的不是项目是否终于交付，而是哪些任务可以扩大到 L3、哪些仍需人类主导、哪些治理资产值得产品化，以及下一阶段必须补强哪项验证、恢复或组织能力。

21.5.8 四种试点结论

裁决	证据特征	下一步
扩大	净收益稳定，护栏未恶化，验证与恢复可靠	扩大同类任务，不立即跨风险域复制
受限继续	局部有效，但 HITL、成本或某项门禁不稳定	保持范围，针对短板再运行一个证据窗口
先补基础	AI 能执行，但测试、数据、知识或 CI/CD 无法支撑证明	暂停扩权，优先补工程基础设施
停止	风险、总成本或质量持续劣于基线	结束该场景试点，保留证据与反例

能够得出“此场景暂不适合”的结论，恰恰说明试点发挥了作用。真正危险的不是试点失败，而是在无法证明收益时仍凭热情扩大自治。

21.6 试点运行节奏

试点期间只保留承担真实决策的节奏：

- 意图与优先级决策，确认本轮要证明什么；
- 例外审查，处理权限扩大、不可逆操作和约束冲突；
- Evidence Review，判断本轮证据是否支持继续、收缩或停止；
- 遥测回顾，识别重复失败、等待和成本变化；
- 治理资产回写，把已验证的规则、测试或恢复动作沉淀为下一轮输入。

这些活动服务于试点假设，不应退化为状态汇报。每一次讨论都要产生明确的行动、责任人、截止时间和下一轮验证方式。

21.7 试点期的变革沟通

试点沟通应说明责任如何变化：重复执行可以交给 Agent，人类把精力转向目标、边界、异常和判断；任何高风险自治都需要更强证据和更清晰责任，而不是更少治理。

管理者应允许团队暴露失败，避免用速度或自动化率为试点定调。否则成员会把治理工件当成汇报材料，把真实问题留在系统之外。长期的治理机制、激励和评价制度见第二十三章。

21.8 本章小结

四维转型不能拆成四个互不相干的项目。人员任务、组织决策权、流程控制和工具能力必须围绕同一条价值流共同改变，并接受同一组业务与护栏证据检验。首周跑通真实闭环，30 天验证可重复提效，60 天验证跨场景迁移，90 天形成规模化决策；90 天不是交付周期，而是首轮组织能力验证窗口。

本章最小实践：为一条真实价值流填写三层四维转型画布，评估各层各维度的 L1-L4 当前水平，并设计课堂/首周、30 天、60 天和 90 天四个证据闸门。

完成证据：四维转型画布、成熟度画像、基线与提效假设、护栏指标、阶段证据和最终四类裁决规则。

延伸实践：推荐学习《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》，把四维重构、任务证据和 90 天行动放入同一张转型画布；线下工作坊进一步完成企业首轮扩展判断。

第二十二章 | 企业转型路线：从任务运行到价值网络与战略意图组合

第 21 章的 90 天首轮验证解决一个问题：一个任务运行闭环是否值得继续投入。它并不自动证明方法可以复制，更不证明企业已经具备规模化自治能力。本章沿着三层分辨率继续向上：先把任务运行闭环迁移为责任单元，再把多个相互依赖的价值流连接成价值网络，最后把价值网络放回战略意图与投资组合中，形成可授权、可验证、可收缩的扩展路线。

22.1 三层分辨率：任务运行、价值网络与战略意图层

Agentic Agile 的三层不是传统汇报层级，也不是把每一层都复制一套同样的表格，而是同一套控制语法在三个分辨率上的运行：

战略意图层：企业要放大什么价值，拒绝什么风险，继续投资什么方向

↓ 约束继承、投资边界、能力与预算假设

价值网络层：哪些价值流和责任单元围绕一个客户/经营结果协同，怎样集成、发布和回退

↓ 能力编排、跨流协议、依赖与证据聚合

任务运行层：一项真实工作如何由人和 Agent 分工、协同、验证、恢复和裁决

↑ Evidence 向上聚合，Telemetry 进入价值网络与战略慢外环

这与传统 LPM 的问题意识相近，但不是引入同名层级、角色和仪式。Agentic Agile 只保留需要解决的控制关系：方向与投资、跨价值流协同、任务运行与证据。组织可以把它映射到现有的组合、解决方案、产品线、平台、团队或项目结构，不必为每一层新设部门。

分辨率	主要问题	最小组织对象	核心工件	主要裁决
任务运行层	这项真实工作能否由人和 Agent 在边界内完成并证明？	一个可授权任务与责任单元	Intent Contract、Constraint Matrix、Evidence Bundle	接受、补证、返工、回退或停止
价值网络层	多条价值流和责任单元能否共同兑现一个结果？	一个跨产品/团队/平台的价值网络	Value Network Charter、跨流 Work Graph、协议、聚合 Evidence	集成、排序、依赖解除、扩大或收缩
战略意图层	哪些价值值得继续投资，哪些风险必须拒绝？	一个战略意图、投资假设与价值组合	Strategic Intent Graph、Investment Contract、Measurement Contract	投资、调向、限权、暂停或停止

三层之间有四条不可省略的关系：上层约束向下继承，下层只能增加限制不能放宽红线；下层 Evidence 按统一口径向上聚合，不能把一次 PASS 直接升级为组织成功；每层都有自己的 Decision Owner；上层只裁决本层问题，不接管下层每个执行动作。

什么时候需要建立价值网络？

不是多个团队一起工作，就一定需要建立价值网络。满足以下任意条件时，应把任务运行闭环提升为价值网络管理：

- 多个产品、平台或团队必须共同交付一个客户或经营结果；
- 一个价值流的完成状态不能独立证明，必须等待其他价值流集成或共同发布；

- 共享数据、接口、模型、环境、合规约束或发布窗口形成关键依赖；
- 单个团队局部提效后，等待、返工或风险转移到了其他价值流；
- 需要对端到端结果、联合回退或跨流残余风险作出裁决。

如果价值流之间没有真实依赖，只需在任务运行层或单价值流层运行，不要为了“规模化”人为建立价值网络。

22.2 三层如何具体落地

组织落地不是先画组织架构图，而是先建立一条向上升级、向下授权的事实链：

1. **战略意图层**选择一个业务结果或风险主题，写明投资假设、非目标、红线、预算边界和停止条件。
2. **价值网络层**把结果拆成必须共同完成的能力或价值流，指定网络 Owner、依赖、集成证据、联合发布和回退边界。
3. **任务运行层**从价值网络边界中领取一个具体目标，形成 Intent Contract，完成受控执行、独立验证和 Evidence。
4. **向上聚合**时保留任务类型、样本量、风险、时间窗、缺口和来源；只有达到本层证据门槛，才进入价值网络或战略裁决。
5. **向下调整**时把战略红线、解决方案约束、授权范围和验证要求写回下层工件；不得只通过会议口头传递。

最小落地不是“搭一套企业级平台”，而是选一个已有的战略结果，绑定一个由多个相互依赖价值流组成的价值网络，跑完一个完整的聚合与裁决周期。

22.2.1 三层责任不是三套审批会

每层都使用 IO、OA、AS 的责任语义，但分辨率不同：

层级	IO 负责	OA 负责	AS 负责
任务运行层	当前任务价值、范围、非目标与不可逆取舍	上下文、约束、编排、验证与恢复设计	在授权范围内执行、证明、修复并提交 Handoff
价值网络层	端到端结果、跨流排序与联合风险	网络协议、依赖图、集成证据与跨流恢复	执行具体价值流、完成集成节点并回传事实
战略意图层	价值方向、投资与风险红线	组合级能力、遥测口径、约束继承与投资证据	执行已授权的共享能力或分析任务，不自行改变战略目标

同一个人可以在不同层级承担不同角色，但每个关键裁决必须标明所属层级、责任人和有效期。价值网络层 IO 不能替任务运行层 IO 签署每个任务；战略意图层 OA 也不能用平台规则代替业务 Owner 的价值判断。

22.2.2 三层工件的最小字段

工件	至少包含	向上/向下连接
Strategic Intent Graph	业务结果、价值主题、投资假设、红线、非目标、组合关系	向下产生约束与投资边界，向上接收组合遥测
Investment Contract / Measurement Contract	投资范围、预算、目标、反指标、样本口径、停止条件、Owner	把“继续投资”变成可复核假设，而不是口号
Value Network Charter	端到端结果、参与价值流、依赖、协议、集成门、联合回退、网络 Owner	继承战略边界，约束任务领取与跨流协作
Cross-stream Work Graph	跨流节点、输入输出、依赖、并行组、证据要求和升级路径	将解决方案计划连接到各价值流任务
Intent Contract	单任务目标、AC、约束、授权、验证与回退	接收上层边界，向上贡献可追溯 Evidence

22.2.3 对照例子：同一个报销规则，什么时候需要建立价值网络

第 18 章已经用一个具体任务说明了任务运行闭环：中国区差旅报销单笔含税人民币金额超过 5000 元时，增加成本中心总监审批。下面不换案例，只改变系统边界，比较“无需建立价值网络”和“必须建立价值网络”两种情况。

情况 A：无需建立价值网络

假设当前系统已经具备以下条件：

- 只有一个报销规则服务负责计算审批链；
- Web、移动端和财务后台都读取同一个规则服务的结果，不各自复制规则；
- 成本中心总监目录、审计事件和功能开关已经是这个服务的现有能力；
- 本次只改规则判断，不改接口、不改数据模型、不改其他系统的发布节奏。

这时它就是一个任务运行闭环。团队可以直接沿用第 18 章的路径：

IO 确认规则与边界

→ OA 建立 Intent Contract / Constraint Matrix

→ AS 在 expense-rule-engine 中先写可信 RED

→ Prove 5000.00、5000.01、外币、缺失总监、历史草稿

→ Verify Evidence 与功能开关回退

→ 两个试点部门灰度

任务契约至少绑定：5000.00 不触发、5000.01 触发、外币按提交日汇率折算、找不到总监进入人工财务队列、历史草稿不被改写、关闭开关后恢复旧路径。一次合格 Evidence 可以支持这个任务的灰度或发布裁决。

此时不需要另建 Value Network Charter。因为没有跨价值流依赖，新增一层只会制造空审批：任务层的 IO/OA 已经能够对完整结果负责，其他系统也没有需要共同签署的集成边界。

情况 B：必须建立价值网络

现在业务要求变成：“这套超过 5000 元的政策要同时覆盖 Web、移动端、预算校验、审批目录和审计报表，并在下季度全面启用。”系统事实变为：

参与价值流	独立负责的结果	真实依赖
报销规则服务	根据含税人民币金额生成审批节点	需要成本中心和汇率数据
预算/账务服务	校验预算并冻结或释放额度	需要规则结果和审批状态
Web 与移动端	展示新增审批节点和状态	需要统一规则版本与接口
组织目录服务	提供总监和代理人	缺失或冲突时必须进入人工队列
审计与报表	记录规则版本、审批轨迹和异常	需要所有服务使用同一事件语义

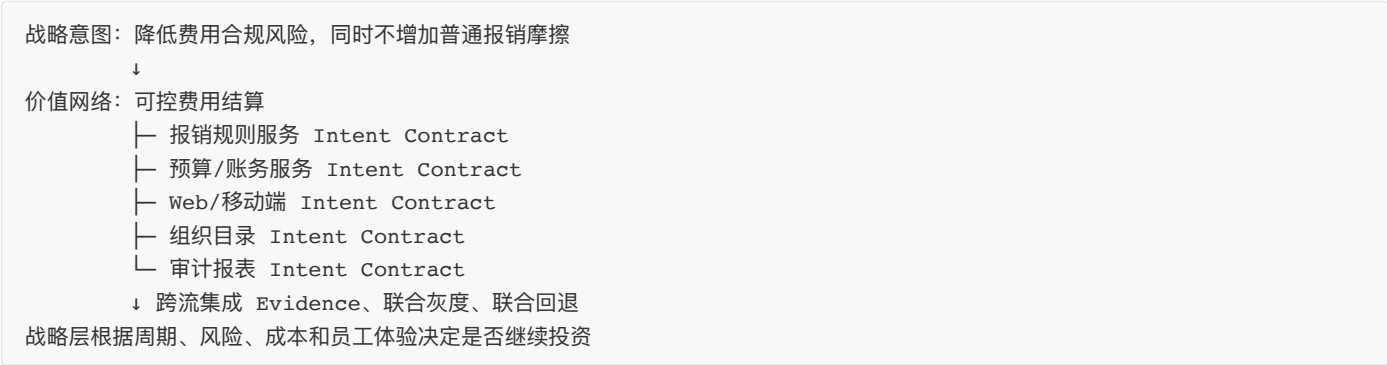
这时单个规则服务即使完成第 18 章的全部测试，也只能证明“规则服务任务通过”。它不能证明：移动端没有继续使用旧阈值，预算服务不会在审批前错误扣款，目录冲突不会自动放行，审计报表能否重建完整轨迹，或者所有服务是否能够一起回退。问题已经从“修改一个规则”变成“多个价值流共同交付可控费用结算”，因此必须建立价值网络。

价值网络层新增的不是一套更大的审批表，而是四个具体控制对象：

1. **Value Network Charter**：写清端到端结果、参与价值流、网络 Owner、接口责任、统一规则版本、联合发布范围和联合回退条件。
2. **Cross-stream Work Graph**：规定规则服务、预算服务、客户端、目录和审计的依赖；哪些可以并行，哪些必须等契约或制品就绪。

3. **集成验收与聚合 Evidence**：至少证明五件事：`5000.01` 在所有入口得到同一结果；审批状态能驱动预算状态；目录异常不自动通过；审计事件可追溯；功能开关可以让全链路恢复旧路径。
4. **解决方案级裁决**：网络 Owner 不能因为四个价值流各自 PASS 就宣布完成。只要一个关键集成证据是 `UNKNOWN`，就只能补证、缩小灰度或停止联合发布。

运行关系变成：



两种情况的差别不是团队数量，而是**是否存在一个单个价值流无法独立证明的端到端结果**。没有这种依赖，就停留在任务运行层；一旦存在共同结果、跨流依赖或联合回退，就必须把价值网络显式化。任务 PASS 只证明任务，网络证据证明端到端集成，战略证据才支持投资组合裁决。

22.3 规模化扩展的单位不是 Agent 数量，而是责任单元

一个成功试点往往依赖少数既懂业务、又懂工程治理的人。他们可以用口头补足契约、上下文和异常处理的缺口。把更多 Agent、更多许可证或更多任务投入这样的系统，只会把这份隐性经验放大成新的瓶颈。

企业真正需要扩展的是**责任单元**：它围绕一类价值流，能够独立完成意图签署、约束裁剪、受控执行、证据裁决、事后回溯和学习回写。一个责任单元至少有清晰的 IO、OA、执行能力、异常升级路径和可查看的运行证据。

判断一个单元是否成立，不问“使用了多少 AI”，而问：原推动者不在场时，它能否解释当前授权；出现规则冲突时，它能否停止并找到有权者；一次发布后，它能否回到契约、证据和回退对象。

22.4 从首个闭环到第二责任单元

首个闭环证明“这条价值流可以运行”；第二责任单元才开始证明“这套方法可以迁移”。第二个场景应有相近的治理语义、但不能只是同一团队复制同一份模板。它可以是相邻模块、不同产品线或另一类规则密集任务。

迁移时应保留稳定的语义，重新裁剪本地参数：

应保持一致	必须重新判断
意图、约束、证据和裁决的基本语义	业务目标、风险等级、数据和工具权限
IO、OA、AS 的责任分离	当前 Decision Owner 与升级路径
契约、Evidence、回溯的可追溯要求	AC、验证强度、灰度与回退方式
UNKNOWN 必须显式保留	本地系统、团队能力和历史债务

第二单元不是“照抄成功模板”的验收。它必须能解释哪些资产直接复用、哪些需要修改、哪些假设不再成立。若每个问题都只能回到首个试点的专家，组织复制的仍是个人能力。

22.5 多责任单元的共享治理资产要像内部产品一样运营

当两个以上责任单元反复使用同一类模板、检查、约束、Handoff 协议或恢复动作时，这些内容就不再只是项目文件，而是组织的治理资产。它们需要 Owner、版本、适用范围、发布说明、兼容策略、使用数据和淘汰机制。

优先产品化的通常不是大平台，而是摩擦最高、复用最多的最小资产：

- 常见任务类型的 Intent Contract 与 Example Mapping 模板；
- 可复用的 Constraint Matrix、Tools Manifest 与例外策略；
- 关键业务规则、集成接口和回退路径的验证器；
- Evidence Bundle、Release Manifest 与事件字段的稳定 Schema；
- 经验证的 Context、Memory、Skill 和 Failure Mining 模式；
- 用于下钻而非排名的遥测口径与 Dashboard。

资产产品化不等于中央团队垄断所有决定。中央或平台团队维护协议和默认能力；价值流团队拥有业务语义和风险取舍。前者保证可连接，后者保证不脱离现场。

22.6 什么时候值得建设平台管理共享治理资产

平台不是转型起点，而是对重复摩擦的投资。以下信号同时出现时，平台化才有明确理由：多个责任单元需要同一项能力；本地重复建设正在制造证据或权限不一致；人工汇聚、审计或跨工具交接已成为端到端瓶颈；继续依赖手工协调的成本高于维护共享能力的成本。

平台建设也应有自己的意图契约。它要明确服务哪些价值流、消除哪种可观察摩擦、哪些系统仍是事实源、如何渐进迁移、何时停止旧路径，以及成功后怎样降低每个任务的边际治理成本。一个统一门户、知识库或 Agent 编排器，如果不能解决已证实的问题，只会增加另一层等待和锁定。

当多个责任单元开始共享对象、状态、权限和动作时，本体（Ontology）就不再只是数据建模概念，而会成为平台能否支撑业务运行的关键。它对企业 AI 转型最重要的启示是：

企业真正缺的不是更多数据，也不是更大的模型，而是一个能够被人和 Agent 共同理解、查询、判断和操作的“组织运行世界”。

如果企业只有数据，没有统一的对象和状态；只有模型，没有可执行的动作和责任；只有 Agent，没有权限和证据，那么 AI 只能在局部环节提供聪明的回答，不能稳定参与企业运行。本体的价值，正是在数据、业务语义、动作、权限和反馈之间建立一层共同的操作语言。

22.6.1 从“描述世界”到“让组织能够行动”：本体的来龙去脉

“本体”（Ontology）原本是哲学中对“世界上有什么、这些东西如何存在和相互关联”的研究。进入软件与企业系统后，它不再主要讨论抽象的存在论，而是回答一组非常具体的问题：

企业中有哪些重要对象？
对象之间是什么关系？
对象当前处于什么状态？
哪些动作可以改变状态？
谁可以执行这些动作？
动作产生什么后果和证据？

企业在数字化过程中，通常经历了几层表达方式：

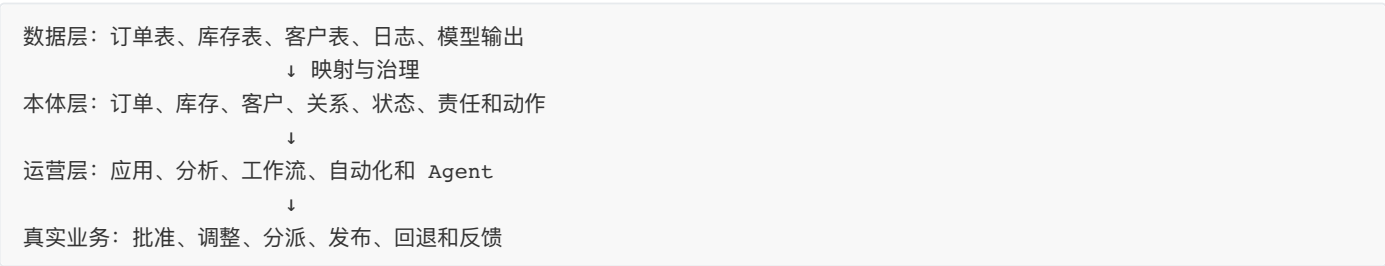
层次	主要回答	局限
数据表与字段	数据存在哪里、字段是什么	不一定表达业务含义和责任
数据模型	表之间如何关联、数据如何存取	仍然偏向系统实现，不一定对应真实业务对象
分类与术语表	同一个词是什么意思	能统一语言，但不能表达完整状态和动作
知识图谱	对象、关系和事实如何连接	通常擅长查询，未必能安全改变业务状态
企业本体	对象、关系、状态、动作、逻辑和权限如何共同运行	需要持续治理，不能一次建成

因此，本体不是“把所有数据画成一张关系图”，也不是给知识库换一个更有吸引力的名字。它把企业的**名词**和**动词**同时建模：名词是客户、订单、设备、合同、员工和任务；动词是批准、分派、暂停、发货、回退、升级和关闭。只有当对象和动作连接起来，AI才不只是理解企业信息，还能够在授权范围内参与企业运行。

22.6.2 Palantir 的实践：Ontology 是数据之上的运营层

Palantir Foundry 的 Ontology 做法值得借鉴的地方，不是某个产品界面，而是它对企业系统分层的处理：底层保留数据集、代码、模型和外部系统；上层用对象类型、属性、关系、动作类型、函数和权限，把这些资产连接到真实业务世界。[Palantir Ontology Overview](#)

可以用一个简化结构理解：



Palantir 的关键实践有四点：

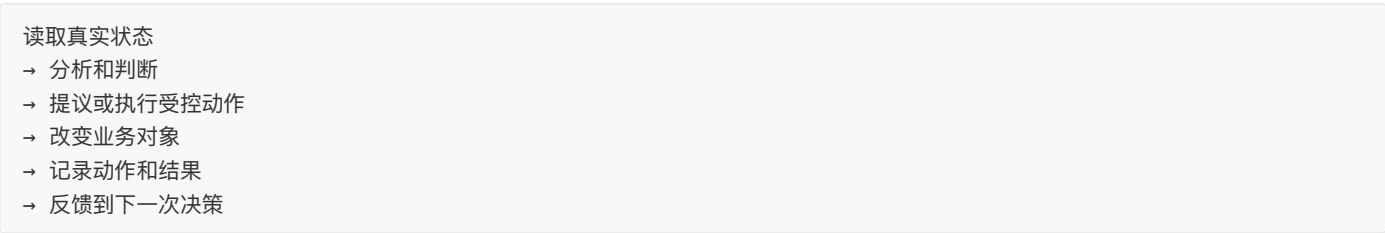
第一，把真实业务对象放到统一语义层。数据表中的一行订单，被提升为具有身份、属性、关系和状态的业务对象；不同系统中的订单、客户和库存不再只能通过人工解释拼接。

第二，把动作建模为受控事务。Action 不只是修改某个字段，而是一个有业务含义的动作，可以同时改变多个对象，并触发校验、通知、外部系统调用或其他副作用。[Object edits and materializations](#)

第三，把权限放进本体。权限不仅决定谁能访问一个应用，也可以细化到对象类型、关系类型、具体对象和具体动作。这样，“Agent 能不能调用这个工具”就不再是唯一问题，还要问“它能不能对这个对象执行这个动作”。[Object permissioning](#)

第四，把动作记录成可分析的组织事实。Action Log 将动作提交和相关决策记录下来，使组织能够回看谁在什么条件下做了什么改变，并把这些记录用于后续监控和决策。[Action Log](#)

这四点共同构成一条读写闭环：



这也是 Palantir 本体实践与普通数据平台、报表平台和静态知识库的根本差异：它不仅帮助组织“看见发生了什么”，还试图让组织在同一套语义和权限边界内“改变正在发生的事情”。

22.6.3 Ontology对企业 AI 转型的五个启示

Palantir 的实践真正值得企业借鉴的，不是复制一套平台产品，而是重新理解 AI 转型的对象。企业 AI 转型至少有五个启示：

第一，AI 转型首先是组织世界建模。企业不应从“把所有文档接入 RAG”开始，而应先选择一条真实价值流，明确其中的业务对象、关系、状态、责任和变化条件。Agent 需要知道的不只是“订单表里有什么”，还包括什么是订单、订单与合同如何关联、订单当前处于什么状态以及谁有权改变它。

第二，要从读数据走向受控改变现实。传统数据平台主要回答“发生了什么”，企业 Agent 还需要参与“下一步可以做什么”。但 Agent 不应直接获得任意 API 或数据库写权限，而应通过有业务语义的动作执行，并检查前置条件、授权、影响对象、失败处理和回退方式。

第三，权限必须绑定对象和动作。“这个 Agent 能不能调用工具”通常不够精确。真正的问题是：它能否查看这个对象、修改这个状态、代表这个角色批准这项动作，以及能否跨越组织和数据边界。权限要跟随对象、关系、动作和上下文，而不能只停留在系统入口。

第四，本体必须形成读写反馈闭环。如果本体只用于查询和可视化，它仍然是静态知识资产。受控动作改变对象后，动作、结果、异常、人工裁决和回退都应进入可分析的记录，反过来改善下一次决策、规则和编排。企业要从“看见问题”走向“改变状态，并知道改变是否有效”。

第五，本体是人与 Agent 的共同工作语言。业务人员、工程师、分析师、管理者和 Agent 可以使用不同界面，但应围绕同一组对象、状态和动作协作。这样，产品人员说的是“订单延期”，工程师看到的是状态和接口，Agent 处理的是可执行动作，管理者审查的是价值流结果，彼此不再依赖每次临时翻译。

这五个启示共同指向一个判断：企业 AI 转型不是给旧系统增加一个聊天入口，而是把企业从“系统记录数据、人员解释数据、人工推动动作”的模式，推进到“人和 Agent 共享业务世界，并在治理边界内协同改变它”的模式。

22.6.4 本体与 Agentic Agile：不是替代关系，而是上下层关系

Ontology 与 Agentic Agile 解决的问题不同，但可以组合成一个更完整的企业 AI 运行系统：

本体回答	Agentic Agile 回答
企业中有什么对象和关系？	组织希望改变什么价值？
对象当前是什么状态？	本次任务的真实基线和未知是什么？
哪些动作可以改变对象？	本次任务允许执行哪些动作？
谁能执行某个动作？	谁拥有意图、编排、验证和最终责任？
动作改变了什么？	什么证据足以支持接受、回退或扩大？
历史动作和结果是什么？	哪些经验应回写为契约、约束、测试或知识？

可以进一步映射到 3-4-3：

本体能力	Agentic Agile 机制
Object / Link	业务对象、意图图谱和跨模块关系
State	Baseline、任务状态和运行上下文
Action	Agent 可执行的业务动作与 Work Graph 节点
Function	业务规则、模型、验证器和编排逻辑
Dynamic Security	Constraint Matrix、授权矩阵和最小权限
Action Log	Evidence Bundle、Telemetry 和决策记录

但不能因此把二者混为一谈。Ontology 不自动决定“什么值得做”，也不自动承担残余风险；它为组织提供一个可被理解和操作的世界模型。Agentic Agile 则规定：

- 谁定义目标和价值取舍；
- 谁签署本次意图契约；
- Agent 能在什么范围内执行；
- 哪些判断必须升级给人；
- 如何用 Evidence 判断动作结果；
- 失败和经验如何进入下一轮治理。

一句话说：

本体告诉 Agent“组织世界中有什么、它们如何关联、哪些动作可以发生”；Agentic Agile 决定“为什么做、谁授权、做到哪里、如何证明以及谁承担后果”。

22.6.5 什么时候企业值得建设本体平台

企业不应一开始就建设所谓“全企业本体”。以下条件同时出现时，才值得把局部对象模型升级为共享平台：

1. 两个以上责任单元反复使用同一组业务对象和状态；
2. 不同价值流之间存在共享状态、跨系统动作、联合证据或联合回退；
3. 各团队正在重复建设对象定义、权限判断、动作接口和审计记录；
4. 手工解释和跨系统协调已经成为端到端瓶颈；
5. 对象 Owner、状态语义、动作责任和事实源已经能够被明确指认。

前四项说明存在重复摩擦，最后一项决定平台是否有可能建立可信语义。没有 Owner 的对象模型只能成为新的争论场；没有稳定状态和动作语义的本体，无法支撑 Agent 安全执行。

平台化的判断顺序应当是：

一个真实价值流闭环

→ 第二个相邻责任单元复用

→ 发现共同对象、状态和动作

→ 证明共享模型减少了实际摩擦

→ 再建设本体平台

平台建设本身也必须有 Intent Contract，至少说明：服务哪些价值流，统一什么对象语义，哪些系统仍是事实源，哪些动作允许写回，如何迁移，何时停止旧路径，以及成功后如何降低每个任务的边际治理成本。

22.6.6 一个最小本体的落地范围

以费用报销价值流为例，第一版不需要覆盖企业全部财务系统，只需定义一组能够支撑真实决策的对象：

员工、报销单、发票、成本中心、审批节点、付款单、财务规则

再定义它们之间的关系和关键状态：

员工提交报销单
报销单包含发票
报销单属于成本中心
报销单进入审批节点
审批节点产生付款单
报销单状态：草稿 → 已提交 → 审批中 → 已批准 / 已驳回 → 已支付

最后只开放少量有明确责任的动作：

提交报销、补充材料、批准、驳回、转人工、生成付款、撤回

每个动作都必须绑定：前置条件、授权角色、影响对象、业务规则、Evidence、失败处理和回退方式。这样建立的不是一张漂亮的图，而是一块能够支持人和 Agent 协作的运营基础设施。

22.6.7 不应该建设本体平台的情况

以下情况不适合立即建设企业本体：

- 只有一个孤立试点，没有跨价值流复用需求；
- 各部门对同一业务对象仍然没有共同解释；
- 对象状态经常被口头修改，无法追溯事实源；
- 没有明确的动作 Owner、授权人和回退责任；
- 平台只能提供查询，不能承载受控动作和反馈；
- 只是为了“拥有企业级 AI 平台”而建设平台。

本体平台不是转型的起点，而是责任单元经过验证之后，对共同语义和重复控制进行产品化的结果。最危险的不是没有本体，而是把错误的业务假设固化成全企业共享的本体。

22.7 连接责任单元：从局部闭环到价值网络

单个责任单元成立并具备复制条件后，扩展的重点不再是增加 Agent，而是识别多个责任单元之间已经出现的真实依赖，并把它们组织成可以共同承担结果的价值网络。

当多个单元开始共享客户、数据、接口、制品或发布窗口时，组织面对的已不是单元复制，而是价值网络协作。此时需要补充跨单元契约、统一身份与权限语义、证据聚合、依赖发布顺序和联动回退；这些工程机制将在第二十七章展开。

企业层面的关键不是强迫所有团队使用同一种工具，而是让它们能够交换同一种责任与证据语义：一个发布能够回到具体意图，一个例外能够找到责任人，一个跨模块故障能够定位到真实依赖，而不是在多个看板之间传递解释。

价值网络不是组织图上把几个团队框在一起，也不是平台接入数量达到某个规模后的自动结果。只有当跨单元的目标、依赖、授权和证据能够被持续管理，局部闭环才真正连接成了端到端的价值流。

22.8 经营价值网络：扩大、保持、收缩与停止

价值网络建立后，也不能默认进入全面推广。每个责任单元和每条跨单元价值流，都应在证据窗口后得到一种经营决定：

决定	适用事实	管理动作
扩大	净收益可重复，护栏稳定，恢复和责任边界清楚	扩大同类任务或迁移到相邻单元
保持	价值成立，但能力或成本尚不稳定	维持授权范围，修复一个明确短板
收缩	人工接管、例外或失败开始超过可接受范围	缩小权限、降低影响面、回到更强验证
停止	风险、总成本或业务效果持续劣于基线	关闭该路径，保留反例、证据和可复用教训

停止不是失败标签。能够用证据否决一项不值得扩大自治的路径，说明组织正在把资源从热情转向判断。真正危险的是把试点成功、平台建设和全面推广当作同一个不可逆项目。

22.9 分阶段扩展：一条可执行的路线

前一节回答价值网络形成后如何经营；本节回答组织何时进入下一层级。扩展不应一次性覆盖全员，而应把首年分为三个连续窗口。每个窗口都要完成对应分辨率的证据闭环，不能因为任务层成功就直接跳到战略层投资：

- 0—90 天：证明任务运行闭环。**运行第 21 章的试点，形成继续、受限继续、先补基础或停止的裁决；此时只证明选定任务类型，不宣称组织转型成功。
- 90—180 天：证明价值网络可协同。**选择一个真实端到端结果，连接至少两个有依赖的价值流，建立 Value Network Charter、跨流 Work Graph、集成证据和联合回退；根据端到端结果决定扩大、保持、收缩或停止。
- 180—360 天：证明战略投资值得继续。**将多个价值网络及其价值流放回 Strategic Intent Graph 和 Investment Contract，比较业务结果、风险、总成本、能力复用和机会成本，只为反复出现的摩擦建设共享资产或平台。

每个窗口结束时都应留下可供下一次投资决策使用的证据，而不是一份宣告“转型进入下一阶段”的汇报。扩展速度取决于价值、风险、恢复能力和责任成熟度，而不是管理层预先设定的 Agent 覆盖率。

22.10 本章小结

企业转型不是把首个成功案例复制到所有团队，而是先证明任务运行闭环，再把责任单元连接成价值网络，最后把价值网络放回战略意图与投资组合中分阶段扩展。每一步都要根据证据决定扩大、保持、收缩或停止；先运营共享资产，再投资平台。组织最终扩展的不是 Agent 数量，而是能在速度提升时仍保持责任、证据与恢复能力的运行方式。

本章最小实践：选择一个已有战略结果，分别写出一份 Strategic Intent / Investment Contract、一份 Value Network Charter，以及两份下属任务 Intent Contract；标出约束继承、证据聚合、三层 Decision Owner、证据窗口和四类经营决定。

完成证据：三层对象关系图、跨流依赖与联合回退、资产复用与差异清单、分层 Evidence 索引，以及扩展/保持/收缩/停止规则。

第二十三章 | 管理与评价体系：从个人产出到人机系统能力

AI 进入研发后，最容易被复制的是工具，最容易被破坏的是评价方式。若组织仍然用代码行数、工单数量、AI 使用率、Token 消耗或最低人工介入率评价个人，团队会主动选择简单任务、隐藏失败、削弱门禁，最后得到一组漂亮但无效的数字。

管理者真正需要回答的是：这套人机协同系统是否创造了更多有效价值，又是否把成本、风险和验证压力转移到了别处。

23.1 管理对象发生了变化

传统管理主要观察人、岗位、团队和项目。Agentic Agile 增加了任务、Agent、工具、契约、证据和组织记忆，但管理对象不应因此变成“每个人调用了多少次 AI”。真正应观察的是一条价值流：目标是否准确，执行是否受控，失败是否及时暴露，结果是否被正确接受，能力是否能够复用。

个人活动 → 任务结果 → 价值流表现 → 组织能力变化

越接近左侧的数字，越容易被操纵，也越不适合直接作为绩效结论；越接近右侧，越能说明系统是否形成了真实能力，但也越需要结合场景和责任边界解释。

管理对象从个人活动转向价值流表现之后，指标也必须从“工具用了多少”转向“结果是否可靠”。

23.2 用系统指标替代工具指标

不应单独追求	更值得观察
AI 调用次数	有效任务首次成功率
生成代码行数	从意图到可接受结果的周期
Token 越少越好	每个有效结果的总成本
HITL 越少越好	人工介入是否发生在正确风险节点
测试全绿次数	逃逸缺陷、回退成功率和证据完整度
Agent 数量	单位协调成本下的净交付能力

这些指标也不能脱离基线、任务类型和风险等级比较。一个资金规则任务比一个文案调整需要更多人工判断，并不代表它的自治水平更低；一次主动阻止高风险发布，也可能是治理能力提升而不是效率下降。

三个数字不能混为一个结论

围绕 AI 的管理讨论经常把以下三个数字混成“效率”：

AI 生成了多少代码？	→ 执行贡献
任务完成得有多快？	→ 研发执行
组织交付了多少有效价值？	→ 组织效能

第一项上升，可能只是实现环节更快；第二项上升，可能仍被需求等待、集成、审批或发布瓶颈抵消；只有第三项能够说明局部提效是否穿过了整条价值流。Anthropic 相关案例的启示正是如此：即使 AI 贡献了很高比例的生产代码，也不能据此推出研发效率或组织效率按同一比例增长。

管理者应把代码贡献率作为过程信号，而不是绩效目标；把端到端周期、首次成功率、返工、缺陷、残余风险、人工判断质量和业务结果放在同一组指标中观察。尤其要确认：人是否因为 Agent 加速而获得了更多高质量判断时间，而不是被迫在更大的生成量后面做更多救火。

指标不会自动改进系统。必须有人解释这些信号，并据此调整授权、能力建设和工具投资。

23.3 新管理者的五项责任

管理者不再只是分配人力和检查进度，还要承担五项系统责任：

- 1. 选择适合自治试点的价值流，并明确不应自动化的边界；
- 2. 确保 IO、OA、AS 的责任能够被独立指认；
- 3. 保护团队报告失败、UNKNOWN 和停止事件的权利；
- 4. 根据 Evidence 与 Telemetry 调整授权、能力建设和工具投资；
- 5. 防止组织把遥测变成人员监控或绩效排名工具。

管理者的核心问题不应是“为什么这个 Agent 还没有全自动”，而应是“当前证据说明哪里可以扩大授权，哪里必须收缩，下一轮应该改变什么”。

管理责任确定后，还需要把判断落到不同层级，避免用一次局部任务的结果直接评价个人，或用一个组织平均数掩盖价值流问题。

23.4 绩效评价的三层结构

可以把评价拆为三层，避免把组织结果简单归因给个人：

层级	评价对象	示例
任务层	本次任务是否完成并可接受	AC、证据、返工、残余风险
价值流层	同类任务是否持续改善	周期、等待、缺陷、成本、客户结果
组织能力层	组织是否形成可复用能力	契约复用、知识晋升、授权扩大、跨场景迁移

个人辅导可以使用任务层证据，但不应只依据单一指标下结论。评价时还应考虑任务难度、风险、上下文质量、权限约束和团队协作条件，否则系统问题会被错误地转化为个人问题。

三层评价解决了“看什么”和“在哪一层看”的问题；如果激励方向错误，组织仍然会主动制造好看的数据。下面用一个反例说明这种异化如何发生。

23.5 一个反例

某团队把“AI 自动完成率”设为季度目标，结果出现三种行为：成员把复杂任务拆成大量简单任务；遇到未知项时直接选择默认路径；为了减少人工介入，关闭了原本必要的安全审查。仪表板显示自治率上升，但逃逸缺陷和返工同时上升。

修正方式不是换一个更复杂的自动化率公式，而是撤销单指标目标，把观察改为“风险调整后的有效交付”：首次成功率、残余风险、回退成功率、人工介入质量、总成本和知识复用率共同解释结果。任何指标改善都必须同时检查质量、责任和恢复能力是否恶化。

评价体系必须帮助组织暴露问题，而不是替问题遮挡。只有当指标进入组织学习会议并触发下一轮调整，它们才开始产生管理价值。

23.6 从遥测评审到组织行动

前一节的反例说明，指标只有进入真实行动，才有管理价值。组织学习会议因此不应成为逐项念数字的会议，而要把遥测中的变化转成下一轮可验证的调整。一次有效评审至少回答：

- 哪类任务的首次成功率明显变化？
- 哪些失败重复出现，说明约束或上下文存在系统缺口？
- 哪些人工升级是必要控制，哪些是流程等待？
- 哪些 Agent 能力可以扩大授权，哪些应收缩？

- 哪项知识、工具或平台投资能够降低下一轮总成本？

会议输出必须是行动：修改契约模板、增加测试、调整权限、停止某类自治、补充培训或迁移到下一场景。只有形成责任人、期限和验证方式，Telemetry 才真正进入管理系统。

学习会议可以改进一条价值流；跨团队风险、共享资产、重大例外和自治范围，则需要组织级裁决机制处理。

23.7 从局部行动到组织级裁决

学习会议解决的是单条价值流如何调整；当问题跨越团队、风险等级或共享资产时，就需要组织级裁决。治理委员会不审批每一行代码，也不接管日常低风险任务。它负责组织级规则：确定风险分级、批准高影响约束和例外、处理跨团队冲突、复盘重大异常，并依据证据决定自治范围扩大还是收缩。

委员会的输入应是 Evidence 与趋势，而不是汇报型 PPT。一个有效委员会看趋势和例外，把一次裁决沉淀为可复用政策；如果所有任务都必须排队等待委员会，说明授权矩阵和机械门还没有建设好。

治理资产也应按内部产品运营：有所有者、版本、发布说明、使用数据、问题反馈、兼容迁移、淘汰机制以及贡献审查流程。契约模板、约束、Skill、Dashboard 和恢复 Playbook 只有在持续维护时，才能跨团队复用。

组织评审应与三层运行对象对齐，不能用一次“AI 转型汇报”同时替代三种裁决：

评审分辨率	输入	主要问题	输出
任务运行层	当前任务 Evidence、失败、回退和人工升级	这项工作是否可以接受、补证、返工或停止？	任务裁决与下一次授权
价值网络层	多条价值流的聚合 Evidence、依赖、集成结果和联合风险	端到端结果是否成立，哪个依赖需要解除或收权？	集成、排序、扩大或收缩决定
战略意图层	价值网络趋势、业务结果、总成本、风险和机会成本	哪些方向值得继续投资，哪些应调向、暂停或停止？	投资、能力建设和红线调整

任务运行层评审可以高频自动化，价值网络层需要围绕集成和依赖运行，战略意图层按投资窗口运行。上层评审不能把缺少下层来源的平均数当成事实，也不能越级批准下层未验证的执行。

治理机制确定了谁可以调整规则，也把局部发现提升为组织政策。但规则一旦进入绩效、资源和晋升体系，就可能被重新解释，甚至诱导人们为了指标而优化指标。

23.8 让激励与评价不被异化

错误激励会破坏真实遥测。把“自动化率最高”“HITL 最少”或“Token 最低”设为单一目标，会诱导团队绕过必要检查、隐藏失败或挑选简单任务。

更健康的评价组合包括：

- 目标是否准确；
- 关键约束是否稳定；
- 失败是否如实记录；
- 是否减少重复缺陷；
- 是否沉淀可复用资产；
- 是否在相同风险下提高效率；
- 是否让价值决策更透明。

这些指标用于解释系统能力，不应用于简单排名个人。必要的人工介入、主动阻止高风险发布和如实暴露 UNKNOWN，都可能是治理有效的信号。

即使指标和激励设计正确，如果成员不相信遥测不会被用来惩罚自己，真实反馈仍然不会出现。也就是说，激励设计解决“组织希望什么行为”，但还没有解决“成员是否愿意提供真实信号”。因此，管理体系的最后一道边界是信任。

23.9 以信任换回真实反馈

要让前面的指标、行动、治理和激励形成闭环，组织需要公开说明遥测评估的是系统，而不是把每一次 AI 调用转为人员监控；专业知识不会被抹去，而会转化为更有影响力的约束、测试、规则和裁决资产；轻量任务使用轻量治理，自治扩张必须由证据支持。

当团队能够安全报告失败、UNKNOWN 和停止事件，管理系统才会得到真实信号。若成员担心数据被用于惩罚，他们会把问题藏在系统之外，最终使组织失去学习能力。

当管理对象、指标、责任、评价、治理和信任被放进同一条链路，组织才有可能从“统计 AI 使用情况”转向“经营人机系统能力”。

23.10 本章小结

AI 时代的管理不是追踪机器使用量，而是经营一套人机协同生产系统。健康的管理与评价体系从个人活动上移到任务结果、价值流表现和组织能力：用治理机制维护共同规则，用指标组合防止局部优化，用学习会议把遥测转成行动，用信任保护真实反馈。

本章最小实践：删除一个正在驱动错误行为的 AI 指标，为同一价值流建立包含结果、质量、成本、风险和学习的指标组合，并明确组织级审查与行动闭环。

第二十四章 | 风险与行业裁剪：操作系统相同，参数与责任边界不同

24.1 不建立一套适用于所有行业的阈值

金融、车载、政企、互联网和内部工具的风险不同。图书不能替代法律、监管、安全和行业专家，也不应给出未经验证的统一留存年限、自治上限或发布阈值。

3-4-3 提供的是裁剪框架：识别风险，建立约束，定义证据，把不可自行决定的事项交给正确的人。

这里需要区分两个经常被混淆的概念：**治理原则可以复用，治理参数不能照抄**。无论在哪个行业，目标都要有人负责，权限都要有边界，关键行为都要留下证据，不可逆操作都不能由 Agent 自行批准；但什么数据属于敏感数据、什么变化需要双人复核、证据保存多久、灰度比例多大、谁有资格签署，必须由所在组织依据业务影响、适用法规和工程能力确定。

行业裁剪也不是简单地“删掉几份模板”。真正的裁剪，是回答四个问题：哪些风险在当前场景中真实存在，哪些控制可以机械执行，哪些证据足以支持下一步行动，哪些剩余风险必须交给具备责任资格的人。轻量化可以减少工件数量和审批层级，但不能删除责任、边界、验证和恢复。

因此，同一个“生成代码”任务在不同场景中可能对应完全不同的治理重量。内部演示工具可能只需要轻量契约、沙箱和自动测试；涉及客户资金的规则变更则可能需要职责分离、独立对账、合规审阅、灰度窗口与可追溯发布。差异不来自模型名称，而来自失败后果和组织承担责任的方式。

24.2 六个裁剪问题

- 1. 任务会接触什么数据？
- 2. Agent 可以使用哪些工具和权限？
- 3. 哪些行为不可逆？
- 4. 哪些失败会造成安全、资金或合规影响？
- 5. 哪些证据才能支持发布？
- 6. 哪些决策必须由具备责任资格的人批准？

六个风险问题确定治理重量，四维检查则保证裁剪能够进入企业运行方式：

转型维度	行业裁剪必须确认
人员	哪些判断需要执业资格、双人复核或领域专家，哪些执行可交给 Agent？
组织	最终责任主体、Decision Owner、例外批准和事故指挥权属于谁？
流程	哪些状态必须独立验证、监管报送、留痕、灰度或人工接管？
工具	数据驻留、模型路由、权限、证据保全、供应链和恢复能力需要什么等级？

只提高扫描强度而不改变责任与流程，不是行业裁剪；只增加审批而没有可执行工具控制，也不是。相同的 3-4-3 操作系统可以复用，但参数、证据门槛、自治范围和责任资格必须由具体场景决定。

这六个问题不应只在项目启动会上讨论一次，而应被转成可追踪的“风险裁剪卡”，分别写入治理工件：

裁剪问题	应落入的治理资产	最低可验证结果
数据是什么	数据分级、Tools Manifest、模型路由策略	未批准数据无法离开允许边界
工具与权限是什么	工具白名单、最小权限、审批策略	越权调用被阻断并留痕
哪些动作不可逆	Work Graph、停止条件、HITL 节点	不可逆步骤必须等待责任人批准
失败影响是什么	风险记录、威胁模型、故障模式	每个高影响失败都有预防、发现与恢复控制
什么证据足够	AC、验证计划、Evidence Bundle	发布结论可以回溯到独立证据
谁承担责任	RACI、签署矩阵、例外政策	Agent 不能替代具备资格的责任主体

裁剪过程可以按“场景事实 → 风险等级 → 治理 Profile → 增量约束 → 证据要求 → 人类责任人”展开。若信息不足，应保留 UNKNOWN 并提高治理重量，而不是为了快速启动把未知项默认解释为低风险。随着证据增加，治理可以升级；任何降级都应有明确理由、批准人和可撤销条件。

24.3 金融场景

可能需要强化：

- 数据最小化和脱敏；
- 资金与权限双重审批；
- 规则版本和审计链；
- 确定性计算与对账；
- 更严格的回退和发布窗口；
- 法务、风险和合规签署。

示例阈值必须由具体机构依据适用监管和风险政策确定。

例如，Agent 辅助修改授信规则时，真正的治理对象不是“生成了一段规则代码”，而是这段变化是否会影响客户资格、额度、定价或公平性。Intent Contract 应明确允许调整的规则范围和禁止触碰的政策；Constraint Matrix 应把金额边界、职责分离、数据用途和审批责任写成 MUST；验证至少包含历史样本回放、边界值、反例、差异分析与独立对账。Evidence Bundle 不能只展示单元测试通过，还应说明影响了哪些客群、是否出现异常分布、使用了哪个规则版本，以及谁批准进入下一阶段。

对于支付、清算和账务场景，系统还要回答“正确一次”之外的问题：重复执行是否幂等，部分失败如何补偿，账实不符怎样停止，重试会不会重复扣款，人工修复是否留下双向审计链。Agent 可以发现异常、生成候选修复和组织证据，但资金动作、关键参数和例外放行仍应由授权角色裁决。

24.4 车载与安全关键系统

可能需要强化：

- 安全等级与系统边界；
- 仿真、台架和实车分层验证；
- 时序、不变量和故障模式；
- 供应链和软件物料清单；
- OTA 灰度、熔断和恢复；
- 功能安全与网络安全责任人审批。

Agent 可以辅助生成和验证，但不能绕过安全生命周期要求。

在这类系统中，代码通过通常只是验证链的起点。需求、安全目标、系统边界、接口假设、故障模式、测试环境和发布对象必须可追溯。Agent 生成的测试也需要证明其覆盖了安全不变量，而不是只覆盖正常路径。仿真通过不能自动替代台架验证，台架通过也不能自动推导实车安全；每一层证据只能支持其适用范围内的结论。

一个典型的裁剪结果可能是：开发环境允许 Agent 自动修改并运行仿真；进入安全相关分支前必须锁定依赖和配置；台架、道路与 OTA 阶段设置独立 HITL；任何安全约束变化都触发重新评估影响面。发布证据还应包含软件物料清单、工具链版本、已知偏差、回滚条件和车辆批次范围，避免把一次实验成功扩张成全量安全结论。

24.5 政企与涉敏环境

可能需要强化：

- 网络和模型部署边界；
- 数据分级分类；
- 工具和外部连接白名单；
- 国产化环境兼容；
- 审批、留痕和证据保全；
- 涉密场景的人类责任链。

政企场景常见的误区，是只关注“模型是否私有化部署”，却忽略插件、MCP、日志、遥测、缓存、向量库和运维通道同样可能形成数据出口。真正的边界应覆盖完整数据流：数据从哪里进入，经过哪些 Agent 和工具，被哪些中间系统保存，谁能够检索，何时删除，发生异常时怎样证明没有继续扩散。

在隔离网络或涉敏环境中，Tools Manifest 应采用显式允许而不是默认开放；依赖、模型和 Skill 更新需要离线校验与版本固定；Evidence Bundle 应避免复制敏感正文，而用摘要、哈希、分级标识和受控引用证明结果。对于法定审批、公共资源分配和行政决定，Agent 可以提供检索、比对与建议，但最终决定必须保留清晰的人类责任链、理由和申诉入口。

24.6 互联网与高频业务

高频交付不等于低合规。互联网产品同样涉及隐私、账号、支付、内容和用户权益。

可以重点设计：

- 小流量灰度；
- 业务漏斗遥测；
- 自动回退；
- 用户影响边界；
- 大促资源预算；
- 实验伦理与隐私保护。

速度可以压缩范围和反馈周期，不能压穿底线。

互联网场景更适合把治理做成“快速反馈的控制回路”，而不是在发布前堆积一次性审批。团队可以用功能开关、用户分群、流量上限、错误预算和自动回退，把一个大风险拆成多个可观察、可停止的小步骤。但灰度本身不是免责机制：若实验涉及未成年人、敏感画像、价格歧视、内容安全或难以撤回的用户权益，即使只影响 1% 用户，也可能仍是高风险任务。

例如，推荐算法或运营文案由 Agent 高频迭代时，Evidence Bundle 除技术指标外，还应包含目标人群、实验假设、负向指标、投诉与伤害信号、停止阈值和实验后处置。只看点击率上升，可能会奖励夸张内容、成瘾设计或对弱势群体不公平的策略。价值遥测必须与护栏指标一起解释，不能把短期增长自动等同于业务正确。

24.7 医疗与生命科学

医疗场景需要先区分科研辅助、行政支持、临床决策支持和直接诊疗控制。四类任务看似都在“处理医疗信息”，但责任、证据和准入要求完全不同。越接近患者诊疗、剂量、诊断或生命支持，越不能用通用软件测试替代专业验证。

可能需要强化：患者数据最小化与用途限定；数据集代表性和偏差分析；模型、Prompt、知识库与指南版本追踪；临床专家复核；适用人群和禁用场景；错误建议的升级与纠正；对患者可理解的说明。Agent 可以汇总病历或提示潜在冲突，但不能掩盖不确定性，也不能把“模型高置信度”当作临床证据。

24.8 工业、能源与生产控制

工业场景需要区分办公建议、生产排程、设备参数优化和直接闭环控制。Agent 越接近物理设备，验证就越需要覆盖实时性、环境边界、故障安全、人工接管和网络隔离。

一个稳妥的渐进路径是“只读观察 → 离线建议 → 人工确认执行 → 受限自动执行”，每一级都用真实运行证据决定是否扩大自治。数字孪生和历史回放可以降低试验成本，但不能证明所有现场条件；进入生产控制前，还需要明确参数上下限、联锁保护、异常停机、手动接管和恢复演练。

这些行业的治理重量虽然不同，背后却可以归结为同一组变量：失败后果、可逆性、数据敏感度、验证能力和责任资格。把行业差异落到这些变量上，组织才能形成风险等级、控制链和可执行的恢复机制，而不是为每个行业重新发明一套方法。

24.9 把行业差异落到风险分层

风险	推荐治理
低风险、可逆、内部使用	轻量契约、核心约束、自动验证
中风险、影响用户、可灰度	完整 Evidence Bundle、灰度、遥测和回退
高风险、不可逆、资金或敏感数据	强约束、独立验证、人工批准和审计
安全关键或监管关键	行业生命周期、专家签署、隔离环境和正式合规证据

风险层级不是给整个项目贴一个永久标签。更准确的做法是按任务、数据、工具和动作分别判断，并取其中足以决定控制强度的高风险项。同一产品中，“生成测试数据”可能是低风险，“修改生产权限”是高风险，“自动执行资金或物理控制”则可能属于监管或安全关键。若只按项目平均风险治理，高风险动作很容易被低风险背景稀释。

治理 Profile 也不是成熟度荣誉。选择 high-risk、legacy 或 multi-module，并不代表团队落后，而是说明当前失败后果、未知项或协作复杂度需要更多控制。只有当历史证据证明特定控制可以被更可靠的自动机制替代，组织才应考虑减轻人工步骤；减少文档不是目标，降低剩余风险和决策成本才是目标。

风险分层决定治理需要多重；要把控制放对位置，还要知道风险从哪里进入，又会怎样跨层放大。

24.10 Agentic 风险的七类来源

意图风险

目标模糊、激励错误、非目标缺失、规则冲突。主要通过批判性思维、Grill-Me、契约和签署治理。

上下文风险

信息过期、来源冲突、Prompt Injection、Context Poisoning 和不可信内容获得过高指令权。主要通过分层上下文、来源优先级、内容隔离和指令信任治理。

数据与隐私风险

源代码、设计、客户数据或身份信息被发送到未批准模型、地区、日志、缓存、向量库、Memory、MCP 或第三方工具；供应商训练、留存、删除和子处理边界不清；Telemetry 与 Evidence 过度复制敏感正文。主要通过数据分级、目的限定、最小化、模型路由、DLP、合同条款、Tools Manifest、保留删除策略与负向验证治理。任务级控制见第十一章；本章处理行业治理重量、责任资格和事故边界。

工具风险

权限过大、任意命令、生产访问、供应链工具不可信。主要通过 Tools Manifest、沙箱、批准和审计治理。

执行风险

死循环、资源争用、重试风暴、跨模块漂移。主要通过 Work Graph、预算、超时、熔断和 Protocol 治理。

验证风险

测试确认偏差、Judge 未校准、证据过期、UNKNOWN 被隐藏。主要通过独立验证、证据映射、一致性和时效检查治理。

组织风险

职责模糊、OA代签、指标游戏、例外泛滥、事故数据美化。主要通过角色责任、机械门、真实遥测和治理委员会治理。

七类风险往往不是独立发生，而是形成链式放大。一个过期的外部页面首先是上下文风险；其中隐藏的指令诱导 Agent 调用高权限工具，转化为工具风险；自动重试扩大影响，成为执行风险；测试只验证正常返回，又形成验证风险；团队以“无人干预率”奖励 Agent 不上报异常，最终演变为组织风险。行业裁剪因此不能只列单点控制，还要检查风险如何跨层传播。

建议为高风险任务记录“风险链”：触发源、传播路径、放大条件、发现信号、阻断点、恢复动作和责任人。它能帮助 OA 判断控制应放在哪里——不是在每一层重复审批，而是在最早、最可靠且最便于验证的位置切断传播。

风险来源地图帮助我们看见风险如何传播；但要决定具体放置哪条约束、哪项观测和哪种恢复动作，还需要把它展开成逐项可验证的威胁模型。

24.11 把风险链转成威胁模型

每个高风险任务可以按以下路径分析：

资产是什么

→ 谁或什么 Agent 可以访问

→ 可能发生什么错误或攻击

→ 哪条约束负责预防

→ 哪项观测负责发现

→ 哪个动作负责阻断

→ 怎样恢复

→ 谁承担最终责任

威胁建模不只针对恶意攻击，也包括模型误解、错误工具调用和组织性绕过。

威胁模型至少应产出一张可执行映射，而不是停留在头脑风暴：

资产或目标	威胁/失效方式	预防约束	发现证据	阻断与恢复	责任人
客户敏感数据	被送往未批准模型或日志	模型路由、DLP、最小权限	出站审计、数据流检查	断开连接、撤销令牌、删除与通报	数据责任人
生产配置	Agent 误改或批量扩散	只读默认、审批、变更范围	配置差异、异常指标	熔断、回滚、隔离批次	系统责任人
发布结论	证据不完整却被判定通过	AC 映射、独立验证、时效门	Evidence 审计、UNKNOWN 列表	阻断发布、补证、重新裁决	IO / 验证责任人

威胁建模完成后，应把结果分别回写到 Constraint Matrix、Tools Manifest、Work Graph、验证计划和事故预案。只保留一份孤立的风险表，不能改变 Agent 的实际行为。

威胁模型让控制点进入任务运行，但模型、Skill、MCP、依赖和配置的变化，也可能从系统外部重新打开已经关闭的风险入口。这些组件共同构成了 Agentic 系统的供应链。

24.12 把供应链纳入风险控制链

Agent 可能自动安装依赖、调用 MCP Server、下载脚本或使用外部 Skill。团队需要检查：

- 来源和许可证；
- 版本锁定；
- 权限需求；
- 网络访问；
- 是否读取敏感上下文；
- 更新是否会改变行为；
- 是否能够离线或降级运行；
- 产生的证据是否可信。

开源不自动等于安全，可运行也不等于可授权。

Agentic 供应链还包括 Prompt、Skill、Agent 配置、MCP Server、模型版本、Judge、数据集和自动下载脚本。它们未必进入传统软件物料清单，却可能直接改变系统决策。组织需要维护“Agentic BOM”：组件来源、版本摘要、能力、所需权限、数据访问、更新策略、验证状态和责任人。

升级不能只验证“是否还能运行”，还应进行行为差异测试：工具权限是否扩大，默认模型是否变化，Prompt 是否改变决策边界，Judge 是否产生系统性偏差，遥测是否新增数据出口。关键组件应支持固定版本、可信摘要、隔离验证和快速回退；无法说明来源或权限的组件，不应仅因效果演示良好就进入生产 Harness。

供应链控制降低了外部变更带来的不确定性，但任何控制都可能失效。组织必须预先规定失效后的停止、隔离、取证、通知和恢复路径，而不是等事故发生后临时分工。

24.13 把控制失效纳入事故响应

Agentic 事故响应至少包括：

1. 停止相关 Work Graph；
2. 撤销工具和数据权限；
3. 隔离受影响产物；
4. 保存契约、上下文、工具调用和证据；
5. 判断是否需要回滚；
6. 通知责任人；
7. 修正 Evidence Bundle 和遥测；
8. 运行 Bug 回溯；
9. 更新约束、图谱和治理资产。

事故不能只归因于“模型幻觉”。需要沿治理链寻找为什么错误能够发生、扩散并被放行。

组织可以按影响范围和可逆性定义事故等级，但分级标准必须由本组织确定。低等级事件可以由预设 Runbook 自动隔离并通知；涉及敏感数据、资金、用户权益、安全控制或大范围生产影响的事件，应立即触发责任人、法务、安全或行业专家参与。事故期间首先保护人、数据和系统，其次保护证据，最后才是恢复交付速度。

恢复服务也不等于事故关闭。关闭前至少要回答：受影响范围是否确认，错误产物是否全部隔离，权限是否恢复到最小状态，客户或监管通知是否完成，原 Evidence Bundle 的哪些结论已失效，新的防复发控制是否经过真实测试。定期演练比只写预案更可靠，尤其要演练 Agent 无法停止、权限撤销失败、证据不完整和回滚本身造成二次影响等困难情形。

事故响应把“可停止、可恢复、可追责”从原则变成运行能力。只有这些能力经过真实演练，组织才有依据讨论自治范围能否继续扩大。

24.14 在可验证边界内扩展自治

更强模型、更长上下文、自适应路由、机器可读政策和动态软件可能继续改变研发。但以下原则不会因为技术进步自动消失：

- 目标需要责任主体；
- 权限需要边界；
- 不可逆操作需要批准；
- 证据需要独立性；
- 系统需要可观测、可停止、可恢复；
- 法律与伦理责任不能交给概率模型。

未来可以更自治，但必须先更可验证。

24.15 本章小结

行业不同，约束和证据强度不同；治理原则保持一致。真正可靠的行业适配不是套用一组神奇阈值，而是由了解业务、技术和监管的人共同签署一套可执行边界。

裁剪后的结果应当能够回答一条完整链路：当前场景有哪些事实与 UNKNOWN，为什么选择这一治理 Profile，新增了哪些行业约束，需要什么独立证据，哪些动作必须 HITL，失败后如何停止和恢复，最终由谁承担责任。如果这些问题仍只能靠会议口头解释，说明行业要求还没有真正进入 Harness。

本章最小实践：选择一个真实行业任务，完成六个风险问题和人员、组织、流程、工具四维裁剪，并沿“事实 → 风险 → Profile → 约束 → 证据 → HITL → 恢复”形成一页风险裁剪卡。

完成证据：行业约束增量、四维责任与能力差异、风险链、威胁模型映射、最低证据集、HITL 节点和恢复条件。

延伸实践：推荐学习《Agentic Agile FDE 进阶：从客户现场到企业级生产交付》，把本章的风险裁剪、行业边界和恢复机制迁移到复杂客户环境。

第五部分 | 工程扩展与规模化实践

单项任务能够形成可信闭环之后，工程能力还要面对三种扩展：进入既有系统时如何找到可靠事实，任务结束后如何让经验安全回流，以及多个责任单元如何共同组成一个可发布的系统。

本部分阅读路径

1. **第二十五章 | 在任务开始前建立可信上下文**：既有系统中哪些事实可以作为依据，哪些内容仍然未知？
2. **第二十六章 | 让任务经验安全回流下一次任务**：哪些经验可以影响下一次行动，怎样避免一次局部经验污染全局？
3. **第二十七章 | 让多个局部闭环组成可信交付**：多人、多工具、多模块如何在共同事实、契约、证据和发布边界上协作？

管理者可以先读第四部分；工程负责人可把本部分作为实施参考。规模化的前提不是接入更多 Agent，而是小责任单元已经能够持续产生可信证据。

第二十五章 | Knowledge Engineering：在任务开始前建立可信上下文

当 Agent 进入既有系统，最危险的不是“找不到文档”，而是找到了过期、越权或不适用的内容，却把它当成事实。Knowledge Engineering 解决的是知识如何被发现、裁剪、验证、授权和失效，而不是简单建设一个 RAG 或向量数据库。

25.1 三种视图：Document Map、Code Map 与 Trace Link

一项任务至少需要三种视图：

视图	回答的问题	典型对象
Document Map	为什么这样做，什么才算完成	需求、规则、AC、ADR、约束
Code Map	在哪里实现，改动影响什么	仓库、模块、符号、调用、测试
Trace Link	需求、代码和测试是否闭环	Requirement ID、Symbol ID、Test ID

Document Map 不应猜测代码，Code Map 也不应臆造业务意图。Trace Link 用统一 ID 把二者连接起来：

```
requirement_id: REQ-PAY-017
acceptance_criteria: [AC-PAY-017-01]
implemented_by: [payments.refund_service.create_refund]
verified_by: [tests/refund/test_refund_window.py]
```

三种视图中，Document Map 与 Code Map 是两张基础地图：前者整理业务依据，后者呈现实现结构。Trace Link 是连接层，把两张地图中的对象连成可验证关系。它们不是要求团队再建设一个“大知识库”，而是把现有文件和代码组织成可查询、可追溯的结构。

能力	推荐落点	负责什么	不负责什么
Document Map	IWE	从本地 Markdown、Front Matter、显式链接中建立需求、规则、决策和依赖关系	不替代需求签署、Owner 裁决和 Evidence
Code Map	codebase-memory-mcp	从代码结构中建立文件、符号、调用、接口和测试关系，并按项目持久化索引	不解释业务意图，不自动决定是否可以修改
Trace Link	项目内可版本化的 ID 与关系记录	把需求、代码、测试和证据连接起来	不用模糊相似度猜测对应关系

IWE 适合做 Document Map 的结构底座：需求、业务规则、验收标准、术语、ADR 和会议结论仍然是人可以审阅、Git 可以版本化的普通文件。Front Matter 可以固定 req_id、状态、Owner、适用范围和版本；显式链接可以表达 depends_on、supersedes、constrained_by 和 verified_by。文档重命名或重构时，链接关系也能被检查和维护。

Document Map 也不等于企业本体。Document Map 主要保存“为什么这样做、哪些规则仍然有效、哪些决策由谁作出”；企业本体进一步把业务对象、关系、状态和可执行动作组织成运营层。可以把二者理解为：Document Map 维护组织的解释与依据，本体承载组织正在运行的对象与动作。若要把局部知识工程扩展为企业级平台，必须先在责任单元中验证对象语义、状态和动作责任，再进入第二十二章 22.6 所讨论的平台化阶段。

codebase-memory-mcp 适合做 Code Map 的工程底座：它通过代码解析建立函数、类、调用链、路由和跨模块关系，支持 Agent 在需要时查询结构和影响范围，而不是把整个仓库一次性塞进上下文。代码地图必须绑定仓库、分支或 Commit；代码变化后要增量刷新，否则“地图上的事实”可能早已不是当前事实。

Document Map 和 Code Map 都是可选的 Context Provider，不是 Agentic Agile 的强制依赖；Trace Link 是连接层，不是另一个知识库或地图。工具可以替换，分工不能混乱：IWE 不应被当成业务审批系统，Code Map 不应被当成需求解释器，Trace Link 也不能用相似度猜测对应关系。任何工具的索引结果都不能绕过契约、约束和 Verify。

最小落地可以按四步完成：

- 1. **先整理 Document Map**：为当前任务补齐需求、规则、AC、非目标、ADR 和 UNKNOWN 的稳定 ID；不要先导入所有历史文档。
- 2. **再建立 Code Map**：以当前 Commit 初始化或刷新项目索引，确认关键模块、符号、调用者、消费者和测试能够被定位。
- 3. **补 Trace Link**：将 `REQ`、`RULE`、`AC`、`Symbol`、`Test` 和 `Evidence` 连接起来；自动工具找不到的关系明确标成 UNKNOWN，不用猜测填空。
- 4. **按任务形成 Context Slice**：只把当前契约、相关规则、影响范围内的代码和证明所需测试交给 Agent，并记录实际使用过的来源。

例如，`REQ-PAY-017` 不应该只链接到一个“退款模块”目录，而应逐步落到 `refund_service.create_refund`、退款窗口测试和对应 Evidence。这样下一次变更既能从需求找到代码，也能从代码反查哪些需求和测试会受到影响。

地图能回答“事实在哪里、对象如何关联”，却不能单独判断一条知识是否适用于当前任务。为此，还要区分原始事实、结构索引、语义知识、任务上下文和经过验证的组织经验。

25.2 五层知识结构

L0 事实源：代码、规则、运行记录、正式制度

L1 结构索引：模块、符号、接口、依赖和链接

L2 语义知识：术语、决策、模式、事故和边界

L3 任务上下文：本次任务实际允许读取的切片

L4 验证记忆：经过证据支持、带范围和有效期的组织经验

索引可以重建，事实源必须保留；语义知识可以帮助检索，但不能越过签署和证据成为正式规则。高相似度不代表最新、适用或有权访问。

分层解决了知识处于什么位置；来源、责任人、适用范围和有效期，则决定一条内容能否进入 Agent 的上下文。

25.3 知识条目的最低身份

每条进入 Agent 上下文的知识至少要有：来源、Owner、适用范围、版本、创建时间、有效期、可信状态、引用关系和权限标签。状态至少区分 `candidate`、`verified`、`superseded`、`expired` 和 `disputed`。

```
knowledge_id: RULE-PAY-017
source: governance/decisions/ADR-042.md
scope: payment/refund
version: 3
status: verified
owner: finance-platform
expires_at: 2027-01-01
verified_by: EB-T-137
```

没有来源和适用范围的“经验”，只能作为线索；没有证据和责任人的模型总结，不能晋升为组织事实。

身份信息解决了“这条知识能不能相信”的问题，下一步还要解决“本次任务究竟应该把哪些知识交给 Agent”。这就是 Context Slice 的职责。

25.4 从检索结果到 Context Slice

Agent 不应获得全部仓库、全部聊天记录和全部历史决策。OA 应根据任务契约裁剪：

1. 先读取 Goal、Non-goals、AC 和约束；
2. 根据影响图找到相关模块、接口和测试；
3. 只注入适用版本和授权范围内的知识；
4. 对冲突、过期和 UNKNOWN 显式标记；
5. 在任务证据中记录实际使用过的上下文来源。

上下文越大不代表理解越深。无关内容会稀释关键边界，过期内容会把历史假设伪装成当前规则，越权内容则可能造成数据泄露。

一个真实任务的 Context Slice

以“调整人民币退款窗口”为例，OA 不应把整个支付仓库和全部历史讨论交给 Agent，而应组装一个可解释的切片：

```
当前 Intent Contract 与 AC
+ 当前生效的退款规则、例外和术语
+ Code Map 找到的退款服务、调用者和相关测试
+ 当前版本的接口契约与运行证据
+ 明确的 Preserve、Change Envelope 和 UNKNOWN
- 已废弃的 v1 设计稿
- 无关的外币批处理模块
- 未经授权的生产数据与个人信息
```

切片应留下来源清单，而不是只留下最终 Prompt。来源清单至少包括：文档或代码 ID、版本或 Commit、状态、访问权限、被用于哪一个判断。这样 Prove 失败时，团队可以判断是代码错了、知识过期了，还是 Context Slice 选错了。

一个实用的刷新规则是：**事实源变化，先刷新地图；地图变化，重新生成切片；切片变化，重新绑定任务证据**。不能只更新向量索引，却继续复用旧的任务摘要。

Context Slice 把可信知识变成本次任务可以使用的输入，但它并不自行赋予执行权。这些输入仍要进入 SCOPE-V，接受约束、验证和裁决。

25.5 知识如何进入 SCOPE-V

- Specify 使用 Document Map 澄清术语、历史决策和非目标；
- Constrain 检查读取权限、版本范围和敏感信息；
- Orchestrate 为不同节点分配不同 Context Slice；
- Prove 用代码、测试和运行结果验证知识是否仍然适用；
- Evolve 将经过验证的变化标记为候选知识；
- Verify 决定哪些知识可以进入当前裁决依据；
- Telemetry 观察过期、冲突、错误召回和返工模式。

Knowledge Engineering 是 SCOPE-V 的基础设施，不是第五个动态工件，也不能绕过契约和 Verify 直接给 Agent 授权。在既有系统中，这套原则首先表现为一次面向当前变更的 Recon：先确认事实和未知边界，再提出改变。

25.6 既有系统 Recon：先确认事实，再提出改变

对百万行既有系统，不需要先解释全部代码，也不能因为检索没有命中就断言“没有影响”。开始业务变更前，应先完成一次面向任务的 Recon，把已知事实与未知边界分开：

Recon 产物	要回答的问题
Baseline	当前行为、版本、测试和运行信号是什么？
Preserve	哪些已观察行为在本次变更中不得破坏？
Unknown	哪些依赖、数据语义或例外尚未确认？
Change Envelope	本次允许修改哪些模块、接口、配置和数据？

影响图只根据已验证的代码、测试、配置和运行事实建立。对关键旧行为，先用 Characterization Test 固定观察结果；对尚未确认的依赖保留 UNKNOWN，并在契约中限制发布范围。Agent 可以据此提出改动和补充调查，不能把推测写成系统事实。

这就是局部治理岛：只为一项真实变化建立足够可靠的事实面。它的价值不在于画出一张“完整系统地图”，而在于让当前变更更少依赖猜测，并能说明哪些行为已知、哪些仍然未知。Recon 完成后，如果来源、版本、权限和失效机制没有持续维护，这个事实面仍会迅速失真。

25.7 常见失败

- 把向量检索命中当作事实正确；
- 把全部聊天记录直接注入上下文；
- 只保存摘要，不保存来源和版本；
- 代码地图变化后仍使用旧索引；
- 把局部经验无条件推广到全组织；
- 没有权限标签，导致可检索等于可见；
- 知识进入图谱后没有失效、撤回和冲突处理。

这些失败表面上发生在检索、索引或记忆工具中，实质上都破坏了同一条底线：Agent 只能使用经过确认、处于授权范围且仍然有效的事实。

25.8 本章小结

可信知识不是“存得更多”，而是来源清楚、范围明确、可按任务裁剪、经过证据验证并能够失效。Knowledge Engineering 让 Agent 知道该看什么、为什么相信、哪些内容不能使用。

本章最小实践：选一个真实变更，用 IWE 建立最小 Document Map，用 codebase-memory-mcp 建立当前 Commit 的 Code Map，补齐三条 Trace Link，再生成一份带来源和版本的 Context Slice。工具不可用时，可以用 Markdown 和项目内关系清单替代，但不能省略身份、范围、状态和证据。

第二十六章 | Agent Memory Engineering：让任务经验安全回流下一次任务

Knowledge 解决“组织知道什么”，Memory 解决“任务经历了什么、下一轮应该保留什么”。二者不能混为一谈：知识是相对稳定的事实与关系，记忆是带时间、来源、范围和失效条件的运行状态。

26.1 State、Context、Memory、Knowledge

对象	含义	生命周期
State	当前节点和任务的运行状态	随任务变化
Context	本次执行实际注入的材料	随节点裁剪
Memory	对过去任务的结构化保留	经过晋升、衰减和撤回
Knowledge	可被组织复用的事实和关系	带版本、范围和 Owner

保存全部聊天记录不等于拥有记忆。聊天包含猜测、重复、过期信息和敏感内容，默认不应直接进入下一次任务。

区分记忆与知识之后，真正困难的问题是：一次任务经历中，什么值得留下？记忆的价值不在数量，而在于它能否正确影响下一次行动。

26.2 什么值得记住

优先保留能够改变下一次行动的内容：

- 已验证的失败模式和触发条件；
- 经过批准的规则、例外和回退方式；
- 某类任务的有效 Context 组合；
- 工具、接口或环境的稳定限制；
- 事故根因、修复和适用范围；
- 尚未解决但必须在下一轮继续追踪的 UNKNOWN。

不应默认保留：无证据的模型推断、与任务无关的个人信息、已经失效的临时 workaround、整段聊天原文和无法判断适用范围的口号。

确定了“什么值得记住”，还必须规定“什么条件下才能写入”。否则一次偶然成功或未经验证的推断，就可能被误认为组织经验。

26.3 记忆写入门（Memory Write Gate）

Agent 不能随意把一句总结写入组织记忆。最小写入门检查：

- 这条记忆来自哪个 Task、Commit、Evidence 或事故？
- 是事实、假设、建议还是待验证问题？
- 适用于哪些模块、版本、环境和任务类型？
- 谁拥有确认、修改、撤回和删除责任？
- 什么时候复核或自动失效？
- 是否包含敏感信息，是否需要脱敏或限制可见范围？

```
memory_id: MEM-T-001-003
kind: verified_pattern
source_task: T-001
claim: "金额比较必须使用人民币本位币字段"
scope: expense/reimbursement
evidence: EB-T-001
status: candidate
owner: finance-platform
review_after: 2026-11-01
```

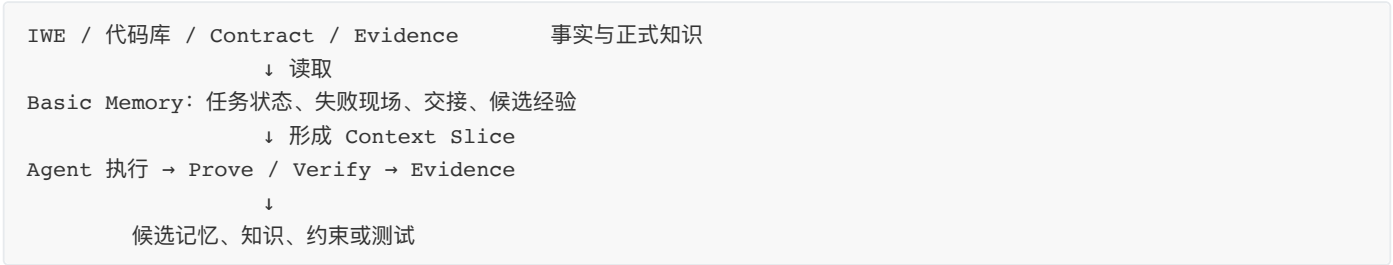
`candidate` 只能被提示和人工审阅，不能自动改变关键决策；经过后续任务验证和责任人批准后，才可以晋升为 `verified`。

Basic Memory：把任务经验放进可治理的记忆地图

Basic Memory 这类工具适合承载跨会话的任务经验和交接信息。它的价值不是替 Agent 保存更多聊天，而是让一次任务中已经发生过的事实，能够在下一次任务中被有范围地检索。这里的“Memory Map”应理解为任务记忆的地图，不是业务知识的唯一事实源：

内容	可以放入任务记忆	不应直接升级为组织知识
任务进展	已完成节点、阻塞点、下一步动作	未经验证的“已经完成”声明
失败经验	触发条件、现场、尝试过的动作和结果	脱离版本与环境的通用结论
交接信息	当前 Commit、修改范围、开放 UNKNOWN、证据位置	一段没有来源的聊天总结
候选模式	可能复用的测试、排障或 Context 组合	只出现一次且没有证据支持的推断

推荐把 Basic Memory 放在任务记忆层，把 IWE 和正式治理工件放在事实与知识层：



一条写入 Basic Memory 的记录至少要能回答“发生了什么、在哪里发生、凭什么相信、下一步是什么”。例如：

```
memory_id: MEM-T-101-004
kind: episodic
scope:
  project: payment-platform
  module: refund-service
  environment: staging
source_task: T-101
source_revision: 8f31c2a
summary: "v2 退款窗口测试因时区配置失败；改用 UTC 固定时钟后通过"
evidence:
  - EB-T-101
status: candidate
next_action: "在生产镜像上复跑时区边界测试"
review_after: 2026-11-01
supersedes: null
```

这条记录可以帮助下一位执行者快速恢复现场，但它还不是“所有退款服务都必须使用 UTC”的组织规则。只有完成当前版本验证、排除反例并由责任人确认后，才可以把稳定结论回写到 IWE 的规则文档、Constraint、自动化测试或 Skill 中；原始记忆保留为事件来源。

Basic Memory 的最小使用流程是：

- 1. **任务开始前读取**：按项目、模块、版本、任务类型和权限筛选，优先读 `verified`，再读明确标注的候选线索。
- 2. **任务执行中更新**：只写关键决策、失败现场、开放 UNKNOWN、已验证结果和交接状态，不同步整段聊天。
- 3. **任务结束后整理**：把一次性状态归档，把可复用模式标为候选，把稳定结论回写正式工件，并在记忆中留下 `promoted_to` 关系。
- 4. **事实变化时失效**：代码、接口、环境或规则变更时，关联记忆降权、标记过期或重新验证；删除也必须传播到索引、缓存和派生摘要。

Basic Memory 可以作为可选 Provider 接入，不应成为 3-4-3 的硬依赖。即使只用项目内 Markdown，也应保持同样的记忆写入门、记忆读取门、作用域和失效规则。

通过写入门，只说明一条经历可以留下，并不代表它在下一次任务中仍然适用。重新进入任务之前，记忆还要接受读取门检查。

26.4 记忆读取门（Memory Read Gate）

记忆即使已经存在，也不应默认进入下一次任务。读入前需要检查：

- 1. 来源和证据是否仍可访问，状态是否为 `verified` 或明确允许作为候选线索；
- 2. 模块、版本、环境和任务类型是否落在适用范围内；
- 3. 是否已过期、被替代、出现反例，或与当前契约冲突；
- 4. 当前任务是否有权读取，其中的敏感内容是否需要脱敏；
- 5. 这条记忆会影响哪一个具体判断、测试或执行边界。

不通过记忆读取门的条目可以作为待调查线索，但不能作为规则、授权或发布依据。记忆影响行动时，任务记录应保留实际读入的条目和版本；这样出现偏差时，团队能够追溯是实现错误、当前事实变化，还是旧经验被错误套用。

读取门判断一次使用是否安全；晋升、衰减、失效和撤回，则决定这条记忆能存续多久、还能影响哪些任务。

26.5 记忆生命周期

候选 → 校验 → 批准 → 注入 → 观察 → 复核 / 降权 / 失效 / 撤回

时间不是唯一的衰减条件。接口替换、规则改版、反例出现、责任人否决、适用范围变化和证据失效，都应触发降权或撤回。记忆条目必须能够反向传播删除，不能只从主库删除而残留在向量索引、缓存和已生成上下文中。

时间会让记忆失效，新的事实也会与旧经验发生冲突。两条互相矛盾的记忆不能同时进入上下文，更不能由 Agent 自行挑选一条执行。

26.6 记忆冲突与上下文污染

当新旧记忆冲突时，Agent 不能自行选择“看起来更合理”的一条。系统应展示来源、版本、适用范围和状态，由 OA 判断是否需要回到 Specify 或由领域责任人裁决。

上下文污染通常来自三种情况：过期记忆被当成当前规则，临时例外被推广成通用原则，未经验证的模型推断与正式知识混在一起。解决办法不是继续扩大上下文，而是显式标记状态、降低默认注入范围并保留冲突。

冲突有了处理边界后，还需要把记忆放回完整任务控制回路，明确它如何影响 Specify、验证、演进和裁决。

26.7 记忆如何贯穿 SCOPE-V

- Specify 读取相关历史决策和未解决问题，但不把它们当作新目标；

- Constrain 检查记忆的权限、范围和失效状态；
- Orchestrate 只向相应节点注入所需记忆；
- Prove 用当前测试和运行结果检验记忆是否仍成立；
- Evolve 产生候选记忆并保留失败来源；
- Verify 判断某条记忆能否支持当前行动；
- Telemetry 观察错误召回、记忆命中后的返工和跨任务复用效果。

一条记忆只有进入 SCOPE-V 并接受当前证据检验，才会真正影响行动。下面的支付系统案例展示一次失败如何被记录、验证、拆解并回写。

26.8 既有系统案例

一个支付系统团队把事故复盘“某接口偶发超时”直接写入全局记忆，后续 Agent 因此为所有调用增加重试，反而放大了重复扣款风险。经过治理后，记忆被拆成：适用接口、触发条件、证据、禁止动作、回退方式和复核日期。只有满足同样条件的任务才会收到这条记忆，并且必须运行幂等性测试。

这说明记忆的价值不在于让 Agent “记住更多”，而在于让过去的经验以正确范围影响未来行动。

如何从一次失败到可复用资产？同一案例可以按以下路径处理：

超时导致支付重试

- Basic Memory 记录接口、环境、版本、现场和重复扣款风险
- 当前任务补做幂等性测试并形成 Evidence
- Verify 判断“所有接口都重试”是错误泛化
- 将正确结论拆成接口级限制、测试和回退规则
- IWE 记录正式规则，Basic Memory 保留原始失败经验

这条路径区分了“发生过什么”和“以后应该怎样做”。前者适合记忆地图，后者必须进入有 Owner、有版本、有证据的正式工件。

这条路径是否可靠，可以用一组最低条件逐项检查。

26.9 验收清单

- 每条记忆都有来源、Owner、范围和状态；
- 候选记忆不会自动获得组织事实资格；
- 记忆写入、读入、晋升、失效和撤回有门禁；
- 敏感信息可按权限隔离和删除传播；
- 冲突记忆会升级，而不是由 Agent 静默选择；
- 记忆能够通过 Evidence 和 Telemetry 被持续校验；
- 下一任务能够说明实际使用了哪些记忆。

满足这些条件，才说明记忆真正成为可治理的运行资产，而不是另一个无法追责的聊天存档。

26.10 本章小结

可治理记忆不是聊天记录仓库，而是有来源、有范围、有状态、有责任和有失效条件的组织运行资产。它帮助系统从失败中学习，同时防止一次局部经验污染所有未来任务。

本章最小实践：用 Basic Memory 或项目内 Markdown 记录一次真实任务的三类信息——进展、失败现场、开放 UNKNOWN；下一次任务按范围读取，完成后把其中一条候选经验通过 Evidence 晋升为正式规则、测试或 Skill，并保留回写关系。

第二十七章 | 规模化协作：从单任务闭环到多人、多工具、多模块可信交付

单个任务的边界足够清楚时，意图、写入范围、证据和验收责任可以在一个责任单元内闭合。规模化真正带来的难题，不是 Agent 数量，而是那些不能通过拆分消除的依赖：共享状态、接口语义、发布顺序、工具交接和多个意图主体同时指挥同一执行会话。

规模化协作的目标，是让多个局部闭环组成一次可信交付。人员配置、转型节奏和管理制度已经在第四部分展开，任务内部仍按第三部分的 SCOPE-V 运行；这里关注的是局部闭环之间必须共享的责任、协议、顺序和证据。

规模化的第一动作，是缩小必须协作的部分。

27.1 先拆责任域，再增加协作机制

共享状态越少，并行越可信。面对复杂项目，先判断能否拆开模块、隔离写入范围、稳定接口，或把一个大任务改成多个独立闭环。能够通过拆分消除的协调，不应再用会议、群聊或全局 Agent 来弥补。

以下情况才需要进入跨模块控制：

- 多个模块共享接口、数据、配置或发布窗口；
- 同一责任域出现并发写入；
- 不同工具接力同一任务；
- 局部证据必须合成为一项发布结论。

治理重量由共享状态、依赖强度和失败传播范围决定，不由参与人数决定。两人改同一核心规则，通常比十人维护十个独立模块更需要强控制。

协同场景	默认方式	必需控制
不同模块、无实质依赖	各自闭环并行	独立 Task ID、Change Envelope、模块证据
不同模块、有接口依赖	受约束并行	所有权、XC、集成 DAG
共享数据或配置	分阶段推进	数据所有权、迁移顺序、系统级回退
同一责任域	单写入者，其他角色并行验证	明确写入边界、合并验证

“单写入者”不是只允许一个人参与，而是同一责任域、同一时间窗口只有一个活跃修改者。测试设计、反例构造、安全审查和证据核验仍可并行。文本可自动合并，不代表业务语义可以自动合并。

27.2 所有权：局部自主，边界统一

跨模块项目至少应分清两类责任：模块所有者负责模块内的契约、上下文、约束、实现和 Evidence Bundle；集成责任人负责端到端意图、共享约束、跨模块契约、集成顺序和发布裁决。

后者不是额外的管理层。只有模块之间存在无法消除的依赖时，才需要这项责任；它不接管模块日常执行，只处理共同边界。

每项跨模块工作都应明确以下事实：

事实	责任
哪个模块可以写入哪一类状态	模块所有者
接口字段、时序和错误语义由谁解释	接口所有者
哪些变更会影响其他模块	变更提出者与集成责任人
哪些局部结果可以进入发布结论	集成责任人
发生冲突、超时或回退时谁裁决	预先指定的责任人

所有权不清时，不应以“大家先做起来”替代裁决。它会把决策延后到集成失败之后，并让所有人都以为责任属于别人。

27.3 Protocol 与跨模块契约 XC

Protocol 是跨主体协作的最小共同语言，约定模块清单、术语、版本策略、Handoff 格式、失败升级方式和发布证据规则。它不替代模块内部文档；它只解决模块之间必须共享的事实。

跨模块契约（XC）则把一个具体边界写成可验证对象。它至少说明：

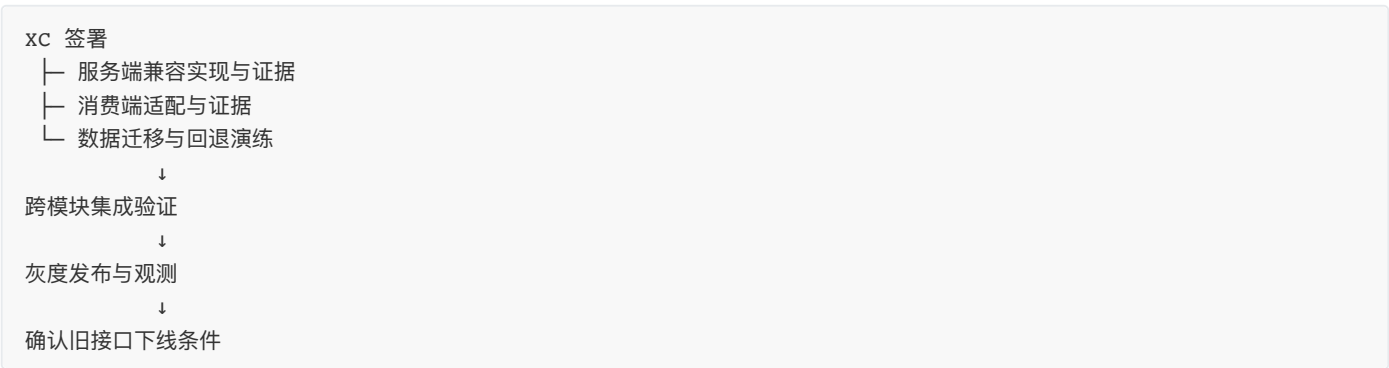
- 请求、响应和事件的字段及版本；
- 业务语义、幂等性、时序与错误处理；
- 数据所有权、兼容窗口和破坏性变更规则；
- 谁提供什么证据、由谁在什么环境验证；
- 失败后的补偿、回退和旧版本退出条件。

字段结构相同只能证明语法兼容，不能证明两个模块对“成功”“完成”或“允许重试”的理解一致。XC 的作用正是把这些语义从口头约定变成可验证边界。

以报销审批结果同步到财务付款为例，审批服务、规则服务和财务接口分别通过本地测试，仍可能因 `approverRole` 与 `approver_role` 的字段差异而无法集成。真正缺失的不是新一轮模块单测，而是字段、版本、错误语义和兼容策略都没有进入 XC。

27.4 集成 DAG：把局部完成变成系统完成

任务清单说明要做什么；集成 DAG 说明哪些产物必须先成立，后续工作才能安全开始。每个节点都应有输入、输出、责任人、关联 XC、完成条件、失败路由和证据位置。



节点状态为“完成”不足以解锁下游；下游还必须确认产物属于同一契约版本，相关 MUST 已通过，UNKNOWN 没有被隐藏，证据仍在有效期内。迁移、兼容实现、消费者升级和旧接口下线的顺序必须显式化。否则每项局部任务都可能完成，系统却在发布时失败。

当节点失败时，DAG 让团队知道哪些下游必须阻塞，哪些并行工作仍可继续，以及回退会不会破坏已经形成的系统状态。回退不是单模块动作：它必须检查数据兼容和下游依赖，避免一个模块“恢复成功”却造成二次故障。

27.5 多工具接力：统一事实源，不统一界面

跨模块依赖明确后，任务还可能在不同人员和工具之间接力。每次接力都可能丢失契约、边界、版本和未知项，因此工具可以不同，事实源不能分裂。

团队可以同时使用不同 IDE Agent、模型、CI Bot 和内部平台。工具不必统一，但它们不能各自保存一份真相。稳定的事实应落在可追溯资产中：Intent Contract、约束、Task ID、Change Envelope、证据位置和 Handoff 记录。

一次工具交接至少携带：当前 Task ID、契约与约束版本、已修改范围、已验证与未验证的内容、开放 UNKNOWN、下一步动作和证据位置。新的执行者必须回到这些事实源，而不是把前一工具的聊天总结当成唯一依据；权限较高也不构成扩大执行范围的理由。

有效 Handoff 是可验证的状态转移，不是“我做完了”的自然语言声明。下游无法确认输入版本、MUST 状态、未知项和证据有效期时，应拒绝接手，而不是静默补齐上游责任。

双地图与任务记忆如何支撑接力

多人、多工具协作时，最容易丢失的不是代码，而是“为什么这样改、当前看到的是哪个版本、下一位能不能继续”。三类资产应各自承担明确职责：

协作需要	主要来源	交接时携带什么
解释目标、边界和决策	IWE Document Map	Contract、规则、AC、非目标、决策链接
定位实现和影响范围	Code Map	仓库、分支或 Commit、模块、符号、调用者、测试
恢复任务现场	Basic Memory 或等价任务记忆	已完成节点、失败现场、开放 UNKNOWN、下一步、证据位置

交接包不应复制三张地图的全部内容，而应保存稳定 ID 和查询入口。下游工具先根据交接包读取当前版本，再按自己的权限生成 Context Slice。这样 IDE Agent、CI Bot、审查工具可以使用不同界面，却不会各自维护一份互相冲突的上下文。

一次跨工具交接可以采用如下顺序：

```
上游任务写入 Handoff
→ 固定 Contract / Constraint / Commit
→ 从 IWE 读取目标与规则
→ 从 Code Map 查询影响范围
→ 从 Memory Map 恢复失败现场与 UNKNOWN
→ 下游确认 Read Gate
→ 执行、Prove、Verify 并回写 Evidence
```

这里有一条重要边界：Basic Memory 中的“下一步建议”不能直接成为下游执行指令；IWE 中的链接也不能自动扩大 Change Envelope；Code Map 找到的调用关系更不能替代业务 Owner 对影响的确认。三者共同减少寻找事实的成本，但执行授权仍来自当前契约和责任人。

规模化时可以按项目、模块和任务建立记忆作用域。项目级记忆保存分支和环境规则，模块级记忆保存接口与失败模式，任务级记忆保存当前交接。作用域越大，晋升门槛越高；跨项目复用必须有证据证明适用，而不是因为关键词相似。

27.6 多人共享会话：协作权不能自动变成执行权

工具接力解决“下一个执行者能否接手”；多人共享会话则要解决“更多人参与讨论时，谁仍然拥有执行授权”。

多人围绕同一个聊天窗口讨论，确实有价值：可以补充上下文、提出反例、审阅证据和共同决策。但当多个人直接向同一批 Agent 连续发出执行指令时，首先冲突的是意图与授权，而不是代码。

共享会话需要把协作平面与执行平面分开：前者允许提出建议、风险和澄清；后者只接受当前任务 IO 或被授权 OA 确认过的指令。至少遵守六条规则：

- 1. 每条消息保留身份、时间和角色，不能压缩成无来源 Prompt；
- 2. 每个运行中的任务只有一个 IO，其他输入默认是建议；
- 3. 消息先分类为建议、澄清、正式变更、新任务或讨论；
- 4. 目标、AC、优先级或 Change Envelope 改变时，暂停执行并形成新版本或 Amendment；
- 5. 每批执行绑定 Task ID、契约版本、约束版本和 Context Slice，不跟随聊天持续漂移；
- 6. 出现冲突、责任不清或授权 UNKNOWN 时停止并升级，不由模型折中。

多人输入

→ 分类与确认

→ 稳定的契约版本或独立任务

→ 绑定执行快照

→ 执行、验证、验收

共享会话可以增加参与，不应稀释责任。产品若不能提供身份、唯一 IO、版本快照、暂停和追溯，就不应把共享聊天直接连接到具有写入权限的 Agent。

27.7 证据聚合与发布边界

责任、契约、DAG、工具接力和共享会话，最终都要回答一个系统级问题：这些局部结果是否足以支持端到端发布？每个模块都应保留自己的 Evidence Bundle。发布前，集成责任人把模块契约版本、XC 验证、集成测试、全局 MUST、开放 UNKNOWN、发布版本和系统级回退方案聚合为 Release Evidence Bundle。

聚合不是把绿色数量相加。它必须回答四个问题：

- 1. 每个模块是否满足自己的责任；
- 2. 模块之间是否满足 XC；
- 3. 端到端行为是否满足全局意图和约束；
- 4. 出现问题时，系统是否能在授权边界内停止和恢复。

任何模块的 UNKNOWN、证据过期或契约版本不一致，都不能被汇总页上的绿色平均值掩盖。不同语言和测试框架可以并存，但输出应能够关联 Task、契约或约束 ID、制品、时间和原始结果；统一的是证据语义，不是工具栈。

27.8 何时继续扩大，何时退回小闭环

协作机制本身也有成本。随着协议、等待、冲突和证据聚合增加，组织需要判断扩大并行是否仍然带来净收益。

并行 Agent 不一定带来净收益。除执行成本外，还应观察上下文准备、协议维护、等待、冲突、重复验证、证据聚合和人类监督成本。只有任务可分解、接口足够稳定、验证可自动化且失败影响可控时，规模化才值得继续。

跨项目视图可用于识别共同瓶颈和可复用治理资产，但不能把不同风险、不同类型、不同工程基础的项目放在同一排行榜。组织级信号必须能够下钻到任务和原始证据；它服务于减少耦合、改进协议和复用有效控制，而不是衡量谁调用了更多 Agent。

成熟的组织不以“同时运行多少 Agent”证明规模化成功。更可靠的信号是：越来越多任务能够在小责任单元内独立闭环，少数真实依赖有明确所有权、契约、顺序和恢复边界。

27.9 本章小结

规模化协作从来不是把更多人和 Agent 放进同一个上下文。先拆开责任域；对拆不掉的依赖，明确所有权，用 XC 固定语义，用集成 DAG 管理顺序，用 Release Evidence Bundle 支持系统级裁决；跨工具交接回到共同事实源，多人共享会话则把协作权与执行权分开。

这些机制的目的只有一个：局部完成能够组成端到端价值，失败能够被阻断和恢复，发布结论能够回溯到责任与证据。

本章最小实践：把一个跨模块需求拆成模块责任、两个 XC 和一张集成 DAG；用 IWE 保存共同的需求与规则链接，用 Code Map 固定实现影响范围，用 Basic Memory 记录每个节点的交接状态和 UNKNOWN；最后为每个节点写明输入、输出、责任人、完成条件、失败路由和证据位置。

第六部分 | AI 组织案例

前五部分回答了为什么重构、重构成什么、任务怎样运行、组织怎样转型，以及工程能力怎样规模化。本部分把这些框架放回真实企业，观察 AI 进入核心价值流后，人员、组织、流程与工具究竟发生了什么。

这些案例不是成功企业的宣传册，也不是 Agentic Agile 的事后证明。每个案例都区分企业披露、外部报道与本书分析，既呈现结果，也保留转型代价、证据缺口和不可直接复制的条件。

本部分阅读方法

阅读每个案例时，持续追问五个问题：

- 1. 企业原来依靠什么组织结构和运营方式创造价值？
- 2. AI 首先进入了哪个环节，为什么局部应用不足以解决原问题？
- 3. 人员、组织、流程和工具分别发生了什么变化？
- 4. 哪些结果有独立证据，哪些只是企业自报或仍待验证？
- 5. 哪些机制可以迁移，哪些做法依赖企业特有的业务、数据和风险条件？

本部分章节

- 1. **第二十八章 | Kavak：从二手车平台到 Agent 原生组织**：一家高度垂直一体化企业，怎样从职能交接转向客户长期 Agent，并尝试建立评估驱动、自我改进的人机组织？
- 2. **第二十九章 | EY 与 Factory Droids：从编码助手到 Agent 原生工程组织**：大型专业服务组织如何让编码 Agent 连接工程标准、代码库与合规门禁，并将工程师从直接执行者推向人机编排者？
- 3. **第三十章 | 明略科技：从数据智能到 Agentic Services 组织**：一家中国数据智能公司，如何将 Agent 嵌入职能、分析师与研发团队，并尝试从交付洞察走向按结果收费的数字劳动力服务？
- 4. **第三十一章 | Spotify：从敏捷研发到 Agentic Fleet Engineering**：一家以软件产品为核心的敏捷公司，如何面对代码规模七倍于工程师增长的压力，把研发从逐个维护组件转向 Agent 驱动的全舰队变更？
- 5. **第三十二章 | GitLab：研发操作系统厂商如何把 Agent 纳入统一控制面**：一家研发平台公司，如何把意图、上下文、约束、Agent 执行、对抗评审和人类批准放进同一个可追溯的研发控制面？

第二十八章 | Kavak：从二手车平台到 Agent 原生组织

2022 年末，拉美二手车平台 Kavak 正处在高速增长之后的转折点。一边是跨国扩张、数十亿美元估值和数千名员工，另一边是成本削减、裁员、客户投诉与盈利压力。创始人兼 CEO Carlos García Ottati 在内部信中承认，公司必须“**focus on doing fewer better things**”，即聚焦更少、但做得更好的事情。[Reuters, 2022](#)

三年多以后，Kavak 对外描述的已经不是一家给员工配置 AI 助手的公司，而是一套由 Agent 承担大部分客户交互和交易、由人处理实体工作和异常情况的组织。它为活跃客户实例化长期 Agent，让 Agent 记住客户历史、调用企业 API、协调销售与金融能力，并在无法继续时向人类请求帮助。公司还宣称，96% 的客户交互和约 95% 的交易已经由 Agent 完整处理。[a16z, 2026](#)

这组数字很醒目，但它不是本案例最重要的部分。真正值得研究的问题是：一家原本依靠专业分工、跨团队交接和大规模人工运营的企业，怎样把组织的基本协调单元从“岗位处理任务”改成“长期 Agent 服务客户”？

本章基于截至 2026 年 8 月的公开资料。Kavak 是非上市公司，没有公开完整的组织图、分阶段投资、样本口径和审计后的 Agent 经营数据。文中凡涉及 96%、2.1 倍、NPS 提升三倍、AI CEO 增利 50% 等结果，均明确标为公司自报，不能视为独立因果证明。

28.1 Kavak 是谁：它从来不只是一家二手车网站

Kavak 于 2016 年 10 月在墨西哥成立，创始人为 Carlos García Ottati、Loreanne García 和 Roger Laughlin，最初团队约 15 人。创立动因来自 Carlos 本人的二手车交易经历：卖车耗时很长，买到的车又存在机械问题。他由此看到拉美二手车市场中信息不透明、欺诈风险高、质量没有保障和融资不足等问题。[Forbes, 2022](#)

Kavak 采用的不是轻资产信息撮合模式，而是高度垂直一体化的 C2B2C 模式：从个人手中收购车辆，完成定价、检测和整备，再销售给另一位消费者，同时提供融资、质保和售后服务。



Kavak 创始人 Carlos García Ottati 置身车辆整备现场。这个场景直观揭示了 Kavak 与普通信息平台的区别：它不只处理线上交易，还要管理车辆检测、维修、库存和交付等实体作业。图片来源：[Bloomberg Línea, 2022](#)。

个人卖车

- 在线估价与收购
- 车辆检测
- 整备与维修
- 库存和物流
- 展示与销售
- 金融、保险与置换
- 交付、质保与售后

这条价值流决定了 Kavak 的组织复杂度。它不仅要像电商平台一样获客和交易，还要像汽车经销商一样管理库存，像维修企业一样检测与整备，像物流企业一样调度车辆，像金融科技公司一样评估风险和提供贷款。公司还需要建立拉美市场缺失的车辆数据与信用基础设施。

Kavak 从成立之初就使用数据和机器学习进行车辆定价、信用评估和物流优化，因此它不是在 2023 年才第一次接触 AI。2022 年时，公司已经拥有自己的估价算法、240 项车辆检查流程、金融产品和大规模整备中心。[Forbes, 2022](#)

这里必须区分两种变化：

- **算法进入业务**：用机器学习改善定价、风控和调度；
- **Agent 重构组织**：让 AI 持有目标和上下文，跨越原有部门边界持续推进客户旅程。

前者优化了局部决策，后者才开始改变公司的运行方式。

28.2 转型前的组织与运营：垂直一体化，但客户仍在职能之间流动

28.2.1 高增长形成的专业化组织

Kavak 在墨西哥验证模式后快速扩张。2020 年，它成为墨西哥首家独角兽；2021 年私人估值达到 87 亿美元。到 2022 年，公开报道显示公司拥有超过 5000 名员工、40 个车辆整备中心，并进入墨西哥、阿根廷、巴西等多个市场。[Forbes, 2022](#)

随着业务扩张，Kavak 形成了典型的专业职能结构：获客、销售顾问、车辆采购、定价、融资、保险、检测、整备、物流和售后分别积累专业能力，国家或地区团队负责落地运营。公开资料不足以还原一张正式组织图，但 Kavak 首席产品与 AI 官 Alejandro Maza Ayala 回顾 2020 至 2021 年的销售过程时称，一个客户完成买车可能需要跨越约 15 类专业能力，与融资、车型咨询、收车估价和保险等不同团队分别沟通。[a16z 访谈转录, 2026](#)

因此，转型前的基本协调逻辑可以概括为：

客户提出需求

- 前台识别需求
- 转交专业团队
- 各团队完成局部任务
- 再由下一团队接力
- 人工汇总结果并推进交易

这种结构并非错误。面对车辆、金融和合规等复杂问题，专业分工可以提高局部质量。但当客户旅程持续三四个月、需要跨越多个专业团队时，组织会付出三种代价：

1. 客户需要重复说明需求，关系与上下文在交接中损耗；
2. 各团队优化自己的任务指标，却没有单一主体持续负责客户长期结果；
3. 业务量上升时，服务能力依赖招聘、培训和增加管理层级。

28.2.2 增长掩盖不了运营断点

2022 年，外部环境和内部运营问题同时出现。Kavak 在巴西裁减约 150 个岗位，随后又宣布显著削减支出和团队规模。公司将其归因于利率、通胀和经济放缓，并把盈利速度、库存控制、客户留存和车辆周转列为 2023 年重点。[Reuters, 2022](#)

客户体验问题更加直接地暴露了组织断点。Carlos 在内部信中承认，客户很难联系到公司，团队也不能在第一次交互中高效提供正确解决方案。Reuters 同年还报道了墨西哥市场中广受关注的客户服务投诉。Carlos 对此回应：“**We have a group of users that we definitely could serve better.**”[Reuters, 2022](#)

来自外部的声音更尖锐。Bloomberg Línea 采访的多名前员工把问题归因于决策过度集中、管理低效和快速扩张后的运营失配；一名前售后负责人估计，投诉主要与车辆检查和交付后的问题有关。由于这些内容来自匿名或离职员工，具体比例无法独立核验，但它们至少说明：Kavak 当时面对的不只是宏观经济压力，也包括组织协同和客户服务能力不足。[Bloomberg Línea, 2022](#)

这构成了 AI 转型前的真实基线：

维度	转型前主要特征	暴露的问题
人员	大量专业人员按职能处理客户旅程	能力绑定岗位，扩容依赖招聘与培训
组织	多专业团队、国家运营和管理层级	决策集中，跨团队交接损耗上下文
流程	客户依次经过销售、金融、检测和售后	周期长，首次解决率不足，责任易漂移
工具	定价、风控、物流等局部算法系统	算法支持局部决策，却没有持续协调客户旅程

28.3 四阶段转型：从垂直平台到 Agent 原生组织

Kavak 没有公开一份正式的四阶段路线图。以下阶段是本书根据公司披露、外部报道和 2026 年高管访谈重建的分析框架，日期存在重叠，不应理解为严格串行的项目计划。

阶段	大致时期	核心变化	组织基本单元
第一阶段	2016—2022	用数据与算法建立垂直一体化平台	职能团队与交易
第二阶段	2022—2023	在经营压力下重置目标，开始围绕 Agent 改造系统	Agent 化业务能力
第三阶段	2023—2025	以 Eval 驱动销售、金融和运营 Agent 进入核心价值流	工作流与专业 Agent
第四阶段	2025 年末以来	放弃部分既有多 Agent 图，转向“一个客户一个长期 Agent”	客户 × 长期 Agent

28.3.1 第一阶段：算法增强垂直一体化业务

Kavak 的第一阶段不是生成式 AI，而是数字平台和机器学习。它用数据进行车辆估价、信用评分、库存与物流决策，同时把检测、整备、融资和售后纳入统一业务体系。

这一阶段建立了后来 Agent 化所需的三个前提：

- 1. **业务闭环**：公司能够从获客一直触达交易、融资、交付和售后结果；
- 2. **数据闭环**：交易、车辆、客户和履约过程持续产生数据；
- 3. **行动接口**：关键业务能力已经数字化，后来才可能被 Agent 调用。

但此时 AI 主要服务于职能，组织仍以岗位和流程为主。算法可以给出价格，却不能持续理解客户为什么犹豫；风控可以计算贷款条件，却不会主动协调车辆选择、置换和保险；每个系统都更聪明，客户仍然需要在组织中移动。

阶段启示：Agent 原生转型并非从模型开始。没有数字化业务能力、可靠数据和可调用接口，Agent 只能停留在聊天层，无法承担真实价值流。

28.3.2 第二阶段：从“采用 AI”转向“围绕 Agent 重建”

经营压力迫使 Kavak 重新审视增长模式。2022 年前后的成本削减与服务问题，提供了一个不能被忽略的背景：AI 转型不是发生在没有约束的创新实验室里，而是在公司必须改善盈利、客户留存和运营效率时展开。

根据 Alejandro 的回顾，Kavak 作出了三项战略选择：

1. 不保持原组织不变后给员工发放 ChatGPT，而是围绕 Agent 重建系统和 API；
2. 不只自动化简单任务，而是把 Agent 放进最难、最接近经营结果的问题中；
3. 不再只按买入、卖出多少辆车衡量公司，而是转向客户关系、满意度、转化率和终身价值。

他把第一项选择概括为：“**redesign your whole company around the agents**”。[a16z 访谈转录, 2026](#)

这句话真正困难的部分不在愿景，而在基础设施。Agent 要推进交易，就必须读取客户和车辆状态，调用估价、预约、贷款、保险和履约能力，并受到身份、权限和规则约束。Kavak 因此需要重建大量 API 和系统，使原本只供内部应用使用的能力可以被 Agent 可靠调用。

阶段启示：给员工配置 AI 只改变局部执行速度；当 Agent 要承担端到端结果时，企业必须把知识、规则和业务动作改造成可读取、可调用、可观测的运行能力。

28.3.3 第三阶段：让 Eval 成为速度、授权与管理系统

从 2023 年起，Kavak 已经使用 AI Agent 管理车辆发现、融资和检测预约等客户对话。[Meta, 2026](#) 但把 Agent 放到销售和贷款等高影响场景，不能只依赖演示效果。Kavak 的答案是把 Eval 放在系统中心。

Alejandro 披露，公司用于建设 Eval 的工程时间、Token 和资金，与建设 Agent 本身大体相当。它首先检查的不是 Agent 说得像不像人，而是业务结果：客户是否获得价值、是否完成交易、是否满意，以及之后是否愿意继续互动。

Agent 候选能力

- 离线场景与反例评估
- 受控进入真实客户旅程
- 观察转化、满意度、风险与成本
- 人工处理异常
- 将失败转成新场景、规则或能力
- 再次评估与扩大

Kavak 没有把 Agent 定位成回答常见问题的客服，而是直接建设销售 Agent。公司称，传统买车过程需要融资、保险、置换估价和车型咨询等约 15 类专业能力；新的 Agent 则把这些能力组合成面向客户的“超级专家”。

公司自报的结果包括：

- 客户 NPS 和满意度提高到原来的约三倍；
- Agent 的销售转化率最初比人工团队高约 50%，后来达到约 2.1 倍；
- 车贷审批通常缩短到三分钟以内；
- 截至 2026 年，96% 的客户交互、约 95% 的交易由 Agent 完整处理。

这些数据没有披露样本量、时间窗口、客户分配方式和完整对照实验，因此只能说明公司报告了显著改善，不能证明所有改善都由 Agent 单独造成。尤其是车贷审批时间，还依赖 Kavak 的垂直业务数据、风险政策和车辆资产处置能力，不是换一个模型就能复制。

这一阶段还把 Agent 推进到实体运营。Kavak 为技师提供名为 El Mike 的辅助 Agent，用于车辆检查和维修指导；公司自报相关质保事件下降约 26%。这里的人机关系不是替代：Agent 提供知识和步骤，技师继续承担需要触觉、视觉和实体操作的工作。

阶段启示：Eval 不是上线前的一张测试报告，而是连接目标、授权、运行事实和改进动作的管理系统。真正决定 Agent 能走多远的，不只是模型能力，而是组织能否发现错误、限制影响并把异常转化为下一轮能力。

28.3.4 第四阶段：从 workflow Agent 到“一个客户一个长期 Agent”

Kavak 最初建设的仍是 workflow 和多 Agent 系统：不同 Agent 按图执行融资、推荐、购买等步骤。到 2025 年末，这套系统已经有数千乃至数万 Agent 运行，并被公司认为推动了增长与盈利。

但随着更强模型出现，Kavak 作出了一项高成本选择：放弃已经建设约两年的部分多 Agent 架构，转向更薄的 Harness。新的基本单元不是“一个任务一个 Agent”，而是“一个客户一个长期 Agent”。

每个活跃客户 Agent 拥有：

- 独立运行环境或虚拟机；
- 跨时间保存的客户历史与记忆；
- 面向客户满意和终身价值的长期目标；
- 调用公司工具、API 和 Skill 的能力；
- 持续运行、休眠、唤醒和安排下一步行动的机制；
- Eval、遥测和人工求助通道。

公司称，每天会实例化约 10 万至 20 万个此类 Agent。它们可能运行几分钟、数小时或数天，再根据下一任务自行唤醒。与传统工作流相比，关键变化是协调中心发生了转移：过去由客户穿过公司内部流程，现在由客户 Agent 调用公司能力。

转型前：客户 → 多个职能团队 → 拼接一次交易

转型后：客户 → 长期 Agent → 调用专业 Agent、API 与人类能力

当 Agent 遇到无法处理的情况时，Kavak 没有只把客户转给二线团队后结束 Agent 任务，而是设计了“I need help”接口：Agent 请求人工能力，人工解决问题，处理过程再回到 Agent 的学习与评估闭环。

这就是“AI 调用人”最具体的含义。它不意味着 Agent 成为法律或伦理责任主体，也不意味着人类只是工具。它表示在运行结构上，Agent 持续保存客户目标和任务状态，人类以领域判断、实体操作或例外裁决的方式进入任务；责任仍必须由有权的人和组织承担。

阶段启示： 自治不是移除人，而是重新安排人出现的位置。Agent 负责连续执行，人负责目标、规则、例外和不可转移责任；一次人工介入只有被记录、验证并回写，才会成为组织学习，而不只是一次救火。

28.4 组织怎样改变：从岗位协作到人机能力网络

28.4.1 团队变平，但不是组织自动消失

Alejandro 描述的当前 Kavak，是由更扁平、更资深和更有授权的团队构成，团队内部同时包含工程、AI 和运营能力。员工的工作大致分为三类：建设 Agent、为 Agent 提供能力，以及在物理世界面对客户和车辆。

这种结构减少的是围绕信息传递和任务分配形成的中间层，不是所有管理工作。目标设定、风险偏好、预算、权限、例外和绩效判断仍然存在，只是其中一部分被编码进 Agent 的目标、Eval、API 和运行规则。

因此，“Agent 成为老板”更适合作为运行比喻，而不是治理结论。Agent 可以分配每日任务、请求人工帮助和跟踪进度，但它不能自行取得组织授权，也不能承担贷款、客户权益、劳动关系和事故后果。

28.4.2 Jedi Academy：转型不只培训 AI 工程师

Kavak 建立了内部 Jedi Academy，培训对象从 CEO、工程师和财务人员一直延伸到技师。Alejandro 称，课程持续更新，参与者经过六周训练后要把 Agent 推向生产。

这个做法背后的重点，不是要求每个人转岗为 AI 工程师，而是建立共同的 Agent 协作素养：

- 业务人员能够把专业知识转成目标、规则和评估场景；
- 工程人员能够把企业能力封装为 Agent 可调用的接口；
- 一线人员能够识别错误、请求接管并反馈真实运行情况；
- 管理者能够围绕业务结果和风险，而不是 AI 使用量作出投资判断。

Kavak 的内部表达也相当强硬：员工可以选择学习并适应新的现实，否则这家公司可能不再适合他。这种自上而下的方向感有助于打破观望，但也带来组织伦理问题：培训机会是否公平，员工能否真实表达反对意见，岗位变化和裁员之间如何区分，劳动责任是否被“技术必然性”掩盖？公开资料没有充分回答这些问题。

28.4.3 AI CEO：大胆实验不等于治理移交

2026 年，Kavak 在墨西哥 Cuernavaca 进行了一项为期约六周的“AI CEO”实验。Agent 分析当地经营指标，为销售、检测等一线人员安排每日任务，并通过语音反馈跟踪执行。公司自报，首月利润未达到翻倍目标，但提高了约 50%，客户满意度、库存周转和融资渗透率也有改善。

这个实验说明 Agent 可以承担高频经营分析和任务编排，却不能据此推出“CEO 已被替代”：

- 1. 实验发生在单一城市，不等于公司级治理；
- 2. 目标和指标由人设定，Agent 没有决定公司使命；
- 3. 人类仍完成车辆检查、交付等实体工作；
- 4. 公司没有公开基线、对照组、利润口径和风险事件；
- 5. “AI CEO”仍由 Kavak 管理层授权、监督并承担后果。

更准确的命名是**城市级经营 Agent**。它验证的是经营控制环中的分析、计划、调度和反馈能否被 Agent 承担，而不是公司治理责任能否交给机器。

28.5 结果：经营改善与证据边界必须同时看

2026 年 2 月，Kavak 宣布获得 3 亿美元新一轮融资，其中 a16z Growth 投资 2 亿美元。公司披露，2025 年完成近 12 万笔交易，同比增长约 40%，并在 2025 年 12 月首次实现单月全球合并盈利；公司还称，其金融科技业务成立以来累计提供超过 10 亿美元融资。[Kavak, 2026](#)

这些经营结果与 AI 转型在时间上重叠，但不能简单写成“AI 导致盈利”。同期发生的库存收缩、市场聚焦、成本削减、融资结构和宏观环境变化，同样可能产生影响。较稳妥的判断是：Kavak 已经把 Agent 放进核心价值流，并报告了与转型方向一致的经营改善；AI 对总结果的独立贡献比例仍未知。

内部与外部声音对照

时间与声音	原话或公开立场	它揭示了什么	阅读时的限制
2022 年，创始人 Carlos	聚焦“更少、但做得更好”，并承认部分客户本可得到更好的服务	转型前存在盈利与客户服务双重压力	内部信和采访反映管理层解释
2022 年，多名前员工	指向决策集中、管理低效和快速扩张后的运营失配	外部视角挑战了“问题只有宏观环境”这一解释	多为匿名或离职员工，细节无法全部核实
2026 年，首席产品与 AI 官Alejandro	公司必须围绕 Agent 重建，而不是只提高 AI 采用率	管理层把转型定义为组织重构	属于转型负责人的事后总结
2026 年，Kavak 公司公告	强调客户长期价值、经营改善和 AI 投资	AI 已成为公司增长叙事的一部分	融资公告具有企业传播目的
2026 年，a16z 访谈	把 Kavak 描述为“自我改进公司”	投资方认可其 Agent 原生方向	a16z 已投资 Kavak，不是独立评价机构

这些声音并不需要被压缩成一个“正确版本”。2022 年的运营危机解释了转型为什么具有紧迫性，2026 年的管理层叙事则解释了公司希望转向哪里。二者之间是否存在充分因果关系，仍需要更长时间和更多独立数据验证。

声明	来源性质	可以得出的结论	不能直接得出的结论
96% 交互、95% 交易由 Agent 处理	高管访谈，公司自报	Agent 已进入大部分客户旅程	所有交易均无人监督且没有风险
销售转化率为人工 2.1 倍	高管访谈，公司自报	公司观察到 Agent 组表现更高	架构是唯一原因，其他企业可复制同等结果
NPS 和满意度约为原来三倍	高管访谈，公司自报	客户体验指标被纳入 Eval	指标口径、样本和持续时间已独立验证
城市利润提高 50%	六周内部实验，公司自报	经营 Agent 值得继续实验	AI 可以替代公司 CEO 或承担治理责任
2025 年交易增长约 40%，单月合并盈利	公司融资公告	公司经营状况明显改善	改善完全由 AI 转型造成

外部声音也提醒我们保持克制。2022 年，Reuters 和 Bloomberg Línea 记录了客户投诉、裁员和管理争议；2026 年的主要 AI 成效则来自 Kavak 高管以及已经投资 Kavak 的 a16z。a16z 的访谈提供了罕见的一线细节，却同时存在投资关系。这些说明不能因为后来的增长，就把早期代价和证据冲突从历史中删除。

28.6 Kavak 与 Agentic Agile：相通的是运行逻辑，不是名词

Kavak 没有宣称采用 Agentic Agile，也没有公开 3-4-3、SCOPE-V 或四大动态工件，这是两者机制层面的对照。

Kavak 实践	Agentic Agile 视角	相通之处	仍需追问的边界
从交易量转向客户终身价值	意图锚定	Agent 围绕长期业务结果运行	客户权益、利润和风险冲突时谁裁决
一个客户一个长期 Agent	动态责任单元与持续上下文	目标、记忆和执行不随交接丢失	记忆来源、权限、失效和隐私如何治理
重建 API 与 Skill	Harness 与可执行约束	企业能力成为可调用动作	动作权限、不可逆边界和审计证据是否完备
Eval 投入接近 Agent 投入	Prove、Verify 与 Evidence	完成由结果和证据判断	Eval 是否独立，是否覆盖合规与反例
“I need help”人工接口	人类异常裁决	未知和越界触发人类介入	Agent 是否能识别自己不知道，谁承担最终责任
人工处理回到学习闭环	Evolve 与 Telemetry	失败成为下一轮能力输入	单次经验如何验证后再晋升为全局规则
扁平跨职能团队	人机责任网络	能力动态组合，责任节点仍需稳定	扁平化是否造成责任模糊或劳动压力

这些对照归纳为六个核心论点。它们不是对 Kavak 做法的照搬，而是对其转型逻辑的提炼。

28.7 六个核心论点：Kavak 究竟改变了什么

Kavak 案例的核心不是“用 AI 卖车”，而是以下六项组织逻辑的变化。

1. 从“旧组织加 AI”转向“围绕 Agent 重构组织”

AI 转型最常见的误区，是组织结构、岗位、流程和 KPI 都保持不变，只给员工增加一个 AI 助手。这类做法可能提高局部速度，却不会自动减少部门交接、目标冲突和客户上下文丢失。它就像早期工厂只用电动机替换蒸汽机，却没有围绕电力重新设计生产线。

Kavak 选择的不是简单叠加工具，而是同时改造客户旅程、API、团队结构、评估体系和经营指标，重新安排客户、Agent、员工和业务系统之间的关系。转型对象因此不再是一个工具，而是整个生产系统。

2. 从“一个任务一个 Agent”转向“一个客户一个长期 Agent”

传统 Agent 往往围绕一次任务创建，任务完成后，上下文也随之结束。Kavak 则尝试为每位活跃客户实例化一个持续服务的 Agent，使其拥有长期记忆、客户上下文和调用企业系统的能力。

这使组织的基本服务单元开始从“岗位处理任务”转变为：

客户 × 长期 Agent × 企业能力网络

Agent 的目标不只是完成一次咨询，而是在客户利益、合规、风险和满意度约束下，持续推进客户目标并提升长期价值。

3. 从复制普通劳动力转向复制经过验证的最佳实践

Agent 的价值不只是降低重复劳动成本，还在于把优秀员工的判断方法、业务知识和异常处理经验转化为可复制能力。Kavak 的关键不是复制顶尖销售的一段话术，而是识别他们在什么条件下如何判断、何时转向另一种策略，以及哪些情况必须停止或求助。

当一种方法经过验证后，可以快速应用到大量 Agent；当一次错误被发现，也可以通过更新规则、上下文和 Eval 减少同类错误。这种规模效应同时会放大风险：未经验证的经验如果被全局同步，错误也会被同样快速地复制。因此，经验必须经过“候选—验证—限定范围—责任确认—可回退发布”后，才能晋升为组织能力。

Kavak 自报的部分实验中，Agent 的销售转化表现达到人工团队的 2.1 倍。这一数字可以说明公司的转型方向，但它仍属于公司口径，不能直接推广到其他企业。

4. 从“人调用 AI”转向“Agent 编排人机能力”

在传统模式中，人掌握任务，并调用 AI 完成局部工作。在 Kavak 的模式中，Agent 可以持续保存目标、上下文和任务状态；当自身能力、权限或信息不足时，再通过“I need help”机制向人类请求帮助。

这并不意味着人变成了“工具”，更意味着责任可以转移给机器。更准确的分工是：

Agent 负责持续执行、协调和反馈；人类负责价值判断、异常处理、实体操作与最终责任。

人工介入之后，处理过程还可以被整理为新的评估案例、规则或训练样本。只有当这些经验经过验证并回写系统时，一次人工帮助才会变成组织能力的增量。

5. 从 Prompt 管理转向 Eval 驱动的管理系统

Agent 管理的重点不应是调用了多少次模型、打了多少电话或节省了多少分钟，而应是它是否产生了正确且可持续的业务结果：

- 客户问题是否真正解决；
- 车辆是否完成交易；
- 转化率和利润是否提高；
- 客户满意度是否改善；
- 合规、质量和风险是否保持在边界内；
- 失败后能否停止、升级或恢复。

因此，Eval 不只是模型上线前的测试工具，而是连接业务目标、Agent 行为、运行证据和管理决策的控制系统。它用来决定 Agent 能否上线、能获得多大权限，以及何时必须停止或回退。

6. 从固定流程自动化转向可治理的自我改进

传统自动化依赖预先设计好的固定 Workflow。随着模型能力增强，企业可以让一部分预设流程变薄，让 Agent 根据目标和环境选择行动，但流程变薄不等于治理消失。组织的管理重点反而要转向：

- 正确且可观察的目标；
- 真实、连续并经过治理的上下文；
- 清晰、最小且可撤回的权限；
- 独立的评估和验证机制；
- 人工介入、异常升级和停止条件；
- 持续反馈、经验沉淀和可回退发布机制。

系统的每次执行、失败和人工介入都可能产生新的改进信号，推动 Agent、规则和组织能力共同演化。但所谓“自我改进”必须是可验证、可限制、可追溯和可恢复的，不是让 Agent 自行改写全局规则。

六项变化共同指向一个结论：**AI 转型不是给旧公司加装工具，而是重新设计公司感知客户、调用能力、完成工作、处理异常和积累经验的方式。**

28.8 从案例到行动：企业短期可以做什么

Kavak 的“一个客户一个 Agent”并不适合所有企业。它成立于几个特殊条件：客户价值高、决策周期长、跨专业协调复杂、企业掌握端到端数据与服务能力，而且一次更好的转化可以覆盖较高的模型与工程成本。

其他企业值得迁移的不是 Kavak 的表面架构，而是一组可以从小范围开始的行动。

1. **盘点现有 AI 用例。** 判断它们是否只是“旧流程加 AI 工具”，还是已经改变了价值流、责任关系和反馈机制。
2. **选择一条高价值客户旅程。** 不要一开始就重构整家公司。选择上下文容易断裂、交接频繁、结果可度量且失败可控制的旅程，同时建立周期、质量、成本和客户体验基线。
3. **试验“一个客户一个长期 Agent”。** 为 Agent 提供持续身份、必要记忆和受控 API，但必须同步定义客户权益、数据边界、权限范围、停止条件和人工接管机制。
4. **建立业务结果导向的 Eval。** 除任务完成率外，同时衡量转化、利润、满意度、合规、返工、人工介入和失败成本。先建立停止、接管和恢复机制，再扩大 Agent 的自治范围。
5. **把人工介入变成学习入口。** 记录 Agent 为什么求助、人如何解决、问题是否重复，以及能否沉淀为规则、测试或新能力。只有经过验证的经验才能进入组织规则。
6. **寻找可复制的优秀实践。** 提炼顶尖员工的判断依据和异常处理方法，不只复制最终话术；同时保留适用条件、反例和风险边界。
7. **逐步调整组织分工。** 让 Agent 承接连续执行，让人从重复操作转向目标定义、能力建设、异常裁决和系统改进。扩大、保持、收缩或停止的依据，应是端到端业务结果，而不是 AI 使用量。

在每一步中，企业都应同时评估员工影响、劳动责任、客户知情、隐私和算法公平，而不是等到规模化之后再补治理。

企业还应主动避免四种误读：

- 不要把裁员自动解释为 AI 生产率；Kavak 2022 年的裁员早于公开的 Agent 成效，且公司当时主要归因于宏观环境和盈利压力。
- 不要把高转化率解释为 Agent 可以无限自主；金融和高客单价交易更需要权限、证据和责任边界。
- 不要把“AI CEO”解释为公司治理已经自动化；目前公开材料只支持一次城市经营实验。
- 不要把自我改进理解为未经验证的全局同步；复制经验之前必须先防止复制错误。

28.9 本章小结：公司开始像软件一样演化，但责任不能像代码一样自动部署

Kavak 从二手车平台成长为同时经营车辆、物流、金融与售后的垂直一体化公司，但增长也带来了跨团队交接、客户上下文丢失、服务难以扩容和管理复杂度。2022 年的经营与客户体验压力，使这些问题无法继续被增长掩盖。

此后的 AI 转型不是简单增加聊天工具。Kavak 重建 API，以 Eval 控制 Agent 进入销售、金融和实体运营，再从工作流 Agent 转向客户长期 Agent，并同步改变团队结构、人员能力和管理指标。它真正激进的地方，是把客户关系、任务协调和组织学习重新放到 Agent 周围设计。

但多数亮眼成效来自公司自报，投资方访谈也不是独立审计；客户记忆、贷款决策、劳动影响、错误同步和最终责任仍有未公开边界。Kavak 展示的是一种组织可能性，不是已经验证的通用答案。

真正可迁移的结论是：

未来公司可能像软件一样持续升级，但升级的不是一个模型，而是目标、能力、规则、证据和人机责任共同构成的运行系统。执行可以交给 Agent，责任不能随版本自动转移。

本章最小实践：选择一条跨三个以上岗位的客户旅程，画出转型前的交接链；再设计一个能够持续持有客户目标和上下文的 Agent 责任单元，明确它可调用的 API、必须遵守的约束、Eval、人工求助接口和最终责任人。不要先讨论替代多少岗位，先回答客户是否更早获得可靠结果。

28.10 主要资料与证据说明

- [Kavak, "Who we are"](#): 公司成立背景、业务定位与客户价值主张。
- [Forbes, "How This Mexico-Based Used-Car Seller Became The Most Valuable Startup In Latin America", 2022](#): 创立过程、业务模式、人员规模、估值、垂直一体化能力与创始人观点。
- [Reuters, "SoftBank-backed Kavak lays off staff, makes significant cuts", 2022](#): 成本削减、裁员、2023 年经营重点和内部信。
- [Reuters, "Mexican used-car startup Kavak expands outside Latin America", 2022](#): 国际扩张、巴西裁员与客户投诉回应。
- [Bloomberg Línea, "Kavak falla en atención al cliente...", 2022](#): 前员工对管理、组织和客户服务问题的外部视角；匿名来源内容仅作争议信号。
- [Bloomberg Línea, Kavak 墨西哥管理变动报道, 2022](#): 章节配图的刊载来源；正式出版前应再核实图片授权范围。
- [Meta, Kavak success story](#): 自 2023 年起的客户旅程 Agent、WhatsApp 前置触达和试点数据；页面明确注明结果为企业自报。
- [The a16z Show, "The Self-Improving Company | Kavak's AI Playbook", 2026](#): Kavak 首席产品与 AI 官对架构、Eval、组织与经营实验的系统说明。a16z 已投资 Kavak，存在利益相关。
- [访谈完整转录与摘要](#): 用于核对访谈细节；属于第三方整理的 YouTube 字幕，不替代企业审计资料。
- [Kavak, Series F announcement, 2026](#): 2025 年交易、盈利、融资与 AI 投资方向，均为公司披露。

第二十九章 | EY（安永）与 Factory Droids：从编码助手到 Agent 原生工程组织

2026 年 3 月，EY（安永）全球客户技术工程负责人 Stephen Newman 在接受 VentureBeat 采访时，提出了一个比“AI 能写多少代码”更尖锐的问题：即使 Agent 能在几分钟内生成数千行代码，如果这些代码无法集成、不符合内部标准，或者需要更多返工，那么前端的“快”并没有产生真实价值。

“You can generate a ton of code, but it doesn't mean really anything ... It's got to be code that is integratable, that is compliant.”

“你可以生成大量代码，但这本身并不意味着什么……代码必须能被集成，并且符合合规要求。”

—— Stephen Newman, EY Global Client Technology Engineering Leader

这句话道出了大型企业引入编码 Agent 的核心矛盾：**模型可以加速代码生成，却不会自动获得组织的工程上下文、质量标准和行动权限。**

EY 后来试用了 Lovable、Replit 和 Factory 等多种 Agent 平台。开发者自发向 Factory 的 Droids 集中后，使用量迅速上升，EY 不得不限制流量和可连接的代码库，等待安全与合规审批。真正的转折不是选中了哪个工具，而是 EY 开始把 Agent 连接到代码库、工程标准、源代码目录和合规控制，并重新划分哪些工作可以高度自治，哪些必须由人掌握。

本章研究的不是“EY Droids”这个不准确的产品名称。**Droids 是 Factory 的软件工程 Agent 产品，EY 是企业用户之一。**本章的对象，是 EY 的产品工程团队如何从辅助式 AI 走向半自主 Agent 执行，以及这种转变如何改写上下文、流程、权限和工程师角色。

本章基于截至 2026 年 8 月的公开资料。EY 没有公开 Droids 的团队数量、代码库范围、测量口径和细化质量数据。本章中 15%—60% 和最高 4—5 倍的数据来自 EY 负责人访谈，并非独立审计结果。

29.1 EY（安永）是谁：一家以专业标准和人才组织交付的全球网络

EY 的历史可追溯至 19 世纪和 20 世纪初的多家会计事务所。1989 年，Ernst & Whinney 与 Arthur Young 合并为 Ernst & Young，后来统一使用 EY 品牌。

EY 不是一家由全球总部直接经营所有客户业务的单一公司，而是由 Ernst & Young Global Limited 成员所组成的全球网络。各成员所是独立法律实体，在共同品牌、战略、方法与专业标准下协作。这种结构既能贴近各国市场与监管，也使全球技术标准、数据边界和审批流程更加复杂。

2025 财年，EY 披露的全球合并收入为 532 亿美元，全球人员约 40.6 万，在 150 多个国家和地区提供服务。其业务横跨审计、咨询、税务、战略与交易。审计、税务和金融平台不只要“可用”，还要满足数据隔离、可追溯、稳定性、专业方法和监管要求。



EY 全球创新市场激活负责人 Edwina Fitzmaurice。她在 EY 的 Agent 实践中强调“边做边学”，并亲自构建编辑、质疑者和倡导者等 Agent。这条更广的 EY Agent 转型线，为工程团队接受 Droids 提供了组织背景。图片来源：[Microsoft Customer Stories, 2025](#)。

29.2 转型之前：软件交付不是写代码，而是穿越整个工程系统

EY 的产品工程团队为审计、税务和金融等业务构建平台。在这类环境中，一个功能从需求到上线，需要穿越产品规划、架构、安全、开发、测试、代码审查、验收、发布和运维。工程标准分散在代码库、文档、流水线、审批规则和资深工程师的经验中。

Factory 用这张图描绘大型组织软件开发生命周期中的依赖、延迟和反馈回路。它不是 EY 的实际流程图，但可以解释为什么只加速“Coding”节点未必能加快端到端交付。图片来源：[Anthropic 对 Factory 的客户案例](#)。

因此，通用编码模型面对的不只是代码生成问题，而是组织记忆缺失问题。它不知道“我们在这里如何工作”，生成的代码越多，后续检查和返工反而可能越多。

EY 的产品工程交付并不是开发者单独完成的链条。产品负责人决定业务目标和范围；架构师判断系统边界与长期演进；工程师实现功能；安全、隐私和合规团队检查数据与访问风险；测试和发布团队验证变更能否进入生产。全球网络结构又增加了一层现实约束：同一种能力可能要适应不同国家的监管、客户数据边界和成员所流程。

- Agent 能否读取客户代码和内部文档?
- Agent 创建的变更由谁承担安全责任?
- 代码审查能否由 Agent 完成初筛，什么情况下必须升级给专家?
- 一个跨系统改动的“完成”，由功能团队还是发布责任人确认?

因此 EY 面对的不是一个购买决策，而是一项组织授权决策。工具可以由工程师试用，但代码库、权限和生产路径不能由个人采用行为自动打开。

29.3 三阶段转型：从个人辅助到半自主工程执行

EY 没有公布一份名为“Droids 三阶段路线图”的正式文件。以下阶段是本书根据 EY 负责人采访、EY 与 Microsoft 的客户案例及 Factory 产品资料重建的分析框架。

阶段	主要做法	组织能力的变化
第一阶段	以 Copilot 类工具建立辅助式 AI 习惯	工程师学会与 AI 协作，但任务仍由人持有
第二阶段	并行试用 Lovable、Replit 和 Factory Droids	用真实采用和产出信号选工具，暴露安全与授权瓶颈
第三阶段	连接上下文宇宙，对工作负载分级授权	Agent 进入半自主执行，工程师转向编排、验证和裁决

29.3.1 第一阶段：先让工程师熟悉辅助式 AI

EY 先从 GitHub Copilot 类工具开始，让工程师在代码补全、解释和生成中学习如何给 AI 提供指令。Newman 把“自下而上的有机采用”视为一个重要经验：管理者没有强行规定所有人使用同一种方式，而是先让工程师在工作中发现价值。

这一阶段解决的是信任与能力问题，却没有改变任务边界。AI 仍在工程师的 IDE 中完成局部动作，人仍要自己查找上下文、分解任务、运行测试并推动交付。随着使用深入，局部代码生成的提升开始触及上限。

阶段启示： 组织学会使用 AI 是 Agent 转型的必要前提，但使用率不等于交付能力。当任务、上下文和责任关系没有变化时，辅助式 AI 主要改善个人动作。

29.3.2 第二阶段：不由领导指定答案，让真实使用产生选择信号

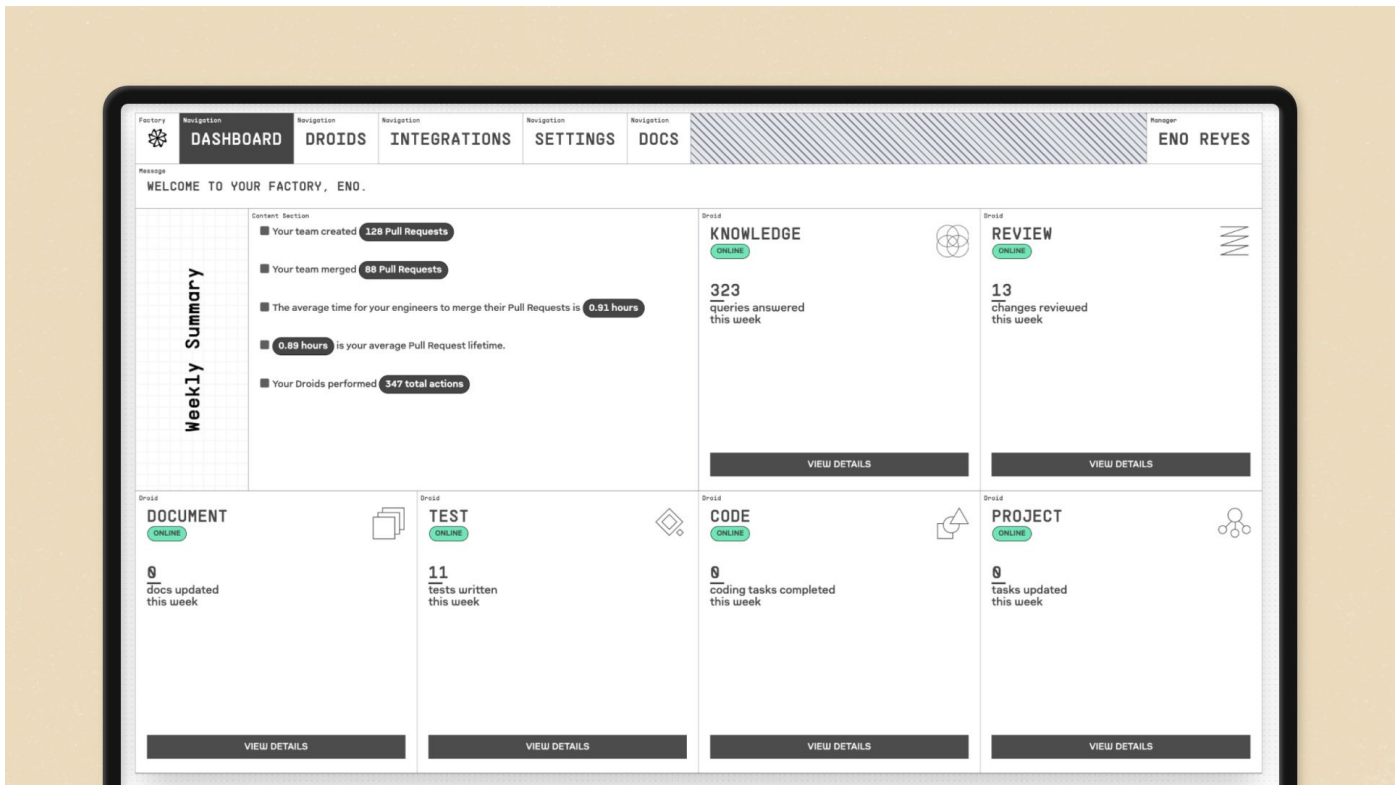
工程师希望 AI 不只生成代码，还能构建、验证并推进更完整的任务。EY 因此并行评估 Lovable、Replit 和 Factory Droids，并观察采用、使用和产出表现。Newman 明确表示，领导团队不希望过早规定一种工具，再把它简化成所有人被迫使用的标准答案。

开发者逐步向 Factory 集中，使用量在进入试点后迅速上升。但“自发采用”并不意味着可以立即开放所有权限。EY 一度需要节流，并限制 Droids 可以连接的代码库，等待合规和安全审批。

这形成了案例中最重要的采用冲突。工程师看到的是等待时间缩短和任务吞吐增加；平台与安全团队看到的是连接面扩大、数据外流风险和无法解释的责任链；管理层则必须在“不要压制创新”和“不要让未经验证的 Agent 进入关键代码库”之间做出取舍。

EY 的节流不是简单限制调用次数，而是把“采用”和“授权”拆开：允许团队继续在低风险场景试用，同时暂缓高敏感代码库、生产凭证和跨系统动作的连接。这个动作保留了真实使用信号，也为安全团队争取了建立权限、审计和验证规则的时间。

这也解释了为什么自下而上的采用并不等于无治理。开发者可以帮助组织发现最有价值的工作负载，但不能单独决定 Agent 的最大影响范围。



Factory 早期 Droids 产品界面，展示 Knowledge、Review、Document、Test 和 Code 等不同工程 Agent。这是 Factory 的产品示意，不是 EY 内部部署截图。图片来源：[Anthropic 对 Factory 的客户案例](#)。

Factory 将 Droids 定位为软件工程 Agent。Review Droid 可以分析拉取请求、提供上下文并留下行级评论；Code Droid 可以从 Jira 或 Linear 等系统接收任务，完成代码修改并创建拉取请求。与只在编辑器中建议下一段代码相比，它开始承接一个跨多步的工程责任单元。

阶段启示： 工具选型可以利用开发者的真实行为作为信号，但采用热度不能替代风险裁决。Agent 越能执行真实动作，组织越需要在连接代码库之前定义范围、权限和证据。

29.3.3 第三阶段：连接“上下文宇宙”，再按工作负载授权

Newman 把 Agent 需要连接的组织知识称为“context universe”，即“上下文宇宙”。它至少包括：

- 代码库与历史变更；
- 工程标准和架构约束；
- 源码目录和组件依赖；
- 测试、构建和部署工具；
- 安全、合规和访问控制；
- 团队的交付惯例与质量门禁。

当 Droids 可以读取正确上下文、调用工具并生成可验证的拉取请求时，Agent 才从通用代码生成器变成了组织内的受约束执行者。

EY 没有对所有工作给予同等自治权。团队将任务区分为两类：

可给予较高自治度	仍需要强人工监督
代码审查	大规模重构
文档生成与更新	架构决策
缺陷修复	跨系统集成
绿地功能	高影响、边界不清的变更

这个分类并不是对任务名称的永久判决。一个“缺陷修复”如果涉及敏感数据或共享架构，仍然可能是高风险任务。其真正含义是：授权必须根据可观察性、可验证性、影响范围和可逆性动态调整。

可以把 EY 的授权过程理解为四个连续问题：

- 1. **Agent 看得到什么？** 代码、文档、历史变更和客户数据是否属于同一敏感等级。
- 2. **Agent 能改什么？** 是只能提出建议，还是可以修改分支、创建 PR，甚至触发流水线。
- 3. **谁来证明改对了？** 是确定性测试、人工专家、客户验收，还是多种证据共同成立。
- 4. **失败如何收回？** 能否停止任务、撤销权限、回滚变更，并追踪影响范围。

因此“代码审查”可以获得较高自治度，并不代表“架构决策”也能获得同样授权；“绿地功能”比较容易建立边界，也不代表“跨系统集成”可以沿用同一套规则。授权的对象不是某个工具，而是某个 Agent 在特定代码库、任务类型和证据条件下能够执行的动作集合。

阶段启示： Agent 能力的上限由模型与上下文共同决定，Agent 自治的上限则由验证与治理能力决定。组织不是一次性地“信任 Agent”，而是对具体工作负载设定具体权限。

29.4 组织怎样改变：工程师从 Doer 走向 Orchestrator

Newman 将 EY 在部分产品上的变化概括为“horizon model development”：半自主 Agent 大规模执行，团队从直接做事的人转向编排者，Agent 则被连接到上下文宇宙。

这不意味着工程师不再需要理解代码。恰恰相反，当 Agent 可以并行生成更多变更时，人类需要承担更高层次的工程责任：

- 1. 把模糊需求转化为可执行、可验证的任务；
- 2. 为 Agent 选择正确的代码库、数据和工具；
- 3. 设定架构、安全、成本和时间约束；
- 4. 检查运行证据，而不只阅读最终代码；
- 5. 处理跨系统取舍、未知依赖和高影响例外；
- 6. 根据失败更新测试、标准和 Agent 可用能力。

这将工程管理的关注点从“每个人写了多少代码”转向“人机系统能否稳定产生可集成、合规且可运行的变更”。Agent 承接执行并不会消灭责任，而会迫使责任从隐性经验变为显性约束和可审查决策。

一个容易被忽视的问题：初级工程师如何成长

传统工程师会通过写文档、修复缺陷、小型功能和代码审查逐步建立系统理解。而这些正是 EY 划入较高 Agent 自治度的任务。如果组织只是把基础任务交给 Agent，却不重新设计学习路径，就可能同时获得短期速度和长期能力断层。

因此，Agent 原生工程组织还需要建立新的成长机制：让初级工程师阅读 Agent 的计划和运行轨迹，验证变更，诊断失败，完成反例设计，并在受控范围内逐步扩大对系统的责任。

29.5 结果与证据边界：“4 倍”不是一个可独立归因的数字

EY 披露，在早期采用阶段，不同开发者角色的效率提升介于 15% 至 60%。在完整实施新模式的部分团队中，公司观察到最高 4 至 5 倍的生产率表现，并将其视为后续向审计、税务和金融平台扩展的标杆。

但 Newman 同时承认，4 至 5 倍无法单独归因于编码 Agent。它是工具、工程标准集成、试错、团队文化和行为变化共同作用的结果。公开资料也没有回答：

- 生产率的分母是工时、周期时间、发布量还是其他指标；
- 对照基线和观察周期是什么；
- 缺陷泄漏、回滚、安全问题和维护成本是否同步改善；
- 多少团队达到最高倍数，多少团队仍处于早期区间；
- 人工验证和后续返工的成本如何计入。

因此，这个案例最有价值的证据不是一个孤立倍数，而是 EY 对“什么才算有用的代码”的重新定义：可集成、合规，且不把前端速度转化为后端清理工作。

内部声音与证据属性

声音或主张	说明了什么	需要保留的边界
Stephen Newman：代码必须可集成、合规	价值指标从代码量转向可交付结果	来自 EY 负责人访谈
开发者自发选择 Factory	真实采用可以作为选型信号	不等于已证明综合成本最低
使用量上升后先节流和限制代码库	组织没有把采用热度当成授权依据	未披露审批时间和控制细节
工程师从 doers 转向 orchestrators	工作单元和角色开始变化	尚无完整人才结构与员工体验数据
最高 4—5 倍生产率	完整模式在部分团队中可能产生大幅改善	为公司自报且无法单独归因于 Droids

29.6 与 EY 整体 Agent 战略的关系：一个工程案例，不是全部转型

EY 同时在推进多条 AI 路线，它们不应被混为一个产品：

1. **Microsoft 365 Copilot 与 Copilot Studio**：EY 已向超过 15 万名员工部署 Copilot，并鼓励业务人员构建特定 Agent。
2. **EY.ai Agentic Platform**：EY 于 2025 年宣布与 NVIDIA 合作构建企业 Agent 平台，首批计划以 150 个税务 Agent 支持 8 万名税务专业人员。
3. **Factory Droids**：EY 产品工程团队采用的第三方软件工程 Agent，是本章的直接案例。
4. **EY 自身的 Agentic Workforce Architecture**：EY 在 2025 年的架构文件中提出运行环境、监控、治理、成本和多 Agent 编排的全局框架。

Droids 案例的价值，在于它提供了一个比全局宣言更具体的窗口：Agent 如何从辅助工具进入真实代码库，组织如何在采用激增时暂停，以及工程标准如何从人的背景知识变成 Agent 可执行的上下文。

29.7 EY Droids 案例的六个核心论点

1. 编码速度不是交付速度

代码生成只是价值流中的一个节点。如果变更无法集成、通过测试和审批，局部提速就会变成下游积压。因此，编码 Agent 的管理指标必须穿越代码量，进入周期时间、一次通过率、缺陷、回滚、维护成本和真实业务结果。

2. Agent 缺的通常不是 Prompt，而是组织上下文

EY 的“上下文宇宙”表明，可部署代码来自模型、代码库、标准、工具、依赖和权限的组合。一段更精巧的 Prompt 无法替代可读取的架构决策、可执行的质量门禁和可追溯的历史。

3. 不要把采用热度误当成授权证据

开发者对 Droids 的自发选择说明产品产生了用户价值，但不证明它已经可以访问所有代码库和执行所有动作。EY 在采用增长后先节流和限制连接，是这个案例中最值得注意的治理动作之一。

4. 自治度应该跟随工作负载，而不是跟随工具名称

同一个 Droid 可以用于文档、缺陷修复或架构重构，但这些任务的不确定性、影响范围和可逆性完全不同。授权对象不应是“某个 Agent”，而应是“某个 Agent 在某类任务、某个代码库和某组约束下的某些动作”。

5. 工程师从执行者转向编排者，但责任不会转移

Agent 可以读取任务、修改代码、运行测试并创建拉取请求，人则转向目标、上下文、约束、验证和例外。但 Agent 不承担客户、监管和系统失败的最终责任。“Orchestrator”不是更轻的角色，而是要对更大的人机执行面承担判断。

6. 生产率改善是社会技术系统的结果，不是模型的单独功劳

EY 的最高倍数来自 Agent、上下文集成、流程、标准、文化和工程师行为的共同变化。这正是为什么其他企业不能购买同一个工具后，就预言相同结果。要复制的不是“4 倍”，而是让 Agent 从通用模型进入可治理生产系统的条件。

这六个论点共同指向一个结论：

Agent 原生工程不是让 AI 写更多代码，而是让意图、上下文、执行、验证和责任围绕一个可交付变更重新组合。

29.8 EY Droids 与 Agentic Agile：相通的是上下文与受控执行

EY 没有宣称 Droids 实践使用 Agentic Agile。以下对照是本书的机制分析，不是实施认证。

EY Droids 实践	Agentic Agile 视角	相通之处	仍需追问
从代码量转向可集成、合规变更	Intent 与 Value	完成由端到端结果定义	生产率指标是否包含长期质量
上下文宇宙	Context 与 Harness	Agent 获得组织标准与工具	上下文的所有者、时效和冲突如何治理
按工作负载分级自治	Constraint 与 Act	权限与风险相匹配	分类是否在运行时动态校验
测试、审查和合规门禁	Prove 与 Verify	代码必须产生可验证证据	裁决与执行是否足够独立
工程师从 doer 转向 orchestrator	人机责任网络	人聚焦意图、例外与责任	初级工程师的能力如何形成
从试点到扩展	Telemetry 与 Evolve	采用与运行数据支持决策	失败、回滚和维护成本是否完整记录

EY 案例对 Agentic Agile 最重要的印证是：上下文不是给 Agent 的背景资料，而是决定它能否正确执行的生产基础设施。与此同时，权限、验证和恢复也必须成为基础设施，否则更多上下文只会让 Agent 更有能力地制造更大影响。

29.9 从案例到行动：工程组织可以如何开始

- 1. 先建立真实基线。记录需求到上线的周期、一次通过率、缺陷泄漏、回滚、返工和人工介入，不要用生成代码量做主要基线。
- 2. 让多种工具在受控场景中竞争。同时观察开发者选择、任务完成、质量、成本和风险，不由领导层只根据演示效果选型。
- 3. 建立最小上下文宇宙。从一个代码库开始，连接有效标准、依赖、测试命令、代码所有者和安全边界，并为上下文设定来

源、版本和过期规则。

4. **按工作负载建立自治矩阵。** 根据数据敏感度、依赖范围、可逆性和验证强度，定义哪些任务只能建议，哪些可以修改，哪些可以创建拉取请求。
5. **把验证变成不可绕过的路径。** 让测试、静态分析、安全检查、架构规则和人工批准根据风险分级自动触发，而不是在 Agent 完成后靠人记得检查。
6. **用失败改进系统，不只修正当前代码。** 区分错误来自意图、上下文、工具、模型、验证还是权限，将经过验证的修复回写到对应层。
7. **重新设计工程师成长路径。** 不要让 Agent 吞掉所有初级任务。让新人通过验证 Agent 计划、诊断失败、设计反例和维护上下文，建立对系统的真实理解。

企业还应避免四个误读：

- 不要把 Factory Droids 写成 EY 自研产品；
- 不要把最高 4—5 倍写成全部团队的表现；
- 不要把工程师转向编排者解释为人不再需要技术判断；
- 不要认为连接更多上下文天然更安全；访问面扩大必须伴随更细的权限与审计。

29.10 本章小结：从会写代码的 Agent，到可交付变更的工程系统

EY 的 Droids 实践并不是一个“买了工具，生产率翻倍”的简单故事。它从辅助式 AI 开始，经历多工具试用和开发者自发选择，在使用量激增时又主动节流、限制代码库连接。只有当 Agent 连接代码库、工程标准和合规门禁，并根据工作负载获得不同权限后，它才开始进入真实产品工程。

这个案例与 Kavak 构成了一组清晰对照：Kavak 让长期 Agent 持续承接客户旅程，EY 则让工程 Agent 在组织上下文与质量门禁中承接软件变更。一个重构客户责任单元，一个重构工程执行单元。两者共同说明：当 Agent 进入核心价值流时，企业必须同时重构上下文、权限、验证和人的责任。

代码可以由 Agent 大规模生成，但“什么可以进入生产”仍然是一个组织判断。真正的 Agent 原生工程，是把这个判断变成可执行、可验证、可追溯和可恢复的系统。

本章最小实践： 选择一个低至中风险代码库，选取文档更新、缺陷修复或测试生成中的一类任务，为 Agent 明确需求、可读上下文、可用工具、禁止动作、验证门禁和人工批准人。连续观察两个交付周期，比较端到端时间、一次通过率、返工和人工介入，再决定是否扩大权限。

29.11 主要资料与证据说明

- [EY, Global Network](#)：EY 成员所网络的法律与协作结构。
- [EY, FY2025 Global Revenue Announcement](#)：2025 财年收入、人员、地区与业务结构。
- [VentureBeat, “EY hit 4x coding productivity by connecting AI agents to engineering standards”, 2026](#)：Droids 试用、工具选择、节流、上下文宇宙、工作负载分类、角色变化与成效口径的主要来源。内容基于 EY 负责人采访，非独立审计。
- [Anthropic, “Factory is building Droids for software engineering with Claude”](#)：Factory 成立时间、Droids 定位、Review Droid 与 Code Droid 的工作方式，以及两张产品图片的刊载来源。
- [Factory, Code Droid Technical Report, 2024](#)：Code Droid 的技术定位、工程基准与能力主张；属于产品方资料。
- [Microsoft Customer Stories, EY, 2025](#)：EY 向超过 15 万名员工部署 Copilot、业务人员构建 Agent 和变革管理路径；同时为角色配图来源。
- [EY, EY.ai Agentic Platform announcement, 2025](#)：EY.ai Agentic Platform 的技术组成与首批税务 Agent 部署计划。这是 EY 公司公告，计划不等于已实现结果。
- [EY, “Architecting an agentic workforce at the EY organization”, 2025](#)：EY 对多 Agent 运行、治理、成本和规模化的全局架构主张。

第三十章 | 明略科技：从数据智能到 Agentic Services 组织

过去，一份社交媒体深度复盘报告的交付，可能从一张数据需求表开始。分析师确认品牌、时间和渠道范围，从不同平台采集信息，清洗数据，设置标签，判断话题与情感，再把图表、解释和建议组合成报告。客户收到的是一份“看懂数据”的成果，至于下一步如何调整内容、选择达人、投放并验证结果，仍然需要客户自己完成。

这是明略科技长期擅长的 Data Intelligence 模式：连接数据，提供洞察，帮助人做决策。它曾支撑明略从在线广告测量走向知识图谱、线下门店运营和企业数据智能，也形成了一支由销售、项目经理、分析师、数据工程师和研发人员组成的专业服务组织。

但到了 2024 至 2025 年，明略开始追问一个更深的问题：

如果 AI 已经能够理解任务、调用工具和操作软件，公司为什么还只把“报告”交给客户，而不直接帮客户把事情做完？

这个问题推动的不只是产品升级。它开始改变明略向客户交付什么、客户为什么付费、一线人员如何工作，以及一家专业服务公司如何获得规模效应。

2025 年年报中，明略把这项转变概括为：

从帮助客户“看懂数据”，到帮助客户“拿到结果”。

本章基于截至 2026 年 8 月的招股书、上市公告、年度业绩、管理者演讲与公开报道。明略内部 Agent 数量、20 倍人效、报告自动化率和营销效果等数据主要来自公司披露，并非独立实验。本章将公司口径、财务数据与本书分析分开处理。

30.1 明略科技是谁：二十年都在连接数据与业务

明略科技的起点是 2006 年创立的秒针系统。创始人吴明辉从互联网用户行为和广告效果测量切入，建立了广告监测、预算分配和社交聆听等产品。2014 年，团队创立明略数据品牌，将业务从线上商业扩展到政府、金融、工业和城市数据中台。2019 年，明略科技集团成立，业务逐步回到以营销智能和营运智能为主的企业服务主线。



明略科技创始人、CEO 兼 CTO 吴明辉。他把明略二十年的主线概括为对数据、知识与企业决策的持续连接。图片来源：[明略科技官网](#)。

2024 年，明略科技收入约 13.81 亿元，业务结构如下：

业务	收入	占比	主要价值
营销智能	7.31 亿元	52.9%	广告测量、社媒分析、营销策略与效果管理
营运智能	5.23 亿元	37.9%	会话智能、门店运营、质量和工单管理
行业解决方案	1.28 亿元	9.2%	面向特定行业的数据平台与项目交付

这些业务的共同基础，是把结构化和非结构化数据转化为可供人理解的洞察。公司称已服务 135 家财富世界 500 强、约 2100 家品牌客户和超过 24 万家企业用户。这些长期项目累积了数据、标签体系、行业知识和业务工具，后来成为 Agent 进入真实业务的上下文基础。

30.2 转型之前：工具已经数字化，交付仍然依赖人海

明略的传统交付链大致如下：

- 销售与客户确认需求
- 项目经理分解范围和交付物

→ 数据工程师连接与治理数据

→ 分析师配置模型、标签和分析方法

→ 资深专家复核异常与结论

→ 制作报告、看板或系统

→ 客户再根据洞察采取行动

这个模式的瓶颈不只是某个人工步骤太慢，而是“洞察”和“行动”分属两个组织。明略看到客户的数据，却未必能调用客户的全部工具；客户拿到报告，却仍要重新解释、协调并执行。反馈需要等到下一次项目或复盘才能返回，学习周期很长。

同时，这是一种高度依赖专业人员的模式。客户越多、报告越深、定制化程度越高，公司就需要投入更多分析师、项目经理和工程师。软件可以复制，项目经验却大量停留在人的头脑、表格和临时沟通中。

这个结构已经面临经营压力。2022—2024 年，明略科技经调整净亏损分别约为 10.99 亿元、1.74 亿元和 4511 万元。同期，研发、销售和行政支出明显收缩。这说明 AI Native 转型不是在无限资源下的概念实验，它也承担着降低交付成本、恢复盈利和寻找新增长曲线的现实任务。

30.3 四阶段演进：从数据工具到数字劳动力

明略的转型同时沿着两条线展开，必须分开观察。第一条是**内部 AI Native 组织线**：职能、分析师和研发团队如何改变工作方式；第二条是**外部 Agentic Services 业务线**：公司如何把这些能力包装成面向客户的结果服务。前者解决“自己能不能这样工作”，后者解决“客户是否愿意为这样的结果付费”。两条线相互支持，但不能用内部 Agent 数量证明外部商业成功，也不能用新业务收入反推内部组织已经完成重构。

阶段	时间	核心能力	交付的东西
第一阶段	2006—2014	大规模广告数据采集、测量与分析	指标、数据与营销洞察
第二阶段	2014—2023	数据中台、知识图谱、线下营运智能	平台、报告与专业服务
第三阶段	2024	在既有组织中平行内嵌 AI Native 体系	由人与专业 Agent 共同交付
第四阶段	2025 至今	DeepMiner、Mano、Cito、Octo 与 FDE	可量化结果和 Agentic Services

30.3.1 第一阶段：让广告效果首先变得可观测

秒针系统最早解决的是在线营销的测量问题。它采集大规模广告请求和用户行为数据，通过 Admonitor、MixReach 和 SocialMaster 等产品，帮助广告主理解曝光、触达、社交话题和媒介效果。

这个阶段建立了后来 Agent 最需要的第一层基础：真实、连续和可计算的业务数据。但系统主要回答“发生了什么”，行动仍由人发起。

阶段启示： 自治行动之前先要有可观测的现实。如果组织不能看到真实结果，Agent 就无法学习，Eval 也无从建立。

30.3.2 第二阶段：从线上数据走向企业知识和线下运营

2014 年以后，明略将大数据技术扩展到政府、金融、工业和城市治理，并建立数据中台和领域知识图谱能力。2018 年以后，公司又进入线下门店、餐饮、汽车和消费品运营场景。与百胜中国成立合资公司，是一个具代表性的动作：数据智能不再只服务营销部门，也开始进入员工赋能、门店运营和供应链。

这些项目使明略积累了不只是数据，还包括“什么是好结果”、“异常应该如何处理”和“哪些动作可以在什么条件下执行”。然而，这些 know-how 仍大量存在于项目团队中，难以低成本复制。

阶段启示： Agent 需要的不只是一个大模型，还需要数据、领域知识、工具和对结果的判断标准。明略的二十年积累，是它后来能够做 Agentic Services 的条件，但条件不会自动变成新组织。

30.3.3 内部转型线：在 1500 人组织中平行内嵌 AI Native 体系

2024 年，明略开始内部 AI Native 实验。管理层没有先建立一个完全独立的“AI 部门”，也没有立即推倒旧组织，而是在现有结构中平行内嵌一套 Agent 协作体系。公司披露的范围包括 109 人左右的职能团队、600 多名分析师和近 800 名研发人员。



明略科技创始合伙人、总裁兼 CFO 姜平在 2026 年虎啸营销论坛介绍“从 SaaS 到数字劳动力”的内部实验。图片来源：[明略科技, 2026](#)。

这项实验在三类团队中呈现出不同形态：

- **职能团队：**Agent 处理财务数据整理、报表和例行信息工作，人聚焦异常、解释和管理决策。
- **分析师团队：**Agent 承接采集、标签、初步分析、图表与报告草稿，分析师转向问题定义、客户品味和策略判断。
- **研发团队：**多个专业 Agent 分工进行需求分析、编码、测试和评审，人负责架构、约束和最终判断。

到 2026 年，明略称已有超过 1400 名员工与 3200 多个 Agent 在 Octo 平台日常协作。这个数字不能直接解释为“每个 Agent 都是稳定的全职数字员工”。它更适合被理解为：明略已经在真实工作流中建立了数千个特定角色和能力的 Agent 实例。其活跃率、自治等级、任务成功率和人工介入比例尚未完整公开。

阶段启示： AI Native 不是建立一个 AI 小组帮所有人做自动化，而是让每个团队重新定义目标、人机分工、验证和失败处理。Agent 数量是输入，价值流如何变化才是转型证据。

30.3.4 外部业务线：从卖软件和项目，走向按结果收费的 Agentic Services

2025 年，明略正式在 Data Intelligence 之外新增 Agentic Services 业务。技术上，它由 DeepMiner 企业级 Agent 平台、Cito 推理模型、Mano GUI Agent 以及 Octo 协作网络支撑。但新模式最重要的不是技术名称，而是交付契约发生了变化。

内部实验解决的是能力供给问题：明略能否让自己的员工与 Agent 协作。外部服务解决的是价值交换问题：客户是否愿意把一段结果责任交给明略，以及双方如何定义基线、归因、权限和失败责任。只有两层同时成立，Agentic Services 才不是把内部工具换一个名称出售：

传统模式：客户为软件、项目、报告和人时付费
Agentic Services：客户逐步为营销效果、内容产出或问题解决等可量化结果付费

以营销为例，新流程不停在洞察报告，而是向下延伸：

市场与消费者洞察

→ 营销策划

→ 文本、图像和视频内容生成

→ 渠道分发与广告投放

→ 效果数据回收

→ 策略调整与再执行

明略并没有认为客户需求可以被一次 Prompt 完整表达。它把 FDE（Forward Deployed Engineer）放在客户与 Agent 网络之间。一个 FDE 带着一组 Agent，进入客户环境，连接业务系统、数据与工具，并在持续合作中获得三种 Agent 无法自己猜到的东西：

1. **Objective**：客户真正要获得的结果；
2. **Context**：客户的业务现实、数据、权限和限制；
3. **Taste**：品牌偏好、经验判断和难以完全编码的质量感。

吴明辉因此强调，AI Native 时代的 FDE 不只是技术实施人员。当通用推理逐步被 AI 承接后，FDE 更重要的能力是建立信任、识别真实目标、获取隐性上下文和把客户的“品味”反馈给 Agent。

阶段启示：从交付软件到交付结果，企业承担的责任更大，而不是更小。它必须同时获得动作权限、建立结果归因、限制风险，并决定当营销效果与品牌安全冲突时谁有最终决定权。

30.4 组织怎样改变：从专业分工链到“FDE + Agent 群”

这一节讨论两条线交汇后的组织变化。内部 AI Native 线改变员工如何完成工作，外部 Agentic Services 线改变客户如何购买结果；FDE 位于两者之间，既把客户目标带回内部 Agent 网络，也把内部能力带到客户现场。

传统专业服务组织依靠层级和职能进行质量控制：初级人员整理数据，高级分析师构建解释，项目经理协调资源，专家复核结论。AI Native 模式把其中一部分“认知劳动”转化为多 Agent 协作：采集 Agent、数据处理 Agent、分析 Agent、内容 Agent、评审 Agent 和投放 Agent 可以并行或串行执行。

人的角色随之上移：

传统角色	AI Native 中的重心
销售确认合同范围	与客户建立可持续的共同目标
项目经理分配人力	设计人机责任、上下文与升级路径
分析师制作报告	定义问题、判断证据和产生策略
数据工程师临时接口	把数据与动作建设成可治理的 Agent 能力
专家人工检查所有内容	设计 Eval、反例、风险门禁和异常裁决
交付后复盘	运行中持续观测、反馈与改进

这里最关键的新角色是 FDE。它不完全属于销售、产品、咨询或交付中的任何一方，而是横跨了从意图到运行结果的责任链。一名 FDE 带领多少 Agent 并不是核心指标；更重要的是，他能否确保 Agent 在正确目标、真实上下文和明确权限中产生可证明的结果。

30.5 从工具到结果：收费模式为什么会反过来塑造组织

按项目或软件许可收费时，供应商的交付边界相对清晰：系统可用、功能验收或报告交付后，客户如何行动主要由客户自己负责。

按效果收费改变了这个边界。如果明略为营销 ROI 改善或内容效率付费，它就必须更深地进入客户的执行链，同时回答三个难题：

- 1. **结果归因：**效果改善究竟来自 Agent、客户产品、市场环境还是媒介价格？
- 2. **权限边界：**Agent 可以生成内容、更改投放和调整预算到什么程度？
- 3. **价值冲突：**短期转化与品牌长期信任冲突时，谁裁决？

因此，按结果收费不是将价格表从“人天”换成“效果”那么简单。它要求企业建立更强的遥测、实验、证据、合规和停止机制，也会把一部分客户经营风险转移到供应商。

30.6 结果：新业务已经产生收入，但距离重写公司仍有距离

2025 年，明略科技实现收入 14.258 亿元，同比增长 3.2%；毛利 7.896 亿元。按香港财务报告准则，公司仍录得 1572 万元经营亏损；按非香港财务报告准则口径，经调整净利润为 4204 万元，从 2024 年的经调整净亏损 4511 万元转正。

Agentic Services 当年收入为 1.002 亿元，约占总收入 7%。这说明新模式已经从演示和内部提效进入商业收入，但明略目前仍然主要是一家 Data Intelligence 公司。转型是真实的，却还不能写成已完成。



2025 年 11 月 3 日，明略科技在香港交易所上市，股票代码 2718.HK。上市为模型研发和 Agentic Services 扩展提供了资本，也使新业务必须接受公开财务数据的持续检验。图片来源：[流媒体网, 2025](#)。

公司还披露了一组运行结果：

- 营销智能交付效率最高提升 4 倍；
- 营运智能工单解决时间缩短超过 30%；
- Agentic Services 营销流程以近 3 倍运营效率，帮助客户平均提高 20% 营销效果；
- 部分内部财务报表处理从 3 天缩短到 10 分钟；
- 部分分析师报告的自动化程度达到 99%，社媒深度复盘报告人效提升最高达 20 倍。

这些数据的证据强度不同。收入、毛利和利润来自上市公司年度业绩；具体效率与效果指标主要由公司自报，公开材料尚未完整披露样本、基线、持续时间和失败成本。

可以相对确定	仍需持续验证
Agentic Services 已经形成收入	按效果收费的长期毛利率和续约率
2025 年经调整口径扭亏	改善有多少来自 Agent，有多少来自历年费用压缩
公司已将 Agent 部署到多类内部团队	3200 个 Agent 的活跃率、自治度和成功率
新交付模式从报告延伸至执行	营销效果的归因方法与风险分担
FDE 已成为公司对外阐述的关键角色	FDE 的真实数量、能力模型和绩效机制

30.7 六个核心论点：明略案例究竟改变了什么

1. 从“交付洞察”转向“交付闭环结果”

数据报告的完成不等于客户问题的解决。明略的新模式尝试跨过“知道”与“做到”之间的断点，将策划、生成、分发、投放和反馈组成连续循环。这会产生更大价值，也会让供应商承担更多结果责任。

2. 从 SaaS 工具转向数字劳动力服务

SaaS 通常向人提供一个操作工具；Agentic Services 则把目标、上下文和能力组装成可持续执行的数字角色。客户购买的不再只是软件使用权，而是某类工作被完成的能力。这会重写定价、成本、责任和续约逻辑。

3. 从一个 Copilot 转向组织级 Agent 协作网络

个人 Copilot 围绕一个人的工作提供建议；Octo 试图让多个具有不同角色的 Agent 与人共享任务、上下文和结果。组织问题因此从“每个人是否使用 AI”转向“数千个人与 Agent 如何分工、发现冲突、升级异常和共享经验”。

4. 从项目经理转向 FDE 责任节点

传统项目经理管理范围、人员和交付物；FDE 则需要持续连接 Objective、Context 和 Taste，并为 Agent 网络的结果承担前端责任。它把销售、产品、技术和交付的部分责任压缩到一个更接近客户价值的节点上。

5. 从按输入收费转向按可量化结果收费

人天、许可和项目金额主要衡量供应商投入了什么；效果收费要求证明客户获得了什么。这会迫使 Eval 从技术测试变成经营基础设施，也要求组织建立可双方接受的归因和证据。

6. 从“向客户讲 AI”转向“先把自己变成 AI Native 组织”

明略将职能、分析师和研发团队作为内部实验场，不只是为了节约成本，也是为了在向客户出售数字劳动力之前，先经历角色、流程、权限和学习方式的冲突。这种“先自我应用，再对外交付”提高了实践密度，但内部提效仍不能自动证明外部客户价值。

这六项变化共同指向：

AI 原生专业服务不是用 Agent 降低人天成本，而是把原本分散在工具、专家和客户之间的目标、上下文、行动与反馈重新组成一个可对结果负责的系统。

30.8 与 Agentic Agile 的关系：FDE 是责任节点，不是超级个人

明略没有宣称实施 Agentic Agile。两者的相通之处是机制，不是名词。

明略实践	Agentic Agile 视角	相通之处	必须保留的边界
从看懂数据到拿到结果	Intent 与 Value	以可观察客户结果定义完成	结果必须受客户利益、品牌与合规约束
Objective + Context + Taste	意图图谱与上下文工程	Agent 不能只依赖一次 Prompt	Taste 不能只停留在 FDE 的个人感觉中
FDE + Agent 群	动态责任单元	人持有意图与例外，Agent 扩展执行	FDE 不能成为不可替代的单元
按效果收费	Prove、Verify 与 Evidence	证据直接连接交付与收入	Eval 和归因不应由利益单方独立定义
Octo 中人与多 Agent 协作	Work Graph 与 Harness	角色、任务、能力和交付关系显性化	多 Agent 会带来协调债、错误传播和权限叠加
结果反馈驱动模型与 Agent 迭代	Telemetry 与 Evolve	真实运行进入学习闭环	经验必须验证、限定范围并可回退

FDE 的核心价值不是一个人掌握所有技术和业务，而是确保责任链不会在客户、平台、Agent 和专业团队之间丢失。如果所有上下文和判断又集中到一名 FDE 头脑中，组织只是用一种新角色重建了旧的信息瓶颈。

30.9 值得迁移的七个动作

- 1. 从一条“洞察与行动断开”的价值流开始。不要先追求全公司有多少 Agent，而要找到一个已经看到问题、却无法及时行动的场景。
- 2. 同时建立结果和风险基线。记录周期、人工工时、质量、效果、异常、客诉和返工，避免只用产出量证明 AI 有效。
- 3. 把 Objective、Context 和 Taste 分开治理。目标需要可衡量，上下文需要有来源与时效，品味则要通过样例、反例和人工裁决逐步显性化。
- 4. 设立类 FDE 责任节点。让一个稳定角色连接客户目标、数据、Agent 能力和交付结果，但通过工件和协议避免其成为单点。
- 5. 先设计人机责任，再设计多 Agent 编排。每个 Agent 必须有明确输入、输出、权限、Eval、求助条件和最终责任人。
- 6. 将按效果收费当成治理项目。在合同中定义基线、归因、数据权利、失败责任、最大暴露和停止条件，而不只是设置一个奖励比例。
- 7. 先在自己的真实工作中验证。内部使用可以暴露权限、质量、协调和人才问题，但必须进一步通过外部客户结果验证商业价值。

30.10 不能回避的四个挑战

挑战一：Agent 数量容易变成新的虚荣指标

3200 个 Agent 很容易吸引注意，但一个 Agent 可能只是一个低频工作流，也可能是高频接触客户和资金的自主系统。没有任务量、成功率、人工介入、失败影响和经营结果，Agent 数量不能证明组织成熟度。

挑战二：按效果收费会刺激局部优化

如果目标只是点击、转化或短期 ROI，Agent 可能用损害品牌、用户体验或隐私的方式获得指标。因此，“对结果负责”必须同时包括不可突破的负向约束。

挑战三：自动化可能削弱专业人才的成长路径

分析师过去通过数据清洗、标签和报告逐步学会判断。当这些工作被 Agent 承接后，组织必须让新人通过验证 Agent、诊断错误、设计 Eval 和处理异常获得等价的学习经验。

挑战四：经营改善不能全部归功于 AI

明略在 2022—2024 年大幅降低研发、销售与行政费用，2025 年又同时出现传统运营智能增长和 Agentic Services 新收入。因此，经调整扭亏是多个因素共同作用的结果，不能由结果反推“全部来自 Agent”。

30.11 本章小结：当专业服务开始承诺结果

明略科技的转型不是从一次大模型发布开始的。它的起点是近二十年的营销数据、企业知识、线下运营和项目交付。这些积累使它知道数据在哪里，也知道一份报告之后通常还有哪些工作。

真正的转折发生在公司不再满足于帮客户“看懂”，而是尝试让 Agent 把洞察、策划、内容、执行和反馈连成闭环。为了验证这个模式，明略先将 Agent 嵌入自己的职能、分析师和研发团队，再将“FDE + Agent 群”转化为对外交付方式。

这个转型最深的地方，不是报告生成变快了，而是公司的交易对象发生了改变：过去客户购买工具和专业投入，现在开始购买一个由人和 Agent 共同产生的结果。一旦承诺结果，明略就不能只对模型能力负责，还必须对目标是否正确、上下文是否真实、权限是否合适、证据是否可信以及失败能否被限制负责。

Agentic Services 已在 2025 年产生 1.002 亿元收入，但只占总收入约 7%。这一数字恰好使案例保持了真实：明略已经跨过概念验证，却还没有完成公司的重写。未来的判断标准不是 Agent 数量能否继续翻倍，而是结果付费能否形成稳定收入、可持续毛利、更好的客户结果和可信的责任体系。

专业服务的 AI 原生化，不是把分析师换成 Agent，而是把从问题到结果的整条责任链重新设计。执行可以规模化，目标、品味和最终责任必须有人承担。

本章最小实践：选择一份你的团队周期性交付的报告、分析或建议，不要先问 Agent 能否快速生成它。先画出报告交付后客户还要做哪些工作，再选择一个可逆、可验证的后续动作交给 Agent。为它定义 Objective、Context、Taste 样例、权限、Eval、人工升级和最终责任人，用客户问题是否更早被解决来评估试点。

30.12 主要资料与证据说明

- [明略科技官网, 公司介绍与发展历程](#)：成立时间、历史阶段、业务扩展、管理团队和客户口径；人物配图来源。
- [明略科技招股书, 2025](#)：业务结构、收入、客户、成本、研发与风险因素。
- [明略科技 2025 年度业绩公告](#)：2025 年收入、毛利、利润、Agentic Services 收入、交付效率与营销效果口径。财务结果与公司运营主张应区分证据强度。
- [明略科技, “AI Native 驱动业务升级, 1500 人的协同实验”, 2026](#)：内部组织实验、职能/分析师/研发团队的 Agent 应用与经营成效口径；姜平演讲配图来源。
- [明略科技, “全球 AI 公司押注的 FDE, 为什么是 AI 落地的真正解法?”, 2026](#)：吴明辉对 FDE、Objective、Context、Taste、Agentic Services 与 Octo 内部应用的说明。
- [财新, “营销营运 SaaS 公司明略科技登陆港交所”, 2025](#)：上市、股价与公司资本市场背景的外部报道。
- [流媒体网, 明略科技上市报道, 2025](#)：上市现场配图的刊载来源。

本章三张图片均来自公司官网或公开新闻报道，用于识别人物和还原案例现场。正式商业出版前应再核实原图片授权范围。

第三十一章 | Spotify：从敏捷研发到 Agentic Fleet Engineering

当一个软件产品的代码规模增长速度远远超过工程师人数，研发组织最先失去的往往不是写新功能的能力，而是维护旧系统的余力。Spotify 面临的正是这个问题：依赖升级、API 迁移、漏洞修复和组件改造不断占用工程师时间。

Spotify 成立于 2006 年，总部位于瑞典斯德哥尔摩，核心业务是数字音乐和播客服务。它长期采用小型、跨职能、相对自治的敏捷团队，并通过平台工程连接大量团队。随着规模扩大，团队越多，依赖越复杂；代码越多，统一升级越困难。

本章基于 Spotify Engineering 公开文章、工程负责人演讲摘要和公开技术材料。Spotify 披露的指标属于公司口径；本章将其作为案例证据，而不是未经检验的普遍规律。

31.1 敏捷团队很快，舰队维护很慢

Spotify 的产品价值流是：用户问题 → 产品实验 → 代码变更 → 测试 → 发布 → 用户反馈。

但工程维护流往往是：依赖升级或 API 迁移 → 找出受影响组件 → 设计脚本 → 各团队排队 → 分别修改与验证 → 逐个提交 PR。

这两条流遵循不同的经济规律。产品功能通常有明确的用户假设，可以用一次实验验证价值；维护任务的收益却常常表现为“没有发生什么”：没有服务中断，没有漏洞公开后的紧急返工，也没有某个周末突然爆发的技术债。收益延迟且不显眼，维护任务很容易输给产品路线图。

这里的“慢”不只是代码写得慢，而是五种速度同时受限：

1. **发现慢**：不知道哪些服务使用了旧 API，组件目录和依赖关系不完整。
2. **决策慢**：每个团队都要重新判断迁移方式，重复讨论同一类技术问题。
3. **执行慢**：同一种改动在不同仓库中反复手工完成。
4. **验证慢**：测试环境、技术栈和质量标准不一致，失败原因难以比较。
5. **收口慢**：PR 分散在不同团队，没人能快速判断哪些可以合并，哪些需要升级。

所以增加程序员并不能线性解决问题。工程师增加后，代码和服务也可能继续增加；如果依赖、标准、测试和发布机制没有一起改善，新增人员只会复制更多重复工作。

Spotify 公开称，几年前生产代码库增长速度约为工程师人数增长速度的七倍，维护工作逐渐挤压产品开发，迁移工作成为工程师主要的挫折来源之一。

这暴露了一个矛盾：**敏捷解决了团队如何快速学习和交付的问题，却没有自动解决全组织如何安全传播一次技术变更的问题。**

如果每个团队都自己处理同一种升级，组织得到局部自治，却付出重复劳动；如果中央团队统一处理，组织又会形成瓶颈。Spotify 的选择不是取消团队自治，而是在自治团队之上增加一个舰队级能力层：团队保留自己的服务和判断，平台负责识别范围、传播变更、汇总证据和暴露例外。

31.1.1 为什么原有敏捷机制没有自动解决这个问题

Squad 擅长围绕用户价值快速做出局部决策，但跨 Squad 的维护任务往往没有天然的产品 Owner。它不是单个团队的功能，也不是一次性项目；它穿过服务边界、团队边界和技术栈边界。

这会形成三种错位：

- **责任错位**：谁提出升级，谁通常只负责发起，并不负责所有下游组件完成。
- **收益错位**：投入由当前团队承担，收益却可能在未来由整个平台获得。
- **证据错位**：一个团队的测试全绿，只能证明自己的组件，不代表整条链路没有变化。

因此 Fleet Management 不是敏捷的反面，而是敏捷组织缺少的另一半。敏捷团队负责发现和交付价值；Fleet 层负责把共性工程意图变成可传播、可验证、可追踪的组织动作。

31.1.2 从“很多小任务”重新定义为“一次舰队变更”

没有 Fleet 视角时，一次 API 迁移看起来像几百个团队任务；有了 Fleet 视角，它首先是一个整体变更，再被分解为多个组件上的候选 PR。

旧对象：每个团队各自完成一个升级任务。

新对象：组织发起一次 Fleet Change，系统追踪全量影响和完成证据。

前者容易产生“我的 PR 已经完成”；后者必须回答“目标范围内还有哪些组件没有完成、为什么没有完成、谁来处理”。这正是从局部敏捷走向系统敏捷的转折点。

31.1.3 Fleet Management 到底是什么

Fleet Management 不是传统意义上的“管理部门”，也不是给 Squad 分派任务的项目办公室。这里的 Fleet，指由大量软件组件、服务、仓库和依赖组成的整个软件舰队；Fleet Management 是管理这些组件如何被统一发现、批量变更、验证和持续维护的一套平台能力与工作方式。

它主要承担五项职责：

- 1. **建立组件视图**：知道组织有哪些服务、由谁负责、使用什么技术、依赖哪些库，以及哪些组件会受到一次变更影响。
- 2. **定义变更范围**：把“升级某个依赖”或“迁移某个 API”转换为可执行的目标集合。
- 3. **编排批量执行**：按组件、技术栈、风险或团队分批生成变更，避免一次错误修改扩散到全部系统。
- 4. **汇总验证证据**：统一收集构建、测试、静态分析、部署和 PR 状态，让组织看到全局完成度。
- 5. **处理例外和回流学习**：识别无法自动迁移的组件，把失败交给责任团队，或沉淀为新的规则、脚本和测试。

可以把它理解成研发组织的“全舰队变更控制面”：

Fleet Management 负责：发现范围 → 安排批次 → 调度执行 → 汇总证据 → 暴露例外。

它不负责：替 Squad 决定用户价值，替团队承担产品责任，或绕过代码评审和发布门禁。

31.1.4 Fleet Management 与 Squad 的关系

二者解决的是不同层级的问题：

层级	Squad 关注什么	Fleet Management 关注什么
价值	用户问题、产品目标和业务结果	共性工程变更为什么需要全局推进
范围	自己负责的产品、服务或组件	哪些仓库、服务和依赖会受到影响
执行	在自己的代码和运行环境中交付	跨团队编排变更，分批调度 Agent 或自动化任务
验证	本团队功能和服务是否正确	全舰队是否完成、哪些例外未完成
责任	对产品行为和服务质量负责	对传播机制、证据汇总和异常路由负责

Fleet Management 不是 Squad 的“老板”，而是 Squad 之上的横向能力层。它不夺走团队自治，而是处理单个 Squad 无法独立解决的系统性问题。

举例来说，一个基础库升级可能影响 800 个服务：

Squad 负责：判断自己的服务是否适用，检查业务例外，审查本服务变更，并接受本服务风险。

Fleet Management 负责：识别 800 个服务，分组排序，生成候选 PR，运行统一验证，汇总完成情况，并将异常交回对应 Squad。

如果没有 Fleet Management，800 个 Squad 需要各自发现、判断、编写迁移方案；如果 Fleet Management 越权替 Squad 直接合并所有改动，又会破坏团队对服务的责任所有权。Spotify 的关键设计正在两者之间：

平台统一传播能力，Squad 保留局部判断和最终服务责任。

这也是它与 Agentic Agile 中 OA 角色相近的地方。OA 不取代 IO 做价值取舍，也不代替 AS 完成所有操作，而是把意图编排成可执行任务，持续收集证据，识别异常，并把需要人判断的部分交回责任人。

31.2 四阶段演进

阶段	核心做法	组织变化
第一阶段	小团队持续交付和内部工程平台	提升反馈速度与团队自治
第二阶段	Fleet Management 与 Fleetshift	让确定性维护变更跨组件传播
第三阶段	自动化 PR、大规模验证与合并	降低逐个处理的人工成本
第四阶段	后台 Coding Agent 与自然语言定义变更	让更多工程师发起复杂全舰队变更

31.2.1 从敏捷团队到平台化研发

小团队围绕用户价值持续实验，平台团队提供共同的交付基础设施。这个阶段的优势是反馈短、决策近、责任清晰；代价是组件、依赖和技术选择不断增加。

阶段启示：Agent 时代需要的不是消灭团队自治，而是为自治团队提供共同的上下文、约束和反馈基础设施。

31.2.2 Fleet Management 把维护变成组织能力

Spotify 建立 Fleet Management，并用 Fleetshift 执行跨大量软件组件的变更。依赖升级、API 迁移和安全修复，可以被描述为可重复的 Fleet Change，再由系统生成和验证 PR。

Spotify 公开称，Fleetshift 已合并超过 250 万个自动化维护 PR，其中大部分可以在没有人工介入的情况下自动合并。关键不在于自动合并，而在于自动化只覆盖目标明确、影响可预测、验证条件清晰的变更。

阶段启示：高自治的第一步通常不是让模型自由行动，而是先把可确定的工程规则写进系统。

31.2.3 从脚本迁移到 Agent 迁移

复杂迁移需要理解上下文：不同组件使用不同 API，历史代码存在例外，测试覆盖也不均匀。Spotify 随后把后台 Coding Agent 接入 Fleet Management。工程师可以用自然语言描述代码转换，Agent 负责理解仓库上下文、修改代码、运行验证并提出 PR。

Adding Honk to our Fleet Management tooling

Fleetshift

Find + target

Honk

Code, build, test

Fleetshift

Monitor + track



Spotify 官方架构示意：Fleetshift 负责目标识别、调度和进度跟踪，Honk 负责实际代码修改，并通过 CI 验证。图片来源：[Spotify Engineering](#)。

Agent 降低的不是写一段代码的成本，而是把复杂迁移变成可重复 Fleet Change 的门槛。它通过 Slack、GitHub Enterprise 和 MCP 等入口连接日常协作与工程系统。

阶段启示：Agent 放大的不是单个开发者的手速，而是组织把一次判断传播到整个代码舰队的能力。

31.2.4 Agentic Fleet Engineering

Spotify 2026 年公开称，超过 99% 的工程师每周使用 AI 编码工具，94% 认为 AI 提高了生产力，PR 频率提高 76%。这些数据说明采用非常广泛，但不自动说明质量和业务价值同步提高。

更重要的是，Spotify 将 AI 放进已有 Fleet Management，而不是让每个工程师在本地孤立使用 Agent。变化分成三层：工程师定义一次跨组件变更的意图；Fleet Management 识别范围、安排批次、调用 Agent 并汇总结果；Squad 判断本服务的业务语义和例外。Agent 获得的是执行权，不是产品决策权，也不是不受约束的全局写入权。

这不是简单地把“写代码的人”换成“写代码的 Agent”，而是把研发任务拆成三层：工程师定义一次跨组件变更的意图；Fleet Management 识别范围、安排批次、调用 Agent 并汇总结果；Squad 判断本服务的业务语义和例外。Agent 获得的是执行权，不是产品决策权，也不是不受约束的全局写入权。

31.3 四维转型

这里的“四维”不是四种工具清单，而是同一个问题的四个观察面：谁承担工作、谁拥有能力、工作怎样流动、系统靠什么保证结果。只有四个面同时改变，Fleet Change 才不会退化成一个更快的脚本。

31.3.1 人员：从组件维护者到变更系统设计者

工程师从重复修改每个组件，转向定义变更意图、识别例外和风险、设计测试与回滚条件、处理 Agent 无法判断的异常，并把经验沉淀回系统。代码产出减少手工劳动，责任判断却更加集中。

31.3.2 组织：产品团队加平台控制面

产品团队仍围绕用户价值运行；Fleet Management、开发者平台和基础设施团队承担跨团队共性能力。产品团队决定为什么改、改什么；平台团队负责怎样跨舰队执行、验证和观测。

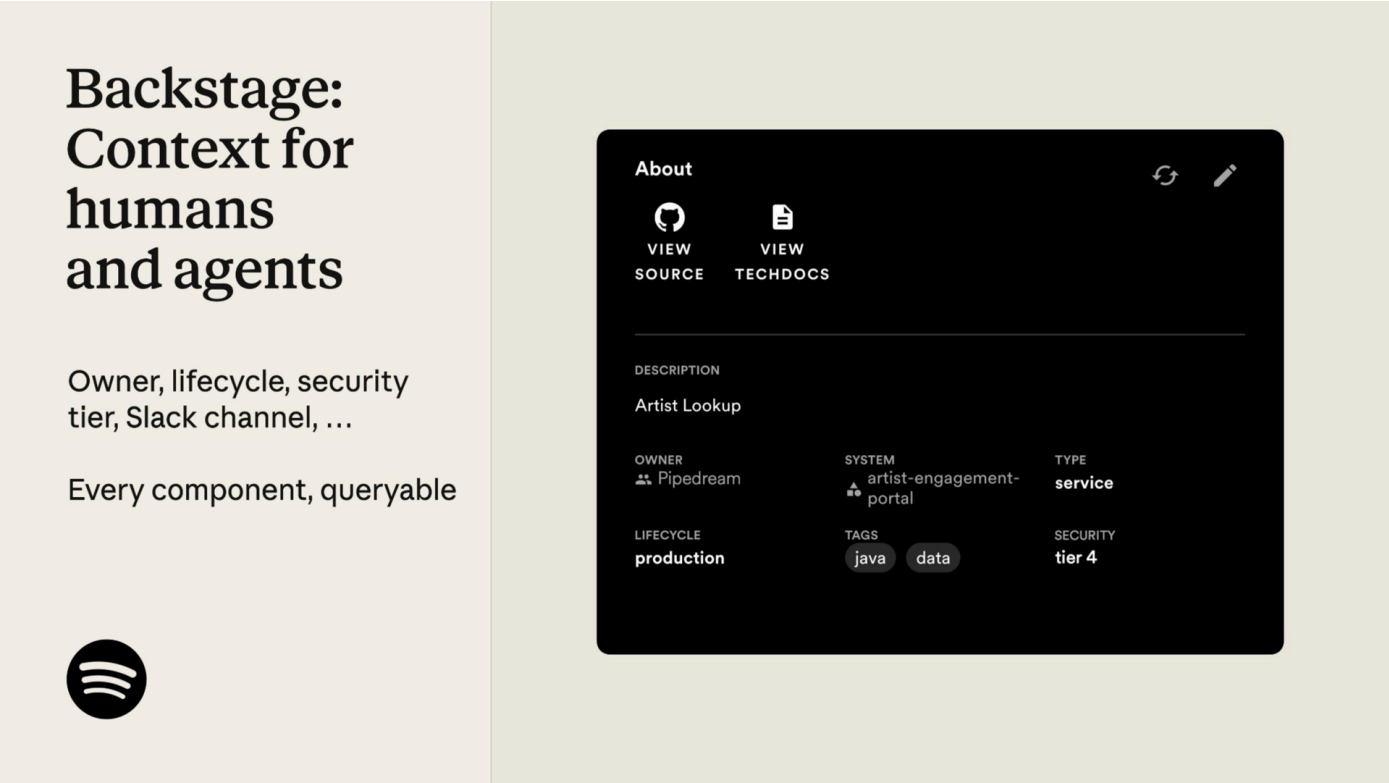
31.3.3 流程：从单仓库 PR 到全舰队闭环

变更意图先识别受影响组件，再由 Agent 生成候选变更，经过自动测试、静态检查和 PR 评审，最后自动合并或人工升级。运行结果与失败模式回流下一轮。

工作单元从函数或文件变成跨多个仓库的可验证变更。验证不再是最后一道门，而是编排过程的一部分。

31.3.4 工具：从脚本平台到 Agent Harness

Fleetshift 提供执行能力，组件目录提供上下文，CI 和测试提供约束，PR 提供审查和证据，MCP 提供 Agent 与工程系统的连接。可靠性不是模型单点属性，而是模型、代码库、测试、权限、反馈和回滚机制共同形成的系统属性。



Spotify 将 Backstage Software Catalog 同时作为工程师和 Agent 的组件上下文入口。图片来源：[Spotify Engineering](#)。

31.4 与 3-4-3 的机制映射

3-4-3	Spotify 对应机制
IO	产品团队或工程师定义 Fleet Change 的目标、范围和价值
OA	Fleet Management、Fleetshift 和平台工程团队负责编排与验证
AS	后台 Coding Agent 执行转换、生成 PR 和处理重复任务
意图图谱	组件、依赖、仓库、PR、测试、团队和发布关系
意图契约	迁移目标、影响范围、排除条件、验收和回滚条件
约束矩阵	仓库规则、测试、CI、权限、自动合并和人工升级规则
证据包	diff、测试结果、PR、合并状态、失败日志和运行反馈
意图注入	自然语言 Fleet Change、工程文档和仓库上下文
对抗自净化	测试失败、静态检查、跨组件验证和异常反馈
人类异常裁决	高风险组件、无法验证的迁移、例外规则和生产影响判断

31.5 用 SCOPE-V 重放一次 Fleet Change

假设 Spotify 需要将一个已弃用 API 迁移到新版本：

- 1. **Specify**：IO 说明为什么迁移、目标 API、必须保持的行为、排除服务和完成标准。
- 2. **Constrain**：OA 读取组件目录和仓库规则，设置批次、权限、测试门禁和自动合并条件。
- 3. **Orchestrate**：Fleet 平台识别范围，按技术栈和风险分批调度；AS 修改代码、生成 PR、运行 CI。
- 4. **Prove $\Rightarrow$ Evolve**：测试失败返回 Agent；同类失败反复出现时暂停批次，先更新规则、测试或上下文。
- 5. **Verify**：Squad 检查本服务业务语义，平台责任人检查全局完成度、传播风险和回滚能力，再决定合并或停止。

批量不意味着一次性放大权限。更好的做法是分批执行、逐步扩大范围，让失败在传播前暴露。

31.6 真实声音与证据边界

Spotify 工程负责人曾指出，生产代码增长速度约为工程师人数增长速度的七倍，迁移工作成为开发者主要挫折来源。Fleetshift 的累计自动化维护 PR 超过 250 万个，说明 Spotify 在 Agent 之前已经拥有大规模自动化执行底座。

Spotify 2026 年又用一句话概括变化：

“Coding is no longer the constraint.”

编码不再是约束。

约束发生转移：组织接下来必须回答哪些变更值得传播、哪些测试足以支持自动合并、哪些错误会同步扩散，以及如何让人类关注高风险例外。

99% 使用率、94% 生产力感知和 76% PR 增长属于采用与吞吐指标，不等于质量、客户体验或长期可维护性指标。

31.6.1 这些数字为什么不能直接证明研发提效

“PR 增长 76%”首先说明可提交的变更变多了，而不是有价值的交付变多了。它可能意味着 Agent 让有价值的工作更快完成，也可能意味着一个大变更被拆成更多小 PR，或者低成本生成了更多尚未产生价值的实验。

“99% 的工程师每周使用 AI”说明工具进入日常工作，却没有回答使用深度、任务类型、失败率和团队差异。94% 的生产力感知很重要，但它属于体验证据，不等于质量证据。

Spotify 案例真正值得学习的不是三个漂亮百分比，而是指标背后的控制结构：代码规模压力推动 Fleetshift；Fleetshift 的自动化底座承接 Honk；Backstage 提供上下文和标准；PR 增长之后，人类决策又成为新的约束。只有把这些放在一起，才能看见转型的因果链。

31.6.2 关键转折：瓶颈从编码转移到判断

当 Agent 可以在几分钟内生成候选变更，研发组织会遇到一个反直觉结果：最稀缺的资源不再是写代码的时间，而是判断哪些代码值得合并的注意力。

组织因此必须重新设计评审：

1. 对低风险、规则明确、验证充分的变更，允许自动合并；
2. 对跨服务、权限、数据迁移和用户行为变化，提升人工评审等级；
3. 对证据不足或失败模式未知的变更，停止传播并进入异常处理；
4. 将评审发现的重复问题沉淀为规则、测试或组件标准。

如果只增加 Agent，而不改变风险分级和评审机制，组织会从“写不完代码”进入“看不完 PR”。Spotify 的公开材料已经承认这一点：编码速度提升后，人的决策能力成为新的约束。

31.6.3 Spotify 案例的适用边界

Fleet Engineering 最适合三类任务：影响范围可以识别、目标可以描述、结果可以自动验证。依赖升级、API 替换、漏洞修复和组件标准化通常符合这些条件。

它并不自动适合产品方向选择、用户体验取舍、核心架构重构和高风险数据迁移。这些任务的难点不是把同一种改动传播得更快，而是判断应该不应该改、牺牲什么、风险由谁承担。

因此，Spotify 不是“所有研发都交给 Agent”的案例，而是“把适合规模化的工程变更交给 Agent，同时把价值判断和异常裁决留给 人”的案例。

31.7 六个核心启示

这六点是一条因果链，而不是并列口号：

1. **敏捷扩大了局部自治，也制造了跨组件协调债。** Squad 能快速交付自己的服务，却不能独立完成全舰队升级。
2. **协调债要求横向控制面。** Fleet Management 负责发现范围、安排批次、传播变更、汇总证据和暴露例外。
3. **控制面应先承接确定性自动化。** 目标明确、影响可预测、结果可验证的任务，才适合自动合并。
4. **复杂例外推动 Agent 进入执行环。** Agent 扩大可处理任务范围，也扩大错误传播风险。
5. **风险扩大后，人员角色必须升级。** 工程师从逐个修改文件，转向定义意图、设计验证、处理异常和维护 Harness。
6. **最终标准从吞吐转为系统结果。** PR 数量增加，只有在质量、维护成本、用户价值和风险没有恶化时才算提效。

31.8 案例总结

Spotify 的关键不是“用了 AI”，而是它先解决了 AI 最容易放大的三个问题：组件分散、上下文不足、规则无法跨团队执行。Fleetshift、Backstage 和工程标准化先形成底座，Honk 才有可能进入这条责任链。

- **人员**从逐个维护者转向变更系统设计者；
- **组织**形成产品团队与平台控制面的新分工；
- **流程**从单 PR 交付转向 Fleet Change 闭环；
- **工具**从确定性脚本平台演进为 Agent Harness。

它补充了 3-4-3 的一个重要场景：很多高价值任务并不围绕一个客户或一条产品需求，而是跨多个组件的基础设施演进。此时，意图图谱、约束矩阵、批量证据和异常裁决，比单次 Prompt 更重要。

这个案例也给出一个限制条件：Agent 不是敏捷的替代品。没有清晰的产品意图、稳定的组件目录、可执行的工程约束和可靠的验证机制，Agent 只会让更多错误更快进入更多仓库。

敏捷让团队更快学习和交付；**Agentic Fleet Engineering** 进一步让一次经过验证的工程意图，能够在边界内被整个软件舰队安全吸收。

案例资料

- [Coding Is No Longer the Constraint: Scaling Developer Experience to Teams and Agents at Spotify](#)
- [1,500+ PRs Later: Spotify's Journey with Our Background Coding Agent](#)
- [Let's Talk Agentic Development: Spotify x Anthropic Live](#)

第三十二章 | GitLab：研发工具厂商如何把 Agent 纳入统一控制面

一个开发者打开 Issue，写下客户问题、验收标准和不可触碰的边界，然后将任务分配给 Agent。Agent 读取代码、历史 Issue、架构文档和 `AGENTS.md`，修改多个文件，运行测试，并创建 Merge Request。

流水线失败了。Agent 读取日志、修复问题、再次提交。CI 转绿后，另一个评审角色专门寻找反例，发现一条被忽略的权限路径，实现被退回。最后，人类 Maintainer 核对意图、风险和证据，决定是否合并。

这不是对未来研发的科幻描述，而是 GitLab 在 2026 年《AI-Assisted Development Playbook》中写出的基本循环：

Issue 与需求

- 共同制定计划
- 技术规格
- Agent 实现
- CI 与测试验证 ⇄ 修复
- 对抗评审 ⇄ 修复
- Merge Request
- 人类评审与合并

当人和 Agent 都能改变软件时，谁定义意图，谁执行，谁证明，谁批准，失败又由谁承担？

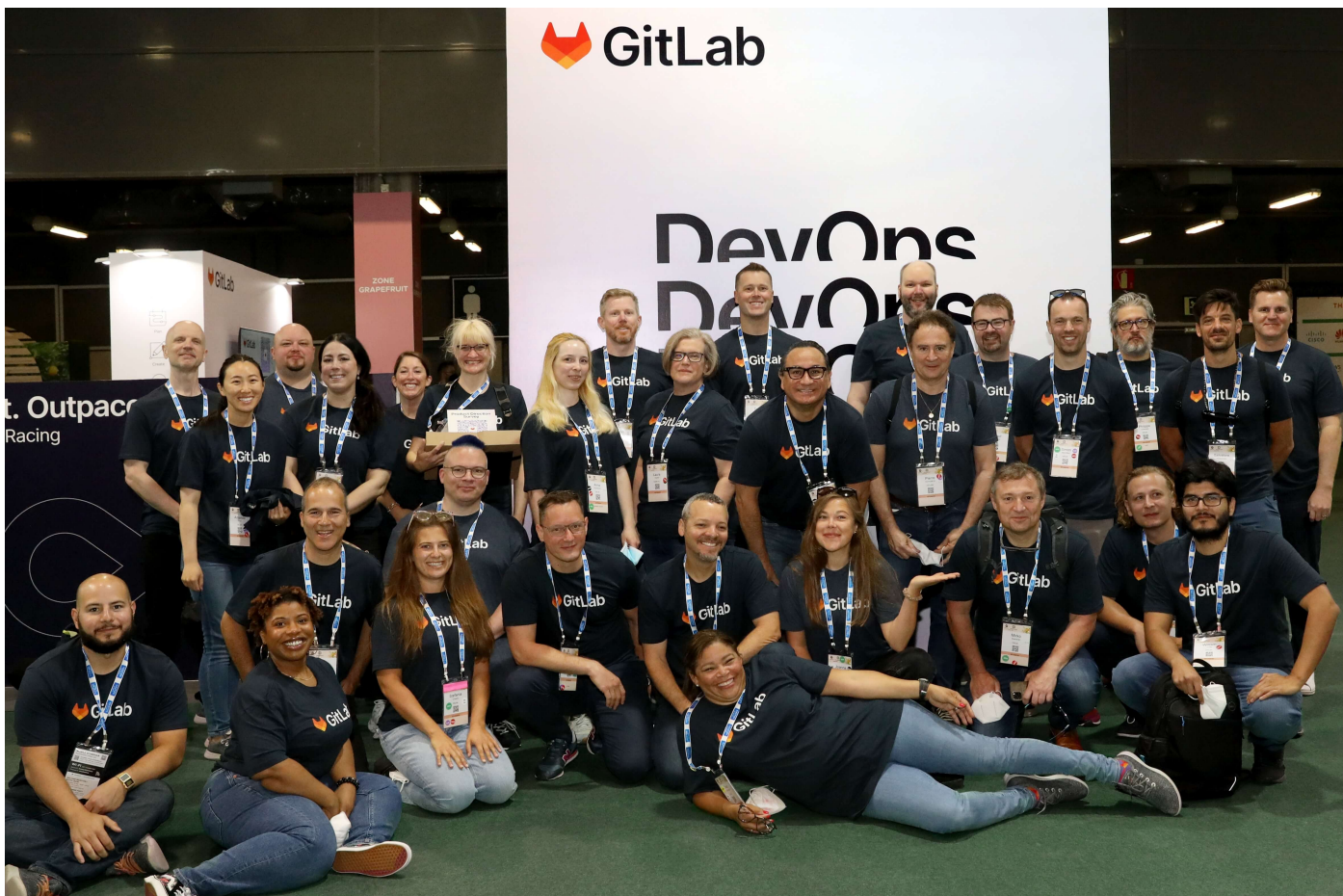
本章基于截至 2026 年 8 月的 GitLab 官方网站、公开 Handbook 与产品文档。需要特别说明：GitLab 没有宣布采用 Agentic Agile 3-4-3。本章做的是机制对照，而不是品牌归属；产品能力、内部规范和规模化效果也将分开陈述。

本章的案例定位不同于上一章的 Spotify。Spotify 主要展示一个软件产品组织如何因规模压力重构研发价值流；GitLab 主要展示一个研发工具厂商如何把 Agent、上下文、权限、验证和审计纳入产品控制面，并用自己的研发团队进行 dogfooding。它适合说明“控制面如何形成”，不适合单独证明“四维组织转型已经完成”。

32.1 GitLab 是谁：把自己的研发方式做成产品

GitLab 起源于 2011 年的开源项目。Dmitriy Zaporozhets 和 Valeriy Sizov 最初想做一个能在本地自由使用的 Git 协作工具。2013 年，Sid Sijbrandij 与 Dmitriy 开始建立商业实体；2014 年公司正式注册；2015 年加入 Y Combinator，并将 GitLab Handbook 公开到网站仓库；2021 年在纳斯达克上市。

它的产品沿着自身研发痛点逐步长出来。最初只有代码管理；为了自动测试 GitLab，团队建立 GitLab CI；随后把计划、代码、安全、CI/CD、部署和观测逐步收进同一平台。这种开放核心模式既服务开源社区，也通过 SaaS 和企业订阅获得收入。



分布在不同国家和地区的 GitLab 团队成员在线下活动相聚。GitLab 从成立之初就采用 all-remote 模式；截至 2026 年 6 月，公司称有 2500 多名团队成员、5500 多名代码贡献者和超过 5000 万注册用户。图片来源：[GitLab 公司网站](#)。

全远程是理解 GitLab 的关键。当同事不在同一间办公室，依赖口头默契的管理很快就会失效。因此，工作原则、角色责任、开发流程和决策记录被持续写入 Handbook。这些文档原本为人类异步协作准备，后来却成为 Agent 可读的组织上下文。GitLab 既是 Agentic 研发工具的制造者，也是使用自己产品的“客户零号”。

32.2 Agent 之前：组织已经共享一条数字交付链

GitLab 在 AI 转型前，已经将大部分研发活动放在同一个可见系统中：

```
Epic / Issue 定义价值与范围
→ 设计与技术方案
→ 分支与代码变更
→ Merge Request 汇集讨论与批准
→ CI/CD 执行测试、安全检查和构建
→ 部署与运行数据
→ 新 Issue 与后续改进
```

人仍是主要执行者，但意图、变更、批准和运行证据已有可追溯关系。如果需求、代码、测试和批准散落在互不相通的工具中，Agent 即使能写代码，也看不到完整责任链。

- **工作对象化**：Issue、MR、Pipeline、Deployment 和 Vulnerability 都是可引用对象。
- **规则代码化**：测试、静态分析、权限和批准规则进入 CI/CD 与仓库。
- **决策可追溯**：从 Issue 到 MR 再到部署，可以重建“为什么改、改了什么、谁批准”。

核心启示：Agent 不会自动消除工具断点和组织断点，它通常会放大这些断点。先让人类交付链可计算，才能让 Agent 可治理。

32.3 两条演进线：产品能力与内部实践

GitLab 的转型不能只写成一条产品发布时间线。产品能力回答“客户可以怎样使用 Agent”，内部实践回答“GitLab 自己怎样改变研发工作”。两条线相互促进，但证据性质不同。

阶段	产品能力线	内部实践线	证据性质
第一阶段	2011—2022：代码管理、CI/CD、DevSecOps 平台逐步统一	all-remote、Handbook-first，先建立可见的异步协作系统	长期组织与产品事实
第二阶段	2023—2024：Duo 进入代码建议、解释、测试和评审	工程、技术写作和产品团队开始 dogfood	产品能力与内部规范
第三阶段	2024—2025：Duo Workflow 可从需求生成 MR	Agent 从建议者变为异步执行者，但仍进入 MR 批准流程	产品演示与局部实践
第四阶段	2025 至今：Agent Platform、AI Catalog、Flows、Orbit	内部 Playbook 规定自治等级、Harness 和 AI 贡献披露	产品路线与规范状态

32.3.1 第一阶段：先把工作变成机器可读的事实

GitLab 长期实行每月 22 日发布新版本的节奏。分布式远程团队要持续发布，不能依赖“找那个懂的人问一下”。任务状态、代码评审、运行手册和决策方法必须外化。

这个阶段没有 Agent，却在建设 Agent 最稀缺的东西：可定位的意图、版本化的知识、明确的责任人和自动化反馈。

阶段启示：AI Native 的前置工程，往往是让组织先停止依赖隐性知识。

32.3.2 第二阶段：Duo 进入人的工作环

GitLab Duo 最初主要提供代码建议、解释、测试生成、MR 摘要和漏洞解释。这一阶段仍是“人调用 AI”：人拥有任务，AI 缩短局部操作。

GitLab 同时在真实任务中 dogfood Duo，把摩擦变成 Issue。Developer Experience 团队后来要求 MR 作者使用 `devex-ai-assistance::1-5` 标签，披露 AI 参与程度。

阶段启示：使用量不是转型结果，却是建立真实反馈的起点。只有 AI 贡献可见，组织才能比较不同自治等级下的质量、评审成本和失败率。

32.3.3 第三阶段：从建议代码到承接完整任务

2024 年，GitLab 公布 Duo Workflow，将其从反应式助手推向自主 Agent。Agent 可以理解需求、读取上下文、修改多个文件、运行命令并创建 MR；遇到不确定性时请求指导，产出仍进入原有 MR 批准流程。

人不再介入机器内环的每一步，却必须守住需求、异常和批准等责任节点。

Developer Advocacy at GitLab / projects / Cheap flights / Issues / #15

Open

Implement Analytics Dashboard and Admin Panel

Edit

Popular Routes: Most searched and booked flight routes

Revenue Metrics: Booking values and conversion rates

Performance Metrics: Search response times, error rates

User Behavior: Search patterns, abandonment rates

Reporting Features

- Exportable reports (CSV, PDF)
- Custom date range filtering
- Real-time vs historical data
- Automated weekly/monthly reports
- Alert system for anomalies

Data Visualization

- Interactive charts and graphs
- Geographic heat maps for popular routes

Read more

0 of 8 checklist items completed

0

0

Add design

Create merge request

Generate MR with Duo

Child items 0

Add

No child items are currently assigned. Use child items to break down work into smaller parts.

Labels

None

Edit

Parent

None

Edit

Weight

None

Edit

Milestone

None

Edit

Iteration

None

Edit

Dates

Start: None

Due: None

Edit

Health status

None

Edit

Affected-systems

None

Edit

GitLab 官方示例：开发者从 Issue 启动 Developer Flow，Agent 在后台完成变更并创建待评审的 MR。这不等于自动获得合并权。图片来源：[GitLab Duo Agent Platform 指南](#)。

阶段启示：Agent 可以成为执行 Owner，却不能因此成为风险 Owner。执行权与最终责任必须分开设计。

32.3.4 第四阶段：把 Agent 收入组织控制面

Duo Agent Platform 把 Agent、Flow、模型、上下文、权限和运行记录收入统一平台。组织可以管理专业 Agent 和多步 Flow，规定其运行位置、模型和工具。每次执行形成 Session，记录触发条件、工具调用、输出和 CI/CD 日志。

GitLab^{Duo} Agent Platform

Build, customize, and connect AI agents for your workflow

Teams interact with Agents & Flows

Duo Agentic Chat

Purpose: **Conversations and tasks**

How to access: Chat panel or IDE extension

Execution: Real-time

Model selection: User selects

Agent type: **Foundational or Custom**

Custom Flows

Purpose: **Complex tasks and automation**

How to access: Mention or assign in Issues/MRs

Execution: Runner

Model selection: User selects

Flow type: **Custom**

Foundational Flows

Purpose: **Common development tasks**

How to access: Built-in action buttons

Execution: Runner

Model selection: Pre-configured

Flow type: **Foundational (Maintained by GitLab)**

External Agents

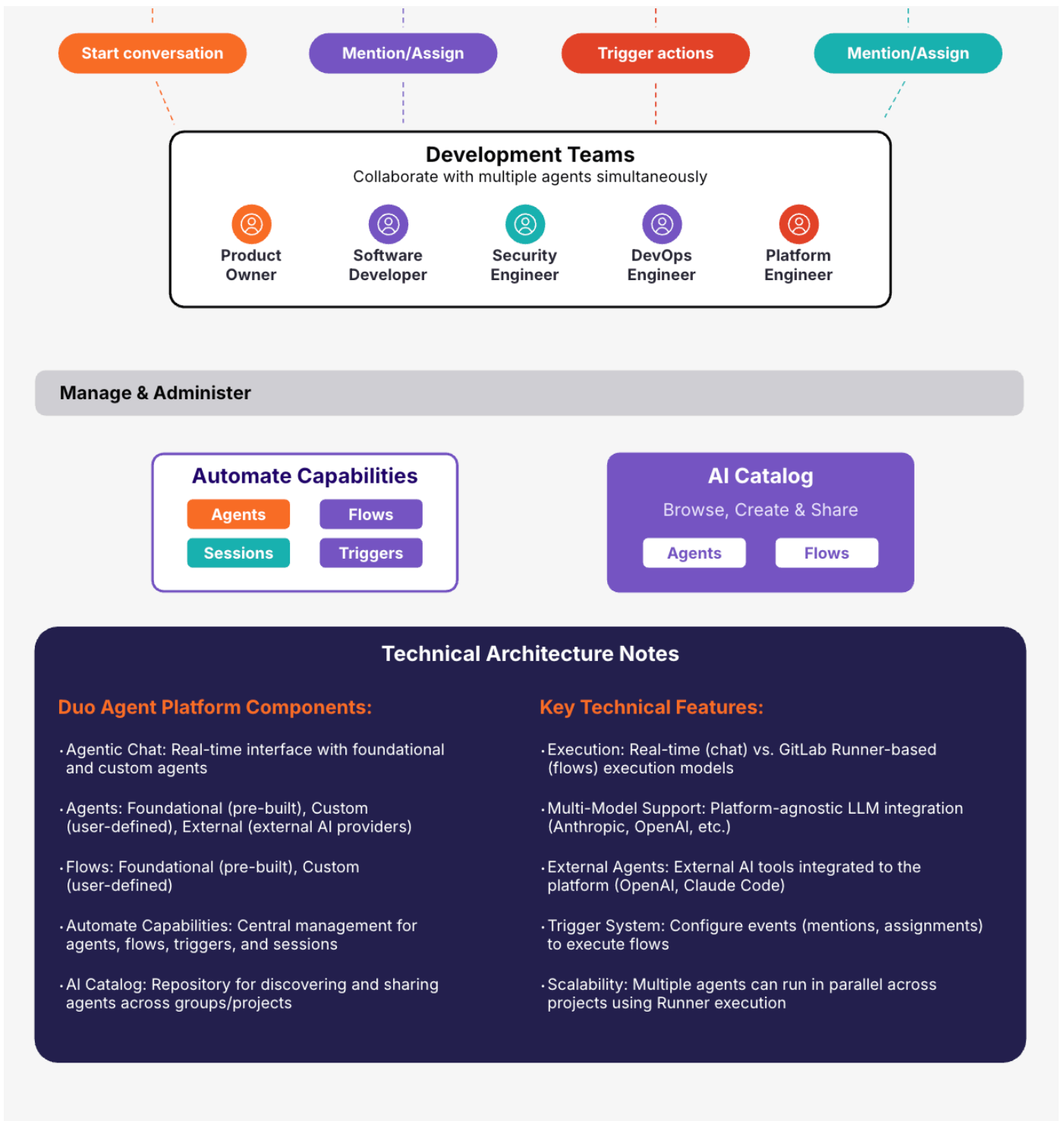
Purpose: **External AI tool providers**

How to access: Mention or assign in Issues/MRs

Execution: Runner

Model selection: External tool defines

Agent type: **External**



Duo Agent Platform 把 Chat、CLI、内置与外部 Agent、Flows、AI Catalog 和运行 Session 连接起来。图片来源: [GitLab, 2026](#)。

GitLab Orbit 则将代码、Issue、MR、Pipeline 和 Deployment 组成持续更新的上下文图，并通过 MCP 向 Duo、Claude Code 和 Codex 提供基于索引事实的查询。它的目标不是让模型“猜”依赖，而是让依赖成为可遍历关系。

阶段启示：当 Agent 可以异步、并行和跨工具执行时，组织需要的不是更好的聊天窗口，而是 Agent 控制面。

32.4 四维转型：改变的不只是工具

32.4.1 人员：从直接编码者到意图、架构与裁决者

GitLab 将 AI 自治分为五级：Baseline、Pair、Conductor、Orchestrator 和 Harness。到 Conductor，人引导单个 Agent；到 Orchestrator，人管理多个异步 Agent；到 Harness，人主要设定架构和质量标准，其余执行交给系统。

这不是“人离开回路”，而是人从操作回路上移到责任回路。GitLab 在招聘实验中也开始评估候选人表达意图、定义边界、验证输出和借助 AI 判断的能力。

32.4.2 组织：从 AI 小组到横跨产品和平台的能力网络

GitLab 的 AI Engineering 已分化出 Agent Foundations、AI Coding、AI Core Infrastructure、Model Services、Clients 和 Chat 等能力组。共享平台团队建设控制面，产品、安全和开发者体验团队则在自身价值流中使用并反馈。

32.4.3 流程：从“写完再审”到双重反馈环

- 1. 自动验证环：CI 或测试失败，实现立即返回 Agent。
- 2. 对抗评审环：即使测试全绿，仍主动寻找计划和实现的缺陷。

测试回答“已知性质是否成立”，对抗评审追问“我们遗漏了什么”。两者通过后，变更才进入 MR 和人类批准。

32.4.4 工具：从 AI 插件到研发控制面

AGENTS.md 和仓库文档提供局部上下文；Issue 和技术规格提供任务意图；CI/CD 把安全、测试和合规规则变成可执行约束；Orbit 连接跨生命周期事实；Session 记录 Agent 行为；MR 承载评审和最终批准。

32.5 为什么它与 3-4-3 高度同构

三个超级角色

Agentic Agile 角色	GitLab 中的对应机制	差异
IO（意图责任人）	Issue 发起人、Product Manager、DRI 或 Maintainer	未统一使用 IO 概念，意图签署强度因团队而异
OA（编排与保证者）	Conductor / Orchestrator、Flow、Pipeline、MR 批准规则	编排与独立保证未必由不同主体承担
AS（Agent 执行系统）	Duo Agent、外部 Agent、Developer Flow、CI 与安全 Agent	自治等级仍受仓库成熟度限制

四大动态工件

3-4-3 工件	GitLab 中的近似形态	未完全覆盖的部分
意图图谱	Epic、Issue、代码、MR、Pipeline、Deployment 及 Orbit 图	价值因果与战略权重仍需补足
意图契约	Issue、验收标准、技术规格和失败测试	尚非统一的可签署契约，非功能边界可能分散
约束矩阵	AGENTS.md、CODEOWNERS、分支保护、CI 门禁、安全与评审指令	规则分散，冲突优先级不总是显式
证据包	MR diff、Pipeline、测试、安全报告、批准和 Agent Session	有证据元素，但不等于逐项契约化的 Evidence Bundle

三大自治机制

3-4-3 机制	GitLab 中的对应	成熟度判断
意图注入	Issue、Spec、仓库文档和 AGENTS.md 进入 Agent 上下文	较强，但取决于仓库知识质量
对抗自净化	失败测试、CI 返工环、对抗评审和专业审查 Agent	已进入 Playbook，独立性仍需按项目检查
人类异常裁决	Agent 求助、MR 人工评审、CODEOWNERS 与保护分支批准	节点存在，但要防止橡皮图章式审批

GitLab 已有很多 3-4-3“部件”，但部件存在不等于每个任务都形成了完整闭环。

32.6 用 SCOPE-V 重放一次 GitLab 式交付

假设团队要修复“特定权限用户无法查看流水线日志”的缺陷：

1. **Specify**：Issue 记录场景、期望结果、不得放大的权限和验收示例；失败测试把“完成”转为可执行预言。
2. **Constrain**：AGENTS.md 指明架构、命令和禁改区；CODEOWNERS 要求权限专家评审；分支保护阻止 Agent 直接写入主分支。
3. **Orchestrate**：Planner 完善方案，Developer Agent 修改实现，CI 执行测试与安全扫描，Reviewer 用反例检查权限泄漏。
4. **Prove** ⇌ **Evolve**：失败测试和评审意见返回实现环；通用缺口沉淀为新测试、静态规则或 MR 评审指令。
5. **Verify**：人类 Maintainer 不只看“Pipeline 全绿”，还核对客户问题、权限边界、变更影响和回滚能力，再决定是否合并。

GitLab 提供的平台不能自动保证每个组织都正确定义了意图、拥有独立证据，或真正完成风险裁决。

32.7 真实结果：有高强度样本，还没有全局答案

GitLab Handbook 披露了一个有冲击力的内部样本：Orbit 知识图谱项目由 4 名工程师在 2 周内完成 259 个 MR，约 13.5 万行 Rust 代码中 95% 由 AI 生成。GitLab 的解释不是“模型足够聪明”，而是项目从第一天就有 CI、AGENTS.md 和架构文档。

这个样本证明高自治研发在特定边界内可以产生极高吞吐，但它不能证明：

- 效果可迁移到历史包袱深、测试薄弱的老系统；
- 95% AI 代码等于 95% 价值或责任由 AI 承担；
- MR 数量和代码行数可以替代缺陷、安全、成本与客户结果；
- 局部实践已在 2500 多人的公司中达到同等成熟度。

更严谨的结论是：GitLab 已提供一个强度很高的 Agentic 研发样本，但仍在把局部高自治经验转化为组织级稳定能力。

32.8 内部声音：快不是唯一价值，责任也没有转移

GitLab 对分工的概括很直接：

“Software will be built by machines, directed by people.”

软件将由机器构建，由人来定向。

这里的 directed 不应被简化为“写 Prompt”。GitLab 为人保留的责任包括架构、客户理解、品味和权衡。Developer Experience 团队还明确写道：

“AI-assisted review, not AI-replaced review.”

用 AI 辅助评审，而不是用 AI 取代评审。

其 AI 开发手册的一条原则指向组织学习：

“Fix the environment, not the prompt.”

出现系统性失败时，不要只改这一次 Prompt，而要把学习写进测试、Lint、文档或门禁。

这与 Evolve 非常接近：一次失败的价值，不只在当次被修好，而在于它是否改变了下一次执行的环境。

32.9 六个核心论点

1. **Agentic 研发的起点不是 Agent，而是可计算的责任链。** 只有意图、代码、验证、批准和运行证据可以关联，Agent 才能看到完整任务。
2. **高自治由仓库和组织成熟度授予。** 没有 CI、测试、上下文和评审成熟度时，直接跳到 Orchestrator 或 Harness 只会放大技术债。
3. **约束应进入环境，而不只留在 Prompt。** 测试、分支保护和 CI 门禁比临时指令更稳定。
4. **对抗评审是高速生产的必要配对。** Agent 越快，系统越需要主动反驳方案、寻找遗漏和构造反例。
5. **人可退出操作内环，不能退出责任回路。** 人仍要守住意图、权限扩大、重大异常、不可逆动作和剩余风险。
6. **自我改进意味着失败改变下一次执行环境。** 人工介入只有转化为测试、规则、文档、工具或评估样本，才会成为组织能力。

32.10 对研发组织的行动指南

1. **先选一个仓库。** 选择责任人清晰、发布频繁、结果可验证的代码库。
2. **评估 Harness 成熟度。** 检查测试、CI、架构文档、分支保护、CODEOWNERS、回滚和观测，再决定自治级别。
3. **把任务改写为可验证契约。** 写明功能、Preserve、禁改区、验收证据和停止条件。
4. **建立精简的 AGENTS.md。** 放入项目结构、实际命令、关键约定和 off-limits 区，并由团队评审。
5. **先加一条确定性约束。** 从失败测试、Secret Detection、Lint 或测试数量保护中选择一项。
6. **分离执行与批准。** Agent 可创建 MR，但不能扩大权限、降低门禁或批准自己的变更。
7. **记录 AI 贡献与人工介入。** 除生成比例，还记录成功率、返工、评审时间、逃逸缺陷和求助原因。
8. **把重复失败修进系统。** 每次人工救火后追问：能否沉淀为测试、规则、上下文或新验证能力？

32.11 还没有解决的四个问题

第一，吞吐遥测不是价值遥测

AI 代码比例、MR 数量和节省时间容易计算，却可能鼓励生产更多变更。价值遥测还必须连接客户结果、缺陷逃逸、可维护性、运行成本与安全风险。

第二，人类评审可能成为新瓶颈和假门禁

Agent 能同时生成大量 MR，人的注意力却没有同比增长。没有风险分级、证据摘要和异常路由，“人在环上”容易退化为快速点击同意。

第三，生成、测试与评审的独立性仍需证明

同一模型、相同上下文和类似 Prompt 可能产生共同盲区。多 Agent 不自动等于多元证据。高风险变更仍需要确定性工具、独立上下文或不同责任主体的验证。

第四，上下文图展示事实关系，不是价值判断

Orbit 可以说明代码与 MR、Pipeline 和 Deployment 的关系，却不能替人回答：这项改动是否值得做？剩余风险由谁接受？知识图可以增强判断，不能替代判断。

32.12 案例总结：机器开始构建软件，组织必须重构责任

人定义意图与不可牺牲的边界

→ 平台注入上下文与最小权限

→ Agent 承接持续执行

→ CI 和对抗评审持续反驳

→ MR 汇集变更与证据

→ 人对异常、风险与不可逆动作作出裁决

→ 失败进入测试、规则与下一次执行环境

AI 原生研发组织不是拥有最多编码 Agent 的组织，而是能够让意图、约束、人、Agent、证据和责任在同一条交付链上持续对齐的组织。

机器可以构建软件，却不会因为 Pipeline 转绿就自动获得业务正当性。当执行被大规模交给 Agent，人的价值不会消失；它会集中到更少、更难，也更不可推卸的责任节点上。

案例资料与证据边界

主要资料

- [About GitLab](#)：公司历史、规模与时间线。
- [GitLab 上市时的创始人信](#)：项目起源、GitLab CI 与商业化过程。
- [GitLab Duo Agent Platform](#)：Agent、Flow、AI Catalog、政策和可追溯性。
- [Duo Agent Platform 指南](#)：架构、Session 与从 Issue 生成 MR 的工作流。
- [AI-Assisted Development Playbook](#)：五级自治、Harness、对抗评审与内部样本。
- [AI in Developer Experience](#)：客户零号、AI 贡献标签、人工评审与仓库上下文。
- [GitLab Orbit](#)：软件生命周期上下文图与 MCP 连接。
- [GitLab Mission](#)：人与机器在软件建造中的定位。

证据边界

1. 公开 Handbook 中一些内容是规范与目标状态，不等于所有团队已一致执行。
2. Orbit 项目数据来自 GitLab 自报，是单项目样本，不是公司级对照实验。
3. 部分 Agent Platform 能力在 2025—2026 年才进入可用或规模化阶段，长期质量与人工批准负荷仍需观察。
4. 本章的 3-4-3 与 SCOPE-V 对照属于本书分析，不代表 GitLab 对该框架的认可或采用。

附录与行动入口

附录不是正文的缩写版，而是读者回到真实任务时使用的工具箱。

怎样使用附录

- **原则与概念：**附录 A 是五条宣言；附录 D 用于快速对齐核心术语。
- **工件与检查：**附录 B 定位开源治理资产；附录 E 速查五道门；附录 I 指导四大动态工件如何建立与检查。
- **真实任务：**附录 G 是从任务入口到发布回流的端到端工作表；附录 H 回答实践中最常见的十个质疑。
- **继续学习与行动：**附录 C 说明图书、Portal 与工作坊分别承担什么；附录 F 提供公开资源、实践路径与联系入口。

第一次实践时，直接从附录 G 开始：选择一个影响可控的真实任务，完成意图、约束、证明、裁决和回流。遇到概念、工件或门禁问题，再回到相应附录查询。

附录 A | Agentic Agile 宣言

- 意图锚定，胜过详尽规约。
- 多模态涌现协作，胜过仅依赖跨职能人类团队。
- 算法规则与架构韧性，胜过仅依赖流程纪律与仪式。
- 实时遥测与闭环对抗，胜过仅依赖阶段性复盘。
- 基于验证的系统工程，胜过随性提示与氛围编程。

五条宣言定义的是冲突发生时的优先级：先锚定价值与边界，再组织人机协作；把可重复的控制写入运行环境；用运行事实纠正偏差；最终以独立证据支持行动裁决。

附录 B | 开源治理资产速查

Agentic Agile 3-4-3 开源仓库提供可复用的治理资产。它们不是照抄即用的流程表，而是帮助团队把责任、边界、证据和发布事实落到真实任务中的起点。

资产类别	用途	典型工件
意图与范围	定义目标、非目标、验收与允许变更范围	Intent Graph、Intent Contract、Change Envelope
约束与验证	把规则转成可检查边界和风险匹配的验证	Constraint Matrix、Verification Plan、AI Coding Guide
执行与协作	组织任务依赖、工具权限和跨模块协作	Work Graph、Tools Manifest、Protocol
证据与发布	绑定制品、验证结果、裁决与回退事实	Evidence Bundle、Release Manifest
学习与改进	保存经验证的经验，支持下一轮任务	Loop Memory、遥测与参考示例

从一个真实任务开始，只取当前风险需要的最小工件。低风险单模块任务通常不需要完整 Protocol；涉及敏感数据、不可逆变更或跨模块发布时，再提高约束与证据强度。

- [访问 Agentic Agile 3-4-3 开源仓库](#)
- 具体模板、安装方式和兼容性以仓库当前发布说明为准。

附录 C | 图书与课程学习地图

这本书给出 Agentic Agile 的完整判断框架；真正的能力，要在真实任务、练习反馈和组织共创中形成。Portal 在线课程与线下工作坊共享本书的价值观和操作系统主干，但承担不同任务：在线课程帮助个人跑通能力闭环，线下工作坊帮助团队完成转型判断与首轮行动设计。

四门 Portal 在线课程

访问 [Agentic Agile Portal](#)，可进入在线课程、公开资源与持续更新的实践内容。

课程	适合谁	你将获得什么
《Agentic Agile 认知课：AI 转型为什么不是工具升级》	希望建立 AI First 判断框架的管理者、产品与研发同学	看清局部工具提效为何不能自动成为组织提效，找到真正需要重构的人员、组织、流程与工具问题
《Agentic Agile 3-4-3 基础课：跑通第一次受控自治》	准备亲手实践 AI Coding 的研发、测试、产品与架构师	在本地 AI 工具中完成一次真实工程任务，留下契约、约束、执行证据与裁决结果
《Agentic Agile 组织转型进阶：构建人机协同研发操作系统》	CTO、研发负责人、平台负责人、敏捷教练与转型团队	将任务闭环扩展为责任单元、协作协议、授权边界和组织级行动，形成首轮转型路线
《Agentic Agile FDE 进阶：从客户现场到企业级生产交付》	FDE、交付团队、解决方案架构师与企业实施负责人	在客户现场或明确标记的模拟环境中，完成事实、场景、价值流与交付证据闭环

课程不是“看完视频再做题”。每个 Mission 都围绕任务、判断与阶段产物展开，无敌哥 AI 学习伙伴会辅助你完成深度思考和实战，而非只完成一次观看。

线下工作坊：把共识变成组织行动

《Agentic Agile（智能体敏捷）：从碳基协作到硅基自治——AI 时代研发组织重构与转型沙盘》面向正在进入 AI 转型深水区的企业团队。

工作坊以 AI First / AI Native 和四维重构为主线：先确认组织的真实转型事实、价值流和责任断点；再围绕真实或明确标记为模拟的工程任务，完成意图、约束、证据、裁决与扩展判断；最后形成一张连接真实价值流、任务证据和首轮行动的转型画布。

它不是在线课程的线下复述，也不是一份口号式转型规划。它要求团队面对自己的流程、边界、系统约束和管理取舍，做出可验证的首轮决定。

从阅读到实践

认知课：看清转型断点

↓

3-4-3 基础课：完成一次任务级受控自治

↓

组织转型进阶：把任务闭环扩展为责任网络

↓

FDE 进阶：进入客户现场与企业级交付

↓

线下工作坊：把个人能力变成组织级转型行动

这不是强制的购买顺序，而是一条能力形成路径。管理者可以从认知课和线下工作坊进入；工程人员可从 3-4-3 基础课开始；组织与平台负责人重点进入组织转型进阶；FDE 与交付团队则从 FDE 进阶进入。

图书、Skill、Portal 与工作坊的边界

图书：建立价值观、模型与判断框架

Learning Skill：在本地 AI 工具中引导学习、反馈与生成学习证据

343 Skill：把真实工程任务变成可治理、可验证、可回滚的运行闭环

Portal：提供课程、Mission、进度、公开资源与实践入口

线下工作坊：将个人闭环带回组织，完成四维转型与扩展裁决

从书中获得认同并不等于完成转型。选择下一步：进入 Portal 跑通一次任务级闭环，或带着真实价值流进入工作坊，开始重构你的研发组织。

附录 D | 术语表

术语	含义
Agentic Agile	面向智能体时代，对研发人员、组织、流程与工具进行系统重构的人机协同研发操作系统；治理是其运行机制，不是全部目的
四维重构	对人员、组织、流程与工具四个相互耦合的维度共同重构，而不是只引入 AI 编码工具
3-4-3	三个责任角色、四大动态工件、三大自治机制构成的最小运行架构
IO	意图主理人，负责价值、取舍和签署
OA	编排架构师，负责治理设计和异常升级
AS	在授权边界内执行的自治蜂群
Intent Graph	连接战略、价值目标、用户、能力、规则、任务与证据的意图关系图；按当前决策需要逐步扩展，不追求一次画出全局
Intent Contract	单任务目标、非目标、规则、AC 和签署
Constraint Matrix	机器可执行的边界和门禁来源
Evidence Bundle	支持人类裁决的结构化证据
四大动态工件	Intent Graph、Intent Contract、Constraint Matrix、Evidence Bundle；它们有所有者、状态、版本和触发事件，并随真实运行持续变化
SCOPE-V	Specify、Constrain、Orchestrate、Prove、Evolve、Verify
Verify	依据证据判断当前状态支持发布、条件通过、阻断还是补充验证；它不是“测试通过”的同义词
Telemetry	从已验证任务事实中派生价值、能力、效率和进化信号的慢外环；它不是第五个动态工件
Measurement Contract	规定指标定义、数据源、口径、责任人和失效处理的度量契约；它约束遥测解释，不增加新的运行时工件
Harness	包裹概率性执行的约束、验证和恢复系统
Work Graph	描述任务节点、依赖、输入输出、责任、并行与失败路由的工作关系图；无环任务通常用 DAG 表达，存在重试、修复与回流时可包含受控循环
HITL	需要人类承担判断或批准责任的介入点
Context Engineering	设计 Agent 可获得的上下文、边界、来源和新鲜度，而不只是堆叠提示词
Harness Engineering	把约束、工具权限、测试、门禁、证据与恢复机制工程化地包裹在 Agent 外部
Tools Manifest	声明可用工具、权限、来源、版本和风险边界的清单
Verification Plan	根据风险说明验证层次、方法、环境、通过标准与责任人的计划
Change Envelope	明确本次允许修改与禁止触及范围的变更边界
Release Manifest	将发布制品、版本、证据、审批、环境与回滚事实绑定的清单
Recon	在修改既有系统前重建代码、依赖、行为、测试和风险认知的侦察活动

Preserve	先用特征测试或基线证据固定现有行为，再实施有界变更
UNKNOWN	当前没有足够证据判断，必须保留未知状态，不能自动视为通过
NOT_APPLICABLE	经明确规则和责任人判定后不适用，与“未执行”不同
快内环 / 慢外环	快内环用于单任务执行、验证和恢复；慢外环用跨任务遥测调整规则、能力和组织设计
验证工程	关键行动由可复查证据支持，并允许失败、未知和残余风险被如实表达的工程方式

--

附录 E | 五道门速查

五道门是状态转换的裁决点，不是按顺序移交给不同部门的五个阶段。前置事实不足时，任务不得以“先做再说”的方式越过门槛。

门	时机	核心问题	不通过时
前置门（Pre）	业务编码前	契约是否签署，约束、风险和 AC 是否准备好	停止进入业务编码，补充澄清或签署
编码门（Coding）	实现前	关键测试是否先写，并真实证明过 Red	修正测试或任务理解，不进入实现
验证门（Prove）	Verify 前	测试、AC、构建、约束和证据是否支持当前结论	修复、补证或如实保留 FAIL / UNKNOWN
收尾门（Closing）	宣告完成前	Evidence Bundle、回写与发布事实是否完整	不得宣告完成，补齐事实或撤回结论
Bug 回溯门（Bug）	完成后发现 Bug	原裁决、证据和组织记忆是否需要修正	控制影响，修正记录，回写根因与改进行动

门禁的价值不在于增加审批，而在于让系统在事实不足时停止扩张影响范围。

附录 F | 公共资源与行动入口

公共资源

- [Agentic Agile Portal](#): 在线阅读、课程与公共资源
- [本书 GitHub 仓库](#): 章节 Markdown、PDF、EPUB 与版本发布
- [Agentic Agile 3-4-3](#): 开源治理资产

从今天开始

1. 选定一条真实价值流，找出影响可控、结果可验证的任务。
2. 指定真实 IO，写清目标、非目标、验收条件、约束与 UNKNOWN。
3. 在明确的 Change Envelope 内执行，用独立证据证明实现。
4. 由责任人依据 Evidence Bundle 决定发布、补证、收缩或停止。
5. 将验证过的结果回写为规则、测试、知识或记忆，再决定是否扩大自治。

不要从“全员接入 Agent”开始。从一个可信闭环开始。

联系与共建

- 微信: iloveagile
- Email: 3433839@qq.com
- 欢迎提出问题、实践案例与改进建议；请避免提交客户数据、生产凭证或其他敏感信息。

附录 G | 端到端案例工作表：从一句话需求到可验证发布

这份工作表用于把一个真实需求推进到可验证发布。填写时不必追求文档漂亮，重点是让意图有人签署、边界能够执行、证据可以复查、行动由有权者裁决。

G.1 任务入口

字段	填写内容
Task ID	
业务负责人 / IO	
OA	
目标结果	
截止时间与原因	
受影响价值流	

把原始需求原样保存，再列出至少三条歧义。没有歧义清单，不应直接进入实现。

G.2 意图与边界

```
goal: ""
non_goals:
  - ""
acceptance_criteria:
  - id: AC-01
    given: ""
    when: ""
    then: ""
constraints:
  - id: MUST-01
    rule: ""
    failure_action: stop
unknowns:
  - ""
```

IO 确认 Goal、Non-goals、AC、约束和 UNKNOWN；OA 可以提出修改建议，不能替代责任签署。

G.3 Work Graph 与责任

节点	输入	输出	允许写入	前置条件	异常升级给
测试 / 证明	契约、边界、样例	Red、测试证据	测试目录	契约已签署	OA
实现	契约、Red、代码切片	候选变更	指定文件	Red 为 expected_red	OA
集成	候选制品、接口契约	集成结果	隔离环境	依赖版本固定	模块负责人
证据整理	原始结果、差异、版本	Evidence Bundle	证据目录	所有关键节点完成	OA
裁决	Evidence Bundle、残余风险	发布决定	裁决记录	证据可追溯	IO / 专业责任人

G.4 Prove 证据表

AC / MUST	证明方式	原始证据位置	版本 / 环境	状态
				PASS / FAIL / UNKNOWN
				PASS / FAIL / UNKNOWN
				PASS / FAIL / UNKNOWN

Prove 只确认实现是否满足契约；是否采取发布行动，由 Verify 基于完整证据裁决。

G.5 Verify 裁决

已经证明：

-

尚未证明：

-

残余风险：

-

本次行动：批准 / 条件批准 / 补证 / 拒绝 / 撤回 / 升级

批准范围：

有效期：

监控指标：

自动回退条件：

Decision Owner：

G.6 发布后 Telemetry

信号	基线	观察窗口	触发动作	责任人
业务结果				
错误 / 逃逸缺陷				
周期 / 等待 / 返工				
人工介入与恢复				
成本与资源				

Telemetry 用于判断长期能力和净收益，不能用 Agent 数量、Token 或调用次数直接代替价值结果。

G.7 复盘与回写

本次任务只回写经过证据支持的事实：

- 新增或修正了什么约束？
- 哪类测试或上下文应成为默认模板？
- 哪个失败模式需要进入知识与记忆？
- 哪项授权应扩大、收缩或保持不变？
- 下一项相似任务如何验证迁移，而不是复制表演？

附录 H | 常见质疑与回答

H.1 “这不就是把敏捷流程重新命名吗？”

敏捷解决增量交付和快速反馈；Agentic Agile 处理新的执行现实：Agent 能调用工具、修改多个文件、并行协作并快速扩大错误。治理对象因此延伸到意图、权限、上下文、机器门禁、证据和自治边界。

H.2 “小需求也要写这么多文档吗？”

治理重量应与风险匹配。低风险单模块任务可以只保留最小契约、核心约束、可执行 AC 和证据；跨模块、不可逆或高风险任务才需要完整 Protocol、Work Graph、XC 与批准链。减少的是无关字段，不能减少目标、边界、证据和责任。

H.3 “模型越来越强，以后还需要这些吗？”

模型能力会提升实现质量，却不会自动获得组织授权，也不能承担法律、商业和伦理后果。能力越强、作用范围越大，越需要明确边界、停止条件、证据和恢复方式。

H.4 “TDD 会不会拖慢 AI Coding？”

可信 Red 和验收条件减少的是错误方向上的高速实现与事后争论。应优化测试分层和执行速度，而不是取消证明环节。

H.5 “Evidence Bundle 会不会变成新的形式主义？”

会，如果它只是人工拼接的截图。有效证据应可追溯到原始结果、环境、时间和适用范围，并如实保留未验证事项。它服务于行动裁决，不服务于展示。

H.6 “为什么 IO 必须本人签署？”

目标、非目标和残余风险属于价值责任。OA 可以澄清，Agent 可以执行，但不能代替业务责任人决定什么值得做、什么风险可以接受。

H.7 “多 Agent 一定比单 Agent 快吗？”

不一定。并行收益受任务独立性、共享资源、接口稳定性和验证成本限制。不可分解任务强行并行，只会增加冲突、交接和返工。

H.8 “发生 Bug 是否说明证据失效？”

证据证明的是特定范围、时间和环境下的结论，不是绝对保证。发现 Bug 后，应控制影响、修正原裁决和证据、追溯根因，并把有效教训回写到下一轮任务。

H.9 “已经有 DevOps、CI/CD 和平台工程，还需要 3-4-3 吗？”

这些能力是重要基础，但它们不会自动回答意图由谁签署、Agent 权限如何裁剪、跨 Agent 交接怎样验证、残余风险由谁接受。3-4-3 将现有工程能力连接到意图与责任，不要求推倒重建。

H.10 “怎样证明这套方法值得投入？”

选择一个低风险真实任务建立基线，同时观察歧义发现时点、返工、越权阻断、验证成本、发布后缺陷和人工介入质量。若治理成本没有换来更早风险发现、更少返工或更清晰决策，就应调整方法，而不是美化数据。

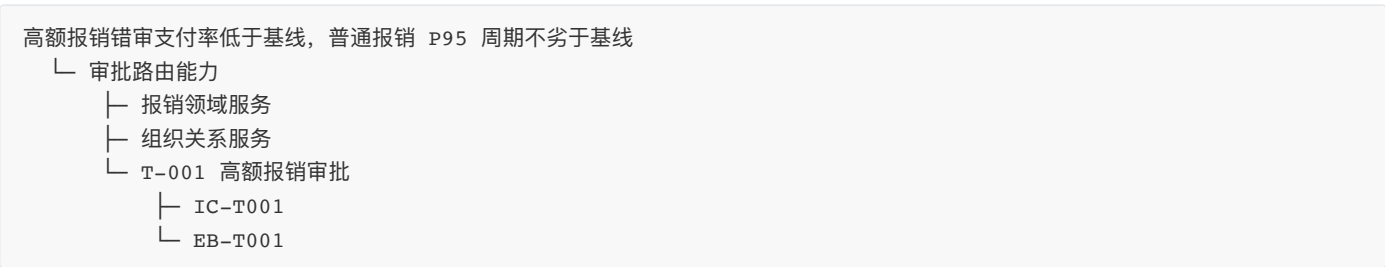
附录 I | 四大动态工件实战指南

四大动态工件组成同一条治理链：Intent Graph 连接目标与依赖，Intent Contract 锁定一项任务的承诺，Constraint Matrix 定义不可突破的边界，Evidence Bundle 支持行动裁决。它们可以由 Markdown、YAML、数据库或平台对象承载；稳定的应是语义、责任与追溯关系。

工件	最小内容	建立时机	完成时应能回答
Intent Graph	目标、能力、模块、任务、证据之间的关系	开始理解价值流或既有系统时	这项任务为什么存在，影响哪些能力与模块？
Intent Contract	Goal、Non-goals、AC、约束、UNKNOWN、签署	进入业务编码前	做什么、不做什么、怎样判定完成、谁负责？
Constraint Matrix	规则、范围、检查方式、失败动作、例外责任	任务契约形成后	Agent 可以做什么，触线后必须怎样处理？
Evidence Bundle	版本、原始结果、AC/MUST 映射、未知项、回退与裁决	执行和验证过程中持续积累	当前证据支持什么行动，什么仍不确定？

I.1 Intent Graph：只画当前任务需要的关系

从一个真实任务向外扩展，不要先建一张“全企业地图”。最小图谱通常只需要业务目标、能力、模块、任务和证据五类节点。



只回写经验证的依赖、字段语义和风险；推测必须标为假设。每个关键节点都应有 Owner、最后验证时间和可能失效条件。

检查：能否从任务追溯到业务目标、约束与证据？事实与假设是否可区分？过期关系能否被发现？

I.2 Intent Contract：写清会改变实现或裁决的内容

契约不是更长的需求文档。它只记录会改变行动的事实：可观察目标、非目标、规则与边界示例、验收条件、已知假设、待决问题、风险与回退要求、数据和权限边界，以及 IO/OA、版本和签署时间。

目标“优化审批体验”不可裁决；“在不改变 5000 元及以下路径的前提下，对中国区新提交且本位币金额大于 5000 元的差旅报销增加成本中心总监审批节点”可以验证。

检查：不同执行者是否会得到相近理解？每个 MUST 是否有判定方法？边界值、异常、非目标与回退是否明确？是否由真实 IO 签署？

I.3 Constraint Matrix：让边界能够执行

每条约束至少包含适用范围、强度、检查方式、失败动作、例外责任人和证据位置。业务规则、数据与隐私、架构与依赖、工具与权限、运行与发布都应有对应边界。

“注意数据安全”不是约束；“Agent 不得读取生产客户明细，失败即停止并通知安全责任人”才是可执行边界。MUST 只用于违反后不能继续的规则；SHOULD 和 MAY 必须保留例外理由与责任。

检查：规则能否由脚本、策略或明确人工动作检查？失败后是停止、降级、回退还是告警？例外是否有时限、责任人和复核？

I.4 Evidence Bundle：组织事实，不包装结论

证据包应持续积累：任务与契约版本、代码和配置变更、AC/MUST 对应结果、原始命令与环境、构建与测试、批准与例外、UNKNOWN、残余风险、回退验证和遥测入口。它可以推荐裁决，不能替代责任人作出裁决。

证据必须允许得出“不发布”。没有证据、证据证明失败、经裁决不适用是三种不同状态，不能用一个绿色标签掩盖。

检查：每个 MUST 是否能找到原始证据？时间、环境和版本是否可识别？残余风险能否被非开发角色理解？发布后发现 Bug 时能否修正并追溯？

I.5 四个工件怎样一起检查

目标一致：Contract 的目标来自 Graph 中有效意图

边界一致：Contract 的规则被 Matrix 覆盖

验证一致：每项 MUST AC 都映射到 Evidence

版本一致：签署、检查和证据针对同一版本

责任一致：批准者与授权边界匹配

回写一致：经验证的新事实回到 Graph 和规则资产

低风险任务可以使用简短工件并继承默认约束；涉及敏感数据、不可逆变更、跨模块发布、新工具或无法自动验证的关键判断时，必须提高边界与证据强度。轻量化减少的是无关字段，不是责任。

结语 | 从碳基协作到硅基自治

AI 已经进入研发的执行链。它可以理解需求、生成方案、修改代码、运行测试、分析故障并提出修复。研发组织正在进入一个新的阶段：人定义方向、作出判断并承担责任，Agent 承接越来越多可以被授权和验证的执行。

真正的分水岭不在于团队是否接入了模型，而在于组织是否重新设计了责任。没有明确意图的速度，只会更快地偏离目标；没有约束的自治，只会更高效地扩大风险；没有证据的“完成”，只是把判断推给下一个人、下一个环节或下一次事故。

Agentic Agile 的立场很清楚：

人定义价值，承担最终责任

Agent 在授权范围内执行

Harness 守住系统边界

Evidence 支持行动裁决

Telemetry 把每次运行变成下一次能力

这不是给旧研发流程加上一层 AI 工具，而是重建研发组织的运行方式。目标不只是更快交付代码，而是在速度提升时，仍能说明目标由谁签署、边界由谁设定、系统做过什么、证据证明了什么、剩余风险在哪里，以及谁有权停止、回退和继续。

未来的竞争，不取决于谁拥有最多模型、插件或 Token，而取决于谁能把速度、责任、证据和自治组织成一个系统。这样的团队不把 Agent 当作临时助手，而是把它纳入清晰的责任网络，让每一次执行都为下一次任务留下可复用的能力。

现在不需要等待一份完美蓝图。

选择一条真实价值流，挑选一个影响可控、结果可验证的任务，指定意图责任人。签署目标与边界，为 Agent 设置可执行约束，先证明再交付，用证据决定发布范围，并把结果回流到下一项任务。

跑通这第一个闭环，你的组织就不再只是“开始使用 AI”。它已经开始形成下一代研发能力。

从今天开始，重构研发组织。

让人定义价值，让 Agent 执行价值，让证据证明价值。

—— Agentic Agile：从碳基协作到硅基自治 ——

1. 参见 Paul A. David, *The Dynamo and the Computer: An Historical Perspective on the Modern Productivity Paradox*。该研究以电气化说明：通用技术的生产率收益，依赖互补性组织创新以及工厂布局、物料流和生产方式的重构。[↗](#)

2. 参见 The Henry Ford, *How did Henry Ford's Model T assembly line work and how did it evolve?*。其档案资料记载，福特于 1913 年形成移动装配线，到 1914 年初可每 93 分钟完成一辆 Model T，而固定装配方式约需 12.5 小时。[↗](#)

3. 参见 Smithsonian National Museum of American History, *Transforming the Waterfront*，以及 NBER, *All Aboard: The Effects of Port Development*。前者记录了 *Ideal-X* 于 1956 年装载 58 个集装箱完成首次航行，也指出装卸环节曾占运输成本的近一半；后者以集装箱运输的引入研究港口发展及其跨区域经济影响。[↗](#)